

STUDY GUIDE

Operating Systems

Chapters 1–5: from the hardware up to concurrency

This printable guide collects the notes, key terms and review questions (with answers and explanations) from the interactive guide. Use it to review after working through the simulations.

1. Computer System Overview
2. Operating System Overview
3. Process Description and Control
4. Threads
5. Concurrency: Mutual Exclusion and Synchronization

Contents

1. Chapter 1: Computer System Overview

- 1.1 Basic Elements
- 1.2 Evolution of the Microprocessor
- 1.3 Instruction Execution
- 1.4 Interrupts
- 1.5 The Memory Hierarchy
- 1.6 Cache Memory
- 1.7 Direct Memory Access
- 1.8 Multiprocessor and Multicore Organization

2. Chapter 2: Operating System Overview

- 2.1 Operating System Objectives and Functions
- 2.2 The Evolution of Operating Systems
- 2.3 Major Achievements
- 2.4 Developments Leading to Modern Operating Systems
- 2.5 Fault Tolerance
- 2.6 OS Design Considerations for Multiprocessor and Multicore
- 2.7 Microsoft Windows Overview
- 2.8 Traditional Unix Systems
- 2.9 Modern Unix Systems
- 2.10 Linux
- 2.11 Android

3. Chapter 3: Process Description and Control

- 3.1 What Is a Process?
- 3.2 Process States
- 3.3 Process Description
- 3.4 Process Control
- 3.5 Execution of the Operating System
- 3.6 UNIX SVR4 Process Management

4. Chapter 4: Threads

- 4.1 Processes and Threads
- 4.2 Types of Threads
- 4.3 Multicore and Multithreading
- 4.4 Windows Process and Thread Management
- 4.5 Solaris Thread and SMP Management
- 4.6 Linux Process and Thread Management
- 4.7 Android Process and Thread Management

- 4.8 Mac OS X Grand Central Dispatch

5. Chapter 5: Concurrency: Mutual Exclusion and Synchronization

- 5.1 Mutual Exclusion: Software Approaches
- 5.2 Principles of Concurrency
- 5.3 Mutual Exclusion: Hardware Support
- 5.4 Semaphores
- 5.5 Monitors
- 5.6 Message Passing
- 5.7 Readers/Writers Problem

Chapter 1 · Computer System Overview

You cannot understand a manager without knowing what it manages. First, meet the machine.

An operating system is software that runs on real hardware and spends its whole life managing that hardware. This chapter opens the box: the four basic parts of a computer, how the processor grew from one simple chip into today's chips that hold several processors (called cores), how it runs one instruction after another, and how interrupts let slow devices call for attention. It then shows why memory is built in layers, from small and fast to big and slow, with a cache at the top; how direct memory access lets a device move bulk data without the processor; and how several processors or cores share one machine.

Learning objectives

- Name the basic elements of a computer and describe how the system bus connects them.
- Describe how the microprocessor grew into multicore systems on a chip.
- Trace the fetch and execute steps a processor repeats for every instruction.
- Explain what an interrupt is and why it keeps the processor from wasting time.
- Use locality to explain the memory hierarchy and how a cache speeds up memory access.
- Compare programmed I/O, interrupt-driven I/O and direct memory access (DMA).
- Describe symmetric multiprocessors and multicore chips.

How Chapter 1 fits together

An operating system is a manager, and this chapter introduces what it manages. Everything here is hardware, but each topic returns later as an OS problem.

The parts (1.1, 1.2)

A computer has four basic elements. The **processor** does the work, using **registers** (tiny, very fast storage slots such as the program counter and instruction register). **Main memory** holds running programs and their data. **I/O modules** connect external devices such as disks, keyboards and network cards. The **system bus** ties them together by carrying addresses, data and control signals. Section 1.2 shows how the processor grew from one simple chip into chips holding several cores, graphics units and other specialised processors.

The heartbeat (1.3, 1.4)

A processor repeats one loop for its whole life: **fetch** the instruction whose address is in the program counter, then **execute** it. Section 1.4 adds a third stage to that loop: after each instruction the processor checks for an **interrupt**. Interrupts let a slow device announce that it has finished, so the processor can run other programs instead of waiting. The same mechanism is how the OS regains control, which makes it the foundation for everything in Chapters 2 to 5.

The speed gap (1.5, 1.6)

Fast memory is expensive and small; cheap memory is large and slow. The **memory hierarchy** stacks them: registers, cache, main memory, then disks. It works because programs show **locality**: they tend to

reuse the same and nearby addresses. A **cache** exploits locality by keeping copies of recently used memory blocks next to the processor, so most accesses finish at cache speed.

Moving data in bulk (1.7)

There are three ways to do I/O. With **programmed I/O** the processor keeps checking the device and wastes its time. With **interrupt-driven I/O** it does other work until the device interrupts, but it still copies every word itself. With **direct memory access (DMA)** a separate module copies a whole block between the device and memory, and the processor is interrupted only once at the end.

More than one processor (1.8)

A **symmetric multiprocessor** has several similar processors sharing one memory and one set of I/O devices, all controlled by one OS. A **multicore** chip puts several such processors (cores) on one chip, often with some cache levels shared. More processors mean more work at once, and also a new problem that Chapters 4 and 5 tackle: keeping them from interfering with each other.

The common theme

The processor is far faster than memory and devices. Interrupts and DMA stop it from waiting on devices, the cache stops it from waiting on memory, and multiple cores add raw capacity. An OS is built on top of all three ideas.

1.1 Basic Elements

You should be able to

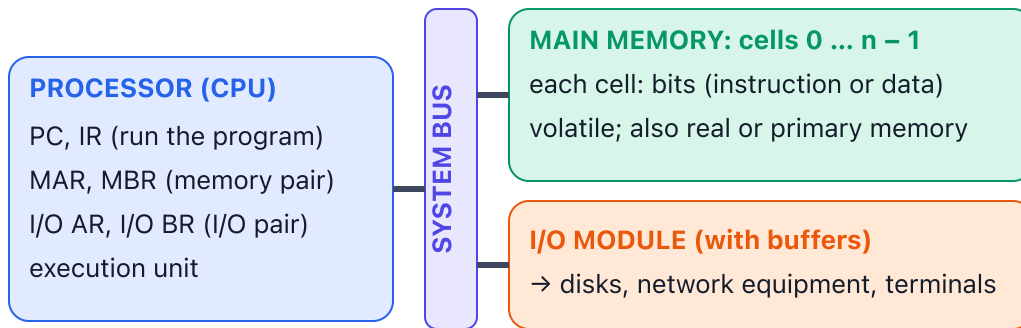
- Name the four main structural elements of a computer and describe the job of each one.
- Explain what the MAR, MBR, I/O AR and I/O BR hold, and which pair the processor uses for memory and which for I/O.
- Describe main memory as cells numbered 0 to $n - 1$ whose bit patterns can be read as instructions or as data.
- Trace how a value travels over the system bus during a memory read, a memory write and an I/O transfer.
- Explain why main memory is volatile and why every I/O module needs internal buffers.

1. Four parts that work as a team

A computer consists of processor, memory and I/O components, connected so they can work together to **execute** (run) **programs**. At the top level there are four main structural elements:

- **Processor.** Controls the operation of the whole computer and performs its data processing. It runs programs by fetching their instructions and carrying them out. When a machine has just one processor it is usually called the **central processing unit (CPU)**.
- **Main memory.** Stores the programs being run and the data they use. It is usually **volatile**: when the power goes off, its contents are lost. Also called **real memory** or **primary memory**.
- **I/O modules.** Move data between the computer and its **external environment**, in both directions. It consists of devices: **secondary memory devices** such as disks, **communications equipment** such as network cards, and **terminals** (keyboard and screen).

- **System bus.** Provides communication among the processor(s), main memory and the I/O modules. It carries **address** signals (where), **data** (what) and **control** signals (read or write, and when).



2. Registers inside the processor

A **register** is a tiny, very fast storage slot inside the processor. The **program counter (PC)** holds the address of the next instruction to fetch and the **instruction register (IR)** holds the instruction being carried out (Section 1.3). The **execution unit** does the arithmetic, comparisons and moves. A real processor has many more registers, but four exist only to exchange data with the rest of the machine:

	Where (address)	What (data)
Memory	MAR (memory address register): the address in memory for the next read or write	MBR (memory buffer register): the data to be written into memory, or the data just read from it
I/O	I/O AR (I/O address register): specifies a particular I/O device	I/O BR (I/O buffer register): the data exchanged between an I/O module and the processor

An address register never holds the data, and a buffer register never says where: the number 9 is a place in the MAR but a value in the MBR. Memory uses the MAR/MBR pair; devices use the I/O AR/I/O BR pair.

3. Main memory as numbered cells

Main memory is a set of locations numbered one after another. Each number is the location's **address**; with n locations the addresses run from **0 to $n - 1$** . Each location holds a binary number, a pattern of **bits**, that can be an **instruction** or **data**. The bits do not label themselves: a pattern is an instruction if the processor fetches it as one, and data if a program uses it as a value. For example, 0001 0000 0000 1000 (hex 1008; one hex digit stands for 4 bits) is the number 4104, but in a toy machine whose instructions use the first 4 bits for the operation and the last 12 for a cell number it means "load from cell 8". A cell is never truly empty; it always holds some bits. (Toy cells here hold 16 bits; real machines usually give each 8-bit byte its own address.)

Reading hex. Long bit patterns are easier to read in **hexadecimal** (base 16). Each hex digit stands for exactly 4 bits and counts 0–9 and then A–F, where A = 10 and F = 15. Each place is worth 16 times the place to its right, so hex 0019 = $1 \times 16 + 9 = 25$, hex 0011 = 17 and hex 002A = $2 \times 16 + 10 = 42$: the toy program's $25 + 17 = 42$, written in hex. **Text** is stored as numbers too: each letter has a number code (H = 72, i = 105), so the 16 bits of hex 4869 can also be read as the two letters "Hi". If the first hex digit of a word is not an operation the machine knows, the word cannot run as an instruction and can only be data.

Worked examples. k address bits give 2^k addresses. 5 bits $\rightarrow 2^5 = 32$ cells, addresses 0 to 31. 10 bits $\rightarrow 1024$ cells, 0 to 1023. A memory with 4096 locations has last address $4096 - 1 = 4095$. Classic mistakes: saying the last address is n (it is $n - 1$), and confusing a cell's address with its contents.

4. Following the data over the bus

1. **Memory read:** address $\rightarrow$ MAR; it goes out on the address lines with a READ signal; memory puts a copy of the cell's contents on the data lines; the value lands in the MBR. The cell is unchanged.
2. **Memory write:** address $\rightarrow$ MAR and value $\rightarrow$ MBR; address, data and WRITE go out together; memory stores the value; the old contents are gone.
3. **Output to a device:** device number $\rightarrow$ I/O AR, data $\rightarrow$ I/O BR; the bus carries them with an I/O WRITE signal; only the module whose number matches takes the data into its buffer, then feeds the device at its own pace.
4. **Input into memory:** device number $\rightarrow$ I/O AR with I/O READ; the module puts the buffered item on the data lines; it arrives in the I/O BR; the processor then writes it to memory using the MAR and MBR. (DMA, Section 1.7, spares the processor this chore.)

5. Why I/O modules have buffers

An I/O module transfers data from external devices to the processor and memory, and back the other way. Devices are slow and unpredictable while the processor is fast and busy elsewhere, so the module holds data in **internal buffers** until the other side is ready. It also holds the control logic for its kind of device, sparing the processor the details. **Worked example:** one key arrives every 0.4 s and the processor empties the buffer every 2 s, so $2 \div 0.4 = 5$ keys arrive between visits. A 4-slot buffer loses $5 - 4 = 1$ key per visit (20%); an 8-slot buffer loses none. Output is the reverse: the processor drops a burst into the buffer at full speed and moves on while the module feeds the slow device. A buffer is temporary, not permanent storage.

6. Volatile memory versus secondary memory

	Main memory	Disk (secondary memory)
Keeps data without power?	No (volatile)	Yes
Speed	Fast	Far slower
Typical size	Gigabytes	Hundreds of GB and up
How the processor reaches it	Directly over the system bus	Through an I/O module

Anything not saved to secondary memory is lost at power-off, even the OS, so every start-up begins by copying the OS from disk into main memory. In "16 GB memory, 512 GB storage", the first figure is main memory and the second is secondary memory.

7. The kitchen analogy

Processor = the chef, who carries out every recipe step. Main memory = the counter: what is in use now, wiped at closing. Disk = the storeroom: huge, keeps things overnight, slow to reach. I/O module = the pass

window, whose shelf (the buffer) lets orders and plates wait. System bus = the one shared aisle every trip uses. MAR = a slip naming a counter spot; MBR = the tray carrying the item.

Key terms

Processor (CPU)

The part of the computer that controls everything the machine does and performs its data processing. When a computer has just one processor, it is usually called the central processing unit, or CPU.

Main memory

The memory that holds the programs being run and the data they use, organised as a long row of numbered cells. It is usually volatile. Also called real memory or primary memory.

Volatile memory

Memory that loses its contents when the power is switched off. Main memory is volatile; a disk is not.

I/O module

The hardware unit that moves data between the computer and its external environment (disks, network equipment, keyboards, screens), holding data in internal buffers on the way.

System bus

The shared set of wires that lets the processor, main memory and I/O modules communicate. It carries addresses (where), data (what) and control signals (read or write, and when).

Register

A very small, very fast storage slot inside the processor that holds one value the processor is working with right now.

Memory address register (MAR)

The processor register that holds the address of the memory cell to be read or written next.

Memory buffer register (MBR)

The processor register that holds the data about to be written into memory, or the data just read out of memory.

I/O address register (I/O AR)

The processor register that names the particular I/O device the processor wants to exchange data with.

I/O buffer register (I/O BR)

The processor register that holds the data travelling between the processor and an I/O module, in either direction.

Program counter (PC)

The processor register that holds the memory address of the next instruction to fetch.

Instruction register (IR)

The processor register that holds the instruction currently being carried out.

Execution unit

The circuitry inside the processor that carries out the work of instructions, such as adding, comparing or moving values. A modern processor core contains several.

Address

The number that identifies one memory cell (or, for I/O, one device). In a memory with n cells the addresses run from 0 to $n - 1$.

Buffer

A small holding area where data waits for a short time while it travels between two parts that work at different speeds or at different moments.

Secondary memory

Large, permanent storage such as a disk or solid-state drive. It keeps its data without power but is much slower than main memory, and the processor reaches it through an I/O module.

Bit

A binary digit: a single 0 or 1. Everything in memory, instructions and data alike, is stored as patterns of bits.

Instruction

One basic command for the processor, such as “add these two numbers”, stored in memory as a pattern of bits.

Data

The values a program works on (numbers, letters, pixels and so on), stored in memory as bit patterns just like instructions.

Review questions with answers

1. Which structural element controls the operation of the computer and performs its data processing?

- A. Main memory
- B. The processor
- C. The system bus
- D. An I/O module

Answer: B. The processor

The processor (usually called the CPU when there is only one) runs programs by carrying out their instructions, and in doing so it controls the machine and performs the data processing. Memory stores, the bus carries, and I/O modules deal with devices.

2. Which of these are among the four main structural elements of a computer?

- A. Processor
- B. Main memory
- C. I/O modules
- D. System bus
- E. The operating system
- F. The keyboard

Answer: A, B, C, D

The four structural elements are the processor, main memory, I/O modules and the system bus. The operating system is software that runs on them, and a keyboard is an external device reached through an I/O module.

3. Match each register to what it holds.

MAR	The address of the memory cell for the next read or write
MBR	Data just read from memory, or about to be written to it
I/O AR	The number of the particular I/O device being addressed
I/O BR	Data passing between an I/O module and the processor
PC	The address of the next instruction to fetch

Address registers (MAR, I/O AR, and the PC for instructions) say *where*; buffer registers (MBR, I/O BR) carry the data itself.

4. Main memory keeps its contents when the computer is switched off.

Answer: False

Main memory is usually volatile: without power its bits are lost. Anything that must survive a shutdown has to be saved to secondary memory, such as a disk.

5. A main memory has 4096 locations, numbered in the usual way. What is the address of the last location?

Answer: 4095

Addresses start at 0, so n locations have addresses 0 to $n - 1$. With $n = 4096$ the last address is $4096 - 1 = 4095$.

6. A memory uses 10-bit addresses, and every possible address names a cell. How many cells can it have?

Answer: 1024 cells

Each extra address bit doubles the count, so k bits give 2^k addresses. $2^{10} = 1024$ cells, numbered 0 to 1023.

7. Put the steps of a memory read in order.

Correct order:

1. The processor places the cell's address in the MAR
2. The address goes out on the bus with a READ control signal
3. Memory places a copy of that cell's contents on the data lines
4. The value arrives in the MBR, ready for the processor to use

The *where* goes out first (MAR $\rightarrow$ address lines, with READ); then the *what* comes back (data lines $\rightarrow$ MBR). Reading leaves the cell unchanged.

8. Where is each thing found?

Processor	Main memory	I/O module
The MAR The I/O BR	A running program's instructions The OS, once loaded at start-up	A key press waiting in a buffer Control logic for one kind of device

Registers such as the MAR and I/O BR live inside the processor; running programs (the operating system included) live in main memory; an I/O module holds data in transit in its buffers and handles the details of its own device.

9. A memory cell holds the bit pattern 0001 0000 0000 1000. What is it?

- A. An instruction, because it starts with 0001
- B. Data, because every bit pattern is a number
- C. It could be an instruction or data; it depends on how the processor uses it
- D. The cell's own address, because every cell stores its address

Answer: C. It could be an instruction or data; it depends on how the processor uses it

Memory cells hold plain bit patterns. A pattern acts as an instruction if the processor fetches it as one, and as data if a program reads it as a value.

10. The processor wants to send the letter A to the printer. Which registers does it fill?

- A. MAR with the printer's number and MBR with the letter
- B. I/O AR with the printer's device number and I/O BR with the letter
- C. PC with the printer's number and IR with the letter
- D. I/O BR with the printer's number and I/O AR with the letter

Answer: B. I/O AR with the printer's device number and I/O BR with the letter

For I/O the processor uses the I/O pair: the I/O AR names the particular device (where) and the I/O BR holds the data being exchanged (what).

11. Why does an I/O module contain internal buffers?

- A. To store files permanently while the power is off
- B. To hold data briefly until the other side (processor or device) is ready to take it
- C. To make the external device itself run faster
- D. To hold the instructions of the program that is running

Answer: B. To hold data briefly until the other side (processor or device) is ready to take it

Processors and devices work at very different speeds and moments. A buffer lets data wait briefly, so neither side has to be ready at exactly the same instant and nothing is lost.

12. Which of these are part of the external environment that I/O modules connect a computer to?

- A. A disk drive
- B. A network card (communications equipment)
- C. A keyboard and screen (a terminal)
- D. Main memory
- E. The MBR

Answer: A, B, C

The external environment includes secondary memory devices such as disks, communications equipment and terminals. Main memory and the processor's registers are inside the computer itself; the processor reaches main memory directly over the system bus.

1.2 Evolution of the Microprocessor

You should be able to

- Explain what a microprocessor is and why putting a whole processor on one chip changed computing.
- Describe how chips evolved: more transistors, faster clocks until the power wall, then more cores, logical processors and deeper caches.
- Explain how SIMD lets a GPU or a CPU vector unit process many numbers with one instruction, and work out the speedup.
- Describe what DSPs and hardware accelerators do and why a system on a chip combines them with CPU cores, caches, radios and memory.
- Match a workload to the kind of processing unit that handles it best.

1. What a microprocessor is

A **microprocessor** is a complete processor (arithmetic, registers and control) on a **single chip** of silicon. Before 1971 a processor was built from many separate chips (earlier still, from transistors or vacuum tubes) spread over several boards. The first commercial microprocessor (1971) held about 2,300 transistors. One chip made processors small, cheap and fast, which made desktop and then handheld computers possible, and microprocessors became the fastest general-purpose processors ever built. Three eras: a processor made of many chips; a one-chip CPU with memory, graphics, sound and modem on separate chips or cards; today's **system on a chip (SoC)**.

2. Fifty years of evolution

- **More transistors.** A transistor is a tiny electrically controlled switch. The count per chip roughly doubles every two years (**Moore's law**): count after n years $\approx \text{start} \times 2^{n/2}$. Example: 1 billion, 10 years later $\rightarrow 2^5 = 32$ billion. From 2,300 in 1971, 25 doublings give ≈ 77 billion by 2021; real chips reached 16 billion (2020) and 92 billion (2023).
- **Faster clocks, for a while.** Clock speed (cycles per second, in hertz) rose about 5,000-fold, from 0.74 MHz (1971) to 3.8 GHz (2004), then stalled near 3–4 GHz: the **power wall**. Power rises steeply with clock speed (faster switching also needs a higher voltage), and chips could not shed the heat.
- **Then more cores.** From the mid-2000s extra transistors became extra cores (2, 4, 8, 16...) and specialised units.
- **Deeper caches.** Early chips had no cache; on-chip caches arrived around 1989. One level (L1) by 1993, two by 2000, a shared third by 2008, because processors sped up far faster than main memory.
- **Two big families.** **x86** began with Intel's 8086 (1978) and dominates PCs and servers (Intel, AMD). **ARM** designs are licensed to many chip makers and run nearly every smartphone, plus a growing number of laptops and servers.

Year	Chip	Transistors	Clock	Cores	Cache levels
1971	Intel 4004	2,300	740 kHz	1	none
1978	Intel 8086 (x86)	29,000	10 MHz	1	none
1985	ARM1 (ARM)	25,000	6 MHz	1	none
2004	Late Pentium 4	125 million	3.8 GHz	1 (2 logical)	L1, L2
2008	Core i7	731 million	3.2 GHz	4 (8 logical)	L1, L2, L3
2023	Apple M3 Max (ARM)	92 billion	4.05 GHz	16	L1, L2, L3

All values approximate and rounded.

3. Inside a modern multicore chip

- **Cores:** each is a complete processor with its own registers, control logic and **execution units** (for example two integer units “Int” for whole-number math, a floating-point unit “FP” for numbers with fractions, and a load/store unit “Ld/St” that moves data between memory and registers).
- **Cache levels:** on a typical recent laptop or desktop chip, each core has a private **L1** (about 32–64 KB, ~4–5 cycles) and **L2** (256 KB–2 MB, ~12–16 cycles); all cores share a large **L3** (8–64 MB, ~30–70 cycles). Main memory, reached through the on-chip memory controller, takes ~80–100 ns: hundreds of cycles.
- **Logical processors:** a **thread** is one independent stream of instructions being carried out. With **simultaneous multithreading (SMT)**, one core holds the registers of two threads and presents two **logical processors** (hardware threads) sharing its execution units. Logical processors = cores × threads per core: $4 \times 2 = 8$; $6 \times 2 = 12$.

Why SMT helps: one thread often leaves units idle, especially while it waits for memory; a second thread fills those slots. In the toy example thread A alone used 11 of 32 slots (34%); with a second thread, 21 of 32 (66%). Real gains are usually around 10–30%, because the threads also compete for the same units and caches. **Common mistake:** two logical processors are not two cores.

4. GPUs and SIMD

SIMD (single instruction, multiple data): one instruction performs the same operation on many data items at once, each in its own **lane**. An ordinary **scalar** instruction handles one item. With L lanes, N items need $N \div L$ instructions: 16 numbers with 4 lanes → 4; 1,024 with 8 lanes → 128; brightening a 4K grey image ($3840 \times 2160 = 8,294,400$ pixels, one add each) with 16 lanes → 518,400.

A **GPU** is built from a very large number of SIMD lanes. Designed for graphics (every pixel gets the same math), GPUs now also serve as general-purpose vector processors for physics simulations and machine learning (huge grids of multiply-adds). Most CPU cores also include **vector units** that do SIMD on 4 to 16 numbers. **Common mistake:** SIMD helps only when the same operation applies to many independent items; branchy code, or steps that each need the previous result, gain little.

5. DSPs and hardware accelerators

A **digital signal processor (DSP)** handles **streams** of samples (audio, radio, video) arriving at a fixed rate. It repeats the same multiply-and-add arithmetic on every sample and must finish each chunk before the next arrives. DSPs are embedded **coprocessors** in modems, in encoding and decoding speech and video (**codecs**), and in encryption and security hardware.

A **hardware accelerator** is a circuit for one specific job, such as encryption, compression, video coding or running AI models. It cannot run other code, but its specialised circuitry (not a faster clock) makes it much faster and far more energy-efficient than software on a CPU core.

A phone call: the microphone is sampled 16,000 times a second (320 samples $\times$ 16 bits = 5,120 bits every 20 ms); the audio DSP removes noise and encodes the chunk to about 250 bits; the modem's encryption hardware scrambles it; the modem's DSPs add error correction and form the radio waveform; the radio transmits. Incoming speech takes the reverse path, and the main CPU cores can sleep through most of the call.

6. The system on a chip

A smartphone **SoC** puts on one die (one piece of silicon): performance and efficiency CPU cores, shared cache, a GPU, an AI engine, a camera image processor, a video codec, an audio/sensor DSP, a security engine, display and I/O controllers, often the cellular modem, and a memory controller. Main memory (DRAM) is a separate die stacked on top in the same **package**. The display controller is on the chip; the touchscreen panel is not. Why one chip: shorter wires are faster and use less energy, one part is cheaper than many, and all units share one pool of memory. Trade-off across the chip: CPU core (runs any program) $\rightarrow$ GPU $\rightarrow$ DSP $\rightarrow$ accelerator (one job, least energy).

7. Matching jobs to units

Unit	Best at	Examples
CPU core	branchy, varied, general-purpose logic	compiling code; the OS scheduler
GPU	the same math over huge arrays	rendering 3D scenes; physics of a million particles; training a neural-network layer
DSP	steady streams of samples with deadlines	decoding call audio; filtering microphone noise
Accelerator	one fixed, very common job	encrypting messages; decoding video

Why the OS cares: every unit is a shared resource. The **scheduler** (the part of the OS that decides which thread runs next, and where) treats each logical processor as a CPU it can run a thread on, and through device drivers the OS queues work for the GPU, DSPs and accelerators.

Key terms

Microprocessor

A complete processor built on a single chip of silicon. The first commercial one appeared in 1971; today microprocessors run everything from phones to servers.

Transistor

A microscopic switch controlled by electricity. Chips are built from transistors, so more transistors means room for more circuitry, such as extra cores or bigger caches.

Moore's law

The long-running observation that the number of transistors that fit on a chip roughly doubles about every two years.

Clock speed (clock rate)

How many clock cycles a processor completes per second, measured in hertz. 3 GHz means 3 billion cycles per second; the processor does its work in steps timed by these ticks.

Power wall

The limit reached in the mid-2000s when raising the clock speed further made chips use too much power and run too hot to cool, so designers began adding cores instead.

Logical processor (hardware thread)

A processor as the operating system sees it. With simultaneous multithreading, one physical core presents two (or more) logical processors that share its execution units.

Simultaneous multithreading (SMT)

A core design that holds the registers and program counter of two or more threads at once and issues their instructions in the same cycles, filling units one thread would leave idle. Intel calls it Hyper-Threading.

Execution unit

A part of a core that performs one kind of operation, such as integer arithmetic, floating-point math, or loading and storing memory. Each core has several.

Cache levels (L1, L2, L3)

The layers of cache on a typical modern chip: a tiny, fastest L1 inside each core, a larger L2 per core, and a big L3 shared by all cores. Each level is bigger and slower than the one above it.

Graphics processing unit (GPU)

A processor with a very large number of simple lanes that apply the same operation to big arrays of data. Built to draw graphics; now also used for science and machine learning.

Single instruction, multiple data (SIMD)

A style of computing in which one instruction performs the same operation on many data items at once, each item in its own lane.

Lane

One slot of a SIMD unit, handling one data item. An 8-lane unit can add 8 pairs of numbers with a single instruction.

Vector unit

Hardware inside a CPU core that performs SIMD operations on short vectors, for example 4, 8 or 16 numbers at a time.

Digital signal processor (DSP)

A processor built for streams of samples, such as audio, radio or video signals. It repeats the same arithmetic on every sample fast enough to keep up with the stream.

Coprocessor

A processor that works alongside the main CPU cores and takes over one kind of work, such as graphics, signal processing or encryption.

Hardware accelerator

A circuit designed to do one specific job, such as encryption, compression or video decoding, much faster and with far less energy than software on a CPU core.

System on a chip (SoC)

A single chip that combines CPU cores, caches, a GPU, DSPs, accelerators and I/O such as radios, usually with main memory in the same package. The heart of every smartphone.

Codec

Short for coder-decoder: hardware or software that compresses (encodes) and decompresses (decodes) audio or video.

x86

The instruction set family that began with the Intel 8086 in 1978. Most desktop PCs, laptops and servers use x86 chips, made by Intel and AMD.

ARM

A family of energy-efficient processor designs that many companies license and build into their own chips. Nearly every smartphone uses ARM cores.

Review questions with answers

1. What made the first microprocessor different from the processors that came before it?

- A. The entire processor was built on a single chip.
- B. It was the first processor able to run a stored program.
- C. It had several cores working in parallel.
- D. It was faster than any earlier computer.

Answer: A. The entire processor was built on a single chip.

A microprocessor puts the whole processor (arithmetic, registers and control) on one chip. Early ones were slow, but being tiny and cheap they made desktop and handheld computers possible.

2. Around 2004–2005, chip makers stopped pushing clock speeds much higher. Why?

- A. Faster clocks made chips use too much power and run too hot to cool.
- B. Transistors could no longer be made any smaller.
- C. Software could not take advantage of faster clocks.
- D. Moore's law ended, so no extra transistors were available.

Answer: A. Faster clocks made chips use too much power and run too hot to cool.

This limit is the power wall. Power rises steeply with clock speed, so designers spent the growing transistor budget on more cores and bigger caches instead.

3. A laptop chip has 6 cores, and each core runs 2 hardware threads using simultaneous multithreading. How many logical processors does the operating system scheduler see?

Answer: 12 logical processors

Logical processors = cores × hardware threads per core = $6 \times 2 = 12$. The OS can hand a thread to each one, even though each pair shares a core.

4. Turning on simultaneous multithreading (two logical processors per core) roughly doubles the work each core gets done.

Answer: False

Both logical processors share one core's execution units and caches. SMT fills slots that would sit idle, often gaining something like 10–30 percent, far less than a second real core.

5. A SIMD unit has 8 lanes. How many add instructions does it need to add two arrays of 1,024 numbers, pair by pair?

Answer: 128 instructions

Each instruction adds 8 pairs at once, so $1,024 \div 8 = 128$ instructions. A scalar core would need 1,024.

6. A chip has 1 billion transistors. If the count doubles every two years, about how many billion transistors will a chip have 10 years later?

Answer: 32 billion

Ten years is five doublings: $1 \times 2 \times 2 \times 2 \times 2 \times 2 = 2^5 = 32$ billion. This steady doubling is Moore's law.

7. Match each processing unit to the job it does best.

CPU core	Running branchy, general-purpose code such as a compiler
GPU	Shading millions of pixels with the same math
DSP	Cleaning up a live stream of audio samples
Hardware accelerator	Encrypting data with one fixed algorithm

Each unit trades flexibility for efficiency: the CPU core runs anything, the GPU repeats math over huge arrays, the DSP keeps up with timed streams, and an accelerator does one fixed job.

8. Does each job suit SIMD (the same operation on many independent items) or not?

Suits SIMD	Gains little from SIMD
Brighten every pixel of a photo Add two lists of a million numbers Multiply the big number grids inside a neural network	Walk a chain of records where each step needs the address found in the one before Parse a typed command full of if-statements

SIMD needs many independent items that all get the same operation. Chains of dependent steps and branch-heavy logic cannot be spread across lanes.

9. Which of these can be integrated into a smartphone's system on a chip (SoC)?

- A. CPU cores
- B. Cache memory shared by the cores
- C. A GPU
- D. DSPs
- E. A cellular modem and radio
- F. Main memory, stacked in the same package
- G. The touchscreen panel itself

Answer: A, B, C, D, E, F

An SoC combines CPU cores, caches, a GPU, DSPs, accelerators and I/O such as radios and codecs, with main memory stacked in the same package. It contains the display controller that drives the screen, but the glass touchscreen panel is a separate part of the phone.

10. Which statement best describes how SIMD hardware is used today?

- A. GPUs, built for graphics, now also run physics and machine learning, and most CPU cores have vector units too.
- B. GPUs only draw graphics; scientific and AI number crunching runs on ordinary CPU cores instead.
- C. Only GPUs can do SIMD; a CPU core always handles exactly one number per instruction.
- D. GPUs have replaced CPU cores in most computers, because SIMD speeds up every kind of program.

Answer: A. GPUs, built for graphics, now also run physics and machine learning, and most CPU cores have vector units too.

SIMD applies one instruction to many data items. GPUs are built from a huge number of SIMD lanes, which suits any big, repetitive array job, and CPU cores add smaller vector units for the same reason.

11. Compared with running the same job as software on a CPU core, a hardware accelerator is usually:

- A. faster and far more energy-efficient, but able to do only its one job
- B. slower, but able to run any program
- C. faster only because it runs at a higher clock speed
- D. the same thing as a GPU

Answer: A. faster and far more energy-efficient, but able to do only its one job

An accelerator is circuitry designed for one task, such as encryption, compression or video decoding. That specialisation buys speed and large energy savings at the cost of flexibility.

12. Most desktop PCs use x86 processors, while nearly every smartphone uses ARM processor cores.

Answer: True

The x86 family began with Intel's 8086 in 1978 and dominates PCs and servers (Intel and AMD build it). ARM designs are licensed to many chip makers and power nearly all phones, and a growing number of laptops and servers.

1.3 Instruction Execution

You should be able to

- Describe the instruction cycle as a fetch stage followed by an execute stage, repeated until the processor halts.
- Explain how the program counter and instruction register work together during a fetch, and how a jump changes the flow.
- Sort instructions into the four categories: processor-memory, processor-I/O, data processing and control.
- Decode a 16-bit instruction word of a simple accumulator machine and trace a short program register by register, including the MAR and MBR.
- Explain why direct memory access (DMA) frees the processor from moving I/O data word by word.

1. A program and the instruction cycle

A **program** is an ordered list of **instructions** stored in main memory. The processor runs it by repeating one loop. The work done for a single instruction is called one **instruction cycle**, and in its simplest form it has two stages:

- **Fetch stage:** read the next instruction from memory into the processor.
- **Execute stage:** **decode** the instruction (work out what its bits ask for) and carry out that action.

Then the next fetch begins automatically. The loop only stops at a **halt**: the machine is switched off, an error it cannot recover from occurs, or the program executes an instruction that tells the processor to stop.

START → **FETCH** the next instruction → **EXECUTE** it → back to **FETCH** ... → **HALT**

Section 1.4 adds a third stage after execute, the **interrupt stage**, in which the processor checks whether a device is asking for attention.

2. The fetch stage: program counter and instruction register

Two **registers** (tiny, very fast storage slots inside the processor) run the fetch:

- The **program counter (PC)** holds the *address* of the next instruction to fetch.
- The **instruction register (IR)** holds the instruction just fetched while it is decoded and executed.

At the start of every cycle the processor fetches the instruction whose address is in the PC and places it in the IR. It then increments the PC (by 1 here, since each instruction fills one word; a byte-addressed machine adds the instruction's length in bytes), so unless something says otherwise, instructions run in address order (100, 101, 102, ...). While an instruction executes, the PC already points at the one after it.

A **jump** is how "otherwise" happens: executing it overwrites the PC with a new address, so the next fetch comes from somewhere else. Loops and if-statements are built this way.

3. The four categories of instruction

Category	What it does	Examples
Processor–memory	Copies data between the processor and main memory, in either direction.	Load the AC from 940; store the AC into 941.
Processor–I/O	Copies data between the processor and an I/O module (the circuit that runs a device).	Send a byte to the printer; read the keyboard's status.
Data processing	Performs arithmetic or logic on data.	Add, subtract, AND, NOT, shift.
Control	Changes the order of execution by loading a new address into the PC.	Jump to 500; skip the next instruction if the AC is zero.

One instruction can combine several categories. “Add the word at 941 to the AC” reads memory (processor–memory) and then adds (data processing).

4. A hypothetical 16-bit accumulator machine

- Memory is a sequence of 16-bit **words**, each with an address.
- Instruction format: a 4-bit **opcode** (which operation) followed by a 12-bit **address field** (which memory word).
- Data format: 1 sign bit (0 = positive, 1 = negative) followed by a 15-bit magnitude (**sign-magnitude**).
- Registers: PC, IR, and one **accumulator (AC)** that holds the data being worked on. PC and MAR hold 12-bit addresses; IR, AC and MBR hold 16-bit words.

Opcode (binary)	Hex	Meaning
0001	1	Load AC from memory
0010	2	Store AC to memory
0101	5	Add to AC from memory

Capacity: 4 opcode bits give $2^4 = 16$ different opcodes. 12 address bits give $2^{12} = 4,096$ (4K) directly addressable words. In general, k bits give 2^k patterns; a 16-bit instruction with a 6-bit opcode would leave 10 address bits, reaching $2^{10} = 1,024$ words.

Hexadecimal is used because each hex digit is exactly four bits. So the first hex digit of an instruction word is its opcode and the last three are its address. Worked example: 1940 hex = 0001 1001 0100 0000 → opcode 0001 (load AC), address 940. Note that 940 is hex: $9 \times 256 + 4 \times 16 = 2,368$ in decimal. As a data word, 8005 hex = 1000 0000 0000 0101 → sign 1 (negative), magnitude 5, value -5 .

Memory cannot tell instructions from data. A word acts as an instruction when the PC fetches it, and as data when an instruction's address field points at it. One consequence: a store whose address points into the program area overwrites an instruction, so the program changes itself. The next time the PC reaches that cell, the processor fetches and runs the new word, not the one the programmer wrote.

5. Tracing the example program (3 + 2)

Memory before the run: program at 300: **1940**, 301: **5941**, 302: **2941**; data at 940: **0003**, 941: **0002**. The PC starts at 300. Three instruction cycles make six stages ([x] means “the word stored at address x”):

#	Stage	What happens	PC	IR	AC	941
1	Fetch	[300] → IR	301	1940	–	0002
2	Execute	Load: AC ← [940]	301	1940	0003	0002
3	Fetch	[301] → IR	302	5941	0003	0002
4	Execute	Add: AC ← AC + [941]	302	5941	0005	0002
5	Fetch	[302] → IR	303	2941	0005	0002
6	Execute	Store: [941] ← AC	303	2941	0005	0005

Result: cell 941 holds 0005 and the PC holds 303. The PC went 300 → 301 → 302 → 303, one step per fetch, and only cell 941 changed. Notice that right after the second fetch the PC is already 302 while 5941 waits in the IR: an instruction stays in the IR until the next fetch replaces it.

The MAR and MBR inside each stage

The processor talks to memory through two more registers. The **memory address register (MAR)** holds the address about to be read or written; the **memory buffer register (MBR)** holds the word just read, or about to be written.

- **Fetch:** MAR ← PC; memory sends the addressed word into the MBR; IR ← MBR; PC ← PC + 1.
- **Execute a load or add:** MAR ← address field of the IR; memory sends the operand into the MBR; then AC ← MBR (load), or the **arithmetic logic unit (ALU)** computes AC ← AC + MBR (add).
- **Execute a store:** MAR ← address field; MBR ← AC; memory writes the MBR's word at that address.

Counting memory traffic in the example: 5 reads (3 instruction fetches plus 2 operand reads, from 940 and 941) and 1 write (the store into 941).

6. Moving data to and from I/O

Processor–I/O instructions exchange data **directly between the processor and an I/O module**. They work like a load or store, except that they name a device (an I/O address) instead of a memory cell. For large transfers this is wasteful: every word costs the processor whole instruction cycles (an input instruction plus a store).

With **direct memory access (DMA)**, the processor instead lets the I/O module read from or write to main memory itself. The processor sends one command to set up the transfer, the I/O module moves the words, and it tells the processor when it is finished (using an interrupt, covered in section 1.4). The processor is relieved of the transfer and keeps executing other instructions. Section 1.7 covers DMA in detail.

Common mistakes

- Thinking the PC points at the instruction currently running. It already points at the next one.
- Calling a jump “data processing”. It changes the PC, so it is a control instruction.

- Thinking the loop needs help to continue. After every execute, the next fetch starts by itself; only a halt stops it.

Key terms

Program

An ordered list of instructions stored in memory. The processor carries them out one after another.

Instruction

A single, very small command the processor knows how to perform, stored in memory as a pattern of bits (for example, "add the number at address 941 to the accumulator").

Instruction cycle

Everything the processor does for one instruction: fetch it from memory, then execute it. The cycle repeats until the processor halts.

Fetch stage

The first half of every instruction cycle: the processor reads the instruction whose address is in the program counter into the instruction register, then increments the program counter.

Execute stage

The second half of every instruction cycle: the processor decodes the instruction in the instruction register and carries out the action it specifies.

Halt

The end of the instruction-cycle loop. It happens when the machine is switched off, when an error it cannot recover from occurs, or when the program runs an instruction that tells the processor to stop.

Register

A tiny, extremely fast storage slot inside the processor. Each one holds a single value, such as an address or a number being worked on.

Program counter (PC)

The register that holds the address of the next instruction to fetch. It is incremented after every fetch unless an instruction changes it.

Instruction register (IR)

The register that holds the instruction just fetched, so the processor can decode it and carry it out.

Accumulator (AC)

The main working register of a simple processor. Numbers loaded from memory and the results of arithmetic are kept here temporarily.

Opcode

Short for operation code: the group of bits in an instruction that says which operation to perform, such as load, store or add.

Address field

The bits of an instruction that name the memory location the instruction works with.

Word

The fixed-size group of bits that the processor and memory store and move as one unit; each numbered memory cell holds one word. In the machine in this section a word is 16 bits.

Hexadecimal (hex)

Base-16 notation using the digits 0–9 and A–F. Each hex digit stands for exactly four bits, so a 16-bit word is written as four hex digits.

Sign-magnitude

A way of storing a signed number: the leftmost bit is the sign (0 = positive, 1 = negative) and the remaining bits hold the size of the number.

Memory address register (MAR)

The register that holds the address of the memory location the processor is about to read or write.

Memory buffer register (MBR)

The register that holds the word just read from memory, or the word about to be written to memory.

Arithmetic logic unit (ALU)

The execution unit that does the actual arithmetic (add, subtract) and logic (AND, OR, NOT) on values held in registers. A simple processor has one; a modern core has several.

Jump

A control instruction that puts a new address into the program counter, so the next fetch comes from somewhere other than the next address in order.

Direct memory access (DMA)

A way of moving data in which an I/O module reads or writes main memory itself, so the processor does not have to handle every word of the transfer.

Review questions with answers

1. A processor has just finished the execute stage of an ordinary add instruction. What does it do next?
 - A. Starts a new fetch, from the address now held in the program counter
 - B. Executes the same instruction again, because it is still in the instruction register
 - C. Halts, because every instruction cycle ends in a halt
 - D. Waits for the operating system to hand it the next instruction

Answer: A. Starts a new fetch, from the address now held in the program counter

The instruction cycle is a loop (fetch, execute, fetch, execute...). The next fetch starts straight away from the address in the PC, and the loop ends only on a halt.

2. On a word-addressed teaching machine where every instruction fills exactly one memory word, the program counter goes up by one after every fetch unless an instruction changes it, so instructions run in address order.

Answer: True

Each fetch moves the PC past the instruction it just read. Here that instruction is one word, so the step is 1; a real processor adds the instruction's length in addressable units (on a byte-addressed machine, 4 for a 4-byte instruction). Only control instructions, such as a jump, break the address order.

3. A tiny accumulator machine runs 300: 1940 (load AC from 940), 301: 5941 (add from 941), 302: 2941 (store to 941). Put its six stages in order.

Correct order:

1. Fetch 1940 from 300 into the IR
2. Load the AC with the word at 940
3. Fetch 5941 from 301 into the IR
4. Add the word at 941 to the AC
5. Fetch 2941 from 302 into the IR
6. Store the AC into 941

Every instruction cycle is fetch then execute, and the PC walks 300 → 301 → 302 → 303 along the way.

4. Sort each instruction into its category: processor–memory (Memory), processor–I/O (I/O), data processing, or control.

Memory	I/O	Data processing	Control
Store the AC into address 941	Read a byte from the keyboard module	Multiply two registers Shift the AC one bit left	If the AC is zero, skip the next instruction

Transfers with memory, transfers with an I/O module, arithmetic or logic on data, and changes to the order of execution (to the PC) are the four categories.

5. An instruction format has a 4-bit opcode field. How many different opcodes can it express?

Answer: 16

Four bits can form $2^4 = 16$ different patterns, so at most 16 operations.

6. An instruction format has a 12-bit address field. How many memory words can one instruction address directly?

Answer: 4096 words

Twelve bits can form $2^{12} = 4,096$ patterns, often written 4K.

7. A different 16-bit machine uses 6 bits for the opcode and the remaining bits for the address. How many words can an instruction address directly?

Answer: 1024 words

$16 - 6 = 10$ address bits, and $2^{10} = 1,024$.

8. On a machine whose opcodes are 1 = load AC, 2 = store AC and 5 = add to AC (the first hex digit of a 16-bit word), what does the word 2941 (hex) do?

- A. Load the AC from address 941
- B. Store the AC into address 941
- C. Add the word at 941 to the AC
- D. Jump to address 941

Answer: B. Store the AC into address 941

The first hex digit is the 4-bit opcode (2 = 0010 = store AC to memory) and the other three digits are the address, 941.

9. In a simple accumulator machine, which of these happen during the **fetch** stage?

- A. The PC's value is copied into the MAR
- B. The instruction word arrives in the MBR and is moved into the IR
- C. The PC is incremented
- D. The AC gets a new value
- E. A data word is written into memory

Answer: A, B, C

Fetch only brings the instruction in and advances the PC. Changing the AC or writing data to memory happens in the execute stage.

10. An accumulator machine runs 300: 1940 (load AC from 940), 301: 5941 (add from 941), 302: 2941 (store to 941), with 0003 at 940 and 0002 at 941. Match each register to the value it holds right after the **second execute** (the add).

PC	302
IR	5941
AC	0005
MAR	941
MBR	0002

The add was fetched from 301, so the PC has already moved on to 302 and 5941 sits in the IR. To execute it, the MAR carried the address field 941 to memory, the MBR received the operand 0002, and the ALU made $AC = 3 + 2 = 0005$.

11. A 16-bit data word uses sign-magnitude: 1 sign bit (1 = negative) and a 15-bit magnitude. What decimal value is the word 8005 (hex)?

Answer: -5

8005 hex is 1000 0000 0000 0101: the sign bit is 1 (negative) and the other 15 bits hold 5, so the value is -5.

12. What is the main benefit of direct memory access (DMA)?

- A. It makes each instruction cycle shorter
- B. An I/O module moves data to or from memory itself, so the processor is freed from the transfer
- C. It lets the processor skip the fetch stage
- D. It stores programs inside the I/O module

Answer: B. An I/O module moves data to or from memory itself, so the processor is freed from the transfer

With DMA the processor only sets up the transfer and is told when it is done; the I/O module moves the words directly to or from memory.

1.4 Interrupts

You should be able to

- Explain why interrupts raise processor utilization when I/O devices are far slower than the processor.
- Classify events into the four classes of interrupts: program, timer, I/O and hardware failure.
- Trace the flow of control and the timing of a program with and without interrupts, for short and long I/O waits.
- List, in order, the hardware and software steps of interrupt processing and show how the PC, PSW and control stack change.
- Compare disabling interrupts with priority-based nesting, and trace a timeline with several interrupts.

Why interrupts exist

An **interrupt** is a signal that makes the processor set aside the running program, run an **interrupt handler** (ISR), then resume the program where it stopped. Interrupts exist mainly to improve **processor utilization**, the fraction of time the processor does useful work. I/O devices are far slower: at one instruction per nanosecond, printing a line (about 20 ms) lasts 20 million instruction-times. Without interrupts the processor idles through it; with them it starts the device and keeps working until the device signals it is done.

The four classes of interrupts

Class	Where it comes from	Examples
Program	The instruction just executed	Arithmetic overflow, division by zero, illegal instruction, reference outside the program's allowed memory
Timer	A clock inside the processor, at regular intervals	OS regains control periodically, even from an endless loop
I/O	An I/O module (device controller)	Operation done, device needs service, or an error (e.g. a disk read error)
Hardware failure	A physical fault	Power failure, memory parity error

Flow of control without and with interrupts

A user program has code segments ①, ② and ③ with a WRITE call between them. Each WRITE calls the OS's I/O program: **prep code** ④ (get the data ready), the **I/O command** that starts the device, and **completion code** ⑤ (check the result).

- **No interrupts.** After the I/O command, the I/O program keeps checking the device until it finishes, runs ⑤, and only then returns. Every WRITE costs a full device time of idle processor.
- **Interrupts, short I/O wait.** After the I/O command, the I/O program returns at once and user code runs *in parallel* with the device. When the device finishes it interrupts partway through user code; the

processor saves its place, runs the handler (which contains ⑤), and resumes. No idle time, and the user program contains no interrupt code.

- **Interrupts, long I/O wait.** If the device takes longer than the user code between WRITES, the next WRITE finds it still busy, so the processor waits for the completion interrupt. Some idle time remains, but less than without interrupts.

Program timing: a worked example

Let each user segment take 10 time units, prep ④ 2, completion ⑤ 2, and the printer D units per line. With interrupts each handler costs 3 units (⑤ plus 1 unit to save and restore the context).

- **Without interrupts:** total = $3 \times 10 + 2 \times (2 + D + 2) = 38 + 2D$, of which $2D$ is idle. $D = 20$ gives 78 units, 40 idle.
- **Short wait, $D = 6$:** 50 units (12 idle) without; with interrupts $3 \times 10 + 2 \times 2 + 2 \times 3 = 40$ units, 0 idle. Saved 10.
- **Long wait, $D = 25$:** 88 units (50 idle) without; 70 with, still 30 idle (at the 2nd WRITE and the last line). Saved 18.

Handler overhead is small next to the waiting it removes; the gain vanishes only when D falls to the 1-unit overhead.

Interrupts and the instruction cycle

The basic cycle is fetch (copy the instruction the PC points to into the **instruction register (IR)** and add 1 to the PC) then execute. To support interrupts, an **interrupt stage** is added after execute: fetch → execute → interrupt check → fetch ...

- If interrupts are **disabled**, the check is skipped and the next instruction is fetched. A request that arrives meanwhile is not lost; it stays **pending**.
- If enabled and **none is pending**, the next instruction is fetched.
- If enabled and **one is pending**, the processor suspends the program, saves its **context** (PC, PSW, registers) and sets the **PC** to the handler's start address, so the next fetch comes from the handler. Later the context is restored and the program resumes.

A device's request never splits an instruction. The handler's instructions are **overhead**, but far cheaper than waiting.

Interrupt processing: the nine steps

Recall: the **PC** holds the address of the next instruction; the **PSW** holds condition codes, the user/kernel mode bit and the interrupt-enable bit. The **control stack** is memory used last-in, first-out to save them.

Two details in the examples. Real processors have many general-purpose registers (R1, R2, R3 ...) instead of the single accumulator of the simple machine in 1.3, so instructions name the register they use, as in LOAD R1, [count]. The processor also has two modes, recorded in the PSW: ordinary programs run in the less-privileged **user mode**, and the OS, including every interrupt handler, runs in the privileged **kernel mode** (sections 2.2 and 3.4 explain modes fully). In the step-by-step example, **N** is the address of the instruction running when the interrupt arrives, **N + 1** is the next one (where the program resumes), and **Y** is the handler's first instruction.

Hardware (the processor, automatically):

1. A device issues an interrupt signal to the processor.
2. The processor finishes executing the current instruction.
3. The processor tests for a pending interrupt, finds one, and sends the device an acknowledgment; the device drops its request.
4. The processor pushes the PSW and the PC onto the control stack; the next step overwrites them.
5. The processor loads the PC with the start address of the right interrupt handler (known from the signal, or by asking the device) and switches to privileged **kernel mode**.

Software (the OS interrupt handler):

6. The handler saves the rest of the processor state: the general registers it will use.
7. The handler processes the interrupt (e.g. reads the device status, sends more data).
8. The saved registers are restored from the stack.
9. The PSW and PC are restored from the stack by a return-from-interrupt instruction, so the program resumes at its next instruction.

Worked example. The program is executing the instruction at $N = 400$, so the PC already holds $N + 1 = 401$. The control stack starts at $T = 1000$ and grows toward lower addresses, one word per item. Step 4 pushes the PSW (to 999) and PC = 401 (to 998): SP goes $1000 \rightarrow 998$. Step 5 sets PC to the handler address $Y = 900$. Step 6 pushes R1, R2, R3: $SP \rightarrow 995$. Step 8 pops them: $SP \rightarrow 998$. Step 9 pops PC and PSW: $SP \rightarrow 1000$, PC = 401. A pop only moves SP; old values stay in memory but count as free. Likewise, PSW, PC and 4 registers pushed from 1000 leave $SP = 994$.

Multiple interrupts

Approach 1: disable interrupts while any handler runs. A request that arrives meanwhile is not lost: it stays pending, whatever its priority, until the handler ends and interrupts are re-enabled. Waiting requests are served strictly one at a time (here, oldest first). Simple, but it ignores priority. Example: printer handler $t = 10$ – 20 , comm line interrupts at $t = 12$: the comm handler cannot start until $t = 20$, risking lost characters.

Approach 2: priorities with nesting. Each source has a priority. A higher-priority request may interrupt a lower-priority handler (its state is saved on the stack); a lower-priority request waits, pending, until higher-priority handlers finish. When a handler ends, the processor serves the most urgent pending request that outranks what it would return to.

Classic example (priorities: printer 2, disk 4, comm line 5; each handler needs 10 units):

Time	Event	What runs
$t = 10$	Printer interrupts the user program	Printer handler
$t = 15$	Comm line interrupts; $5 > 2$, printer handler suspended	Comm handler
$t = 20$	Disk interrupts; $4 < 5$, so it waits (pending)	Comm handler continues
$t = 25$	Comm handler done; disk (4) outranks printer (2)	Disk handler
$t = 35$	Disk handler done; nothing pending	Printer handler resumes
$t = 40$	Printer handler done	User program resumes

With interrupts disabled instead, the same arrivals give printer 10–20, comm 20–30 (a 5-unit wait for the most urgent device), disk 30–40. Nesting serves urgent devices fast; low-priority handlers finish later.

Key terms

Interrupt

A signal that makes the processor set aside the program it is running, run a special routine to deal with some event, and then carry on exactly where it left off.

Interrupt handler

The routine (normally part of the operating system) that runs when an interrupt is accepted. It finds out what the device or event needs and deals with it. Also called an interrupt service routine (ISR).

Processor utilization

The fraction of time the processor spends doing useful work instead of sitting idle.

Program interrupt

An interrupt caused by the instruction just executed: for example arithmetic overflow, division by zero, an illegal instruction, or an attempt to use memory outside the program's allowed space.

Timer interrupt

An interrupt produced by a clock in the processor at regular intervals. It lets the operating system take back control and do jobs on a schedule.

I/O interrupt

An interrupt produced by an I/O module to report that an operation finished, that the device needs service, or that an error occurred.

Hardware failure interrupt

An interrupt triggered by a physical fault, such as the power supply failing or memory detecting a parity error.

I/O command

An instruction the processor sends to an I/O module telling it to start an operation, such as printing a line. The device then works on its own.

Instruction cycle

The repeating routine the processor follows for every instruction: fetch it from memory, execute it, and (with interrupts) check whether an interrupt is waiting.

Interrupt stage

The stage added to the instruction cycle after execute. The processor checks for a pending interrupt and, if one is waiting and interrupts are enabled, switches to its handler.

Pending interrupt

An interrupt request a device has raised that the processor has not accepted yet, for example because interrupts are disabled or a higher-priority handler is running.

Program counter (PC)

The processor register that holds the address of the next instruction to fetch.

Kernel mode

The privileged processor mode in which the operating system (including every interrupt handler) runs: all instructions are allowed, such as those that enable or disable interrupts. Ordinary programs run in the restricted user mode.

Program status word (PSW)

A processor register that holds status about the running program: condition codes (for example, whether the last result was zero or overflowed), whether interrupts are enabled, and whether the processor is in user or kernel mode.

Context (processor state)

Everything the processor needs to continue a program exactly where it stopped: the program counter, the PSW and the contents of the other registers.

Control stack

An area of memory used last-in, first-out, where the processor and the operating system save return addresses and register contents.

Stack pointer (SP)

The register holding the address of the top of the control stack. It moves every time something is pushed onto or popped off the stack.

Disabled interrupts

A processor setting in which new interrupt requests are ignored for the moment. They stay pending and are served once interrupts are enabled again.

Interrupt priority

A ranking given to each source of interrupts so that, when several compete, the more urgent one is served first and may even interrupt a less urgent handler.

Nested interrupts

Handling in which a higher-priority interrupt may interrupt a handler that is already running; the interrupted handler resumes when the higher one finishes.

Review questions with answers

1. What is the main reason computers use interrupts?

- A. To keep the processor doing useful work while slow I/O devices operate
- B. To make I/O devices run faster
- C. To let one user program call another
- D. To stop programs from reading each other's memory

Answer: A. To keep the processor doing useful work while slow I/O devices operate

I/O devices are far slower than the processor. With interrupts the processor starts an I/O operation, runs other instructions, and is told when the device is done, which raises processor utilization.

2. Sort each event into its class of interrupt.

Program	Timer	I/O	Hardware failure
Division by zero Illegal instruction	Regular clock tick	Printer finished a line Disk reports a read error	Memory parity error

Program interrupts come from the executing instruction; timer interrupts from the processor's clock; I/O interrupts from an I/O module (completion or error); hardware-failure interrupts from physical faults such as power loss or parity errors.

3. Put the stages of interrupt processing in order. (The handler's save, process and restore steps are combined into one item.)

Correct order:

1. Device issues an interrupt signal
2. Processor finishes the current instruction
3. Processor tests for a pending interrupt and acknowledges it
4. Processor pushes the PSW and PC onto the control stack
5. Processor loads the PC with the handler's start address
6. Handler saves registers, processes the interrupt, restores registers
7. PSW and PC are restored from the stack

The full list has nine steps. Steps 1–5 are hardware: signal, finish the instruction, test and acknowledge, push PSW and PC, load the handler address. Steps 6–9 belong to the handler: save registers, process the interrupt, restore registers, restore PSW and PC. The stack is last-in, first-out, so things come off in the reverse order they went on.

4. Interrupts are disabled whenever a handler runs. The printer handler (lowest priority) starts at $t = 10$ and needs 10 time units. At $t = 12$ the communications line (highest priority) interrupts. When does the comm line handler start?

- A. $t = 12$, because it has the higher priority
- B. $t = 20$
- C. $t = 22$
- D. Never: a request that arrives while interrupts are disabled is lost

Answer: B. $t = 20$

With the disable approach, requests are handled strictly one after another. The comm line waits 8 units behind a less urgent handler, which is exactly the weakness of this approach: it ignores priority and time-critical needs.

5. A program runs three code segments of 10 time units each, with a WRITE between them (two WRITES). Each WRITE costs 2 units of prep, and the printer then needs 6 units per line. **Without interrupts** the processor waits out the 6 units and then runs 2 units of completion code. **With interrupts** the printer works while user code runs, and each completion costs a 3-unit handler (the completion code plus saving and restoring the context). Each line finishes before the next WRITE. How many time units do interrupts save?

Answer: 10 units

Without: $3 \times 10 + 2 \times (2 + 6 + 2) = 50$. With: the waits disappear, so $3 \times 10 + 2 \times 2$ (prep) + 2×3 (handlers) = 40, with no idle time. Saved: $50 - 40 = 10$ units. The handler costs 1 unit more than plain completion code, but that is far less than the 6-unit wait it hides.

6. Which values does the processor **hardware** push onto the control stack when it accepts an interrupt?

- A. The PSW and the program counter
- B. Every general-purpose register
- C. Only the stack pointer
- D. The whole user program

Answer: A. The PSW and the program counter

The PSW and PC must be saved by hardware because loading the handler address immediately overwrites the PC. The handler then saves whatever other registers it needs.

7. The stack pointer holds 1000 and the stack grows toward lower addresses, one word per item. The processor pushes the PSW and PC, then the handler pushes 4 registers. What address does SP hold now?

Answer: 994

Six items are pushed in total (PSW, PC and 4 registers), so SP moves down from 1000 to $1000 - 6 = 994$.

8. Without interrupts, a program runs three code segments of 10 time units each, with a WRITE between them (two WRITES). Each WRITE needs 2 units of prep, 20 units of printing during which the processor just waits, and 2 units of completion. What is the total time?

Answer: 78 units

Code: $3 \times 10 = 30$. Each WRITE: $2 + 20 + 2 = 24$, and there are two, so 48. Total $30 + 48 = 78$ time units, of which 40 are pure waiting.

9. Priorities: printer 2, disk 4, comm line 5 (higher is more urgent), with nesting allowed. The printer handler is running when the comm line interrupts; then the disk interrupts while the comm handler runs. When the comm handler finishes, what runs next?

- A. The disk handler
- B. The printer handler, since it was interrupted first
- C. The user program
- D. The comm line handler again

Answer: A. The disk handler

On finishing a handler, the processor serves the highest-priority request that outranks what it would return to. The disk (4) beats the suspended printer handler (2); the printer resumes after the disk finishes.

10. Which statements about the interrupt stage are true? Select all that apply.

- A. It follows the execute stage of the instruction cycle
- B. It is skipped when interrupts are disabled
- C. It interrupts an instruction halfway to respond faster
- D. If a request is pending, the context is saved and the PC is set to the handler's start address

Answer: A, B, D

The interrupt stage comes after execute and is skipped when interrupts are disabled. It never splits an instruction; if a request is pending it saves the context and points the PC at the handler.

11. Match each item to its role.

Program counter (PC)	Holds the address of the next instruction
Program status word (PSW)	Holds condition codes, the mode and the interrupt-enable bit
Control stack	Memory where return addresses and registers are saved
Interrupt handler	Routine that services the device that interrupted

The PC and PSW describe where the program is and its status; the control stack stores them while the handler, the OS routine that services the device, runs.

12. A system uses interrupts. A program issues a WRITE, runs some code, then reaches its next WRITE while the printer is still busy with the first line. What happens?

- A. The processor waits until the first line is done (signalled by its interrupt), then starts the second
- B. The second WRITE overwrites the first line in the printer
- C. The OS throws the first line away
- D. The printer prints both lines at the same time

Answer: A. The processor waits until the first line is done (signalled by its interrupt), then starts the second

This is the long I/O wait case. Interrupts let code overlap the printing, but when the program needs the device again before it is free, the processor must wait for the completion interrupt.

1.5 The Memory Hierarchy

You should be able to

- State the three design constraints on memory (capacity, access time, cost) and the three relationships that link them.
- Name the levels of the memory hierarchy, group them into inboard, outboard and off-line storage, and describe the four trends going down it.
- Compute the average access time of a two-level memory from the hit ratio H and the access times T_1 and T_2 .
- Explain locality of reference (temporal and spatial) and why it makes a hierarchy fast and cheap at the same time.
- Describe secondary memory and how the operating system uses part of main memory as a disk cache.

1. The memory designer's dilemma

A memory is judged on three design constraints: **capacity** (how much it holds), **access time** (how long it takes to deliver or store data once asked) and **cost per bit** (the price of storing one bit, usually quoted per GB). Across real technologies three relationships always hold:

1. **Faster access time → greater cost per bit.**
2. **Greater capacity → smaller cost per bit.**
3. **Greater capacity → greater (slower) access time.**

The dilemma: we want large capacity (cheap per bit, room for programs) and fast access, but the big technologies are the slow ones. All-SRAM storage would cost a fortune and forget everything at power-off; all-disk storage would leave the processor waiting. **The solution:** do not rely on one technology. Use a little fast, expensive memory and a lot of slow, cheap memory, stacked in levels, and keep the data in use near the top. This is the **memory hierarchy**.

1 ns = a billionth of a second; 1 μ s = 1,000 ns; 1 ms = 1,000 μ s. Caches use fast, bulky SRAM; main memory uses denser, cheaper DRAM.

2. The levels of the hierarchy

- **Inboard memory:** registers, cache, main memory (reached directly or over the system bus).
- **Outboard storage:** solid-state disks, magnetic disks, optical discs (attached through I/O modules).
- **Off-line storage:** magnetic tape, kept outside the running system and mounted when needed (backups, archives).

Level	Capacity	Access time	Cost per GB
Registers	$\approx$ 1 KB	$\approx$ 0.3 ns	highest
Cache (SRAM)	64 KB–64 MB	1–10 ns	$\approx$ \$1,000
Main memory (DRAM)	8–64 GB	50–100 ns	$\approx$ \$5
SSD	0.5–4 TB	$\approx$ 0.1 ms	$\approx$ 8¢
Magnetic disk	1–20 TB	5–10 ms	$\approx$ 2¢
Optical disc	$\leq$ 100 GB per disc	$\approx$ 100 ms	$\approx$ 5¢
Magnetic tape	10–30 TB per cartridge	seconds–minutes	$\approx$ 0.5¢

Rough mid-2020s figures; the ratios matter most. Registers, cache and main memory are **volatile** (they forget without power); the outboard and off-line levels are **nonvolatile**.

Going down the hierarchy: (a) cost per bit **decreases**; (b) capacity **increases**; (c) access time **increases**; (d) frequency of access by the processor **decreases**. Point (d) is what makes (c) bearable. The trends compare levels. SSD, magnetic disk and optical disc are side-by-side alternatives on the same outboard level, so they need not line up with each other: optical discs ($\approx$ 5¢ per GB) even cost more per bit than magnetic disk ($\approx$ 2¢), because discs are now a niche medium.

Feeling the gaps: if a 0.3 ns register access took 1 second, a cache access would take $\approx$ 3 s, main memory $\approx$ 5.6 min, an SSD $\approx$ 4 days, a magnetic disk $\approx$ 6 months, an optical disc $\approx$ 10 years, and winding a

tape $\approx 6,300$ years. So the OS does not let the processor idle during a disk access: it runs another program and an interrupt announces the data.

3. A two-level memory and its average access time

Level 1 is small and fast (access time **T1**); level 2 is large and slow (access time **T2**). The processor always looks in level 1 first.

- **Hit**: the word is in level 1; the access takes T1.
- **Miss**: the word's **block** (a fixed-size group of neighbouring words) is copied up from level 2 and the word is then read from level 1; the access takes **T1 + T2**.
- **Hit ratio H** = hits $\div$ total accesses, between 0 and 1.

Ts = $H \times T1 + (1 - H) \times (T1 + T2)$, which simplifies to $Ts = T1 + (1 - H) \times T2$.

Worked example ($T1 = 0.1 \mu s$, $T2 = 1 \mu s$, $H = 0.95$): $Ts = 0.95 \times 0.1 + 0.05 \times 1.1 = 0.095 + 0.055 = \mathbf{0.15 \mu s}$. Read by read: a loop reads $a[0] \dots a[3]$ five times (20 reads); only the first misses ($1.1 \mu s$) and 19 hit ($0.1 \mu s$ each), so the total is $3.0 \mu s$, the average $0.15 \mu s$ and $H = 19/20 = 0.95$. Without level 1 the 20 reads would take $20 \mu s$, so level 1 made the loop $\approx 6.7\times$ faster.

Other values: $H = 0.9 \rightarrow 0.2 \mu s$; $H = 0.8 \rightarrow 0.3 \mu s$; $H = 0.99 \rightarrow 0.11 \mu s$; $H = 0 \rightarrow 1.1 \mu s$ (worse than level 2 alone). Ts against H is a straight line from $T1 + T2$ ($H = 0$) to $T1$ ($H = 1$); each 0.01 lost in H adds $0.01 \times T2$. Keeping Ts within $2 \times T1$ needs $H \geq 1 - T1/T2$ (0.9 here): H must be close to 1.

Common mistake: charging only $T2$ for a miss. The processor already spent $T1$ checking level 1.

4. Locality of reference: why the hierarchy works

High hit ratios are possible because of **locality of reference**: over a short period, a program's memory references cluster in a few small areas, and those clusters change only slowly. Causes: sequential instructions, loops, subroutines (the same code called repeatedly, touching a few local variables) and arrays or records processed item by item.

- **Temporal locality**: a location used recently is likely to be used again soon (loop instructions, counters, repeated calls).
- **Spatial locality**: locations near a recently used one are likely to be used soon (straight-line code, array elements). Copying whole blocks exploits it.

A loop hits on later passes only if it fits in the fast memory; straight-line code has only spatial locality (bigger blocks, fewer misses); random addresses almost always miss.

The funnel. Because references cluster, data can be arranged so that the **fraction of accesses going to each lower level is much smaller** than the fraction going to the level above. Example: 1,000,000 references, cache hit ratio 0.95, and 1 in 100,000 main-memory accesses going on to disk. All million use the cache, 50,000 reach main memory and on average 0.5 reach the disk. The same rule with three levels (cache 1 ns, memory 100 ns, disk 5 ms) gives $Ts = 1 + 0.05 \times (100 + 0.00001 \times 5,000,000) = 1 + 5 + 2.5 = \mathbf{8.5 ns}$. With 1 in 10,000 going to disk, Ts is 31 ns, 80% of it disk time: the slower a level, the rarer its use must be.

5. Secondary memory and the disk cache

Secondary (auxiliary) memory, such as SSDs and magnetic disks, sits below main memory. It is **nonvolatile**, so it holds programs and files permanently, but it is thousands of times slower than main memory.

A **disk cache** is a portion of main memory that the OS uses as a buffer for disk blocks. It is a **software** technique (OS code plus ordinary RAM), not a separate hardware memory. (A drive may also carry a small hardware buffer of its own; that is a different thing.) Benefits:

- **Clustered writes:** changed blocks collect in memory and go out later in a few large transfers; a block written twice may reach the disk only once.
- **Re-use:** data referenced again (read or just written) may still be in the disk cache, so it comes from memory instead of the disk.

Example: read 12, read 13, read 12, write 12, write 13, write 12, read 13, write 14, read 12, flush takes 9 disk trips (≈ 45 ms) without a disk cache but 3 (≈ 15 ms) with one: two first-time reads plus one clustered write of blocks 12–14.

The price: main memory is volatile, so changes still waiting in the disk cache are lost if the power fails. The OS therefore writes changed blocks back regularly and flushes everything at shutdown.

Do not confuse the processor cache (hardware, between processor and main memory) with the disk cache (software, a slice of main memory in front of the disk).

Key terms

Memory hierarchy

The arrangement of a computer's storage into levels, from small, fast, expensive memory next to the processor down to huge, slow, cheap storage far from it.

Capacity

How much data a memory can hold, measured in bytes (KB, MB, GB, TB).

Access time

How long a memory takes to deliver (or store) a piece of data after the processor asks for it.

Cost per bit

The price of storing one bit in a memory technology; in practice quoted as dollars per gigabyte.

Inboard memory

The levels the processor reaches directly or over the system bus: registers, cache and main memory.

Outboard storage

Large storage attached through I/O modules, such as solid-state disks, magnetic disks and optical discs.

Off-line storage

Storage media kept outside the running system, such as magnetic tape cartridges, that must be mounted before use; used for backup and archives.

Hit

An access whose data is already in the faster level, so it is served at that level's speed.

Miss

An access whose data is not in the faster level, so it must be fetched from the slower level below (and copied up).

Hit ratio (H)

The fraction of all memory accesses that are hits in the faster memory: $\text{hits} \div \text{total accesses}$, a number from 0 to 1.

Average access time (Ts)

The mean time per access over many accesses. For two levels: $T_s = H \times T_1 + (1 - H) \times (T_1 + T_2)$.

Locality of reference (locality)

The tendency of a program's memory references to cluster: over a short period it uses only a small set of locations, and that set changes slowly.

Temporal locality

A location used recently is likely to be used again soon, as with a loop's instructions or a counter variable.

Spatial locality

A location near one used recently is likely to be used soon, as with consecutive instructions or neighbouring array elements.

Block

A fixed-size group of neighbouring words that moves between two levels as one unit. A miss copies up the whole block, so the neighbours of the word you asked for arrive too.

Secondary memory (auxiliary memory)

Large, nonvolatile storage outside main memory, such as disks, that holds programs and files permanently.

Volatile memory

Memory that loses its contents when the power is turned off, such as registers, cache and main memory.

Nonvolatile memory

Memory that keeps its contents without power, such as solid-state disks, magnetic disks, optical discs and tape.

Disk cache

A portion of main memory that the operating system uses as a buffer for disk data. It is a software technique, not a separate hardware memory.

Solid-state disk (SSD)

Nonvolatile storage built from flash memory chips with no moving parts, also called a solid-state drive; much faster than a magnetic disk but more expensive per bit.

Review questions with answers

1. Which statement about memory technologies is correct?

- A. Faster access time goes with a smaller cost per bit.
- B. Greater capacity goes with a greater cost per bit.
- C. Greater capacity goes with a greater (slower) access time.
- D. Capacity and access time are unrelated.

Answer: C. Greater capacity goes with a greater (slower) access time.

Three relationships hold: faster access → greater cost per bit; greater capacity → smaller cost per bit; greater capacity → slower access time. That tension is why computers use a hierarchy.

2. Put these levels in order from the **top** of the memory hierarchy (closest to the processor) to the **bottom**.

Correct order:

- 1. Registers
- 2. Cache
- 3. Main memory
- 4. Magnetic disk
- 5. Magnetic tape

Registers and cache sit in the processor, main memory completes inboard memory, magnetic disk is outboard storage, and tape is off-line storage at the bottom.

3. Moving **down** the memory hierarchy (from registers toward tape), does each quantity increase or decrease?

Increases	Decreases
Capacity Access time	Cost per bit How often the processor accesses the level

Going down: cost per bit decreases, capacity increases, access time increases, and the frequency of access by the processor decreases.

4. Match each term to its meaning.

Hit ratio	Fraction of accesses found in the faster memory
Temporal locality	A location used recently is likely to be used again soon
Spatial locality	Locations near a recently used one are likely to be used soon
Disk cache	Part of main memory the OS uses to buffer disk blocks
Secondary memory	Nonvolatile storage below main memory, such as disks, that keeps files without power

H measures how well the fast level works, the two kinds of locality explain why it works, secondary memory is the permanent (nonvolatile) store below main memory, and the disk cache is the OS's way of speeding it up.

5. A two-level memory has $T_1 = 0.1 \mu\text{s}$ and $T_2 = 1 \mu\text{s}$. The hit ratio is $H = 0.95$. What is the average access time T_s , in μs ?

Answer: 0.15 μs

$T_s = H \times T_1 + (1 - H) \times (T_1 + T_2) = 0.95 \times 0.1 + 0.05 \times 1.1 = 0.095 + 0.055 = 0.15 \mu\text{s}$, close to the speed of the fast level.

6. A two-level memory has $T_1 = 0.1 \mu\text{s}$ and $T_2 = 1 \mu\text{s}$. What hit ratio H gives an average access time of $0.3 \mu\text{s}$? (Answer as a decimal between 0 and 1.)

Answer: 0.8

$0.3 = 0.1 + (1 - H) \times 1$, so $1 - H = 0.2$ and $H = 0.8$. Dropping H from 0.95 to 0.8 doubles the average time ($0.15 \mu\text{s} \rightarrow 0.3 \mu\text{s}$), which is why hit ratios must stay close to 1.

7. In a two-level memory, how long does an access that **misses** in level 1 take?

- A. T_1
- B. T_2
- C. $T_1 + T_2$
- D. $T_2 - T_1$

Answer: C. $T_1 + T_2$

The processor first checks level 1 (T_1). On a miss, the word is fetched from level 2 into level 1 (T_2) and then read, so the total is $T_1 + T_2$.

8. Why can a small, fast memory level satisfy most of the processor's accesses?

- A. Programs' memory references cluster in small regions for a while (locality of reference).
- B. The OS copies all of main memory into it when the computer starts.
- C. The processor slows down to match the speed of the lower levels.
- D. The lower levels are volatile, so data must be kept in the top level.

Answer: A. Programs' memory references cluster in small regions for a while (locality of reference).

Because references cluster (temporal and spatial locality), the few blocks in use right now can sit in the fast level, and the lower levels are accessed far less often.

9. Classify each example by the kind of locality it mainly shows.

Temporal locality	Spatial locality
A loop's instructions executed again on every pass A counter variable updated in every iteration Calling the same subroutine many times	Reading array elements $a[0]$, $a[1]$, $a[2]$ in order Executing straight-line instructions one after another

Temporal locality means re-using the same location soon (loops, counters, repeated calls). Spatial locality means using neighbouring locations soon (sequential instructions, array elements).

10. The disk cache that the operating system manages is a separate hardware memory chip placed between main memory and the disk.

Answer: False

The OS's disk cache is a portion of ordinary main memory that the operating system sets aside and manages in software as a buffer for disk blocks. No extra hardware is involved.

11. Which are benefits of a disk cache?

- A. Writes can be clustered into fewer, larger disk transfers.
- B. Data referenced again may be found in main memory instead of on the disk.
- C. It makes main memory nonvolatile.
- D. It removes the need for secondary memory.

Answer: A, B

Buffering disk blocks in main memory lets the OS batch writes and serve repeated references from memory. It cannot make RAM survive a power cut, and the disk is still needed to store data permanently.

12. Sort each technology into its group of the memory hierarchy.

Inboard memory	Outboard storage	Off-line storage
Registers Cache Main memory	Solid-state disk Magnetic disk Optical disc (DVD, Blu-ray)	Magnetic tape

Inboard memory is what the processor reaches directly or over the system bus: registers, cache and main memory. Outboard storage is attached through I/O modules: SSDs, magnetic disks and optical discs. Off-line storage, such as tape, sits outside the running system and must be mounted before use.

1.6 Cache Memory

You should be able to

- Explain why processors need a cache (the speed gap, every instruction fetch touches memory) and why a cache works (locality).
- Describe how main memory is divided into blocks, how cache slots and tags record which blocks are present, and trace a read through a hit and a miss.
- Split an address into tag, slot and word fields for a direct-mapped cache, and contrast direct mapping with associative mapping and LRU replacement.
- Explain how cache size, block size and write policy (write-through vs write-back) affect performance and correctness.
- Describe why modern computers stack several levels of cache (L1, L2, L3).

Why a cache is needed

Processor speed has grown much faster than memory speed: a simple instruction can run in about a nanosecond, while a main-memory access takes 50–100 nanoseconds, roughly a hundred times longer. Every instruction cycle touches memory at least once, because the instruction itself must be fetched, so memory speed caps processor speed.

A **cache** is a small, very fast memory between the processor and main memory that holds **copies** of the parts of main memory in current use. It works because of **locality of reference**: programs soon reuse locations they just used (temporal locality) and use locations near them (spatial locality). The hardware normally handles cache fills and replacement by itself, without help from programs or the operating system. The OS affects the cache mostly indirectly: after a process switch the cache is full of the old

process's blocks, so the new one starts with a burst of misses. The OS does occasionally perform explicit **cache maintenance**, for example flushing (writing back) or invalidating lines around a DMA transfer, or when it changes memory mappings, but it never decides which blocks are copied in.

Processor and cache exchange single **words** (fast); cache and main memory exchange whole **blocks** (slower, but the neighbouring words are likely to be used soon).

Organization: blocks, slots and tags

- Main memory has 2^n addressable words (n-bit addresses), divided into fixed-length **blocks** of **K** words, so there are $M = 2^n / K$ blocks.
- The cache has **C slots** (also called **lines**). Each holds one block of K words plus a **tag** identifying which block it holds.
- $C \ll M$, so many blocks take turns in each slot; the tag is the only record of which one is there now.

Example. $n = 16$, $K = 4$: $2^{16} = 65,536$ words and $M = 65,536 / 4 = 16,384$ blocks. At real scale (every byte has an address), 16 GB in 64-byte blocks is about 268 million blocks, while a 32 KB level-1 cache has 512 slots.

The cache read operation

1. The processor generates a read address **RA**.
2. The cache checks (by comparing tags) whether the block containing RA is present.
3. **Hit**: fetch the word from its slot and deliver it to the processor; main memory is not touched.
4. **Miss**: read the block containing RA from main memory; allocate a slot for it (evicting a block if necessary); load the block into the slot and deliver the word to the processor, these last two at the same time.

The **hit ratio** is hits $\div$ total accesses, e.g. 340 hits in 400 accesses = 85%.

Mapping function

The mapping function decides which slot(s) a block may occupy, and so how the cache splits an address into fields.

Direct mapping: each block has exactly one slot: **slot = block number mod C**. The address splits (high bits to low) into **tag | slot | word**: the word field has $\log_2 K$ bits, the slot field $\log_2 C$ bits, and the tag the remaining $n - \log_2 C - \log_2 K$ bits. In block terms: slot = block mod C and tag = block $\div$ C (whole part); the tag tells apart the blocks that share a slot. Only one tag is compared: simple, cheap, fast. Drawback: two busy blocks that map to the same slot keep evicting each other even while other slots are empty.

Examples. (a) 64 words, 4-word blocks, 4 slots: address 37 = 100101 $\rightarrow$ tag 10, slot 01, word 01 (word 1 of block 9; slot $9 \bmod 4 = 1$; tag $9 \div 4 = 2$, remainder dropped). (b) 20-bit addresses, 8-word blocks, 128 slots: word 3 bits, slot 7 bits, tag $20 - 3 - 7 = 10$ bits. (c) With $C = 64$, block 200 goes to slot $200 \bmod 64 = 8$. (d) With 8 slots, blocks 5 and 13 both map to slot 5, so alternating between them misses every time.

Associative (fully associative) mapping: a block may go into **any** slot. The address splits into **tag | word**; the tag is the whole block number, so it is longer than a direct-mapped tag. The hardware must compare it

with every slot's tag at once (one comparator per slot, more complex and costly), and a replacement algorithm must choose a victim when the cache is full. In return, no block is evicted while a slot is free. **Set-associative mapping**, used by most real caches, is the compromise: a block maps to one small set of slots and may use any slot in that set.

Replacement algorithm

When a block must be loaded and there is a choice of slot, the replacement algorithm picks the victim (direct mapping has no choice). The usual one is **least recently used (LRU)**: evict the block unused the longest, betting it is least likely to be needed soon. Example: blocks P, Q, R, S last used at times 8, 3, 6, 4: LRU evicts Q.

Locality in the simulator (4 slots, 4-word blocks): an array walk hits 12 of 16 reads (spatial locality); a loop run 3 times, 22 of 24 (temporal locality); random reads, 3 of 16; alternating blocks 0 and 4, 0 of 12 direct-mapped (both need slot 0), 10 of 12 associative.

Cache size and block size

Cache size: even a small cache has a large effect, and each further doubling helps less. Larger caches also cost more and are slower, so the fastest caches stay small. (Simulated program: best hit ratio 76% at 64 words, 94% at 512.)

Block size: as blocks grow, the hit ratio **first rises**, because each miss brings in neighbouring words that are soon used (spatial locality), **then falls**: with the cache size fixed there are fewer, larger blocks, so data still being reused gets evicted, and much of each large block is never used. (Simulated 256-word cache: 55% with 1-word blocks, 91% with 8, 51% with 64.)

Write policy

	Write-through	Write-back
Main memory updated	On every write, with the cache	Only when a changed block is evicted (or flushed)
Extra hardware	None	A dirty bit (modified bit) per slot, set by a write
Memory traffic	High: a memory write per store	Low: one block write per dirty eviction
Main memory current?	Always	Not always: it can be stale

Example: four stores to x, then its block is evicted: write-through makes 4 memory writes, write-back 1.

Direct memory access (DMA) lets an I/O module use main memory without going through the processor or its cache; if it reads x before the eviction, write-back hands it a stale value, so hardware or the OS must write dirty blocks back first. With several processors, another cache can keep an old copy even under write-through, so hardware must update or invalidate it (**cache coherence**). Write-back never loses data: the newest value stays in the cache until written back.

Multiple levels of cache

One cache cannot be both large and very fast (a bigger memory takes longer to access), so processors stack levels. **L1**: tiny and fastest, in each core, usually split into instruction and data caches. **L2**: larger, a

little slower, usually private to each core. **L3**: several megabytes, shared by all cores. A miss at one level goes to the next; only a miss at every level reaches main memory. Rough total times: L1 about 4 cycles, L2 12, L3 40, main memory 200+.

Average access time = sum over the levels of (fraction of accesses answered there × time to get the word from there). With a 90% L1 hit ratio: L1 only gives $0.9 \times 4 + 0.1 \times 200 = 23.6$ cycles, 20 of them from the rare misses. An L2 that catches 80% of L1 misses gives $0.9 \times 4 + 0.1 \times (0.8 \times 12 + 0.2 \times 200) = 8.56$ cycles. An L3 that catches 75% of what is left brings it to about 6.2 cycles.

The six cache design issues

(1) Cache size, (2) block size, (3) mapping function, (4) replacement algorithm, (5) write policy, (6) number of cache levels.

Key terms

Cache memory (cache)

A small, very fast memory placed between the processor and main memory that holds copies of the parts of main memory in current use. Hardware fills it and chooses what to replace without help from programs or, normally, from the OS.

Word

The unit of data the processor reads or writes in one memory access. Every word in main memory has its own address.

Block

A fixed-size group of K neighbouring words. Blocks are the unit copied between main memory and the cache: a miss brings in a whole block, never a single word.

Slot (cache line)

One place in the cache that can hold exactly one block, together with the tag that says which block it is.

Tag

A few bits stored with each cache slot that identify which main-memory block the slot currently holds.

Cache hit (hit)

A memory access whose word is already in the cache, so it is delivered at cache speed.

Cache miss (miss)

A memory access whose word is not in the cache, so its whole block must first be copied in from slower main memory.

Hit ratio

The fraction of memory accesses that are hits: $\text{hits} \div \text{total accesses}$. A higher hit ratio means the processor waits for main memory less often.

Locality of reference (locality)

The tendency of programs to reuse the same memory locations soon (temporal locality) and to use locations near recently used ones (spatial locality).

Mapping function

The rule that decides which cache slot (or slots) a given main-memory block is allowed to occupy.

Direct mapping

A mapping function in which every block has exactly one slot it can use: $\text{slot} = \text{block number} \bmod C$. Simple and fast, but blocks that share a slot evict each other.

Associative mapping

A mapping function in which a block may be placed in any slot (also called fully associative). Flexible, but every slot's tag must be checked on each access.

Set-associative mapping

A compromise mapping: slots are grouped into small sets; a block maps to one set (like direct mapping) but may use any slot within that set (like associative mapping).

Replacement algorithm

The rule that picks which block to throw out of the cache when a new block must be loaded and there is a choice of slot.

Least recently used (LRU)

A replacement algorithm that evicts the block that has gone unused for the longest time, betting that it is the least likely to be needed soon.

Write policy

The rule for when main memory is updated after the processor writes to a word that is in the cache.

Write-through

A write policy in which every write goes to both the cache and main memory at once, so main memory is never out of date.

Write-back

A write policy in which writes update only the cache; the changed block is copied to main memory later, when it is evicted.

Dirty bit

A one-bit flag per cache slot (also called the modified or update bit) that is set when the block has been written, so the cache knows it must be written back before being replaced.

Multilevel cache

A stack of caches (L1, L2, L3) between the processor and main memory, each larger and slower than the one above it.

Review questions with answers

1. Why does the speed of main memory limit how fast a processor can run, even for a program that touches very little data?

- A. Every instruction must itself be fetched from memory before it can run.
- B. The processor must copy the whole program into the cache before starting it.
- C. The operating system checks main memory after every instruction.
- D. The processor's registers are stored in main memory.

Answer: A. Every instruction must itself be fetched from memory before it can run.

Each instruction cycle starts with an instruction fetch, which is a memory access. So even data-light code needs at least one memory access per instruction, and a slow memory would stall the processor constantly.

2. The operating system decides which blocks of main memory are copied into the cache.

Answer: False

Hardware loads blocks on a miss and picks which ones to replace, with no OS involvement. The OS shapes cache performance indirectly (for example through how often it switches processes), and occasionally it performs explicit cache maintenance, such as flushing or invalidating lines around a DMA transfer or when it changes memory mappings, but it never chooses which blocks are copied in.

3. A memory has 16-bit addresses, so it holds 2^{16} words, and it is divided into blocks of $K = 8$ words. How many blocks M does it contain?

Answer: 8192 blocks

$M = 2^n / K = 65,536 / 8 = 8,192$ blocks.

4. Put the steps of a cache read that turns out to be a miss in order.

Correct order:

1. The processor issues read address RA
2. The cache checks the tags and finds the block containing RA is not present
3. The block containing RA is read from main memory
4. A cache slot is allocated for the block, evicting one if necessary
5. The block is loaded into the slot and the word is delivered to the processor

Receive the address, check for a hit, and on a miss fetch the whole block, find it a slot, then store it while passing the wanted word to the processor (those last two happen together).

5. A direct-mapped cache has $C = 128$ slots. Which slot can main-memory block number 1000 be stored in?

Answer: 104

Direct mapping uses $\text{slot} = \text{block} \bmod C$. $1000 = 7 \times 128 + 104$, so the block can only go in slot 104.

6. A direct-mapped cache is used with 16-bit addresses, blocks of 4 words and 64 slots. How many bits long is the tag field?

Answer: 8 bits

4 words per block need a 2-bit word field; 64 slots need a 6-bit slot field. The tag is what is left: $16 - 2 - 6 = 8$ bits.

7. A processor has an L1 cache and an L2 cache. Getting a word takes 4 cycles when it is found in L1, 12 cycles when it is found in L2, and 200 cycles when it must come from main memory. 95% of accesses hit in L1, and L2 catches 80% of the accesses that miss in L1. What is the average number of cycles per access?

Answer: 6.28 cycles

Weight each time by how often it happens: $0.95 \times 4 + 0.05 \times (0.8 \times 12 + 0.2 \times 200) = 3.8 + 0.05 \times 49.6 = 3.8 + 2.48 = 6.28$ cycles. Without the L2 it would be $0.95 \times 4 + 0.05 \times 200 = 13.8$ cycles, so the second level more than halves the average.

8. Compared with direct mapping, which statements about associative (fully associative) mapping are true?

- A. A block can be placed in any slot.
- B. The address's tag must be compared with the tags of all slots.
- C. A replacement algorithm such as LRU is needed to choose a victim when the cache is full.
- D. Two busy blocks can keep evicting each other while other slots sit empty.
- E. It needs fewer tag comparisons per access.
- F. Its tag field is shorter.

Answer: A, B, C

Freedom to use any slot means the hardware must search every slot at once and must choose a victim when all are full. Two blocks fighting over one slot while others sit empty is the weakness of direct mapping (for example, with 4 slots blocks 3 and 7 both map to slot 3); associative mapping avoids it. Its tag is the whole block number, so it is longer, not shorter.

9. The cache size stays fixed while the block size grows from 1 word to very large blocks. What typically happens to the hit ratio?

- A. It rises at first, then falls.
- B. It rises steadily, because each miss brings in more neighbouring words.
- C. It falls steadily, because each miss takes longer to service.
- D. It stays the same, because the cache holds the same number of words.

Answer: A. It rises at first, then falls.

Bigger blocks exploit spatial locality, so hits increase at first. Past a sweet spot, fewer, larger blocks mean useful data is evicted and much of each block is never used, so the hit ratio drops.

10. Sort each property under the write policy it describes.

Write-through	Write-back
Main memory is updated on every write A loop that stores to one variable 1,000 times causes about 1,000 memory writes An I/O module reading main memory always sees the newest value	Uses a dirty bit to remember which slots changed Main memory can hold stale data for a while A changed block is copied to memory when it is evicted

Write-through keeps memory current at the price of a memory write per store. Write-back saves traffic by writing a block only when a dirty one is evicted, so memory may lag behind the cache.

11. Match each cache design issue to the question it answers.

Mapping function	Which slot or slots may this block occupy?
Replacement algorithm	Which block should be thrown out to make room?
Write policy	When does main memory get updated after a store?
Block size	How many words move on each miss?
Number of levels	How many caches sit between the processor and main memory?

Together with cache size, these are the design decisions every cache designer must make.

12. A 4-slot associative cache uses LRU replacement. It holds blocks A, B, C and D, last used at times 5, 9, 2 and 7. Block E is now needed. Which block is evicted?

- A. A
- B. B
- C. C
- D. D

Answer: C. C

Least recently used means the block with the oldest last-use time: C, last used at time 2. LRU bets that the block idle the longest is the least likely to be needed soon.

1.7 Direct Memory Access

You should be able to

- Describe how programmed I/O, interrupt-driven I/O and DMA each move a block of data, and who does the work.
- Explain why busy waiting wastes processor time and why interrupts help but still route every word through the processor.
- State the two drawbacks shared by programmed and interrupt-driven I/O.
- List the four items the processor gives a DMA module and trace a DMA block transfer from setup to completion interrupt.
- Explain cycle stealing and why DMA still wins for multiple-word transfers.

The problem: who carries the data?

A disk read or a network packet is a **block** of words that must travel between an I/O device and main memory. There are three ways to move it: **programmed I/O**, **interrupt-driven I/O** and **direct memory access (DMA)**. They differ in who moves each word and how much processor time the transfer eats.

1. Programmed I/O

The processor gives the I/O module a command (for example, read the next word). The module carries it out and, when finished, sets bits in its **I/O status register** (such as READY). It then does nothing more: **no interrupt**. The processor must check the status register again and again until the bit shows the operation is complete. This repeated checking is **polling**; waiting this way is **busy waiting**, because the processor is fully occupied yet achieves nothing.

1. Issue a READ command to the I/O module.
2. Read the status register. Not ready? Read it again (loop).
3. Ready: read the word from the module's data register into a processor register.
4. Write that register into the next memory cell.
5. More words? Go back to step 1.

Cost per word = 1 command + the status checks + 1 read + 1 store. If each word needs 25 checks (the 25th finds it ready), a 10-word block costs $10 \times (1 + 25 + 1 + 1) = \mathbf{280 \text{ instructions}}$, of which 240 are checks that found nothing, and no other program runs meanwhile.

2. Interrupt-driven I/O

The processor issues the command and then **goes on with other useful work**. When the module is ready to exchange a word, it **interrupts** the processor. The processor finishes its current instruction, saves the **context** of the running program (program counter, status, registers), and runs the **interrupt handler**. The handler reads the word from the module, writes it to memory, issues the next command, restores the context and returns; the interrupted program continues where it stopped.

This beats polling because the waiting time goes to other programs. But **every word still passes through the processor** and costs a full interrupt. In the guide's model a word costs 11 instructions (save 4, move 2, next command 1, restore 4), so a 200-word block costs 2,200 instructions and 200 interrupts. Worked example (3 words, device needs 16 time units per word): 3 interrupts and 33 instructions of I/O work, while the other program gets 40 instructions done.

The two drawbacks both techniques share

1. **Speed limit:** the I/O transfer rate is limited by how fast the processor can test and service a device. With a very fast device, polling needs at least 3 instructions of service per word and each interrupt needs 11, so the processor, not the device, sets the pace.
2. **Tied-up processor:** the processor must execute a number of instructions for every single word transferred.

Root cause: each word travels through a processor register. Model (device needs D time units per word): polling moves a word every $D + 3$ units using 100% of the processor; interrupts move one every $\max(D, 4) + 7$ units, 11 of them busy. $D = 30$: polling reaches 91% of the device's speed, interrupts 81% while using about 30% of the processor. $D = 3$: only 50% and 27%, with the processor busy 100% of the time.

3. Direct memory access (DMA)

A **DMA module** is hardware that does the processor's bus work for a transfer: it puts addresses and words on the bus and tells memory to read or write. It may be a separate module on the system bus or built into an I/O module. To start a block transfer, the processor sends it four items:

1. whether the operation is a **read or a write** (on the read/write control line);
2. the **address of the I/O device** involved;
3. the **starting location in memory** to read from or write to (kept in the DMA module's **address register**);
4. the **number of words** to transfer (kept in its **count register**).

The processor then continues with other work. The DMA module transfers the entire block, one word at a time, directly to or from memory, **without the words passing through the processor**. After each word the address register goes up by one and the count register goes down by one. When the count reaches zero the module sends **one interrupt**. The processor is involved only at the **beginning** (setup) and at the **end** (the completion interrupt). In the guide's model that is 4 setup instructions + 10 for the completion interrupt = **14 instructions per block**, whatever its size: a 200-word block costs 14 instead of 2,200 with interrupts, saving 2,186. For a single word, though, 14 is more than one interrupt's 11, so DMA pays off for **multiple-word** transfers.

Cycle stealing

The DMA module needs the system bus for every word, and the processor uses the same bus. Rather than waiting for a cycle the processor leaves free, the module usually takes the bus for **one bus cycle** whenever it has a word; if the processor wants the bus in that cycle, it must **pause** for that one cycle. This is **cycle stealing**. It is **not an interrupt**: no context is saved and no handler runs; the processor simply waits, then carries on. So it can happen in the middle of an instruction, wherever the processor needs the bus (fetching an instruction or operand, writing a result), whereas an interrupt is only recognised at the end of an instruction. If the processor does not need the bus in that cycle (decoding, executing internally), the steal costs nothing.

Effect: the processor runs somewhat slower during a DMA transfer. Example (instructions of 3 to 5 stages, about half the cycles using the bus): a DMA word every 4 cycles gives 250 steals in 1,000 cycles; 143 cause a pause (14.3% of cycles lost, 213 instructions finished, where 248 would finish with no DMA at all) and 107 cost nothing. Still, at most one lost cycle per word is far cheaper than many instructions and an interrupt per word, so for multiple-word transfers DMA is far more efficient than the other two techniques.

Comparison

Technique	Who moves each word	How the processor learns it is done	Interrupts for N words	Processor cost
Programmed I/O	Processor	Polls the status register	0	All its time (busy waiting)
Interrupt-driven I/O	Processor (handler)	Interrupt per word	N	Save + move + restore per word
DMA	DMA module	One interrupt per block	1	Setup + one interrupt + stolen cycles

Race, 8 words, slow device (30 units per word): processor time on the transfer is 264 (polling), 88 (interrupts), 14 (DMA, plus 8 stolen cycles); the other program gets 0, 212 and 233 units done. Fast device (3 per word): the device could deliver all 8 by $t = 24$, but polling ends at $t = 48$ and interrupts at $t = 91$ (the processor is the bottleneck); DMA keeps pace and ends at $t = 39$.

Common mistakes

- Thinking the I/O module signals the processor in programmed I/O. It only sets status bits; the processor must look.
- Thinking interrupts keep the data out of the processor. They remove the waiting, not the carrying.
- Thinking DMA interrupts once per word. It interrupts once per block.
- Calling cycle stealing an interrupt. It is a one-cycle pause; nothing is saved.

Key terms

Programmed I/O

An I/O technique in which the processor gives a command to an I/O module and then checks the module's status over and over until the operation is done. The module never interrupts.

Interrupt-driven I/O

An I/O technique in which the processor gives a command, goes on with other work, and is interrupted by the I/O module when it is ready to exchange data. The processor still moves every word itself.

Direct memory access (DMA)

An I/O technique in which a DMA module moves a whole block of words between a device and main memory by itself. The processor only sets up the transfer and handles one interrupt at the end.

DMA module

Hardware, either a separate module on the system bus or logic built into an I/O module, that can take over the bus and move words to or from main memory in place of the processor.

I/O status register (status register)

A small register inside an I/O module whose bits report how the current operation is going, for example ready, busy or error. The processor reads it to find out whether the module has finished.

Data register

A register inside an I/O module that holds the word currently being passed between the device and the rest of the computer.

I/O command

A request the processor sends to an I/O module (by executing an I/O instruction) telling it what to do next, such as read the next word from the disk.

Polling

Asking a device for its status again and again, on the processor's own initiative, to see whether it has finished.

Busy waiting

Waiting by running a loop that checks a condition over and over instead of doing other work or sleeping. The processor looks busy but achieves nothing.

Interrupt handler

The routine, normally part of the operating system, that runs when the processor accepts an interrupt. It deals with the event and then lets the interrupted program continue.

Context (processor state)

Everything the processor needs to continue a program exactly where it stopped: the program counter, the status word and the other registers. It is saved before an interrupt is handled and restored afterwards.

Word

The fixed-size group of bits that the processor, memory and bus move as one unit.

Block transfer

Moving a group of consecutive words, such as one disk sector, between a device and memory as a single job.

Transfer rate

How many words (or bytes) per second actually move between a device and main memory.

Address register

A register in a DMA module that holds the memory address for the next word. It goes up by one after every word, so the block lands in consecutive cells.

Count register

A register in a DMA module that holds how many words are still to move. It goes down by one after every word; at zero the block is finished.

Bus cycle

One use of the system bus: the time it takes to move one word from one module to another over it.

Cycle stealing

The way a DMA module uses the bus: it takes the bus for one bus cycle at a time, and if the processor wanted the bus then, the processor simply pauses for that cycle. Nothing is saved and nothing is interrupted.

Review questions with answers

1. In programmed I/O, how does the processor learn that the I/O module has finished an operation?

- A. It reads the module's status register again and again until the ready bit is set.
- B. The I/O module sends it an interrupt as soon as the word is ready.
- C. A DMA module reports it once the whole block is in memory.
- D. The timer interrupt tells it at the next clock tick.

Answer: A. It reads the module's status register again and again until the ready bit is set.

The module does the work and sets bits in its I/O status register, but it never signals. The processor must poll that register, which is busy waiting.

2. With interrupt-driven I/O, the words being transferred no longer pass through the processor.

Answer: False

Interrupts remove the waiting, not the carrying. The interrupt handler still reads each word from the module into a processor register and then writes it to memory.

3. Which items does the processor send to a DMA module to start a block transfer? Select all that apply.

- A. Whether the operation is a read or a write
- B. The address of the I/O device involved
- C. The starting location in memory to read from or write to
- D. The number of words to transfer
- E. The data words themselves
- F. The address of a handler to run after every word

Answer: A, B, C, D

Those four items are all the DMA module needs. The data never passes through the processor, and there is no per-word handler: the module interrupts once, when the whole block is done.

4. Put the steps of a DMA block transfer in order.

Correct order:

1. The processor sends the operation, device address, start address and word count to the DMA module
2. The processor goes back to other work
3. The DMA module moves the words one at a time between the device and memory
4. The count reaches zero and the DMA module sends an interrupt
5. The processor runs the interrupt handler and continues

The processor takes part only at the beginning (setup) and at the end (the completion interrupt). Everything in between is done by the DMA module.

5. Match each technique or mechanism to its description.

Programmed I/O	Checks a status bit again and again
Interrupt-driven I/O	An interrupt for every word
Direct memory access	One interrupt for the whole block
Cycle stealing	Borrows the bus for one cycle

Polling, one interrupt per word, one interrupt per block, and borrowing single bus cycles: four different answers to the question "who moves the data, and who waits?".

6. During a DMA block transfer, who does each job?

Processor	DMA module
Sending the start address and word count to the module Running the handler when the block is finished	Putting each word's memory address on the bus Adding one to the address register after each word Subtracting one from the count register after each word Raising the interrupt when the count reaches zero

The processor sets the job up and handles the one interrupt at the end. Every per-word job, including updating the address and count registers, belongs to the DMA module.

7. A 512-word block is read from a disk twice: once with interrupt-driven I/O and once with DMA. How many interrupts does the processor receive in total over both transfers?

Answer: 513 interrupts

Interrupt-driven I/O interrupts once per word (512); the DMA module interrupts only once, after the whole block is done (1). $512 + 1 = 513$.

8. Suppose interrupt-driven I/O costs the processor 11 instructions per word (save context 4, move the word 2, issue the next command 1, restore context 4), while a DMA transfer costs it 14 instructions per block (4 to set up, 10 for the one completion interrupt), whatever the block size. How many processor instructions does DMA save on a 200-word block?

Answer: 2186 instructions

Interrupt-driven: $200 \times 11 = 2,200$ instructions. DMA: 14, because the processor works only at the start and the end. Saving: $2,200 - 14 = 2,186$.

9. With programmed I/O, the device needs 25 status checks before each word is ready (the 25th check finds it ready). Each word also needs 1 command, 1 read and 1 store. How many instructions does a 10-word block take?

Answer: 280 instructions

Per word: 1 command + 25 checks + 1 read + 1 store = 28 instructions, so 10 words take 280. Of those, 240 are checks that found nothing (24 per word).

10. Which statement about cycle stealing is correct?

- A. The DMA module uses the bus for one bus cycle, and the processor pauses for that cycle without saving its context.
- B. The DMA module interrupts the processor for each word, and the processor saves its context before giving up the bus.
- C. The processor must finish its current instruction before the DMA module may use the bus.
- D. The processor steals bus cycles from the DMA module, so the transfer never slows the processor down.

Answer: A. The DMA module uses the bus for one bus cycle, and the processor pauses for that cycle without saving its context.

Cycle stealing is a pause, not an interrupt: one bus cycle is handed to the DMA module, and the processor carries on afterwards as if nothing happened.

11. Programmed I/O and interrupt-driven I/O share two drawbacks. Which pair is it?

- A. The transfer rate is limited by how fast the processor can test and service the device, and the processor is tied up managing each transfer.
- B. The device writes straight into memory behind the processor's back, and each word needs its own DMA setup.
- C. Every word is stored twice in main memory, and they can only read from a device, never write to one.
- D. They rely on cycle stealing, so the processor pauses for a bus cycle on every word.

Answer: A. The transfer rate is limited by how fast the processor can test and service the device, and the processor is tied up managing each transfer.

In both techniques every word travels through a processor register, so the processor both caps the speed and pays instructions for every word.

12. Because a DMA module steals bus cycles, the processor runs a little slower during a DMA transfer; even so, DMA is far more efficient than the other two techniques for multiple-word transfers.

Answer: True

Losing one bus cycle per word is tiny compared with many instructions and an interrupt per word. For a single word, though, DMA's fixed setup cost can outweigh its benefit.

1.8 Multiprocessor and Multicore Organization

You should be able to

- Distinguish three ways of building parallel hardware: symmetric multiprocessors (SMPs), multicore computers and clusters.
- List the defining characteristics of an SMP and explain what makes it “symmetric”.
- Explain the four advantages of an SMP over a uniprocessor (performance, availability, incremental growth, scaling) and how the OS keeps the extra processors transparent to users.
- Describe how an SMP and a multicore chip are organized (private caches, shared caches, shared memory and bus), and explain the cache coherence problem and how invalidation solves it.
- Explain why chip makers switched to multicore designs, using the Intel Core i7-5960X as a real example.

1. Why use more than one processor?

A **uniprocessor** has exactly one processor, so it follows one stream of instructions at a time. **Parallel processing** means doing several pieces of work at the same moment on separate processors. Three common designs:

Design	Where the processors are	Shared main memory?	Operating system
Symmetric multiprocessor (SMP)	separate processor chips (one per socket) in one computer	yes, one memory	one integrated OS for all
Multicore computer (chip multiprocessor)	several cores on a single chip (one die)	yes, one memory	one integrated OS for all
Cluster (only mentioned here)	separate complete computers linked by a fast network	no, each has its own and they exchange messages	one copy per computer

The ideas combine: a server with two 8-core chips is an SMP with 16 cores.

2. The five marks of an SMP

1. **Two or more similar processors** of comparable capability (same kind, roughly the same speed), so the OS can treat them as interchangeable.
2. They **share the same main memory** and I/O facilities, connected by a bus or other interconnection scheme, so a memory access takes **about the same time** whichever processor makes it.
3. They **share access to the I/O devices**, either over the same paths or over different paths that lead to the same device.
4. **All processors can perform the same functions**: run user programs, run OS code, handle interrupts. This equality is why the design is called **symmetric**.
5. The system is controlled by **one integrated operating system** that lets the processors and their programs cooperate at the level of jobs, tasks, files and individual data elements.

Common mistake: “symmetric” does not mean running one program in lockstep. It means no processor is special; a design where one boss processor alone runs the OS is asymmetric.

3. Advantages of an SMP over a uniprocessor

- **Performance:** if work can be split into parts, they run at the same moment on different processors, so more gets done per second.
- **Availability:** since all processors can do the same jobs, one failure does not halt the machine; the others carry on, more slowly. A uniprocessor has no spare.
- **Incremental growth:** the owner can make the system stronger by adding a processor to the existing machine instead of replacing it.
- **Scaling:** a vendor can offer a range of products at different prices and performance levels by building them with different numbers of processors.

The multiple processors are **transparent** to the user: nobody chooses a processor. The OS schedules processes and threads across all the processors and synchronizes them when they share data. A program only speeds up if its work can really be divided (see 4.3).

Worked example. 12 independent jobs each need 4 s of processor time: total work $12 \times 4 = 48$ s. One processor needs 48 s. With 2 processors kept busy the jobs finish in $48 \div 2 = 24$ s; with 3, in 16 s; with 4, in 12 s. If one of 2 processors fails, its unfinished job is requeued and the survivor finishes everything, just later.

4. How an SMP is organized

- Each processor is **self-contained**: its own control unit, ALU, registers and pipeline, plus one or two levels of **private cache** (L1, often L2).
- The processors reach **one shared main memory** and the **I/O subsystem** over a **shared system bus**. The bus is simple but carries one transfer at a time, so processors sometimes wait for it.
- Because every processor sees the same memory, processors can **communicate through memory**: one writes a value into a shared location, another reads it.

5. The cache coherence problem

Private caches create copies that other processors cannot see. Suppose $X = 5$ and both P1 and P2 have cached X. P1 writes $X = 6$ (also written through to memory), but P2's cache still says 5, so P2's next read **hits** in its own cache and returns the **stale** value 5. The table follows the story on.

Cache coherence means keeping every cached copy of the same data consistent. It is solved **in hardware**, invisibly to programs and the OS. A common method is **bus snooping** with **invalidation**: every cache controller watches the shared bus; when one processor writes X, an "invalidate X" message makes every other cache mark its copy (the whole cache line) invalid, so the next read of X misses and fetches the fresh value.

Coherence is not mutual exclusion. It keeps copies consistent but does not make $X = X + 1$ indivisible: if two processors interleave their read and write steps, an update can still be lost (a race condition, cured in Chapter 5).

Event	No protocol	With invalidation
P1 reads X, P2 reads X	both caches hold 5	both caches hold 5
P1 adds 1	P1 and memory hold 6; P2 still holds 5 (stale)	P1 and memory hold 6; P2's copy is invalidated
P2 reads X	hit: gets 5 (wrong)	miss: fetches 6
P2 adds 1	writes 6: an update is lost	writes 7 (correct); P1's copy is invalidated

6. Multicore computers

A **multicore computer**, or **chip multiprocessor**, puts two or more processors, called **cores**, on a single piece of silicon, the **die**. Each core has its own registers, ALU, pipeline, control unit and L1 caches (the smallest, fastest level, often split into L1-I for instructions and L1-D for data). There is usually an L2 cache per core, and often an L3 cache that all the cores **share**. To the OS each core looks like a separate processor, so a multicore chip works much like a small SMP.

Why multicore? For decades each new chip got faster largely by raising its **clock speed**. A faster clock also needs a higher voltage, so power, and therefore heat, rises far faster than speed: in a simple model a core's power grows with the **cube** of its clock speed while its work grows only in proportion to it. In the mid-2000s this **power wall** stopped clock growth. Transistor counts kept rising, so designers spent the extra transistors on **more cores and bigger caches**.

Worked example (model: 1 core at 1 GHz = 1 unit of power and 1 of work; cooling limit 64). 1 core at 4 GHz: power $4^3 = 64$ (at the limit), work 4. 1 core at 5 GHz: power 125, too hot. 4 cores at 2 GHz: power $4 \times 8 = 32$, work 8. 4 cores at 2.5 GHz: power $4 \times 15.625 = 62.5$, work 10 (2.5 times the best cool single core). 8 cores at 2 GHz: power 64, work 16. The gain needs software that keeps every core busy: a program that is one single stream of instructions runs on only one core, so on the 8×2 GHz chip it runs at half the speed it had on one 4 GHz core. Also, real voltages cannot drop forever, so slow cores save less than the model says.

7. A real example: Intel Core i7-5960X (2014)

- **Eight x86 cores** on one die; each core has its own L1 caches (32 KB instructions + 32 KB data).
- Each core has a **dedicated 256 KB L2 cache** ($8 \times 256 \text{ KB} = 2 \text{ MB}$ in all, but each core uses only its own).
- All eight cores **share a 20 MB L3 cache**.
- An **integrated memory controller** for DDR4 main memory (four channels) and **PCI Express** lanes for fast devices (graphics cards, SSDs) are built onto the chip.
- Private per core: control unit, ALU, registers, pipeline, L1s, L2. One per chip, shared by all cores: L3, memory controller, PCIe lanes.

Key terms

Parallel processing

Doing several pieces of work at the same moment on separate processors, instead of one after another on a single processor.

Uniprocessor

A computer with exactly one processor, so it can carry out only one stream of instructions at any moment.

Symmetric multiprocessor (SMP)

One computer with two or more similar processors of comparable capability that share main memory and the I/O devices, can each perform any function, and are all controlled by one integrated operating system.

Multicore computer (chip multiprocessor)

A computer whose processor chip holds two or more complete processors, called cores, built on a single piece of silicon (one die).

Die

The single small slab of silicon on which a chip's circuits are built. A multicore chip puts all of its cores, and usually some shared cache, on one die.

Cluster

A group of complete, separate computers, each with its own memory and operating system, linked by a fast network and working together as if they were one bigger machine.

Availability

How much of the time a system is up and doing useful work. In an SMP, the failure of one processor does not stop the machine, because the others carry on.

Incremental growth

Making a system more powerful one step at a time, for example by adding a processor to an existing SMP instead of replacing the whole machine.

Scaling

Offering a family of products at different prices and performance levels by building them with different numbers of processors.

Transparency (transparent)

A mechanism is transparent when users cannot see it and need not do anything about it. In an SMP the OS spreads the work across the processors, so users need not know how many there are.

Private cache

A cache used by one processor or core alone, such as each core's L1 cache. Its opposite is a shared cache, used by all the cores on a chip, such as the L3 on many multicore chips.

Cache coherence

Keeping every cached copy of the same memory data consistent, so that no processor or core keeps working with an out-of-date (stale) value after another one has changed it.

Invalidation

A cache coherence technique: when one processor writes a data item, the hardware marks every other cache's copy of the block (cache line) holding that item invalid, so the next read of it misses and fetches the new value.

Bus snooping (snooping)

A hardware technique in which every cache controller watches the shared bus for other processors' writes, so it can invalidate its own copy of anything that changed.

Control unit

The part of a processor that fetches and decodes instructions and sends the signals that make the ALU, the registers and memory carry them out.

Pipeline

Hardware inside a processor that works on several instructions at once, each at a different stage (fetch, decode, execute, and so on), like an assembly line.

Clock speed (clock rate)

How many clock cycles a processor completes per second, measured in hertz. 3 GHz means 3 billion cycles per second; the processor does its work in steps timed by these ticks.

Power wall

The limit reached in the mid-2000s when raising the clock speed further made chips use too much power and produce more heat than normal cooling could remove, so designers began adding cores instead.

Integrated memory controller

Circuitry built onto the processor chip itself that drives the main-memory (DRAM) chips, so memory requests do not have to pass through a separate chip.

PCI Express (PCIe)

A fast point-to-point connection standard for attaching devices such as graphics cards and solid-state drives. On many modern chips, including the Core i7-5960X, some PCIe lanes come straight out of the processor chip itself.

Review questions with answers

1. Which description fits a **symmetric multiprocessor (SMP)?**

- A. One processor runs the OS and hands out work; the other processors may only run user programs.
- B. Two or more similar processors share main memory and the I/O devices, any of them can perform any function, and one integrated OS controls them all.
- C. Several complete computers, each with its own memory and OS, cooperate over a network.
- D. One very fast processor with a large cache that many programs take turns using.

Answer: B. Two or more similar processors share main memory and the I/O devices, any of them can perform any function, and one integrated OS controls them all.

An SMP is one computer with similar processors that share memory and I/O, can all do the same jobs, and run under one integrated operating system.

2. In an SMP, users must tell the system which processor each of their programs should run on.

Answer: False

The OS schedules processes and threads across the processors by itself. The existence of several processors is transparent to users; they just see work finish sooner.

3. On the Intel Core i7-5960X (8 cores on one die), which of these is there **just one of**, shared by all eight cores? Select all that apply.

- A. The 20 MB L3 cache
- B. The integrated DDR4 memory controller
- C. The PCI Express lanes that connect fast devices
- D. The 256 KB L2 cache
- E. The L1 data cache
- F. The control unit

Answer: A, B, C

Each core owns its control unit, ALU, registers, pipeline, L1 caches and a dedicated 256 KB L2 ($8 \times 256 \text{ KB} = 2 \text{ MB}$ of L2 in all, but no core can use another's). The 20 MB L3, the memory controller and the PCI Express lanes exist once on the die and serve every core.

4. Sort each situation by the SMP advantage it shows.

Performance	Availability	Incremental growth	Scaling
Four customer requests are handled at the same moment on four processors	A server keeps working after one of its processors burns out A faulty processor is taken out of service and users notice no outage	An admin installs a second processor in an empty socket instead of buying a new server	A vendor sells one server design in 2-, 4- and 8-processor models

Performance = more work at once. Availability = surviving a failure. Incremental growth = upgrading the machine you have. Scaling = a product range built from different processor counts.

5. In a classic SMP, why does a memory access take about the same time no matter which processor makes it?

- A. All processors reach one shared main memory over the same interconnection, so none sits closer to memory than the others.
- B. Each processor has its own private main memory of the same size.
- C. The OS copies all of main memory into every processor's cache.
- D. Only one processor runs at a time, so they never compete for memory.

Answer: A. All processors reach one shared main memory over the same interconnection, so none sits closer to memory than the others.

The processors share a single memory through a bus or other interconnection scheme, so every processor has roughly the same path to it and the same access time.

6. Match each part of a multicore chip with its description.

Core	A complete processor: control unit, ALU, registers, pipeline and L1 caches
L1 cache	The smallest, fastest cache, private to one core
Shared L3 cache	A large on-chip cache used by all the cores
Die	The single piece of silicon that holds all the cores
Integrated memory controller	On-chip circuitry that drives the main-memory modules

Each core is a full processor with private L1 (and often L2) caches; a larger L3 on the same die is shared; the memory controller built onto the chip talks to main memory.

7. Processors P1 and P2 each hold a copy of $X = 5$ in their private caches. P1 changes X to 6, and the new value is also written to main memory. There is **no** cache coherence mechanism, and P2's old copy is still in its cache. What does P2 get when it next reads X ?

- A. 6, because main memory now holds 6
- B. 5, the stale copy in its own cache
- C. An error, because two different copies of X exist
- D. Nothing: P2 must wait until P1 releases X

Answer: B. 5, the stale copy in its own cache

P2's read hits in its own cache and returns the old value 5. Keeping every cached copy consistent is the cache coherence problem, and hardware usually solves it.

8. Put these events in order for a machine that keeps caches coherent by invalidation.

Correct order:

1. P1 and P2 both read $X = 5$ into their private caches
2. P1 executes a write of $X = 6$
3. That write sends an "invalidate X " message over the shared bus
4. P2's snooping cache controller marks its copy of X invalid
5. P2 reads X , misses, and fetches the new value 6

Both caches load X ; P1's write puts an invalidate on the bus; P2's snooping controller sees it and invalidates its copy; P2's next read misses and gets the fresh value.

9. In an SMP, only one designated processor may run operating-system code; the rest run application programs.

Answer: False

That would be an asymmetric (master/slave) design. In an SMP every processor can perform the same functions, including running the kernel. That is what "symmetric" means.

10. In the simple power model (power grows with the cube of the clock speed, work in proportion to it), a chip with one 4 GHz core is replaced by a chip with eight 2 GHz cores; both use 64 units of power. An old program that is one single stream of instructions, never split into parts, runs on the new chip. What happens to its speed?

- A. It runs about half as fast: it can use only one core, and that core's clock is half as fast.
- B. It runs 4 times as fast, because the new chip does 16 units of work instead of 4.
- C. It runs at the same speed, because the OS automatically splits every program across the cores.
- D. It cannot run at all until it is rewritten for a multicore chip.

Answer: A. It runs about half as fast: it can use only one core, and that core's clock is half as fast.

One stream of instructions runs on one core at a time, and in this model a 2 GHz core does half the work per second of a 4 GHz core. The chip's extra capacity (16 units of work instead of 4) pays off only when the work is split into pieces the OS can spread over the cores (see 4.3).

11. In a simple power model, a core's power grows with the **cube** of its clock speed, while its work rate grows in proportion to its clock speed. One core running at 4 GHz uses 64 units of power. How many units of power does a chip with **4 cores at 2 GHz** use?

Answer: 32 units

$64 = 4^3$, so in this model 1 GHz costs 1 unit. Each 2 GHz core uses $2^3 = 8$ units, and 4 cores use $4 \times 8 = 32$: half the power of the single 4 GHz core, yet they do $4 \times 2 = 8$ units of work, twice as much, provided the software keeps all four cores busy.

12. An SMP's OS spreads 12 independent jobs, each needing 4 seconds of processor time, over 3 identical processors. If every processor stays busy, how many seconds until all the jobs are done?

Answer: 16 s

Total work = $12 \times 4 = 48$ seconds. Shared by 3 processors, that is $48 \div 3 = 16$ seconds (each processor runs 4 jobs). One processor alone would need 48 seconds.

Chapter 2 · Operating System Overview

What an operating system is for, where it came from, and how real ones are built.

Every operating system (OS) chases three goals: make the computer convenient to use, use its hardware efficiently, and stay able to evolve. This chapter shows how those goals drove seventy years of history, from machines with no OS at all to batch, multiprogrammed and time-sharing systems (each is explained in 2.2). It then covers the big ideas inside every modern OS, how systems survive faults and use many cores, and tours Windows, UNIX, Linux and Android.

Learning objectives

- Summarize the objectives and functions of an operating system.
- Trace the evolution from serial processing to batch, multiprogrammed and time-sharing systems.
- Explain the major achievements: processes, memory management, protection and security, and scheduling.
- Describe the developments that led to modern operating systems and how systems tolerate faults.
- Discuss OS design issues for multiprocessor and multicore computers.
- Compare the structure of Windows, traditional and modern UNIX, Linux and Android.

How Chapter 2 fits together

Chapter 1 described the hardware. Chapter 2 asks what software must sit on top of it, why it looks the way it does, and how real systems are organised.

What an OS is for (2.1)

An OS has three objectives: **convenience**, **efficiency** and the **ability to evolve**. As an **interface** it offers services (program development and execution, I/O and file access, system access, error handling, accounting) through agreed boundaries: the API for source code, the ABI for compiled code and the ISA between software and hardware. As a **resource manager** it hands out processor time, memory, devices and files, even though it is itself a program that must give up the processor and win it back through interrupts.

How we got here (2.2)

Each era fixed the waste of the one before. **Serial processing** lost time to manual setup and booking. **Simple batch** systems added a resident monitor, plus the hardware it needed (memory protection, a timer, privileged instructions, user and kernel mode). **Multiprogramming** kept the processor busy during I/O waits, and **time sharing** gave interactive users quick responses.

The big ideas (2.3, 2.4)

Section 2.3 names the achievements that made this work: the **process**, **memory management** (including virtual memory), **information protection and security**, and **scheduling and resource management**. Section 2.4 adds the developments behind modern systems: microkernels, multithreading, symmetric multiprocessing, distributed systems and object-oriented design.

Staying up and scaling out (2.5, 2.6)

Fault tolerance (2.5) measures reliability and availability, classifies faults as permanent, transient or intermittent, and answers them with redundancy and OS tools such as process isolation and checkpoints. Section 2.6 lists the extra problems when several processors or cores share one OS: running kernel code on many processors at once, scheduling, synchronization, memory management and reliability.

Real systems (2.7 to 2.11)

The chapter ends with case studies: Windows, traditional UNIX, modern UNIX variants, Linux and Android. Look for the same ideas in each: a kernel, user and kernel mode, processes and threads, and a modular structure designed to evolve.

Why it matters next

The process, introduced here as the answer to multiprogramming and time sharing, is the subject of Chapter 3.

2.1 Operating System Objectives and Functions

You should be able to

- Define an operating system and explain its three objectives: convenience, efficiency and the ability to evolve.
- Describe the layered view of a computer system and name the seven kinds of service an OS provides.
- Tell the ISA, ABI and API apart, and use them to predict whether a program will run on another machine as a binary or after recompiling.
- Explain how the OS manages resources even though it is itself a program that must give up the processor and win it back.
- Explain why an OS must keep evolving and why that calls for modular construction with clear interfaces and good documentation.

What an operating system is for

An **operating system (OS)** is a program that **controls the execution of application programs** and acts as the **interface** between those programs and the computer hardware. It pursues three objectives:

- **Convenience:** make the computer easy to use (files and windows instead of disk blocks and device commands).
- **Efficiency:** use the hardware well. Example: a program that computes for 2 ms, then waits 8 ms for the disk, leaves a processor that runs only it idle $8 \div 10 = 80\%$ of the time. The OS fills those waits by running another program.
- **Ability to evolve:** let new hardware, new services and fixes be added without getting in the way of existing services.

The goals can conflict: a rich graphical interface (convenience) costs processor time and memory (efficiency).

The OS as a user/computer interface

Layers, top to bottom: **end user** → **application programs** → **utilities (and libraries)** → **operating system** → **computer hardware**. Each layer uses the one below through an agreed interface and hides it from the layers above.

- **End user:** sees only applications and never needs to know how the hardware works.
- **Application programmer:** writes applications using the utilities (editors, compilers, debuggers, libraries) and the OS's services.
- **OS designer:** must master the hardware below and design services for the programs above.

Utilities are system programs shipped with the OS. They are not the kernel: they ask the OS for services like any application. A **library** holds ready-made routines (such as `fopen` or `fwrite`) that often make system calls for the program.

Saving a document: Ctrl+S → application → library call → **system call** (the processor enters kernel mode, inside the OS) → the OS's **device driver** sends the drive its commands → the drive raises an **interrupt** when done → the result travels back up.

The seven services an OS provides

1. **Program development:** editors, compilers and debuggers. They come with the OS but are, strictly, utilities.
2. **Program execution:** loading instructions and data into memory, preparing files and I/O devices, then starting the program.
3. **Access to I/O devices:** each device has its own commands and control signals; the OS offers one uniform interface (read, write).
4. **Controlled access to files:** the OS understands the storage device and how data is laid out on it, so programs use file names; on shared systems it enforces who may read or change each file.
5. **System access:** on shared systems, controlling who may use the system at all and which resources each user may reach, and resolving conflicts over resources.
6. **Error detection and response:** *hardware errors* (a memory error, a device that fails or malfunctions) and *software errors* (division by zero, an attempt to reach a forbidden memory location, a request the OS cannot grant). The OS responds with the least damage it can: end the program, retry the operation, or simply report the error.
7. **Accounting:** usage statistics for each resource and performance monitoring (such as response time), for tuning, planning upgrades and, on multiuser systems, billing.

Three key interfaces

Interface	Made of	What it gives you
ISA (instruction set architecture)	The user ISA (arithmetic, loads, jumps; any program) plus the system ISA (privileged: interrupts, memory management, I/O; OS only)	The boundary between hardware and software. Machine code runs only on processors with the same ISA.
ABI (application binary interface)	The system call interface to the OS plus the user ISA (and conventions such as which registers carry arguments)	Binary portability : the same executable runs unchanged on any system with the same ABI.
API (application programming interface)	High-level-language library calls plus the user ISA	Source portability : recompile the source on any system that offers the same API.

Most programs reach system calls through library routines. If an ordinary program tries a system-ISA instruction, the processor refuses and hands control to the OS.

Portability rules, with a worked example

- A **binary** runs unchanged only if the target has the **same ISA and the same ABI**.
- **Source code** runs after recompiling if the target offers the **same API** (the target's compiler makes new code for its ISA and ABI).

Built on x86-64 + Linux, moved to...	x86-64 Linux	x86-64 Windows	ARM64 Linux	ARM64 Windows
Compiled binary	runs	no: ABI differs	no: ISA differs	no: both differ
Source, standard C library only	runs	runs after recompiling	runs after recompiling	runs after recompiling
Source that also calls <code>fork()</code>	runs	no: API lacks <code>fork()</code>	runs after recompiling	no: API lacks <code>fork()</code>

`fork()` is a standard call on UNIX-like systems (Linux, macOS) that makes a copy of the running program as a new process; section 3.6 covers it in full. It is part of **POSIX**, the API those systems share. Windows has no `fork()`.

The OS as resource manager

The OS manages **processor time** (who runs next, and for how long), **main memory** (who gets which part, together with memory-management hardware), **I/O devices** (when a program may use one) and **files** (where data lives, who may use it).

It is an **unusual controller**. It is ordinary software executed by the very processor it manages, and it must **give up** the processor so programs can run. While a user program runs, the OS is not running at all; it relies on the hardware to get control back, through a **system call** or an **interrupt** (from a device, the **timer**, or a program error such as dividing by zero). Without a timer, a program stuck in a loop that makes no system call would keep the processor forever.

The **kernel** (or **nucleus**) is the part of the OS that stays in main memory all the time and holds its most frequently used functions; other parts are loaded when needed.

Trace: the OS gives A and B memory, arms the timer, **dispatches** A → A's system call to read a file (OS back) → the OS starts the disk, dispatches B → disk interrupt (OS back; A ready) → B resumes → timer interrupt (OS back) → the OS dispatches A.

Ease of evolution

An OS keeps changing because of **hardware upgrades and new types of hardware** (paging hardware, which hands out memory in fixed-size pages; graphics displays replacing text terminals), **new services** requested by users, and **fixes**: every OS has faults, and a fix can itself introduce new faults.

So an OS should be built from **modules** with **clearly defined interfaces** and **good documentation**, so a change stays inside the module it concerns. In a tangled design, where parts reach into each other's data, a change ripples into many parts, and every extra edit is another chance to add a fault (in this section's illustrative model: 13 edits and a new fault, versus 6 edits and none).

Key terms

Operating system (OS)

A program that controls the execution of application programs and acts as the go-between for those programs and the computer hardware.

Convenience

The first objective of an OS: make the computer easier to use by hiding hardware details behind simple services.

Efficiency

The second objective of an OS: keep the processor, memory and devices doing useful work instead of sitting idle or being wasted.

Ability to evolve

The third objective of an OS: be built so that new hardware, new services and fixes can be added without getting in the way of the services it already provides.

Utilities

System programs that come with the OS, such as editors, compilers, debuggers, command shells and file tools. They help people build programs and run the computer, but they are not the kernel itself.

Library

A collection of ready-made routines (for example the C standard library, with `fopen` and `printf`) that programs call instead of writing that code themselves. Many library routines make system calls on the program's behalf.

Instruction set architecture (ISA)

The complete set of machine instructions a processor can carry out. It is the boundary between hardware and software.

User ISA

The part of the instruction set that any program may use: arithmetic, comparisons, jumps, loads and stores.

System ISA

The privileged part of the instruction set, such as instructions that control interrupts, memory-management hardware or I/O. Only the OS may use it.

Application binary interface (ABI)

The rules a compiled (binary) program follows to use a system: the system call interface to the OS plus the user ISA, including details such as how arguments are passed. Systems with the same ABI run the same binary unchanged.

Application programming interface (API)

The set of calls a program's source code may use: library routines and OS services, on top of the user ISA. Source code written to an API moves to another system with the same API by recompiling it.

System call

A request from a running program to the OS for a service, such as reading a file. A special instruction switches the processor into the OS to handle it.

Kernel (nucleus)

The part of the OS that stays in main memory all the time and holds its most frequently used functions.

Resource manager

The view of the OS as the part of the system that decides how the processor, main memory, I/O devices and files are shared among programs.

Dispatch

To hand the processor to a chosen program by loading its saved state and jumping to its next instruction.

Device driver

The part of the OS that knows the exact commands for one kind of device and turns general requests such as read or write into those commands.

Accounting

The OS service that records how much of each resource is used, for tuning performance, planning upgrades and, on shared systems, billing users.

Portability

How easily software moves to a different machine or OS. Binary portability: the compiled program runs unchanged. Source portability: the source code only needs recompiling.

Module

A self-contained part of a program or OS that hides its insides and is used only through a clearly defined, documented interface.

Review questions with answers

1. Which statement best describes an operating system?

- A. The complete collection of software installed on a computer, including every application program
- B. A program that controls the execution of application programs and acts as the interface between them and the hardware
- C. The circuitry inside the processor that fetches, decodes and executes each machine instruction
- D. A translator that converts programs written in a high-level language into machine code the processor can run

Answer: B. A program that controls the execution of application programs and acts as the interface between them and the hardware

An OS is software with two jobs: it controls how programs run, and it stands between those programs and the hardware, offering services so that programs never have to drive the hardware themselves.

2. Which are the three standard objectives of an operating system? Select all that apply.

- A. Efficiency
- B. Keeping the hardware design secret from programmers
- C. Ability to evolve
- D. Convenience
- E. Running only one program at a time, so programs never compete

Answer: A, C, D

An OS aims to be convenient to use, to use the hardware efficiently, and to be able to evolve. It hides hardware details to make programming convenient, not to keep them secret, and running one program at a time would leave the processor idle during every wait, the opposite of efficiency.

3. A program works in a repeating pattern: it computes for 2 ms, then waits 8 ms for the disk to deliver its next block of data. If the OS runs no other program, what percentage of the time does the processor sit idle?

Answer: 80 %

One cycle lasts $2 + 8 = 10$ ms, and the processor has nothing to do for 8 of them: $8 \div 10 = 80\%$ idle. Handing the processor to another program during those waits is how an OS meets its efficiency objective.

4. Match each everyday event to the OS service at work.

You type a password before you reach the desktop	System access
A program divides by zero and is stopped with an error message	Error detection and response
The same print request works on any printer model	Access to I/O devices
A monthly report shows how much processor time each user consumed	Accounting
Double-clicking an icon loads a program into memory and starts it	Program execution

Logging in controls who may use the system; a division by zero is a software error the OS must handle; a uniform interface hides device details; usage records are accounting; loading and starting a program is program execution.

5. Sort each error the OS must detect and respond to into the right kind.

Hardware error	Software error
A memory chip returns a corrupted value A disk sector can no longer be read A device stops responding	A program divides by zero A program tries to read a forbidden memory location A program requests something the OS cannot grant

Hardware errors come from memory or devices that fail or malfunction. Software errors come from what a program tries to do. In both cases the OS responds with the least damage it can: ending the program, retrying the operation, or reporting the error.

6. Which interface marks the boundary between hardware and software?

- A. The application binary interface (ABI)
- B. The application programming interface (API)
- C. The system call interface
- D. The instruction set architecture (ISA)

Answer: D. The instruction set architecture (ISA)

The ISA is the set of machine instructions the processor carries out. Below it is circuitry, above it is software. Its user part is open to every program; its system part is reserved for the OS.

7. A game was compiled for an x86-64 PC running Linux. On which machine will the **same executable file** run without any changes?

- A. An x86-64 PC running Windows
- B. An ARM64 laptop running Linux
- C. Another x86-64 PC running Linux
- D. Any machine at all, as long as the game's source code used only a standard API

Answer: C. Another x86-64 PC running Linux

A binary runs unchanged only where the ABI matches, which means the same system call interface and the same user ISA.

8. Source code written only to a standard API can be moved to another system that offers the same API by recompiling it, even if the new system has a different processor (ISA).

Answer: True

The compiler for the new system produces fresh machine code for its ISA that follows its ABI. For source code, the interface that must match is the API.

9. Why is the operating system an unusual kind of control mechanism?

- A. It is a program run by the same processor it manages, so it must give the processor away and depend on the hardware to get it back
- B. It runs on a separate processor of its own that watches the main processor and every program continuously
- C. It is permanently wired into the hardware circuits, so it can never be changed or updated after it ships
- D. It never gives up control of the processor, so user programs can only run inside the OS itself

Answer: A. It is a program run by the same processor it manages, so it must give the processor away and depend on the hardware to get it back

A thermostat sits outside the furnace it controls; the OS does not. While a user program runs, the OS is not running at all. It regains control only when the hardware hands it back: a system call, or an interrupt from a device, the timer or a program error.

10. Which statements about the OS as a resource manager are true? Select all that apply.

- A. It runs on a separate processor of its own, so it can watch user programs continuously
- B. It decides how processor time, main memory, I/O devices and files are shared among programs
- C. The whole OS, including every utility, stays in main memory at all times
- D. Its kernel (nucleus), which holds the most frequently used functions, stays in main memory
- E. It decides which machine instructions the processor is able to understand

Answer: B, D

The OS allocates processor time, memory, devices and files. Only the kernel (nucleus) stays resident; other parts of the OS and the utilities are loaded when needed. The OS runs on the same processor it manages, and the instruction set is fixed by the hardware: the OS can only use it.

11. Put these moments in order for an OS running a program that reads a file.

Correct order:

1. The OS gives the program memory and arms the timer
2. The OS dispatches the program
3. The program makes a system call to read a file
4. The OS starts the disk and runs another program meanwhile
5. The disk's interrupt hands control back to the OS

The OS sets things up and hands over the processor, gets it back through the system call, keeps the processor busy while the disk works, and regains control when the disk interrupts.

12. Engineers must add support for a new kind of graphics display. Which OS design makes that change easiest and least risky?

- A. One where every part reads and changes every other part's data directly, for speed
- B. One kept small and simple by skipping documentation of its internal interfaces
- C. One built from modules with clearly defined, documented interfaces
- D. One that is frozen and never updated after it ships, so nothing can break

Answer: C. One built from modules with clearly defined, documented interfaces

Hardware upgrades, new services and fixes keep arriving, and fixes can bring new faults. Modules behind clear, documented interfaces keep each change inside the part it concerns.

2.2 The Evolution of Operating Systems

You should be able to

- Explain why operating systems keep evolving: hardware upgrades, new services and fixes.
- Describe serial processing and its two big problems, scheduling and setup time.
- Explain how a simple batch system works: the resident monitor, job control language, the hardware it needs, and user versus kernel mode.
- Calculate processor utilization and show how multiprogramming raises it compared with uniprogramming.
- Contrast batch multiprogramming with time sharing, and describe how CTSS shared one processor among many users.

Why operating systems evolve

An OS is never finished. It changes because of **hardware upgrades and new kinds of hardware** (multicore chips, new devices), **new services** that users ask for, and **fixes** for faults (which can bring new faults). Its early history is a chain in which each era removed the previous era's biggest waste:

Era	Main idea	Problem left open
Serial processing (late 1940s to mid-1950s)	No OS; programmers run the machine by hand	Wasted booked time, long setup
Simple batch (mid-1950s)	A resident monitor runs jobs back to back	Processor idle during each job's I/O
Multiprogrammed batch (1960s)	Several jobs in memory; switch when one waits	No interaction; long waits
Time sharing (1960s on; CTSS 1961)	Short time slices for many terminal users	Protection, file security, contention

Serial processing

No operating system. The programmer worked the hardware directly: a **console** of display lights and toggle switches, an **input device** such as a card reader (programs in machine code on cards), and a **printer**. After an error, the lights showed the problem. Two problems:

- **Scheduling:** time was booked on a paper sign-up sheet in fixed blocks, so a job either left part of its block idle or was cut off before finishing.
- **Setup time:** loading the compiler and source program, saving the compiled program, then loading and linking it, often with tapes or card decks to mount. An error meant starting over.

Simple batch systems

General Motors developed the first batch monitor for its IBM 701 in the mid-1950s; its successor, GM-NAA I/O (1956), ran on the IBM 704. Users hand jobs to an **operator**, who groups them into a **batch**; a **monitor** program runs them one after another. The **resident monitor** stays in memory (interrupt processing, device

drivers, job sequencing, control language interpreter); the rest is the user program area. The monitor reads a job, branches to it, and gets control back when the job ends or fails, so the processor **alternates between the monitor and a user program** with no human gap.

Job control language (JCL) cards, in reading order: **\$JOB** (start; account to charge) → **\$FTN** (load the FORTRAN compiler) → source cards (compiled to object code) → **\$LOAD** (load the object program) → **\$RUN** (jump to it) → data cards (read through the monitor's input routine) → **\$END**. Out of order (say, \$RUN before \$LOAD) the monitor aborts the job and skips to the next \$JOB.

Hardware a monitor needs:

- **Memory protection:** a program may not alter the monitor's memory; an attempt traps to the monitor, which aborts the job.
- **Timer:** set for each job; when it expires, control returns to the monitor, so no job hogs the machine.
- **Privileged instructions:** I/O and similar instructions run only in the monitor, so a program cannot read the next job's control cards.
- **Interrupts:** absent on early machines; the timer and I/O devices use them to hand control back to the monitor, so the OS can give up the processor and win it back flexibly.

User mode: user programs run with protected memory and privileged instructions off-limits. **Kernel mode** (system or control mode): the monitor may execute privileged instructions and access protected memory. A trap or interrupt switches the processor to kernel mode; the monitor switches back when it resumes a job. **Costs:** the monitor uses memory and processor time, yet utilization is far better than with serial processing.

Multiprogrammed batch systems

I/O is slow next to the processor, so even with automatic sequencing it sits idle. **Processor utilization** = busy time / total time. Example: read a record 15 μ s, execute 100 instructions 1 μ s, write a record 15 μ s: utilization = 1 / 31 $\approx$ **3.2%**.

Uniprogramming: one program in memory; the processor waits for every I/O. **Multiprogramming** (multitasking): several programs in memory; when one waits for I/O, the processor runs another. If each program has its own device, utilization with n programs = $n \times \text{compute} / (\text{read} + \text{compute} + \text{write})$, at most 100%. For 15/1/15: 3.2%, 6.5% and 9.7% with 1, 2 and 3 programs. For 20/10/20 with three programs: 30 / 50 = 60%. With one processor the programs still compute one at a time; what overlaps is one program's computing with the others' I/O. The OS needs **memory management** and **scheduling**; the hardware needs **I/O interrupts** and **DMA** (direct memory access) so devices work on their own and signal when done.

Three-job example. 250 MB of user memory, a disk, a terminal, a printer. JOB1: heavy compute, 5 min, 50 MB. JOB2: heavy I/O on the terminal, 15 min, 100 MB. JOB3: heavy I/O on disk and printer, 10 min, 75 MB. About 6 processor-minutes in all; the jobs barely interfere. Each utilization is the average share in use over the run: processor 6 / 30 = 20% versus 6 / 15 = 40%.

Measure	Uniprogramming	Multiprogramming
Processor use	20%	40%
Memory, disk, printer use	33% each	67% each
Elapsed time	30 min	15 min
Throughput	6 jobs/hr	12 jobs/hr
Mean response time	18 min	10 min

One at a time, the jobs finish at 5, 20 and 30 min: mean $(5 + 20 + 30) / 3 \approx 18$ min. Together they finish at 5, 15 and 10: mean 10 min. Throughput: 3 jobs in 0.5 h = 6/h versus 3 in 0.25 h = 12/h.

Time-sharing systems

The processor's time is shared among many interactive users at terminals, each getting a short **time slice** in turn. With n users each sees about $1/n$ of the machine, yet responses feel quick because people are slow next to a computer.

	Batch multiprogramming	Time sharing
Principal objective	Maximize processor use	Minimize response time
Directives to the OS	JCL commands provided with the job	Commands entered at the terminal

Example: a 20 s job, four users needing 0.3 s each and your 0.1 s command arrive together. Run to completion, you wait 21.3 s. With 0.2 s slices you wait 1.1 s, and the long job ends at 21.3 s instead of 20 s: same work, better order. More users means longer rounds.

CTSS (MIT, 1961; IBM 709, later 7094)

- 32,000 words of 36 bits; the monitor used 5,000, leaving 27,000. Every user program was loaded at word 5,000.
- A clock interrupt about every 0.2 s let the OS regain control and switch users.
- **Swapping:** the old user's program and data went to disk before the next was read in, and came back at its next turn. To cut disk traffic, only words the incoming program would overwrite were written out.

Example: JOB1 10,000, JOB2 24,000, JOB3 4,000, JOB4 14,000 words, turns 1, 2, 3, 1, 4, 2. Small JOB3 displaces only 4,000 words of JOB2, and when JOB2 returns only its 14,000 missing words are read. Total: 128,000 words moved versus 148,000 for whole-program swapping.

New problems: protection among users, file system protection (authorized access only), and contention for resources such as printers and storage.

Key terms

Job

One unit of work handed to a computer: a program, its data, and the instructions for how to run it.

Serial processing

The earliest way of using a computer (late 1940s to mid-1950s): no operating system at all, and programmers took turns operating the hardware directly, one job after another.

Setup time

Time spent getting a job ready to run (loading the compiler, the source program, linking and loading the result) during which the machine produces nothing useful.

Batch system

A system in which users hand their jobs to an operator, who groups them into a batch; a monitor program then runs the jobs one after another with no human in between.

Resident monitor

The controlling program of a simple batch system. It stays in main memory at all times, reads each job, starts it, and takes control back when the job ends.

Job control language (JCL)

A small command language whose statements, such as \$JOB, \$FTN, \$LOAD, \$RUN and \$END, tell the monitor what to do with a job.

Memory protection

Hardware that stops a running program from changing memory it does not own, such as the area that holds the monitor.

Timer

A hardware countdown the OS sets before letting a program run. When it reaches zero it interrupts the program so the OS gets the processor back.

Privileged instruction

A machine instruction (for example, one that starts I/O) that the hardware lets only the OS execute. If a user program tries it, the hardware stops it and hands control to the OS.

User mode

The restricted processor mode in which ordinary programs run: protected memory and privileged instructions are off-limits.

Kernel mode

The unrestricted processor mode in which the OS runs: it may execute privileged instructions and touch protected memory. Also called system mode or control mode.

Uniprogramming

Running just one user program at a time. The processor sits idle whenever that program waits for I/O.

Multiprogramming

Keeping several programs in memory at once so that when one must wait for I/O the processor can run another. Also called multitasking.

Processor utilization

The fraction of time the processor is busy doing useful work: busy time divided by total time.

Throughput

How much work a system finishes per unit of time, for example jobs completed per hour.

Response time

The time from submitting a request or job until its result comes back.

Time sharing

Sharing one processor among many interactive users at terminals by giving each a short turn in rotation, so every user gets quick answers.

Time slice

The short stretch of processor time a program gets before the OS may switch to another program. Also called a quantum.

Swapping

Copying a program's memory image out to disk to make room for another program, and later copying it back so it can continue where it left off.

Compatible Time-Sharing System (CTSS)

One of the first time-sharing operating systems, built at MIT in 1961 for the IBM 709 and later the IBM 7094.

Review questions with answers

1. In the serial-processing era (late 1940s to mid-1950s), how did a programmer get time on the computer?

- A. By reserving a block of time on a paper sign-up sheet
- B. The operating system's scheduler gave each job a time slice
- C. An operator grouped jobs into a batch on tape
- D. By typing commands at a terminal whenever they liked

Answer: A. By reserving a block of time on a paper sign-up sheet

With no OS, programmers booked the machine by hand. A booked block was either partly wasted (finish early) or too short (stopped before finishing).

2. In a simple batch system, put a FORTRAN job's cards in the order the resident monitor reads them (first card first).

Correct order:

- 1. \$JOB
- 2. \$FTN
- 3. FORTRAN source program
- 4. \$LOAD
- 5. \$RUN
- 6. Data for the program
- 7. \$END

\$JOB starts the job; \$FTN loads the compiler, which reads the source cards right after it; \$LOAD loads the compiled program; \$RUN starts it, and it reads the data cards; \$END closes the job.

3. Match each hardware feature to the trouble it prevents in a batch system.

Memory protection	A buggy job overwriting the monitor
Timer	One job keeping the processor forever
Privileged instructions	A job reading the card reader directly and swallowing the next job's cards
Interrupts	The OS having no way to regain control when a device finishes

Protection guards the monitor's memory, the timer bounds each job's time, privileged instructions keep I/O in the monitor's hands, and interrupts let events hand control back to the OS.

4. In kernel mode the processor refuses privileged instructions, so the monitor has to ask user programs to perform I/O for it.

Answer: False

It is the other way round. User mode refuses privileged instructions and protects some memory; the monitor runs in kernel mode, where it may execute privileged instructions and touch protected memory.

5. A program reads a record in 15 μ s, executes 100 instructions on it in 1 μ s, and writes the result in 15 μ s. Running alone (uniprogramming), what percentage of the time is the processor busy?

Answer: 3.2 %

The processor is busy 1 μ s out of every $15 + 1 + 15 = 31 \mu$ s: $1 / 31 \approx 0.032$, or 3.2%. It idles almost 97% of the time.

6. Each program repeats: read a record (20 μ s), compute (10 μ s), write a record (20 μ s). Three such programs are multiprogrammed, each with its own I/O device. What is the processor utilization, in percent?

Answer: 60 %

Each program needs 10 μ s of processor per 50 μ s cycle. Three need 30 μ s of every 50 μ s: $30 / 50 = 60\%$. (It could never go past 100%.)

7. Three jobs arrive together and run one after another (uniprogramming), taking 5, then 15, then 10 minutes. What is their mean response time, in minutes?

Answer: 18.33 min

They finish at 5, 20 and 30 minutes, so the mean is $(5 + 20 + 30) / 3 = 55 / 3 \approx 18.3$ min. Run together under multiprogramming they would finish at 5, 15 and 10: a mean of 10 min.

8. Three jobs take 30 minutes in total under uniprogramming but only 15 minutes under multiprogramming. What happens to throughput?

- A. It doubles, from 6 to 12 jobs per hour
- B. It halves, from 12 to 6 jobs per hour
- C. It stays the same, because the same three jobs ran
- D. It rises from 3 to 6 jobs per hour

Answer: A. It doubles, from 6 to 12 jobs per hour

Throughput = jobs finished per unit of time: 3 jobs in 0.5 h = 6 jobs/h, and 3 jobs in 0.25 h = 12 jobs/h.

9. Does each statement describe batch multiprogramming or time sharing?

Batch multiprogramming	Time sharing
Main goal: keep the processor as busy as possible Instructions come from job control language cards sent with the job Users hand in a job and collect printed output later	Main goal: answer each user quickly Instructions are commands typed at a terminal A clock interrupt takes the processor back every fraction of a second

Batch multiprogramming maximizes processor use and is steered by JCL; time sharing minimizes response time, is steered by commands typed at terminals, and slices time with a clock.

10. Which problems did time sharing bring to the fore? Select all that apply.

- A. Protecting users' programs from one another
- B. Protecting files so only authorized users can open them
- C. Settling contention for shared devices such as printers and disks
- D. Booking machine time on paper sign-up sheets
- E. Loading the compiler by hand for every job

Answer: A, B, C

With many users on one machine at once, the OS must keep them apart, guard their files, and referee shared resources. Sign-up sheets and hand-loaded compilers belong to the serial era.

11. CTSS ran on a machine with 32,000 words of memory, and its resident monitor used 5,000 of them. How many words were left for a user program?

Answer: 27000 words

$32,000 - 5,000 = 27,000$ words, and every user program was loaded starting at word 5,000.

12. When CTSS switched from one user to the next, why did it sometimes write only part of the old program to disk?

- A. To cut disk traffic: only the words the incoming program would overwrite had to be saved
- B. Because the disk was too small to hold whole programs
- C. Because the 0.2-second clock left too little time to copy everything
- D. To give the old program more memory when it resumed

Answer: A. To cut disk traffic: only the words the incoming program would overwrite had to be saved

Words the newcomer did not overwrite were still intact in memory, so saving them and later reading them back would have been wasted disk work.

2.3 Major Achievements

You should be able to

- Name the four major achievements in OS design and the problem each one solved.
- Define a process in four equivalent ways, list its three parts, and explain how the OS uses a process list, base and limit registers to switch between processes safely.
- Explain why multiprogramming, time sharing and real-time transaction systems made the process concept necessary, and recognise the four kinds of errors that appear without it.
- List the five storage-management responsibilities of an OS and translate a virtual address (page number + offset) into a real address or a page fault.
- Classify security problems by the goal they violate (availability, confidentiality, data integrity, authenticity) and describe how the OS uses queues and interrupts to schedule processes fairly and efficiently.

The four major achievements

Serving many programs and users at once forced four ideas every modern OS rests on:

- **The process:** programs take turns on one processor without losing their place.
- **Memory management:** each program gets its own memory, safe from others.
- **Information protection and security:** everyone reaches only what they are allowed to.
- **Scheduling and resource management:** the OS decides who runs next, for how long, and who gets each device.

What a process is

Four definitions, one idea seen from different angles:

1. **A program in execution.** A program is a passive file of instructions; it becomes a process once it is loaded and running.
2. **An instance of a program running on a computer.** One program can run as several processes, each with its own data and position.
3. **The entity that can be assigned to and executed on a processor.** The scheduler's view: the thing it hands the processor to.
4. **A unit of activity characterized by a single sequential thread of execution, a current state and an associated set of system resources.** Thread = the path the PC walks through the code; state = the context; resources = memory, files, devices.

Why the process concept was needed

Three developments all had to switch one processor among many activities, at moments chosen by interrupts rather than by the programs:

- **Multiprogrammed batch operation:** keep the processor and devices busy by switching jobs when one waits for I/O.
- **Time sharing:** answer many interactive users quickly by giving each a short turn in rotation.

- **Real-time transaction systems:** many users query and update one shared database (e.g. airline seats) and expect answers in seconds.

Case-by-case switching code, with no clean model of a half-finished program, produced timing bugs in four families:

Error	What goes wrong	Example
Improper synchronization	A program waits for a signal from another activity, and the signal is lost, duplicated or not waited for.	A lost "disk done" signal leaves a program asleep forever.
Failed mutual exclusion	Two programs use a shared resource at the same time when only one should.	Two clerks both read "seat 14C free" and both sell it.
Nondeterminate program operation	Results depend on how programs happen to interleave, not only on the input.	Same input, different totals on different days.
Deadlock	Programs each hold something and wait for what another holds, so none can continue.	A holds the disk, wants the printer; B holds the printer, wants the disk.

The parts of a process and how the OS stores them

A process has three parts: an **executable program**, the **associated data** it works on, and its **execution context** (or process state): what the OS needs to pause it and resume it exactly where it stopped, such as register values, the program counter, priority and whether it awaits I/O.

A typical implementation:

- The OS keeps a **process list** with one entry per process. The entry points to the process's memory block and holds part or all of its saved context (the rest can live with the process).
- Each process owns a block of memory holding its program and data.
- Processor registers describe the running process: the **process index** (which list entry is running), the **program counter** (next instruction, counted from the start of the block), the **base register** (start address of the block) and the **limit register** (size of the block).
- Each address must be less than the limit, and is added to the base. With base 2000 and limit 600, addresses 0–599 are legal: 3 becomes 2003, while 650 is refused and the OS is interrupted.

Switching from A to B: an interrupt gives the OS control; it saves A's context in A's entry, chooses B, and loads B's context (including base and limit) into the registers. B resumes where it stopped; no program or data is copied, only register values.

Memory management: five responsibilities

1. **Process isolation:** stop processes from reading or changing each other's instructions and data.
2. **Automatic allocation and management:** hand out and reclaim memory as needed; programmers never pick physical locations.
3. **Support of modular programming:** modules can be created, removed or resized as the program runs.
4. **Protection and access control:** allow useful sharing (one library copy for all) while controlling who may read, write or run each piece of memory and each file.
5. **Long-term storage:** keep information for long periods, after programs end and the power is off.

Two tools meet these needs. **Virtual memory** lets each program address memory logically, whatever the real memory size or layout (jobs 1 to 4). Many processes can then share memory, and a program can even be larger than main memory. The **file system** keeps information in named files on secondary storage (jobs 4 and 5).

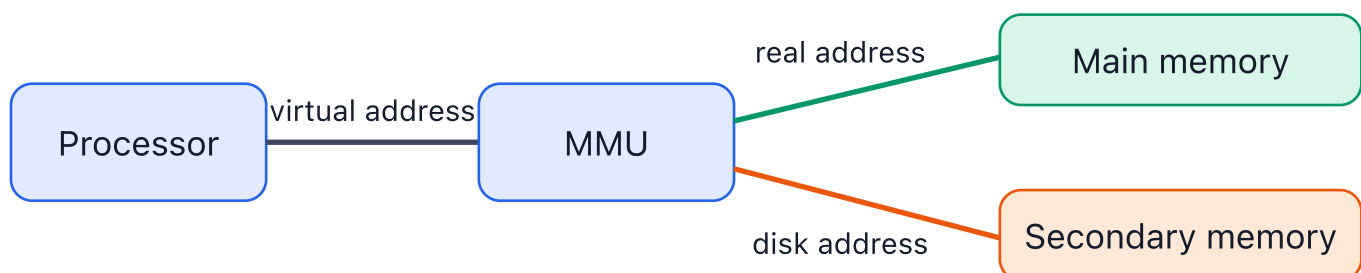
Paging and virtual-memory addressing

Paging divides every process into equal, fixed-size **pages** and main memory into same-size **page frames**. A page can sit in any free frame or stay on disk until needed, so a program's pieces need not be side by side. Each process has a **page table** recording, for every page, the frame that holds it or that it is on disk. A **virtual address** is a page number plus an offset within that page:

```
page   = address ÷ page size (whole part)
offset = address - page × page size
real   = frame × page size + offset
```

Worked example (1,024-byte pages): virtual address 5000 = $4 \times 1024 + 904$, so page 4, offset 904. If the page table says page 4 is in frame 2, the real address is $2 \times 1024 + 904 = \mathbf{2952}$. Since $1,024 = 2^{10}$, the hardware just splits the address: the low 10 bits are the offset, the rest the page number.

The processor issues virtual addresses; the **memory management unit (MMU)** translates each one using the page table. If the page is in main memory, the result is a **real address** in main memory. If the page is on disk, it leads to a disk address instead: the MMU raises a **page fault**; the OS blocks the process, reads the page into a free frame (if none is free, it first evicts a page, e.g. the one in memory longest), updates the table, and the instruction is retried. An address beyond the last page is an error that stops the program.



Information protection and security

The OS must control access to shared programs, data and devices. Four goals:

- **Availability:** the system and its data are usable whenever authorized users need them (broken by a flood of fake requests).
- **Confidentiality:** only those with permission can read the data (broken by a stolen unencrypted laptop).
- **Data integrity:** data is changed only by those allowed to (broken by malware altering records).
- **Authenticity:** users are who they claim and messages come from where they say (broken by a forged email).

Common mistake: *reading* data you should not breaks confidentiality; *changing* it breaks integrity.

Scheduling and resource management

The OS shares the processor, memory and devices among active processes, aiming at three goals:

- **Fairness:** processes competing for the same resource, especially jobs of the same kind, get roughly equal and fair access.
- **Differential responsiveness:** jobs with different needs get different treatment, decided as conditions change; e.g. quickly run a process that holds an I/O device so it frees the device for others.
- **Efficiency:** maximize throughput, minimize response time and, in time sharing, serve as many users as possible.

Key elements of a typical scheduler:

- **Long-term queue:** new jobs waiting to be admitted. Admission gives a job main memory, so the OS admits jobs only when that will not overcommit memory or the processor.
- **Short-term queue:** processes in main memory and ready to run. The short-term scheduler (dispatcher) picks the next one, often by **round-robin**: each gets a short time slice in turn, and a process whose slice ends goes to the back of the line. Another common policy picks by priority.
- **I/O queues:** one per device, holding processes waiting for that device.
- Every event reaches the OS as a **service call** (the running process asks, e.g. to read the disk or exit) or an **interrupt** (timer or device); then the OS picks who runs next.

Key terms

Process

A program in execution: the program's code and data together with everything the OS needs to run it, pause it and later resume it exactly where it stopped.

Execution context

Everything the OS must remember to stop a process and later restart it as if nothing happened: the processor register values (including the program counter), priority, open files, pending I/O and so on. Also called the process state (here "state" means these saved details).

Process list

A table the OS keeps with one entry per process. Each entry points to where the process lives in memory and may hold part or all of its saved execution context (the rest can be kept with the process itself).

Base register

A processor register that holds the starting address of the memory region belonging to the running process. Addresses the process uses are counted from this point.

Limit register

A processor register that holds the size of the running process's memory region. The hardware refuses any address at or beyond this size, so a process cannot reach outside its own region.

Mutual exclusion

Allowing only one process at a time to use a shared resource or run a piece of code that touches shared data.

Deadlock

A situation in which two or more processes each hold something the other one needs and wait for it, so none of them can ever continue.

Virtual memory

A scheme in which each program uses its own logical addresses, as if it had a large private memory, while the OS and hardware map those addresses onto real main memory and onto disk.

Paging

Dividing every process into equal, fixed-size blocks called pages, and main memory into page-sized slots called page frames (frames). Any page can be loaded into any free frame, or kept on disk until it is needed.

Page table

The table the OS keeps for each process that says, for every page, which frame of main memory holds it or that it is currently only on disk. The MMU consults it on every memory access.

Virtual address

An address as a program sees it. With paging it is made of a page number and an offset (the position inside that page).

Real address

An actual location in main memory, found by combining the frame that holds the page with the offset. Also called a physical address.

Memory management unit (MMU)

Hardware between the processor and main memory that translates every virtual address into a real address, and interrupts the OS when the page needed is not in main memory.

Page fault

The interrupt raised when a program uses an address whose page is not in main memory. The OS reads the page in from disk, updates the page table and lets the program retry.

File system

The part of the OS that keeps information for the long term in named files on secondary storage, so it survives after programs end and the power goes off.

Availability (security goal)

A security goal: the system and its data stay usable by authorized users whenever they need them, protected against anything that would interrupt service, such as a flood of fake requests.

Confidentiality

A security goal: only people and programs with permission can read the data.

Data integrity

A security goal: data cannot be changed, added to or deleted except by those allowed to do so.

Authenticity

A security goal: the system can check that users really are who they claim to be, and that messages and data really come from where they say.

Round-robin

A scheduling method that gives each ready process a short turn on the processor in circular order; a process whose turn runs out goes to the back of the line.

Review questions with answers

1. Which of these is **not** one of the four major achievements in operating-system design?

- A. The process
- B. Memory management
- C. The graphical user interface
- D. Scheduling and resource management

Answer: C. The graphical user interface

The four are the process, memory management, information protection and security, and scheduling and resource management. Graphical interfaces matter to users, but they are not on this list.

2. Which statements are valid definitions of a **process**?

- A. A program in execution
- B. The file on disk that holds a program's machine code
- C. The entity that can be assigned to and executed on a processor
- D. A unit of activity with a single sequential thread of execution, a current state and an associated set of system resources

Answer: A, C, D

A program file on disk is passive: just instructions. It becomes a process only when it is loaded and running, with a current position, register values and resources of its own.

3. Which developments pushed designers to invent a clean process concept?

- A. Multiprogrammed batch operation
- B. Time sharing
- C. Real-time transaction systems
- D. Serial processing, where each user booked the whole machine and ran one program at a time

Answer: A, B, C

All three had to switch one processor among many activities at unpredictable moments. Ad hoc switching code led to timing bugs, and the process was the cure. Serial processing ran one program at a time with no switching, so it never faced these problems.

4. Match each bug to its family of error.

Each of two programs holds one lock and waits for the other's	Deadlock
Two clerks' programs both sell seat 14C	Failed mutual exclusion
A lost "I/O done" signal leaves a program asleep forever	Improper synchronization
Same input, different output, depending on timing	Nondeterminate operation

Waiting on each other forever is deadlock. Two programs in a shared record at once is failed mutual exclusion. A lost or badly handled signal is improper synchronization. Results that depend on interleaving are nondeterminate.

5. Put the steps of a switch from process A to process B in order.

Correct order:

1. An interrupt hands control to the OS
2. The OS saves A's context (PC and other registers) in A's process-list entry
3. The OS chooses B as the next process to run
4. The OS loads B's saved context into the registers, including base and limit
5. B carries on exactly where it last stopped

Control must reach the OS first; A's context is saved before anything overwrites the registers; then the OS picks B and restores B's context, so B resumes seamlessly.

6. When the OS switches from process A to process B, it copies A's program code and data out of main memory to make room for B.

Answer: False

A context switch saves and reloads only a few register values (the execution context). Code and data stay where they are in memory.

7. A running process has base register = 4000 and limit register = 500. What is the **largest** address, counted from the start of its block, that it may use?

Answer: 499

The block covers relative addresses 0 to 499 (real addresses 4000 to 4499). Any address of 500 or more reaches the limit, so the hardware refuses it and interrupts the OS.

8. Which tool mainly meets each memory-management need?

Virtual memory	File system
Keeping one process out of another's memory Giving a program more memory while it runs Letting a module grow or shrink while the program runs	Keeping a report after the power is switched off Storing information under a name so it can be found next month

Virtual memory handles isolation, automatic allocation and modular programming while programs run. Long-term, named storage that survives power-off is the file system's job.

9. Pages are 1,024 bytes. A program uses virtual address 3000, and the page table maps that page to frame 7. What real address does the MMU produce?

Answer: 8120

$3000 = 2 \times 1024 + 952$, so page 2, offset 952. Page 2 is in frame 7, so the real address is $7 \times 1024 + 952 = 7168 + 952 = 8120$.

10. A program uses an address whose page the page table marks as **on disk**. What happens next?
- A. The processor reads the byte straight from the disk
 - B. The MMU raises a page fault; the OS loads the page into a frame, updates the page table, and the instruction is retried
 - C. The program is always terminated for a bounds violation
 - D. The MMU uses the virtual address as the real address

Answer: B. The MMU raises a page fault; the OS loads the page into a frame, updates the page table, and the instruction is retried

A page fault is an interrupt: the OS fetches the missing page from disk (sending another page back if no frame is free), fixes the table, and the instruction restarts.

11. Match each security incident to the goal it breaks.

A flood of fake requests keeps real users off a web server	Availability
A stranger reads an unencrypted backup of patient records	Confidentiality
Malware changes the numbers in a company's accounts	Data integrity
An attacker logs in with a stolen password and is treated as the owner	Authenticity

Availability is about access to service; confidentiality about who can read; integrity about who can change; authenticity about genuine identity and origin.

12. Match each scheduling term to its meaning.

Fairness	Competing processes get roughly equal access
Differential responsiveness	Jobs with different needs are treated differently
Efficiency	High throughput, quick response, many users served
Long-term queue	New jobs not yet admitted to main memory
Short-term queue	Ready processes in memory, often served round-robin

The three goals guide the choices; the queues are where waiting processes sit. I/O queues, one per device, hold processes waiting for that device.

2.4 Developments Leading to Modern Operating Systems

You should be able to

- Name the three forces (new hardware, new applications, new security threats) that pushed operating systems to change.
- Contrast a monolithic kernel with a microkernel, and explain why a microkernel is simpler, more flexible and suited to distributed systems.
- Define thread and process, explain when multithreading helps, and explain why a thread switch is cheaper than a process switch.
- Describe symmetric multiprocessing (SMP), its four potential advantages, and the difference between multiprogramming and multiprocessing.
- Explain what a distributed operating system and object-oriented design add to an operating system.

Why operating systems had to change

Early operating systems assumed one slow processor and trusted users. Three forces changed that:

- **New hardware:** machines with several processors, much faster processors, high-speed network connections, and larger, more varied memory.
- **New applications:** multimedia (smooth sound and video), Internet and Web access, and client/server computing (clients send requests, servers do the work and reply).
- **New security threats:** once computers joined the Internet, attackers anywhere could reach them with viruses, worms and ever cleverer hacking techniques.

Five developments answered these forces: **microkernel architecture**, **multithreading**, **symmetric multiprocessing (SMP)**, **distributed operating systems** and **object-oriented design**.

Monolithic kernel vs microkernel

A **monolithic kernel** holds almost the whole OS: scheduling, file systems, networking, device drivers, memory management and more. All of these services run together in kernel mode in one shared kernel address space, so any part can call any other part directly, like a function call, with no messages. "Monolithic" describes this structure, not a single process: the same kernel code runs on behalf of whichever process made a system call, as well as in kernel threads and interrupt handlers. In a microkernel, by contrast, most services are separate user-mode server processes that exchange messages.

A **microkernel** keeps only the essential core in the kernel: **address spaces** (which memory each process may use), **interprocess communication (IPC)** and **basic scheduling**. Every other service (file system, device drivers, networking, the virtual-memory policy) runs as an ordinary **server process in user mode**. Applications and servers cooperate by sending **messages**, and the microkernel delivers each one. The OS offers the same services; they have simply moved out of the kernel.

One file read. Monolithic: one system call into the kernel, where the file system calls the driver directly (1 trip, 0 messages). Microkernel: application → file server → disk-driver process → file server → application,

each hop a message through the kernel (4 messages, 4 trips). That message passing is the classic performance price of a microkernel.

Benefits of a microkernel

- **Simpler kernel:** far less code runs with full privileges, so there is less to get right.
- **Flexibility:** servers can be added, replaced or upgraded without changing the kernel.
- **Suits distributed environments:** a request message can be forwarded unchanged to a server on another machine, so local and remote servers look the same.
- **Contained failures:** a crashed driver server wrecks only its own address space; in a monolithic kernel the same bug can corrupt kernel memory and halt the whole machine.

The three core functions stay in the kernel because each needs privilege: delivering messages, setting up address spaces, switching the processor.

Threads and processes

Multithreading divides one process into several threads that can run concurrently.

- **Thread:** a dispatchable unit of work (what the scheduler runs). It has its own **processor context** (program counter, stack pointer and other registers) and its own **data area for a stack**. It executes its instructions sequentially and is **interruptible**, so the processor can turn to another thread.
- **Process:** a collection of one or more threads plus the **system resources** they share: memory holding code and data, open files and devices.

Thread switch vs process switch. A thread switch within one process saves the old thread's registers and loads the new thread's registers and stack pointer (2 of 4 costs). A **process switch** must also switch the memory map to another **address space** (the range of memory a process may use), after which the processor's cached address translations and data no longer help (all 4 costs). So a thread switch is cheaper.

Multithreading pays off when an application has **independent tasks** that need not wait for each other, such as a database server answering many clients. It helps even on **one processor** (one thread computes while another waits for I/O). It is independent of SMP but combines well with it: threads of one process can then run on different processors at once.

Worked example: a database server

Four requests arrive together; each needs 2 units of processing, a 12-unit disk wait, then 2 more units, on one CPU. Illustrative costs: thread create 1 / switch 1; process create 5 / switch 2.

Design	All done at	Average response	Overhead
Single-threaded	64	40	0, but the CPU is busy only 16 of 64 = 25%
Process per request	44	38	28 units
Thread per request	27	22.5	8 units

Single-threaded total = number of requests × time per request. Example: 5 requests × (3 + 10 + 2) ms = 75 ms, and the processor is busy only 5 of every 15 ms ≈ 33.3%. Threads share the server's data directly;

separate processes need IPC or shared memory to share it, but a crash in one process does not take down the others. Real servers often create a **thread pool** once, at start-up.

Symmetric multiprocessing (SMP)

SMP names both a hardware architecture and the OS behaviour that exploits it: two or more similar processors of comparable capability share main memory and I/O over a bus or other link, with about equal memory access time; each can perform every function, even run the OS (hence symmetric); and one integrated OS schedules processes or threads on all of them, hiding the multiple processors from users.

Potential advantages over a single processor (they appear only if there is parallel work and the OS spreads it):

- **Performance:** work that can run in parallel finishes sooner.
- **Availability:** the failure of one processor does not halt the machine; it continues at reduced performance.
- **Incremental growth:** add a processor to boost performance.
- **Scaling:** vendors offer a range of products with different price and performance based on the number of processors.

Multiprogramming on one processor **interleaves** processes: they take turns, so they only seem to run together. **Multiprocessing** on several processors **interleaves and overlaps** them: some truly run at the same instant. Example: three processes that compute, wait for I/O, then compute again finish at 19 on one CPU, 13 on two, 11 on three (I/O waits limit the gain).

Distributed operating systems

A **cluster** is a group of complete computers on a fast network working together as one resource. A **distributed operating system** gives the illusion of a single main memory and a single secondary-memory space, plus unified access facilities such as a **distributed file system**, so users need not know which machine stores a file. Distributed OSs are still less mature than uniprocessor and SMP operating systems.

Object-oriented design

Objects are modules that hide their internals and are used only through defined interfaces. In an OS this gives a disciplined way to add **modular extensions to a small kernel**, lets programmers **customize the OS without disrupting system integrity**, and **eases building distributed tools** and full distributed operating systems (a remote object can offer the same interface as a local one).

Summary: development → problem it addresses

Microkernel: a huge, fragile kernel. **Multithreading:** one application juggling independent tasks cheaply.

SMP: too little processing power; surviving a processor failure. **Distributed OS:** networked computers that should act as one. **Object-oriented design:** extending the OS safely.

Key terms

Monolithic kernel

A kernel design in which almost the whole operating system (scheduling, file systems, networking, device drivers, memory management and more) runs together in kernel mode in one shared kernel address space, so its parts call one another directly instead of sending messages. The word describes how the kernel is built, not a single process: kernel code runs on behalf of whichever process made a system call, and in kernel threads and interrupt handlers.

Microkernel

A small kernel that keeps only the essential core functions (address spaces, interprocess communication and basic scheduling) and leaves every other OS service to server processes that run in user mode.

Address space

The range of memory addresses a process is allowed to use. The hardware, set up by the kernel, keeps each process inside its own address space, so one process cannot read or damage another's memory.

Interprocess communication (IPC)

Any mechanism that lets separate processes exchange data; in a microkernel it is the kernel delivering messages from one process to another.

Server process

In a microkernel system, an OS service such as a file system or a device driver that runs as an ordinary user-mode process and does its work when request messages arrive.

Client/server computing

Organising work so that client programs send requests to server programs, often on other machines, which do the work and send back replies.

Thread

A dispatchable unit of work inside a process. It has its own processor context (program counter and other registers) and its own stack, runs its instructions one after another, and can be interrupted so the processor can turn to another thread.

Process

A collection of one or more threads together with the system resources they share: memory holding code and data, open files and devices. Informally, a program in execution.

Multithreading

Dividing one process into several threads that can run concurrently, so one application can work on several independent tasks at once.

Thread switch

Moving the processor from one thread to another thread of the same process. Only the processor context and the stack change, so it costs much less than a switch between processes.

Process switch

Moving the processor from one process to another. Besides saving and loading registers, the OS must switch to the other process's address space, after which the processor's cached address translations and cached data no longer help. It is one kind of context switch.

Symmetric multiprocessing (SMP)

A computer with two or more similar processors that share main memory and I/O and can each run any work, including the OS, under one integrated operating system; the term also covers the OS behaviour

that exploits such hardware.

Multiprocessing

Running processes on several processors, so that their execution can both take turns (interleave) and truly happen at the same instant (overlap).

Incremental growth

Raising a system's performance by adding a part, such as one more processor, instead of replacing the whole machine.

Scaling

Offering a family of machines that run the same software but differ in price and performance, for example by the number of processors they contain.

Cluster

A group of interconnected, complete computers that work together as one unified computing resource.

Distributed operating system

An operating system for a cluster of networked computers that gives the illusion of one single main memory and one single secondary-memory space, plus unified services such as a distributed file system.

Distributed file system

A file system spread over several machines that users see as one ordinary tree of files and folders, without needing to know which machine stores each file.

Object-oriented design

Building software from objects: self-contained modules that hide their inner data and are used only through defined interfaces. In an OS it gives a disciplined way to add modular extensions to a small kernel.

Review questions with answers

1. Which functions does a **microkernel** keep inside the kernel?

- A. Address-space management, interprocess communication (IPC) and basic scheduling
- B. File systems, device drivers and networking
- C. Every OS service, running together in kernel mode in one shared address space
- D. None: every OS service runs in user mode

Answer: A. Address-space management, interprocess communication (IPC) and basic scheduling

A microkernel keeps only the essential core (address spaces, IPC and basic scheduling). Everything else runs as server processes in user mode that communicate by messages.

2. Which of these are benefits usually claimed for a **microkernel** design? Select all that apply.

- A. A simpler kernel, with less privileged code to get right
- B. Flexibility: services can be added, replaced or removed without changing the kernel
- C. A good fit for distributed systems, because local and remote servers are both reached by messages
- D. Faster requests, because each request needs fewer trips through the kernel

Answer: A, B, C

The first three are the classic benefits. The last is backwards: passing messages between user-mode servers usually means **more** trips through the kernel, which is the main performance cost of a microkernel.

3. In a microkernel system, put the steps of an application reading part of a file in order.

Correct order:

1. The application sends a request message to the file server
2. The microkernel delivers the message to the file server
3. The file server sends a message asking the disk-driver process for the blocks
4. The disk-driver process reads the blocks and replies to the file server
5. The file server sends the data back to the application in a reply message

Every hop is a message delivered through the microkernel: request to the file server, request to the driver, the driver's reply, then the reply to the application.

4. Which of these does each **thread** keep for itself instead of sharing it with the other threads of its process?

- A. Its processor context (program counter and other registers) and its stack
- B. The program code
- C. The table of open files
- D. The global variables and the heap

Answer: A. Its processor context (program counter and other registers) and its stack

A thread is a dispatchable unit of work: it needs its own saved registers and its own stack. The code, data, open files and devices belong to the process and are shared.

5. Why is switching the processor between two threads of the **same process** usually cheaper than switching between two **different processes**?

- A. The threads share one address space, so the memory map and cached translations stay valid; only the registers and stack pointer change
- B. Threads have no registers of their own, so nothing needs to be saved
- C. Threads cannot be interrupted, so switches only happen at convenient points
- D. Thread switches are carried out entirely by the hardware

Answer: A. The threads share one address space, so the memory map and cached translations stay valid; only the registers and stack pointer change

A process switch must also change the address space, which makes cached memory translations and data stale. Threads of one process skip that work.

6. Five requests reach a single-threaded file server at the same moment, and it handles them strictly one after another. Each request needs 3 ms of processing, then waits 10 ms for the disk, then needs 2 ms more processing. How many milliseconds pass before all 5 requests are finished?

Answer: 75 ms

One thread cannot overlap anything, so each request takes $3 + 10 + 2 = 15$ ms and $5 \times 15 = \mathbf{75\ ms}$. A multithreaded server could process other requests during the disk waits.

7. Multithreading is useful only on a computer that has more than one processor.

Answer: False

Even with one processor, a multithreaded server lets one thread compute while others wait for I/O, so the waits overlap. Multithreading and SMP are independent ideas; on an SMP machine they combine well, because threads of one process can then run on different processors at the same instant.

8. Match each potential advantage of **symmetric multiprocessing** to its description.

Performance	Work that can run in parallel finishes sooner
Availability	If one processor fails, the machine keeps running at reduced speed
Incremental growth	Add a processor instead of replacing the whole machine
Scaling	A vendor sells a range of machines with different numbers of processors and prices

All four come from having several similar processors that can each do any job under one OS. They are potential advantages: they appear only when the OS spreads parallel work across the processors.

9. Three processes run on one machine. What can **multiprocessing** do that multiprogramming on a single processor cannot?

- A. Overlap them: two processes can execute at literally the same instant
- B. Interleave them: switch between them so that all make progress
- C. Keep all three in main memory at the same time
- D. Run another process while one waits for I/O

Answer: A. Overlap them: two processes can execute at literally the same instant

On one processor, processes can only be interleaved. With several processors they are interleaved **and** overlapped: some truly run at the same time.

10. A distributed operating system gives users the illusion of a single main memory and a single secondary-memory space across a cluster of computers.

Answer: True

That is its defining goal, together with unified services such as a distributed file system. Such systems are still less mature than uniprocessor and SMP operating systems.

11. Which force behind modern operating systems does each change belong to?

New hardware	New applications	New security threats
Several processors High-speed networks	Multimedia Client/server computing	Viruses and worms

Hardware changes (processors, speed, networks, memory), new kinds of applications, and Internet-borne attacks all forced operating-system designs to evolve.

12. What is the main contribution of **object-oriented design** to operating systems?

- A. A disciplined way to add modular extensions to a small kernel, so the OS can be customized without disrupting its integrity
- B. Several identical processors running under one operating system
- C. One process containing several threads that run concurrently
- D. A cluster of computers made to look like one machine

Answer: A. A disciplined way to add modular extensions to a small kernel, so the OS can be customized without disrupting its integrity

Objects hide their internals and are used only through defined interfaces, so extensions plug into a small kernel cleanly. The same idea eases building distributed tools and full distributed OSs.

2.5 Fault Tolerance

You should be able to

- Define fault tolerance and explain why it relies on redundancy, and what that redundancy costs.
- Define reliability $R(t)$, MTTF and MTTR, then calculate availability as $MTTF / (MTTF + MTTR)$ and the downtime it allows per year.
- Place a system in the right availability class, from normal availability up to continuous.
- Classify faults as permanent, transient or intermittent, and match spatial, temporal and information redundancy to the faults each one handles.
- Describe how process isolation, concurrency controls, virtual machines, and checkpoints with rollback help an operating system survive faults.

What fault tolerance means

Fault tolerance is the ability of a system or component to keep operating normally even when some of its hardware or software has developed faults. It relies on **redundancy**: spare parts, repeated work or extra information that can take over or undo the damage. Redundancy costs money and often performance (a mirrored disk writes everything twice; a standby server must be kept in step). Spares only cover the faults they duplicate: a second power supply cannot save a server whose OS crashes, while a standby server usually can (unless it hits the same bug).

Measuring it

- **Reliability $R(t)$** : the probability that a system operates correctly for the whole period from time 0 to time t , given that it worked at time 0. $R(0) = 1$ and $R(t)$ never rises.
- **MTTF** (mean time to failure): the average time a system runs, from a fresh start or a repair, until it fails.
- **MTTR** (mean time to repair): the average time to notice, locate and repair a fault.
- **Availability**: the fraction of time the system is up and able to serve requests. $A = MTTF / (MTTF + MTTR)$; downtime per year = $(1 - A) \times 8,760$ h.

MTTF is an average, not a promise: with failures striking at random at a steady rate, $R(t) = e^{-t/MTTF}$, so only about 37% of units still run at $t = MTTF$. Scale makes failures routine: 10,000 disks with an MTTF of 1,000,000 h fail about $10,000 \div 1,000,000 = 0.01$ times an hour, one every 100 hours.

Worked example. MTTF = 990 h, MTTR = 10 h: $A = 990 / 1,000 = 0.99$, so the system is down $0.01 \times 8,760 \approx 87.6$ h a year.

Reliability is not availability: a server that crashes hourly but restarts in one second has poor reliability yet about 99.97% availability. **Only the ratio matters:** $A = 1 / (1 + MTTR/MTTF)$, so from MTTF 1,000 h and MTTR 10 h (99.01%), doubling the MTTF or halving the MTTR gives the same $2,000/2,010 = 1,000/1,005 \approx 99.50\%$.

Availability classes

Class	Availability	Downtime per year
Continuous	1.0	none
Fault tolerant	0.99999 (five nines)	about 5 minutes
Fault resilient	0.9999 (four nines)	about 53 minutes
High availability	0.999 (three nines)	about 8.8 h (often listed as 8.3 h)
Normal availability	0.99 to 0.995	about 44 to 87 hours

Each extra nine cuts downtime tenfold. Five nines with 1-hour repairs needs an MTTF near 100,000 h, so real designs hide failures with redundancy.

Faults

A **fault** is an erroneous state of hardware or software. Causes: component failure, operator error, physical interference from the surroundings (heat, noise, radiation, power dips), design error, program error and data structure error.

- **Permanent**: always present once it occurs, until the part is repaired or replaced (disk head crash, software bug, burned-out part). A bug that only some inputs trigger is still permanent. Retrying is useless; repair it or switch to a spare.
- **Temporary**, of two kinds. **Transient**: happens once (a noise burst garbles a frame, a power flicker, a radiation bit flip); a retry works. **Intermittent**: recurs at unpredictable times (a loose connection, a cracked solder joint); a retry works for now, but the fault returns until fixed.

Three kinds of redundancy

Kind	Idea	Examples	Limits and cost
Spatial (physical)	Several components do the same job at once, or a spare stands by (hot standby).	Triple modular redundancy (TMR) with a majority voter; a backup name server.	Masks any fault in one unit, even a permanent one. Fails if two units are wrong in the same way or all copies share a flaw. TMR triples the hardware.
Temporal	Repeat an operation when an error is detected.	A link resends a frame whose checksum failed.	Works for temporary faults; useless against a permanent one. Needs error detection first. Costs time.
Information	Store extra bits (a code) or copies so errors can be detected and corrected.	Parity; error-correcting code (ECC) memory; RAID.	Costs storage and a little computation. Enough errors can fool any code.

TMR: three units each output 42 (00101010); the voter takes the majority in every bit, so one faulty unit is outvoted and flagged for repair. Two units wrong in the same bit outvote the good one.

Retransmission: each frame carries a checksum. If the receiver's recalculated checksum disagrees, the frame is sent again, masking a transient noise burst at the cost of one extra transmission. A cut cable is

permanent: every resend is lost, and only a second, independent link (spatial redundancy) keeps data flowing.

Parity: a parity bit makes a group's count of 1s even. With one per row and per column of a 4×4 block, a single flipped data bit fails one row check and one column check; their crossing locates it. Several flips are usually detected but not located; four flips on a rectangle's corners go undetected. **RAID:** disks hold 1011, 0110, 0101 and a parity disk their XOR, 1000; if disk 2 dies, $1011 \oplus 0101 \oplus 1000 = 0110$ rebuilds it.

How the operating system helps

- **Process isolation:** each process has its own memory, files and flow of execution, enforced by memory-protection hardware; a stray write is trapped and only that process ends. It cannot contain a crash of the shared kernel.
- **Virtual machines:** each VM runs its own OS on a hypervisor, so even an OS crash stays inside one VM, and a standby VM (ideally on another machine) can take over. Cost: several OS copies.
- **Concurrency controls:** processes sharing data can interleave their steps. If two \$100 deposits to a \$500 balance both copy it before either writes, the result is \$600, not \$700: a lost update from a **race condition** (the result depends on timing). Locks and semaphores enforce mutual exclusion, so every order gives \$700. The OS can also detect deadlock and recover, for example by rolling one process back.
- **Checkpoints and rollbacks:** save the state at a consistent moment, in storage the failure cannot destroy (checkpoint). After a failure, discard the damaged state, restore the checkpoint (rollback) and redo only the work since. Example: transfers between two accounts totalling \$1,000; checkpoint after T3, crash halfway through T6 (total now wrong); roll back and redo T4 to T6 instead of all six. Frequent checkpoints lose less work but cost more time. Databases rely on this.

Key terms

Fault tolerance

The ability of a system or component to keep operating normally even when some of its hardware or software has developed faults.

Redundancy

Extra resources (spare parts, repeated work, or extra information) that are not needed while everything works but can take over or undo the damage when something fails.

Reliability

Written $R(t)$: the probability that a system operates correctly for the whole period from time 0 up to time t , given that it was operating correctly at time 0.

Mean time to failure (MTTF)

The average length of time a system runs correctly, from being started or repaired, until it next fails.

Mean time to repair (MTTR)

The average time it takes to find a fault and repair or replace the failed part so that the system works again.

Availability

The fraction of time a system is up and able to serve requests. Over the long run it equals $MTTF / (MTTF + MTTR)$. The security goal of the same name (section 2.3) makes the same promise against attacks rather than failures.

Fault

An erroneous state of hardware or software. Causes include a failed component, operator error, physical interference from the surroundings, a design error, a program error or a corrupted data structure.

Permanent fault

A fault that, once it occurs, is present all the time until the faulty part is repaired or replaced, such as a disk head crash or a software bug.

Temporary fault

A fault that is not present all the time. It comes in two kinds: transient and intermittent.

Transient fault

A temporary fault that happens only once, such as a bit flipped by an electrical noise spike or by radiation. Trying the operation again usually succeeds.

Intermittent fault

A temporary fault that comes and goes at several unpredictable times, such as a loose connection.

Spatial redundancy

Also called physical redundancy: several physical components perform the same function at the same time, or a spare stands by, ready to take over.

Temporal redundancy

Repeating an operation when an error is detected. It works well against temporary faults but cannot fix a permanent one.

Information redundancy

Storing extra bits (a code) or extra copies of data so that errors in the data can be detected and often corrected.

Triple modular redundancy (TMR)

Three identical units compute the same result and a voter outputs the majority answer, so a single faulty unit is outvoted.

Race condition

A fault that appears when two processes or threads read and write the same shared data and the final result depends on the exact order in which their steps happen to interleave.

Process isolation

The OS, helped by memory-protection hardware, keeps each process inside its own memory, files and flow of execution, so a faulty process cannot damage other processes or the OS.

Virtual machine (VM)

A software-made copy of a whole computer that runs its own operating system. Several VMs share one physical machine but are strongly isolated from one another.

Checkpoint

A saved copy of a program's or system's state, taken at a moment when that state is consistent and kept in storage that the expected failure cannot destroy, so work can resume from there.

Rollback

Recovering from a failure by discarding the damaged current state, restoring the most recent checkpoint, and redoing the work done since then.

Review questions with answers

1. Which statement best describes **fault tolerance**?

- A. A system that never develops hardware or software faults
- B. The ability of a system to keep operating normally despite hardware or software faults
- C. The ability to find and fix a fault quickly after the system has stopped
- D. Detecting a fault and shutting the system down safely before it does damage

Answer: B. The ability of a system to keep operating normally despite hardware or software faults

Fault tolerance means continuing normal operation despite faults. It usually relies on redundancy, which costs money and often some performance.

2. A server has an MTTF of 1,980 hours and an MTTR of 20 hours. What is its availability, as a percentage?

Answer: 99 %

$A = \text{MTTF} / (\text{MTTF} + \text{MTTR}) = 1,980 / (1,980 + 20) = 1,980 / 2,000 = 0.99$, which is 99%.

3. A system has an availability of 0.9999. About how many **minutes** of downtime does that allow per year? (One year = 8,760 hours.)

Answer: 52.56 min

$(1 - 0.9999) \times 8,760 \text{ h} = 0.876 \text{ h}$, and $0.876 \times 60 = 52.56$ minutes: about 53 minutes, the fault resilient class.

4. Reliability, $R(t)$, is best defined as:

- A. the fraction of time the system is available to serve requests
- B. the probability that the system operates correctly from time 0 up to time t , given that it worked at time 0
- C. the average time from a failure until its repair is finished
- D. the number of failures the system has had by time t

Answer: B. the probability that the system operates correctly from time 0 up to time t , given that it worked at time 0

$R(t)$ starts at 1 when $t = 0$ and falls as t grows, because a longer stretch gives more chances to fail.

5. Sort each fault by how long it lasts.

Permanent	Transient	Intermittent
A disk head crash A bug in a program	Radiation flips one memory bit, once A noise burst garbles one transmission	A loose connection that drops at random A cracked joint that fails at random times

Permanent faults stay until repaired (head crash, software bug). Transient faults happen once. Intermittent faults recur at unpredictable times.

6. Match each technique to the idea it relies on.

Three processors vote on every result	Spatial redundancy
A data link resends a frame whose checksum failed	Temporal redundancy
Memory stores check bits that let it fix a flipped bit	Information redundancy
A long job restarts from its last saved consistent state	Checkpoint and rollback

Extra hardware working in parallel is spatial redundancy, repeating work is temporal, extra coded bits are information redundancy, and restoring saved state is checkpoint and rollback.

7. Doubling a system's MTTF raises its availability more than halving its MTTR does.

Answer: False

$A = 1 / (1 + \text{MTTR}/\text{MTTF})$ depends only on the ratio MTTR/MTTF. Both changes halve that ratio, so they give exactly the same availability.

8. Which of these are operating system mechanisms that support fault tolerance?

- A. Process isolation
- B. Concurrency controls such as semaphores
- C. Virtual machines
- D. Checkpoints and rollbacks
- E. Letting any program write anywhere in memory
- F. Switching off the timer interrupt

Answer: A, B, C, D

Isolation, concurrency controls, VMs and checkpoint/rollback all limit or undo damage. Unrestricted memory writes, or no timer, would let one faulty program wreck or monopolise the whole machine.

9. Order these availability classes from the one that allows the **most** downtime per year to the one that allows the **least**.

Correct order:

1. Normal availability (0.99 to 0.995)
2. High availability (0.999)
3. Fault resilient (0.9999)
4. Fault tolerant (0.99999)
5. Continuous (1.0)

Each extra nine cuts yearly downtime by a factor of ten: about 44 to 87 hours, then about 8.8 hours (often listed as 8.3), about 53 minutes, about 5 minutes, and finally none at all.

10. In triple modular redundancy, two of the three units suffer the **same** flipped bit. What does the voter output?

- A. The correct value, because a voter always masks errors
- B. The wrong value, because the majority in that bit is now wrong
- C. Nothing: the voter detects a tie and stops
- D. The value from the one unit that is still correct

Answer: B. The wrong value, because the majority in that bit is now wrong

TMR masks a fault in one unit. When two units are wrong in the same way, they outvote the good one.

11. The only network cable between two buildings has been cut. Which remedy keeps data flowing?

- A. Resend every frame until it gets through
- B. Add a checksum to every frame
- C. Route the traffic over a second, independent link
- D. Save a checkpoint before each transmission

Answer: C. Route the traffic over a second, independent link

A cut cable is a permanent fault: it stays until repaired. Only spatial redundancy, a spare path that is already in place, can mask it.

12. Two processes each deposit \$100 into a shared balance of \$500, with no lock. Each deposit copies the balance into a private variable, adds 100 to the copy, then writes the copy back. Both processes copy the balance before either one writes. What is the final balance, in dollars?

Answer: 600 dollars

Both copy \$500, both compute \$600, and the second write overwrites the first: one deposit is lost. This lost update is a race condition; a lock (mutual exclusion) forces the correct \$700.

2.6 OS Design Considerations for Multiprocessor and Multicore

You should be able to

- Explain what it means for an SMP kernel to run on any processor, with each processor scheduling itself from a shared pool, and why that risks two processors picking the same process or a process being lost.
- Name and explain the five key design issues of an SMP operating system: simultaneous concurrent processes or threads, scheduling, synchronization, memory management, and reliability and fault tolerance.
- Show how locks and reentrant kernel code prevent races, and how an SMP OS degrades gracefully when a processor fails.
- Describe the three levels of parallelism in a multicore system and who exploits each one, including how developers, languages and the OS (for example Grand Central Dispatch) share the work of parallelism within applications.
- Explain the virtual machine approach to multicore: dedicating cores to a process, the OS acting like a hypervisor, and why this cuts context-switch overhead.

Why several processors change the operating system

On a uniprocessor at most one piece of kernel code executes at any instant, and the kernel can guard a table just by briefly disabling interrupts. In a **symmetric multiprocessor (SMP)** the kernel can execute on **any** processor, and disabling interrupts on one processor does nothing to stop the others, so shared kernel data needs **locks**. Usually no processor is in charge: each does **self-scheduling**, running the scheduler itself to take its next process or thread from a shared pool. The kernel can also be built as several processes or threads that run in parallel.

The OS must ensure that no two processors choose the same process and that no process is lost from the queue.

The race on a shared ready queue

```
acquire(lock);           // spin until free, then take it
next = ready[front];     // read the front entry
front = front + 1;       // step past that entry
release(lock);           // let another CPU in
run(next);               // run the chosen process
```

Without the lock the result depends on timing: a **race condition**. CPU 0 reads `ready[0] = P1`, CPU 1 also reads P1, then CPU 0 sets `front = 1` and CPU 1 sets `front = 2`. P1 now runs on two processors at once, both using its one saved context and stack, and P2 is skipped (lost). A lucky order hides the bug; it does not fix it.

With the lock, read-and-advance is indivisible: while one CPU is inside, the others wait, so every process is taken exactly once. It is a **spinlock** (a waiting CPU loops until the lock is free), cheap because the protected code is short. Checking and taking the lock is one **atomic** hardware instruction, such as test-and-set, so two CPUs can never both grab it.

The five key design issues of an SMP OS

Issue	What the OS must handle	Example and fix
1. Simultaneous concurrent processes or threads	Several processors may run the same kernel routine at once. Kernel routines must be reentrant , and kernel tables must be managed so simultaneous use cannot corrupt them or cause invalid operations.	A global scratch buffer makes one call open the wrong file. Fix: per-call data on each caller's own kernel stack, plus a lock on the shared table.
2. Scheduling	Any processor may schedule, so conflicts must be avoided (no two pick the same process). With kernel-level threads, several threads of one process can run on different processors at the same time.	Three ready threads on three free CPUs finish in 1 slot instead of 3.
3. Synchronization	Processes may share address spaces or I/O resources, so the OS must provide mutual exclusion and event ordering. Locks are the most common tool.	Two threads each sell a seat: both read 10 and write 9, so one sale is lost. With a lock: 10, 9, 8.
4. Memory management	All uniprocessor duties, plus exploiting hardware parallelism (e.g. multiported memory serving several processors at once) and coordinating paging across processors, so shared pages stay consistent and replacement is safe.	Each CPU caches recent page locations. CPU 0 evicts shared page 7 while CPU 1 still uses it, so CPU 1 reads another process's data. Fix: replacement considers all CPUs, and every CPU forgets the page's location before its frame is reused.
5. Reliability and fault tolerance	Degrade gracefully when a processor fails: the scheduler and the rest of the OS must recognize the loss and restructure their management tables.	See the worked example below.

Graceful degradation: a worked example

Twelve jobs each need 4 units of work (48 units), and each job saves a checkpoint after every finished unit. With 4 processors the batch takes $48 / 4 = 12$ ticks. Suppose CPU 2 dies after tick 3 while running J3 (3 units done). The OS notices the missed **heartbeat** (each processor regularly checks in; a silent one is declared failed), marks CPU 2 **offline**, stops scheduling on it and requeues J3, which resumes from its checkpoint (without one it would restart from the beginning). Three processors finish at tick 16. Capacity falls to $3 / 4 = 75\%$, but every job completes.

An OS that ignores the loss still lists CPU 2 as online and running J3, so J3 never finishes (11 of 12 done).

Rule of thumb: remaining capacity = working processors ÷ total processors; 2 failures out of 8 leave $6 / 8 = 75\%$.

Multicore: exploiting the parallelism

A multicore chip is an SMP on one piece of silicon, so every SMP issue above still applies. The extra challenge is to **exploit the available parallelism effectively**. It exists at three levels:

1. **Hardware parallelism within each core:** instruction-level parallelism (ILP). A pipeline overlaps several instructions from one stream, and many cores start more than one per cycle. The hardware exploits it (compilers help), not the OS.
2. **Multiprogramming and multithreaded execution within each core:** one core is shared over time among several processes and threads. The OS scheduler handles this, separately for every core.
3. **One application running as concurrent processes or threads across several cores:** the hardest level, needing the developer, language tools and the OS to cooperate.

Parallelism within applications

- The **developer** decides how to split the application into tasks that can run independently.
- **Compilers and programming languages** make those tasks easy to express (blocks, closures, parallel loops).
- The **OS** allocates resources, such as cores, among the parallel tasks efficiently.

Grand Central Dispatch (GCD) on macOS and iOS is a real example. The developer wraps each independent piece of work in a block and adds it to a dispatch queue (e.g. with `dispatch_async`). GCD keeps a **thread pool** sized to the available cores and feeds it the queued blocks; the developer never creates or schedules a thread.

Worked example. Filtering 12 photos takes 8 ms each, 96 ms of work. As one big task it takes 96 ms however many cores exist. As 12 tasks: time = $(12 \div \text{cores, rounded up}) \times 8$ ms, so 48 ms on 2 cores, 24 ms on 4 and 16 ms on 8 (the second round is half empty: 75% busy). As 48 quarter-photo tasks of 2 ms, 8 cores finish in $48 \div 8 \times 2 = 12$ ms with every core busy. Smaller tasks balance better, though each task also carries a small scheduling cost.

The virtual machine approach

With few cores, the classic OS job is **time slicing**: a timer forces a **context switch** on each core every few milliseconds. Each switch costs time to save and load registers, and the newcomer runs slowly while the

caches refill.

With many cores there are enough to go round. The **virtual machine approach** dedicates one or more whole cores to a process and leaves them alone. The OS allocates **cores and memory** instead of time slices, much like a **hypervisor** handing resources to virtual machines, and each process decides how to use what it got. Context-switch overhead almost disappears.

Example (the same work both ways). Processes A to D have 3 runnable threads each (12 threads), and every thread needs 8 slices of work: $12 \times 8 = 96$ core-slices, which exactly fills 8 cores for 12 slices. **Time slicing** from one shared queue: every core switches at every slice boundary, $8 \times 11 = 88$ **context switches**, and because each thread comes back on a different core, all 96 runs start with a cold cache. The OS scheduler runs $8 \times 12 = 96$ times. **Dedicated cores:** each process gets 2 cores and shares them among its own 3 threads. Three 8-slice threads fit on two 12-slice cores only if one thread is split, so the first core runs thread 1 for slices 1 to 8 and then thread 2's last 4 slices, while the second core runs thread 2 for slices 1 to 4 and then thread 3 for slices 5 to 12. Every thread still gets its 8 slices and all finish by slice 12, but there are only 2 switches per process, **8 in all**, each between threads of the same process (no change of address space, so cheaper), and 16 cold starts (the 8 first loads, the 4 third threads starting, and the 4 second threads that changed cores). The OS scheduler ran only for the 8 first loads. **Trade-off:** a dedicated core that its process leaves idle (say, waiting for I/O) is wasted, so the approach pays off when cores are plentiful.

Key terms

Symmetric multiprocessor (SMP)

A computer with two or more similar processors that share main memory and I/O devices and are run by one operating system; any processor can do any job, including running the kernel.

Self-scheduling

The usual SMP arrangement with no boss processor: whenever a processor needs work, it runs the kernel's scheduler itself and takes its next process or thread from a shared pool.

Ready queue

The OS list of processes or threads that are ready to run and are waiting only for a processor (the short-term queue of section 2.3). In a simple SMP design every processor takes work from one shared ready queue.

Race condition

A bug in which the result depends on the exact timing of two or more processors or threads using shared data at the same time. Some orders work, others corrupt the data.

Lock

A shared marker that at most one processor or thread can hold at a time. Code takes the lock before touching a shared structure and releases it afterwards, so two updates can never overlap.

Spinlock

A lock where a processor that finds it taken keeps re-checking in a tight loop (it spins) until the holder releases it. The check-and-take is one atomic (indivisible) hardware instruction, such as test-and-set, so two processors can never both grab it. SMP kernels use spinlocks to guard short stretches of code.

Reentrant code

Code that several processors or threads can be executing at the same moment without interfering, because it never modifies itself and keeps each caller's working data separate, for example on that caller's own stack.

Kernel-level thread

A thread that the kernel itself knows about and schedules. Because the kernel sees each one, it can run several threads of the same process on different processors at the same moment.

Synchronization

Coordinating processes or threads that run at the same time so they use shared memory and devices in a safe order: one at a time where needed, and in the right sequence when one depends on another.

Page

A fixed-size block (commonly 4 KB) of a process's memory. The OS can place each page in any free slot of main memory, or move it out to disk when memory runs short.

Shared page

A page that belongs to the address spaces of two or more processes at once, so they all see the same data, for example shared library code or a shared buffer.

Page replacement

The OS decision about which page to move out of main memory to disk when room is needed for another page.

Graceful degradation

Losing capacity in proportion to what failed instead of crashing: when one of four processors dies, the system carries on at about three quarters of its former speed.

Instruction-level parallelism (ILP)

Overlap among the instructions of one instruction stream inside a single core, for example a pipeline working on several instructions at different stages at once. The hardware finds and exploits it.

Grand Central Dispatch (GCD)

Apple's mechanism on macOS and iOS for parallelism within applications: a program packages work as small tasks placed on dispatch queues, and the system runs them on a pool of threads sized to the available cores.

Thread pool

A set of worker threads created in advance that repeatedly take tasks from a queue and run them, so a program does not have to create a new thread for every piece of work.

Context switch

Saving the registers and state of whatever is running on a processor and loading those of another process or thread. It costs time directly, and afterwards the newcomer runs slowly until the caches refill with its data.

Time slicing

Sharing one processor among several processes or threads by letting each run for a short interval in turn, with a timer interrupt forcing each switch.

Hypervisor

Software that creates and runs virtual machines. It hands each machine a share of the real hardware, such as processor cores and blocks of memory, and keeps the machines isolated from one another.

Virtual machine approach

A multicore OS strategy: instead of time-slicing every core among many processes, dedicate one or more whole cores (plus memory) to a process for a long period and let the process use them as it likes, so the OS behaves more like a hypervisor handing out resources.

Review questions with answers

1. In a typical symmetric multiprocessor (SMP), how does a processor that needs work get its next process or thread?

- A. A master processor assigns it one
- B. It runs the kernel's scheduler itself and takes one from a shared pool
- C. The user chooses a processor for each program
- D. Each processor has a fixed list of processes decided at boot time

Answer: B. It runs the kernel's scheduler itself and takes one from a shared pool

This is self-scheduling: the kernel can run on any processor, and each processor schedules itself from the pool of available processes or threads.

2. Several processors of an SMP take work from one shared ready queue. Which statements are true?

- A. Without a lock, two processors can choose the same process
- B. Without a lock, a process can be skipped and never run
- C. Disabling interrupts on the processor running the scheduler is enough to protect the queue
- D. Checking and taking a spinlock must be one atomic (indivisible) hardware instruction
- E. The hardware automatically makes the read-the-front, then-advance sequence indivisible

Answer: A, B, D

If two processors read the front before either advances it, both take the same process and the double advance skips the next one. Disabling interrupts only affects one processor, and the hardware does not bundle a multi-step sequence by itself; a lock built on an atomic instruction such as test-and-set does.

3. Match each SMP design issue to a situation it covers.

Simultaneous concurrent processes or threads	Two CPUs execute the same kernel routine at once
Scheduling	Threads of one process run on different CPUs together
Synchronization	Two threads on different CPUs update a shared counter
Memory management	Page replacement for a page two CPUs share
Reliability and fault tolerance	Carrying on after one processor fails

Concurrent kernel code needs reentrancy and protected tables; scheduling must avoid conflicts and can run a process's threads in parallel; synchronization protects shared memory and I/O; memory management coordinates paging; reliability means graceful degradation.

4. A kernel routine that keeps its working data in one global variable is safe on an SMP, provided the routine is short.

Answer: False

However short it is, two processors can be inside it at the same moment and overwrite each other's data. It must be reentrant (data on each caller's own stack) or guarded by a lock.

5. With kernel-level threads, an SMP scheduler can run several threads of the same process on different processors at the same moment.

Answer: True

The kernel sees and schedules each thread separately, so it can place threads of one process on several processors at once, which is how a multithreaded program gets faster.

6. CPU 0's memory manager wants to evict a page that a process on CPU 1 is still using through a shared mapping. What should an SMP memory manager do?

- A. Evict it at once; CPU 1 will notice eventually
- B. Coordinate with the other processors: avoid removing a page in use, and make every processor forget its saved location before the frame is reused
- C. Copy the page into CPU 0's registers
- D. Shut CPU 1 down before evicting

Answer: B. Coordinate with the other processors: avoid removing a page in use, and make every processor forget its saved location before the frame is reused

Paging mechanisms on different processors must be coordinated so shared pages stay consistent and replacement never pulls a page out from under a processor that is using it.

7. An SMP has 8 equally fast processors. Two of them fail, and the OS recognizes the loss and restructures its tables. What percentage of the original processing capacity remains?

Answer: 75 %

6 working processors out of 8 is $6 \div 8 = 0.75$, so 75% remains. That proportional loss, rather than a crash, is graceful degradation.

8. Which level of multicore parallelism does each situation show?

1: in a core	2: over time	3: across cores
Pipeline overlaps fetch and execute Two independent adds in one clock cycle	OS swaps one core between two apps Another thread runs during a disk wait	Game threads run on two cores at once Encoder spreads a frame over four cores

ILP overlaps instructions of one stream inside a core; multiprogramming and multithreading share one core over time; the third level splits one application across cores.

9. With Grand Central Dispatch on macOS or iOS, what does the application developer do?

- A. Creates one thread per core and schedules them by hand
- B. Wraps independent pieces of work in blocks and places them on dispatch queues
- C. Writes a new scheduler for the kernel
- D. Pins each block to a particular core number

Answer: B. Wraps independent pieces of work in blocks and places them on dispatch queues

The developer marks what can run in parallel; GCD keeps a thread pool sized to the cores and feeds it the queued blocks.

10. A program queues 12 independent tasks of 8 ms each. A thread pool of 8 threads runs on 8 cores, and each task goes to the next free thread. How many milliseconds until every task has finished?

Answer: 16 ms

The first round runs 8 tasks (8 ms); the remaining 4 need a second round (8 ms more). $8 + 8 = 16$ ms, with half the cores idle in the second round.

11. In the virtual machine approach to multicore, what does the OS mainly hand out?

- A. Short time slices on each core
- B. Whole cores and blocks of memory, like a hypervisor
- C. Individual instructions to pipeline stages
- D. Disk blocks to files

Answer: B. Whole cores and blocks of memory, like a hypervisor

With plenty of cores, the OS dedicates cores (and memory) to a process and leaves them alone, acting more like a hypervisor than a time-slicer.

12. Why does dedicating cores to processes usually improve performance?

- A. It removes the need for main memory
- B. It cuts context switches, so less time is spent switching and caches stay warm
- C. It raises each core's clock speed
- D. It lets two processes share one set of registers

Answer: B. It cuts context switches, so less time is spent switching and caches stay warm

Each context switch costs time directly and leaves the caches cold. A core devoted to one process rarely switches, so more of its time goes to useful work.

2.7 Microsoft Windows Overview

You should be able to

- Trace the Windows NT line from 1993 to Windows 10 and 11 and explain how one design came to run on everything from tablets to servers.
- Name the kernel-mode parts of Windows (the executive and its managers, the kernel, the HAL, device drivers, windowing and graphics) and the four kinds of user-mode processes.
- Explain the client/server model inside Windows, how ALPC messages carry requests, and the four advantages of the design.
- Describe how Windows uses threads and symmetric multiprocessing to keep every processor busy.
- Explain Windows objects: the object manager, handles, security descriptors, named and unnamed objects, and control versus dispatcher objects.

Microsoft Windows: one design, many machines

Windows runs on tablets, laptops, desktops, consoles and servers with 100+ processors, all on one core design, **Windows NT**, built from scratch in the early 1990s as a modular, portable, multitasking OS with security and multiprocessor support.

1. Background

Early Windows (1.0 to 3.x, 95, 98, Me) ran on top of MS-DOS. NT was a fresh design: **NT 3.1** (1993) was 32-bit with preemptive multitasking and protected memory, written mostly in C for portability. **NT 4.0** (1996) moved windowing and graphics into kernel mode for speed. **2000** added plug and play and power management. **XP** (2001) brought NT to home users and ended the MS-DOS line. **Vista** (2007) was an internal overhaul (ALPC replaced LPC). **7** (2009) scaled to many more processors. **8** (2012) added touch and ARM support. **10** (2015): one core for PCs, tablets, phones, Xbox and small devices; Windows Server shares it. **11** (2021) continues on the same NT core.

2. Architecture

Windows is highly **modular**. User programs cross into kernel mode only through system calls. **Kernel-mode components:**

- **Executive:** core OS services, organised as managers (below).
- **Kernel:** thread scheduling, process switching, exception and interrupt handling, multiprocessor synchronization. Unlike the rest of the OS, the kernel's own code does not run in threads.
- **Hardware abstraction layer (HAL):** maps generic hardware commands onto those of one specific platform, isolating the OS from differences in the system bus, DMA controller, interrupt controller and timers. It hides the platform wiring, not the processor: a new processor family (e.g. ARM) also needs recompiling and a small processor-specific part of the kernel.
- **Device drivers:** loadable modules that operate devices or implement file systems; a buggy one can crash the system.
- **Windowing and graphics system:** windows, menus, input focus, drawing.

Executive manager	Job
I/O manager	One interface to all devices; sends I/O request packets (IRPs) to drivers
Cache manager	Keeps recently used file data in main memory
Object manager	Creates, names, tracks, deletes objects; hands out handles
Plug-and-play manager	Detects devices, loads their drivers
Power manager	Sleep, hibernate, powering down idle devices
Security reference monitor	Checks access against security descriptors
Virtual memory manager	Maps virtual to physical memory; pages to and from disk
Process/thread manager	Creates and deletes processes and threads
Configuration manager	Implements the registry
ALPC facility	Fast message passing between local processes

User-mode processes: special system processes (session manager, logon process, authentication subsystem, service control manager); **service processes** (printer spooler, event logger); **environment subsystems** that give programs an OS personality (Win32 is the main one), whose **subsystem DLLs** translate API calls into native system calls; and **user applications** (EXEs and DLLs).

A file-open request: app calls `CreateFile` → subsystem DLL calls `NtCreateFile` in `ntdll.dll` → system call enters kernel mode → I/O manager asks the object manager to resolve the name → I/O manager creates a file object and sends an IRP to the file system driver → if the file's information is not in memory, the disk driver reads it, using HAL routines (e.g. for DMA) → controller transfers data and interrupts; the driver queues a DPC to finish and the kernel wakes the waiting thread → the file system driver has the security reference monitor check the file's security descriptor → the app receives a handle.

3. Client/server model

Many services run as **servers** in their own processes (environment subsystems, services); applications are their **clients**. A client sends a request message; the server does the work and replies. Locally, messages travel through the executive's ALPC facility to the server's **port**. Windows is not a pure microkernel: the executive stays in kernel mode for speed. Advantages:

1. **Simplifies the executive:** new APIs and services are added as servers without changing it.
2. **Improves reliability:** each server is its own process, so a crash is contained (and the service can be restarted).
3. **Uniform communication:** applications reach every service the same way, through remote procedure calls (RPCs). A small **stub** in the client packs each call's parameters into a message, so the call looks like an ordinary function call.
4. **Base for distributed computing:** a server can move to another machine while clients make the same calls.

4. Threads and SMP

On a symmetric multiprocessor, any OS routine can run on any available processor, and different routines can run on different processors at once. Threads of one process can run on different processors simultaneously. Server processes can use several threads to serve several clients at once. Processes share data and resources through flexible IPC.

Worked example: 21 units of work on 4 cores finish at time 6 under Windows' rules (lower bound $21 \div 4 = 5.25$), with cores $21 \div 24 \approx 88\%$ busy. Forcing OS routines onto core 0 (a master/slave design) gives 8; making a process's threads take turns gives 9; both give 11. With 1 or 2 cores all rules tie. Four 6 ms threads on 4 processors finish in 6 ms, but would need 24 ms one at a time.

5. Windows objects

Windows is written in C but uses four object-oriented ideas: **encapsulation** (data hidden; only operations visible), **object class and instance** (each object type is a template; every opened file is a new File object), **inheritance** (only inside the implementation, e.g. objects sharing a common header), and **polymorphism** (one wait function works on processes, threads, events and timers).

Windows uses objects for data that user mode must reach, or that is shared or protected. The **object manager** creates and tracks all objects; it keeps each object's header (name, security descriptor, handle count), while the body holds type-specific data. Programs use them through **handles**, indexes into a per-process handle table, so one number can mean different objects in two processes. A **security descriptor** records who may do what; the security reference monitor checks it when a handle is opened, and the rights granted are stored with the handle. **Named** objects can be opened by name by any permitted process.

Unnamed ones are shared only by passing handles on (duplication or inheritance). An object is deleted when its last handle closes. For example, if A, B and C hold handles and A and B close theirs, 1 remains.

Kernel objects sit underneath executive objects (an executive event is built around a kernel event).

Control objects control the kernel: asynchronous procedure call (interrupts a thread to run a procedure), deferred procedure call (postpones interrupt work), interrupt (links a source to its service routine), process, profile (measures where code spends time). **Dispatcher objects** are used for synchronization: each is signaled or not, and threads wait on them (event, mutex, semaphore, timer, thread). The process object is in both lists, since threads can wait for a process to end.

Common mistakes: the HAL is not a device driver; the kernel is not the executive; Windows is not a pure microkernel.

Key terms

Windows NT

The family of Microsoft operating systems designed from scratch in the early 1990s ("NT" first stood for New Technology). Every modern desktop and server version of Windows, including Windows 10 and Windows 11, is built on it.

Executive

The upper layer of kernel-mode Windows. It holds the core OS services, organised as managers: I/O, cache, objects, plug and play, power, security, virtual memory, processes and threads, configuration, and the ALPC facility.

Windows kernel

The layer beneath the executive that schedules threads, switches between processes, handles exceptions and interrupts, and keeps processors synchronized. Unlike the rest of the OS, its own code does not run in threads.

Hardware abstraction layer (HAL)

A thin layer of kernel-mode code that turns generic hardware requests from the rest of Windows into the exact commands one platform needs, hiding differences in buses, DMA controllers, interrupt controllers and timers.

Device driver

A kernel-mode module, loaded when needed, that knows how to operate one kind of device or file system and turns I/O requests into commands for it.

Environment subsystem

A user-mode server process together with its DLLs that gives programs one operating-system "personality", meaning one set of API calls. Win32 is the main subsystem in Windows.

Dynamic link library (DLL)

A file of shared code that is loaded into a process while it runs, so many programs can use the same functions without each carrying its own copy. Subsystem DLLs translate API calls into system calls.

Win32 API

The main programming interface of Windows: thousands of documented functions, such as CreateFile, that applications call to use OS services.

Service process

A background process that provides a service whether or not anyone is logged in, such as the print spooler or the event logger. The service control manager starts and stops it.

Advanced local procedure call (ALPC)

The executive's fast message-passing facility between processes on the same computer. A client sends a request to a server's port and the server sends back a reply.

Client/server model

A way of structuring software in which a client asks for a service by sending a message, and a server process does the work and sends back a reply.

Remote procedure call (RPC)

A request that looks like an ordinary function call to the caller but is carried out by messages to a server, which may be on the same computer or across a network.

Symmetric multiprocessor (SMP)

A computer with two or more similar processors that share main memory and I/O, where any processor can run any work, including the operating system itself.

Object manager

The executive component that creates, names, tracks and deletes every Windows object and hands out the handles programs use to reach them.

Handle

A small number, private to one process, that stands for an object the process has opened. It indexes the process's handle table; programs never get a direct pointer to the object.

Security descriptor

Data attached to an object that records its owner and which users or groups may do what with it. It is checked whenever someone opens a handle to the object.

Dispatcher object

A kernel object that threads can wait on. It is either signaled or not signaled, and waiting threads are released when it becomes signaled. Events, mutexes, semaphores, timers, threads and processes are examples.

Control object

A kernel object used to control how the kernel itself operates, rather than to synchronize threads: asynchronous procedure call, deferred procedure call, interrupt, process and profile objects. (The process object is also waitable, so it belongs to both families.)

Encapsulation

Hiding an object's data inside it so that outsiders can use the object only through the operations (methods) it offers.

Polymorphism

Using one operation on objects of different types, with each type supplying its own meaning for that operation.

Review questions with answers

1. Which part of Windows turns generic hardware requests, such as "acknowledge this interrupt", into the exact commands one particular platform needs?

- A. A device driver
- B. The hardware abstraction layer (HAL)
- C. The kernel
- D. The I/O manager

Answer: B. The hardware abstraction layer (HAL)

The HAL maps generic hardware commands onto platform-specific ones, so the kernel and drivers barely change from one kind of machine to another.

2. Match each executive component to its job.

I/O manager	Packs requests into I/O request packets and passes them to drivers
Cache manager	Keeps recently used file data in main memory
Security reference monitor	Checks access rights against an object's security descriptor
Configuration manager	Implements and manages the registry
Plug-and-play manager	Detects new devices and loads their drivers

Each executive manager owns exactly one job, which keeps the executive modular: changing one manager leaves the others untouched.

3. Which statement about the history of Windows is accurate?

- A. Windows 10 is built on the Windows NT design that first shipped in 1993.
- B. Windows 10 still runs on top of MS-DOS, like Windows 95 did.
- C. Windows XP was the last release built on NT.
- D. Windows NT was written for Intel processors only, so moving to ARM required a new OS.

Answer: A. Windows 10 is built on the Windows NT design that first shipped in 1993.

Every modern Windows, desktop and server, descends from Windows NT, a portable, modular design started from scratch in the early 1990s.

4. Does each part run in user mode or in kernel mode?

User mode	Kernel mode
Session manager Print spooler service Win32 subsystem DLLs Logon process	Object manager Hardware abstraction layer ALPC facility Device drivers

Special system processes, service processes, environment subsystems and their DLLs, and applications run in user mode. The executive (including the object manager and ALPC), the kernel, the HAL, drivers, and windowing and graphics run in kernel mode.

5. The print spooler service crashes because of a bug, but every other program and the OS itself keep running. Which advantage of the client/server model does this show best?

- A. It simplifies the executive
- B. It improves reliability
- C. It gives applications one uniform way to ask for services
- D. It is a natural base for distributed computing

Answer: B. It improves reliability

Each server runs as its own process with its own memory, so a failure stays inside that server. The service control manager can even restart it.

6. Put the path of a file-open request in order.

Correct order:

1. The application calls CreateFile
2. A subsystem DLL calls the native service in ntdll.dll
3. A system call switches the processor into kernel mode
4. The I/O manager sends an I/O request packet to the file system driver
5. The disk driver sets up the transfer using HAL routines
6. The disk controller finishes and raises an interrupt

The request moves down the layers: application, subsystem DLL, system call, executive (I/O manager), drivers, HAL, hardware. The interrupt at the end is how the hardware reports back.

7. A process has 4 threads. Each needs 6 ms of processor time and never waits for I/O. On a 4-processor machine running Windows, what is the shortest time until all four threads have finished?

Answer: 6 ms

Windows lets the threads of one process run on different processors at the same time, so all four run side by side and finish after 6 ms. If only one thread at a time were allowed, it would take $4 \times 6 = 24$ ms.

8. Which statements about Windows on a symmetric multiprocessor are true?

- A. Any OS routine can run on any available processor.
- B. Two threads of the same process can run on different processors at the same moment.
- C. All kernel-mode code runs on processor 0 only.
- D. A server process can use several threads to serve several clients at once.
- E. The kernel's own code runs in threads, just like the rest of the OS.

Answer: A, B, D

Windows treats processors as equals: OS routines, and threads of the same process, can run anywhere and at the same time, and servers use many threads for many clients. The kernel is the exception to "everything runs in threads": its own code does not.

9. Process A creates a named event and gets a handle to it. Processes B and C then open the same event by name. Next, A and B close their handles. How many handles still refer to the event?

Answer: 1 handles

Three handles were opened (A, B, C) and two were closed, leaving C's. The object manager keeps the event alive until the last handle closes, then deletes it.

10. Process B wants a handle to an event that process A created. In which case can B **not** get it by asking the object manager for it by name?

- A. A created the event without a name
- B. The event's security descriptor lets B's user wait on it
- C. A still holds its own handle to the event
- D. A and B run on different processors

Answer: A. A created the event without a name

Only a named object appears in the object manager's namespace. An unnamed object can reach another process only if a handle is passed on, for example by duplication or inheritance.

11. Sort these kernel objects.

Control objects	Dispatcher objects
Asynchronous procedure call (APC) Deferred procedure call (DPC) Interrupt object Profile object	Event Mutex Semaphore Timer

Control objects steer the kernel's own operation (APC, DPC, interrupt, process, profile). Dispatcher objects are used for synchronization: threads wait on them until they become signaled.

12. Match each object-oriented idea to its Windows example.

Encapsulation	Programs reach an object only through a handle and its operations
Class and instance	Every opened file is a new object of the File type
Inheritance	Mutex, event and semaphore objects share a common header and its code
Polymorphism	One wait function works on processes, threads, events and timers

Windows is written in C, but its design uses all four ideas; inheritance appears only inside the implementation.

2.8 Traditional Unix Systems

You should be able to

- Trace UNIX from its 1970 start at Bell Labs through the PDP-11, the rewrite in C, Versions 6 and 7, and the System V and BSD branches.
- Explain why rewriting UNIX in C was a milestone, and say what still had to be rewritten when it moved to a new machine.
- Name the layers of a UNIX system and explain why the system call interface is the boundary between user programs and the kernel.
- Describe the two main parts of the traditional kernel, the process control subsystem and the file subsystem, plus hardware control.
- Trace a read() call through the kernel and explain why the traditional UNIX kernel is called monolithic.

What traditional UNIX is

UNIX is a multiuser, time-sharing operating system built at **Bell Labs** by **Ken Thompson** and **Dennis Ritchie**. Work began in 1969 on a spare DEC PDP-7 minicomputer; by **1970** it was running and had its name. It stood out because most of it was written in a high-level language (C), it was small and simple (a compact kernel plus many small programs users combine), and its source code was licensed cheaply to universities. Its descendants (FreeBSD, macOS, Solaris, AIX) and look-alikes (Linux, Android) still run much of today's computing.

History and the family tree

When	What happened	Why it mattered
1970	Running on a PDP-7 at Bell Labs, in assembly	A small, practical system built by programmers for themselves
1971	Moved to the DEC PDP-11	Still assembly: the move meant rewriting everything
1973	Kernel rewritten in C	A milestone: almost every OS had been written in assembly
1975	Version 6	First widely used outside Bell Labs; licensed to universities with source code
1978 on	BSD releases, UC Berkeley	3BSD (1979): virtual memory on the VAX. 4.2BSD (1983): TCP/IP networking and sockets. Then 4.3BSD and 4.4BSD
1979	Version 7	Made portable (lessons from the Interdata 8/32 port; UNIX/32V ran it on the VAX); ancestor of nearly every later UNIX
1982, 1983	AT&T System III (first commercial release), then System V	Supported UNIX licensed to computer makers (AIX, HP-UX)
1989	SVR4	Merged System V with popular BSD features (section 2.9)

Two branches grew from the research versions: the **AT&T line** (System III → System V → SVR4 → Solaris, AIX, HP-UX) and the **Berkeley (BSD) line** (1BSD → 3BSD → 4.xBSD → FreeBSD, NetBSD, OpenBSD, parts of macOS). The Berkeley line was especially influential because it brought **virtual memory** (paging, from 3BSD on the VAX) and **TCP/IP networking**, the Internet's protocols, plus sockets (4.2BSD). Linux (1991) is a look-alike that shares no UNIX code (section 2.10).

Why the rewrite in C mattered

Assembly language is tied to one processor family, so an OS written in it must be rewritten line by line for a different processor. **C** is a **high-level language**: a **compiler** translates the same source into instructions for any processor it supports. After the rewrite, only the **machine-dependent code** (trap and interrupt entry, context switching, device drivers) had to be rewritten for a new machine; the rest was recompiled. Each new processor also needed a C compiler, once, after which every C program could move across.

Worked example. An illustrative 10,000-line kernel has 1,000 machine-dependent lines (90% portable). Porting it to four new machines means rewriting $4 \times 10,000 = \mathbf{40,000}$ lines in assembly but only $4 \times 1,000 = \mathbf{4,000}$ lines in C: a ten-to-one saving on every machine. The price was a kernel somewhat larger and

slower than hand-tuned assembly. **Common mistake:** C did not make UNIX portable for free; each machine still needed a compiler and a rewritten machine-dependent part.

The layers of a UNIX system

1. **Hardware:** processor, memory, disks, terminals.
2. **Kernel:** runs in kernel mode; the only software that talks to the hardware directly.
3. **System call interface:** the **boundary with the user**, a fixed set of entry points (open, read, write, fork, exec, wait...) that lets higher-level software reach specific kernel functions. A program enters it with a **trap instruction**, which switches to kernel mode.
4. **Commands and libraries:** shells, compilers, editors, utilities (ls, sort) and libraries such as the C library. All are ordinary user-mode programs; the shell is **not** part of the kernel.
5. **User-written applications.**

User programs invoke OS services **directly** (a system call such as write) or **through a library routine** (such as printf, which makes the system call for them). Either way the request crosses the system call interface.

Inside the traditional UNIX kernel

User programs and libraries **trap** into the **system call interface**, which identifies the call, checks its arguments and passes it to one of two main parts. Both sit on **hardware control**.

Part	Contains	Job
Process control subsystem	Memory management	Gives processes memory, protects them, swaps them out when memory is short
	Scheduler	Scheduling and dispatching: picks the next ready process and switches to it
	Synchronization and IPC	Signals and pipes (System V later added messages, shared memory, semaphores); sleep until an event, then wake up
File subsystem	File management	Names, directories, permissions, which blocks belong to which file
	Buffer cache	Recently used disk blocks kept in main memory; checked first
	Character and block device drivers	Move data as a stream of characters (terminals; no cache) or in blocks (disks; through the buffer cache)
Hardware control	Interrupt handling	Lowest, machine-dependent layer: fields interrupts, talks to the machine

Following one read() call

Miss (14 stops): the program calls read(fd, buf, 512) in user mode → the library wrapper sets up the call → trap to kernel mode; the system call interface checks it → the file subsystem finds the disk block → the buffer cache misses and picks a free buffer → the block driver starts the disk → the process sleeps (Blocked) → the scheduler runs another process → the disk interrupts when done → hardware control runs the driver's handler, the buffer is valid → the process is woken (Ready) → the file subsystem copies 512

bytes into buf → return from the trap to user mode → read returns 512. It takes milliseconds, nearly all waiting for the disk.

Hit (8 stops): the same first four stops, a cache hit, the copy, the return: microseconds. A hit still needs the system call interface, the file subsystem and the buffer cache, but it **skips** the block driver, the disk, hardware control, sleeping and waking (IPC) and the scheduler. That is why the kernel keeps a buffer cache.

A monolithic kernel

All the parts are linked into **one program**, the kernel image, running in kernel mode in **one address space**, where any part can call any other directly. This **monolithic** design is fast and simple but **not very modular**. Adding a driver meant writing it, entering it in the kernel's device switch table, rebuilding the whole kernel and rebooting (logging every user off). A bug in any part, such as a driver writing past its buffer into the scheduler's tables, can crash the whole system with a panic, while a bug in a user program kills only that process. The opposite design, a **microkernel**, keeps drivers and other services outside the kernel as separate user-mode server processes. Modern UNIX (2.9) and Linux (2.10) keep a monolithic core but add modules that can be loaded while the system runs.

Key terms

UNIX

A multiuser, time-sharing operating system created at Bell Labs by Ken Thompson and Dennis Ritchie; it was running on a PDP-7 minicomputer by 1970. The name now covers a large family of descendants.

Assembly language

A low-level language in which each line stands for one machine instruction of one particular processor family, so assembly code written for one processor cannot run on a different kind.

High-level language

A language such as C whose statements (variables, loops, function calls) do not depend on any one processor. A compiler translates them into the instructions of whichever processor you target.

C programming language

The high-level language Dennis Ritchie designed at Bell Labs in the early 1970s; UNIX was rewritten in it in 1973. It is close enough to the hardware for system code, yet it can be compiled for many processors.

Compiler

A program that translates source code written in a high-level language into the machine instructions of one particular processor. Moving C code to a new machine means compiling it again with a compiler for that machine.

Machine-dependent code

Code tied to one kind of hardware (its instruction set, its registers or its devices). It must be rewritten when the system moves to a different machine.

System V

AT&T's main commercial UNIX line, released in 1983 after System III and licensed widely to computer makers. Its fourth release (SVR4) later absorbed many BSD features.

Berkeley Software Distribution (BSD)

The UNIX versions produced at the University of California, Berkeley, from 1978. 3BSD added virtual memory, the 4.xBSD series carried it on and added TCP/IP networking, and these ideas shaped nearly every later UNIX.

Library

A collection of ready-made routines, such as the C library's `printf` and `fopen`, that programs call instead of writing that code themselves. Many library routines make system calls on the program's behalf.

Shell

The command interpreter: an ordinary user-mode program that reads the commands you type (such as `ls` or `cc`) and asks the kernel to run them. It is not part of the kernel.

System call

A request from a running program to the kernel for a service, such as reading a file or creating a process. The program enters the kernel through a controlled gate that switches the processor into kernel mode.

System call interface

The boundary between user programs and the UNIX kernel: a fixed set of entry points, such as `open`, `read`, `write` and `fork`, through which higher-level software asks for specific kernel services.

Trap instruction

A special machine instruction that a program executes on purpose to enter the kernel. It switches the processor to kernel mode and jumps to a fixed kernel entry point, so user code cannot choose where in the kernel it lands.

Process control subsystem

The part of the traditional UNIX kernel that manages processes: memory management, scheduling and dispatching, and synchronization and interprocess communication.

File subsystem

The part of the traditional UNIX kernel that manages files and moves data between main memory and external devices, either in blocks (through the buffer cache) or as a stream of characters.

Buffer cache

An area of main memory where the UNIX kernel keeps copies of recently used disk blocks, so a request for a block that is already there needs no disk access.

Block device

A device, such as a disk or a tape, that stores data in fixed-size numbered blocks. In traditional UNIX its data passes through the buffer cache.

Character device

A device, such as a terminal or a printer, that sends or receives a stream of bytes one after another, with no block structure. Its driver is reached without going through the buffer cache.

Hardware control

The lowest layer of the traditional UNIX kernel: machine-dependent code that handles interrupts and communicates directly with the machine's devices and registers.

Monolithic kernel

A kernel built as one large program whose parts (scheduling, memory management, file system, device drivers and more) all run in kernel mode in one shared address space and call each other directly.

Review questions with answers

1. Where was UNIX first developed, and on which machine was it running by 1970?

- A. At UC Berkeley, on a DEC VAX
- B. At IBM, on a System/360 mainframe
- C. At Bell Labs, on a DEC PDP-7
- D. At MIT, on the Multics computer

Answer: C. At Bell Labs, on a DEC PDP-7

Ken Thompson and Dennis Ritchie built UNIX at Bell Labs. It was running on a little-used PDP-7 by 1970 and then moved to the PDP-11.

2. Why is the 1973 rewrite of UNIX in C seen as a milestone?

- A. C code runs faster than hand-written assembly, so UNIX became the fastest operating system of its day
- B. It showed that a high-level language, not assembly, works for most of an OS, making it easier to understand, change and port
- C. Writing it in C removed every piece of machine-dependent code from the kernel
- D. C let UNIX run programs that had been written for any other operating system

Answer: B. It showed that a high-level language, not assembly, works for most of an OS, making it easier to understand, change and port

Before UNIX, almost every OS was written in the assembly language of one machine. The C rewrite proved a high-level language was good enough for most of an OS, so moving UNIX to a new machine meant recompiling most of it rather than rewriting it.

3. Put these events in the order they happened.

Correct order:

- 1. UNIX runs on a PDP-7 at Bell Labs
- 2. UNIX moves to the PDP-11
- 3. The kernel is rewritten in C
- 4. Version 6 is licensed widely to universities
- 5. AT&T releases System V
- 6. SVR4 merges System V with popular BSD features

PDP-7 (1970), PDP-11 (1971), C rewrite (1973), Version 6 (1975), System V (1983), SVR4 (1989).

4. Which branch of the UNIX family does each item belong to?

AT&T line (System III / V)	Berkeley line (BSD)
System III, the first commercial release Licensed to computer makers, leading to AIX and HP-UX SVR4, which later absorbed BSD features	TCP/IP networking built into the kernel Virtual memory with paging on the VAX Developed at the University of California Lives on in FreeBSD and inside macOS

AT&T sold System III, System V and later SVR4 to companies. Berkeley's releases added virtual memory (3BSD) and TCP/IP (4.2BSD), and continue today in FreeBSD and macOS.

5. Order the layers of a UNIX system from the hardware up to the user.

Correct order:

1. Hardware
2. Kernel
3. System call interface
4. Commands and libraries
5. User-written applications

The kernel sits directly on the hardware. The system call interface is its boundary with everything above: commands and libraries (shells, compilers, utilities, the C library) and then the applications people write.

6. The shell is part of the UNIX kernel, because it runs every command you type.

Answer: False

The shell is an ordinary user-mode program in the commands-and-libraries layer. It asks the kernel to start commands through system calls, as any program could.

7. A user program can reach UNIX kernel services only by calling a library routine.

Answer: False

A program may make a system call directly, or call a library routine that makes the call for it. Either way the request enters the kernel through the system call interface.

8. Which part of the traditional UNIX kernel contains each component?

Process control subsystem	File subsystem
Memory management Scheduling and dispatching Synchronization and interprocess communication	Buffer cache Character device drivers Block device drivers

Process control manages processes: their memory, their turns on the processor and their coordination. The file subsystem moves data between memory and devices, in blocks through the buffer cache or as character streams.

9. Match each part of the traditional kernel with its job.

System call interface	Receives each trap and passes the call to the right subsystem
Buffer cache	Keeps copies of recently used disk blocks in main memory
Scheduler	Chooses which ready process runs next
Hardware control	Handles interrupts and talks to the machine directly
Block device driver	Moves whole blocks between a disk and memory

The system call interface is the way in; the buffer cache and block drivers serve the file subsystem; the scheduler belongs to process control; hardware control is the machine-dependent bottom layer.

10. A program reads a disk block that is already in the buffer cache. Which part is **not** needed for this read?

- A. The system call interface
- B. The file subsystem
- C. The buffer cache
- D. The block device driver

Answer: D. The block device driver

On a cache hit the data is already in memory, so the kernel never needs the disk driver, the disk or an interrupt, and the process never has to sleep.

11. Which statements describe the traditional UNIX kernel?

- A. It is monolithic: one large program in one address space
- B. It is not very modular
- C. Each device driver runs as a separate user-mode server process
- D. Adding a new device driver usually meant rebuilding the kernel and rebooting
- E. A bug in any part of it can crash the whole system

Answer: A, B, D, E

All the parts are compiled into one program that runs in kernel mode. Drivers live inside it, not in separate servers (that is the microkernel idea), so a change means a rebuild and a bug anywhere can bring everything down.

12. A kernel has 10,000 lines of code, of which 1,000 are machine-dependent. It is written in C. How many lines must be rewritten by hand to port it to 3 new machines?

Answer: 3000 lines

Only the machine-dependent part is rewritten for each machine: $3 \times 1,000 = 3,000$ lines; the other 9,000 lines are simply recompiled. Written in assembly, the port would need $3 \times 10,000 = 30,000$ lines.

2.9 Modern Unix Systems

You should be able to

- Explain why the spread of many incompatible UNIX versions pushed designers toward a modular kernel: a small core of common facilities with clean interfaces where new parts plug in.
- Name the six plug-in points of a modern UNIX kernel (exec switch, virtual memory framework, vnode/vfs interface, block device switch, scheduler framework, STREAMS) and say what plugs into each.
- Describe System V Release 4: who built it, the four systems it combined, and the six major features it introduced.
- Trace the BSD line from 4.xBSD to FreeBSD, NetBSD, OpenBSD and macOS, and state BSD's lasting contributions.
- Describe Solaris 11 and its key features: a fully preemptable multithreaded kernel, full SMP support and an object-oriented file-system interface.

Why UNIX needed a new kernel design

AT&T licensed the UNIX source widely, so many versions appeared: Berkeley's BSD (1978) and Microsoft's Xenix (1980), then System V, SunOS, Ultrix and HP-UX by the mid-1980s, and AIX and IRIX by 1989. Each group edited the kernel directly, so code moved between versions only with work and every feature made the kernel harder to change.

The answer was a **modular kernel**: a small core of **common facilities** (system-call entry, processes and threads, interrupts, locking, kernel memory) with modules that connect through fixed interfaces, like appliances in standard wall outlets. The usual mechanism is a **switch table**: one row of function pointers per implementation, which the core calls without knowing the details. The modules still run in kernel mode inside the kernel, so this is not a microkernel.

Traditional versus modern kernel

In a traditional kernel one change spreads: a new file system meant editing system calls, file handling, the buffer cache and the program loader; a new program format meant rewriting exec and memory management; real-time scheduling meant editing the scheduler, clock and sleep code. In a modern kernel each is one new module: NFS behind vnode/vfs, an ELF loader in the exec switch, a real-time class. Drivers were the exception: even early UNIX reached them through a switch table (the block device switch), and modern UNIX applied that trick everywhere. A change kept inside one module is smaller, safer and testable alone.

The six plug-in points of a modern UNIX kernel

Interface	What it does	What plugs in
exec switch	Starts programs: exec offers the program file to each loader until one recognizes its header	a.out (original UNIX format), COFF (Common Object File Format, System V before SVR4), ELF (Executable and Linkable Format, from SVR4; standard today)
Virtual memory framework	An address space is a list of mappings; on a page fault the core calls the faulting mapping's own routine	File mappings (pages from a file, e.g. program code, mmap), device mappings (a window onto device memory such as a frame buffer), anonymous mappings (heap and stacks; start as zeros, go to swap)
vnode/vfs interface	Lets many file-system types coexist; system calls use vnodes, each pointing to its file system's operations; each mounted file system is a vfs object	NFS (Sun's network files), FFS (Berkeley Fast File System), s5fs (System V file system), RFS (AT&T Remote File Sharing)
Block device switch	Table indexed by major device number; the minor number picks the unit	Disk and tape drivers
Scheduler framework	The dispatcher runs the highest-priority ready process; each scheduling class sets its members' priorities	Time-sharing class (ordinary processes, priorities move with behaviour), system class (kernel processes, fixed higher priorities); SVR4 adds real-time
STREAMS	Character I/O as a chain: stream head, optional modules, driver; modules can be pushed and popped while open	Network drivers (with protocol modules above), tty (terminal) drivers

Examples: an ELF program starts → exec switch; first touch of a heap page → VM (anonymous); a remote file → vnode/vfs (NFS); writing a disk block → block device switch; the page-out daemon wakes → scheduler (system class); a key press or network packet → STREAMS.

Adding a file system without touching the core

1. **Load** the module (e.g. pcfs for FAT USB sticks); it registers its type name, vfs operations and vnode operations with VFS.
2. **Mount** the device: VFS calls the module's mount operation and creates a vfs object. Without step 1: "unknown file system type".
3. **Open**: VFS walks the path across the mount point to the file's vnode (v_op points to the module's operations) and returns a descriptor for it. Without step 2 the path is an empty directory on the root file system, so open fails.
4. **Read** (on the descriptor): the vnode's read operation (pcfs_read) fetches blocks from the device.

Core code changed: zero lines. Every file system implements one common set of operations, an object-oriented design.

System V Release 4 (SVR4)

Released in 1989 by AT&T and Sun Microsystems: an almost total rewrite of the System V kernel that merged four systems, drawing on commercial work (SVR3, Xenix, SunOS) and academic work (4.3BSD). It aimed to be one uniform commercial UNIX for machines of every size.

Ingredient	Main contributions
SVR3 (AT&T, 1987)	System V trunk and interfaces; STREAMS; RFS
4.3BSD (Berkeley, 1986)	TCP/IP with sockets; FFS; C shell, job control
Xenix System V (Microsoft)	UNIX on Intel PCs; Xenix programs kept running
SunOS (Sun)	vnode/vfs; NFS; file-mapping virtual memory; shared libraries

Six major new features

1. **Real-time processing support:** a class with fixed priorities above all others.
2. **Process scheduling classes:** 160 priorities; time-sharing 0 to 59, system 60 to 99, real-time 100 to 159.
3. **Dynamically allocated data structures** instead of fixed-size tables set at build time (a full table blocks new processes despite free memory; an empty one wastes it).
4. **Virtual memory management:** mappings of files, devices and anonymous memory.
5. **Virtual file system:** vnode/vfs.
6. **Preemptive kernel:** preemption points where a long kernel path can switch to a more urgent process.

BSD and its descendants

The Berkeley Software Distribution began in 1978. 3BSD (1979) added paged virtual memory; 4.2BSD (1983) built TCP/IP into the kernel with the socket interface and introduced the Fast File System; 4.3BSD (1986) refined it; **4.4BSD** (1993 to 1994) was Berkeley's final release, and its AT&T-free version, 4.4BSD-Lite, became the base of the free BSDs:

- **FreeBSD** (1993): performance and ease of use, widely used on servers; a major source of the BSD code in macOS.
- **NetBSD** (1993): portability to a very wide range of hardware.
- **OpenBSD** (1996): split from NetBSD; security and audited code (OpenSSH).
- **macOS:** built on **Darwin**, whose kernel combines the **Mach microkernel** (Carnegie Mellon) with FreeBSD code, by way of NeXTSTEP (Mach + 4.3BSD).

SunOS was also BSD-based, which is how BSD ideas reached SVR4 and Solaris.

Solaris 11

Sun moved from SunOS to an SVR4 base with Solaris 2 (1992). Oracle bought Sun in 2010 and released Solaris 11 in 2011, a leading commercial UNIX. Features: a **fully preemptable, multithreaded kernel** (kernel work runs as threads and can be preempted almost anywhere); **full SMP support** (all processors are peers; many run kernel code at once, each holding only the small locks it needs); an **object-oriented file-system interface** (vnode/vfs).

Worked examples

Locks. Four CPUs each need 2 ticks of kernel work at once. With one big kernel lock they take turns: $4 \times 2 = 8$ ticks. With per-structure locks on different data, all finish in 2.

Preemption. T1 runs 1 user tick, a 7-tick system call, then 4 user ticks; urgent R wakes at tick 2. No preemption: R waits 6 ticks. Preemption points after 3 kernel ticks (SVR4): 2 ticks. Fully preemptable (Solaris): 0. T1 always finishes at tick 14: preemption changes the order of work, not its amount (apart from a little switching cost). Without preemption, wait = call length – time already spent ($8 - 3 = 5$).

Key terms

Modular kernel

A kernel organised as a small core of shared services plus separate modules that connect to the core through fixed, well-defined interfaces, so a module can be added or replaced without rewriting the core.

Switch table

A table of pointers to functions with one row per implementation (one per device driver, per program file format, per file-system type). The core finds the right row and calls through it, so it never needs the details of any one implementation.

Exec switch

The switch table used when a process starts a new program (the exec system call). Each row is a loader for one executable file format, such as a.out, COFF or ELF, and the kernel uses the loader that recognizes the file.

Executable and Linkable Format (ELF)

The format for program and library files introduced with SVR4. It replaced the older a.out and COFF formats and is standard today on Linux, the BSDs and Solaris.

Virtual memory framework

The part of a modern UNIX kernel that manages each process's address space as a set of mappings. Each kind of mapping (file, device or anonymous) supplies its own code for bringing its pages into memory.

File mapping

A region of a process's address space whose contents come from a file. A page is read from the file the first time it is touched; program code is usually mapped this way.

Anonymous mapping

A region of memory that is not backed by any named file, such as the heap or a stack. It starts out filled with zeros and, if it must leave main memory, it is written to swap space.

Vnode

The kernel's in-memory object for one active file. It looks the same whatever file system the file lives on, and it points to that file system's table of operations (open, read, write, lookup and so on).

Virtual file system (VFS)

The layer that lets many file-system types exist side by side behind one set of file system calls. Each mounted file system is represented by a vfs object and each active file by a vnode.

Network File System (NFS)

Sun's way of using files stored on another computer across a network as if they were on a local disk. Sun created the vnode/vfs interface so NFS and local file systems could work side by side.

Block device switch

The kernel table, indexed by a device's major number, that holds the entry points of every block-device driver, such as disk and tape drivers. Together with its twin for character devices, it is one of the oldest switch tables in UNIX.

Scheduling class

A group of processes that share one scheduling policy and one band of priorities. SVR4 has a time-sharing class, a system class for kernel processes and a real-time class, each plugged into the scheduler framework.

STREAMS

A framework for character I/O such as terminals and network connections. Data flows through a chain of modules between a process and a driver, and modules can be pushed onto or popped off the chain while it is in use.

System V Release 4 (SVR4)

The UNIX release of 1989, developed jointly by AT&T and Sun Microsystems. It merged SVR3, 4.3BSD, Microsoft Xenix System V and SunOS on an almost completely rewritten kernel.

Berkeley Software Distribution (BSD)

The versions of UNIX produced at the University of California, Berkeley, ending with 4.4BSD. BSD added paged virtual memory and TCP/IP networking with sockets, and its code lives on in FreeBSD, NetBSD, OpenBSD and macOS.

Socket

The programming interface, introduced by BSD, through which a program opens a network connection and sends and receives data. It became the standard way programs use TCP/IP.

Preemptive kernel

A kernel that can take the processor away from a process even while that process is running kernel code, so an urgent process does not have to wait for a whole system call to finish.

Multithreaded kernel

A kernel whose own work is carried out by many kernel threads that can run at the same time on different processors, each piece of shared kernel data protected by its own small lock rather than one lock for everything.

Symmetric multiprocessing (SMP)

A design in which two or more similar processors share main memory and devices, and any processor can run any work, including the kernel itself.

Darwin

The open-source core of Apple's macOS. Its kernel joins the Mach microkernel, developed at Carnegie Mellon University, with a large body of code taken from FreeBSD.

Review questions with answers

1. Why did UNIX designers move to a kernel with a small core and plug-in interfaces?
- A. Processors of the time were too slow to run one large, complex kernel
 - B. It let every file system and driver run as a separate user-mode process
 - C. Each version had patched one big kernel its own way, so features were hard to add
 - D. AT&T had stopped licensing the UNIX source code to other groups

Answer: C. Each version had patched one big kernel its own way, so features were hard to add

Vendors and universities had patched the same monolithic code in different ways. A small core with clean interfaces lets new file systems, formats, drivers and policies plug in without rewriting the core.

2. Match each part of a modern UNIX kernel with what plugs into it.

exec switch	a.out, COFF and ELF loaders
Virtual memory framework	File, device and anonymous mappings
vnode/vfs interface	NFS, FFS, s5fs and RFS
Block device switch	Disk and tape drivers
Scheduler framework	Time-sharing and system classes
STREAMS	Network and terminal (tty) drivers

Each interface is a socket for one kind of variety: program formats, memory sources, file systems, block devices, scheduling policies and character streams.

3. Which system contributed each feature to SVR4?

4.3BSD	SunOS	SVR3
TCP/IP networking with sockets Fast File System (FFS)	Network File System (NFS) vnode/vfs file-system interface	STREAMS Remote File Sharing (RFS)

Berkeley brought networking and FFS; Sun brought NFS, vnode/vfs and its virtual memory design; AT&T's SVR3 brought STREAMS and RFS.

4. Which of these were major new features of SVR4?
- A. Real-time processing support
 - B. Process scheduling classes
 - C. Dynamically allocated kernel data structures
 - D. A preemptive kernel
 - E. Fixed-size kernel tables chosen when the kernel is built
 - F. Every device driver moved into user space

Answer: A, B, C, D

SVR4 also brought new virtual memory management and a virtual file system. It replaced fixed-size tables with dynamic allocation, and its drivers still ran inside the kernel.

5. Who developed System V Release 4 (SVR4)?

- A. UC Berkeley and Sun Microsystems
- B. AT&T and Sun Microsystems
- C. AT&T and Microsoft
- D. AT&T and IBM

Answer: B. AT&T and Sun Microsystems

AT&T and Sun built SVR4 together and released it in 1989.

6. 4.4BSD was the final BSD release from the University of California, Berkeley.

Answer: True

Berkeley's research group closed soon after; FreeBSD, NetBSD and OpenBSD carried the code forward.

7. Apple's macOS is built on Darwin. What does Darwin's kernel combine?

- A. The Linux kernel and code from NetBSD
- B. The SVR4 kernel and Microsoft Xenix
- C. The Solaris kernel and the Mach microkernel
- D. The Mach microkernel and code from FreeBSD

Answer: D. The Mach microkernel and code from FreeBSD

Darwin joins the Mach microkernel from Carnegie Mellon with a large layer of FreeBSD code, a path that runs through NeXTSTEP.

8. Which statements describe Solaris 11?

- A. It has a fully preemptable, multithreaded kernel
- B. It fully supports symmetric multiprocessing (SMP)
- C. It has an object-oriented interface to file systems
- D. It is based on the BSD kernel rather than SVR4
- E. It was released by Oracle
- F. It runs on only one processor at a time

Answer: A, B, C, E

Solaris 11 (2011) is Oracle's SVR4-based UNIX with a fully preemptable multithreaded kernel, full SMP support and the vnode/vfs file-system interface.

9. On a single processor, a kernel that cannot be preempted starts a system call needing 8 ticks of kernel work, and the call never blocks. A real-time process wakes up 3 ticks after the call began. How many ticks does the real-time process wait before it can run?

Answer: 5 ticks

The call must run to completion: $8 - 3 = 5$ more ticks. A fully preemptable kernel could switch to the real-time process almost at once.

10. Four CPUs each need 2 ticks of kernel work at the same moment, and one big lock guards the whole kernel. How many ticks pass until the last CPU finishes its kernel work?

Answer: 8 ticks

Only one CPU at a time may hold the lock: $4 \times 2 = 8$ ticks. With fine-grained locks on different data, all four could finish in 2.

11. A new file-system module is added to a running system, and a program then reads a file from it. Put the steps in order.

Correct order:

1. The module is loaded and registers its operations with the VFS layer
2. The device is mounted, creating a vfs object for that file system
3. The program opens the file and the VFS layer finds the file's vnode
4. The read goes through the vnode to the module's own read routine
5. The module fetches the data from its device

Register, mount, look up the vnode, call through the vnode's operation table, then the module talks to its device. The core never changes.

12. In a modern UNIX kernel, supporting a new executable file format means rewriting the exec system call.

Answer: False

You add one loader to the exec switch. exec simply offers the file to each loader until one recognizes it.

2.10 Linux

You should be able to

- Tell the story of Linux: who started it, when, on what hardware, and why a free license and the GNU tools let it succeed.
- Explain why Linux counts as a monolithic kernel even though it is built from loadable modules, and compare it with a microkernel.
- Describe the two key properties of Linux modules, dynamic linking and stacking, and predict when the kernel refuses to load or unload a module.
- Read an entry of the module table: name, size, usecount, flags, exported symbols and dependencies.
- Name the main components of the Linux kernel and trace which ones handle a key press, an arriving network packet and a file read.

Linux in brief

Linux is a free, open-source, UNIX-like operating-system kernel: it follows the design and commands of UNIX but contains no UNIX code. It is **monolithic, yet built from loadable modules**, and it runs on routers, phones, laptops, cloud servers and every one of the world's 500 fastest supercomputers. Its winning traits: free and open source, highly modular, easily configured, and able to run on many platforms.

History

- **1983–1985:** Richard Stallman launches the **GNU project** (a complete, free UNIX-like system) and founds the **Free Software Foundation (FSF)**. By 1991 GNU has a compiler (GCC), a shell (bash), a C library, editors and utilities, but no finished kernel.
- **1991: Linus Torvalds**, a computer science student at the University of Helsinki in Finland, writes a UNIX-like kernel for his PC with a 32-bit **Intel 80386** processor and posts it on the Internet. Volunteers worldwide join in.

- **1992:** Linux is released under the **GNU GPL**. With the GNU tools it forms a complete free system, and the first distributions (kernel plus tools, ready to install) appear.
- **1994:** version 1.0 for 386 PCs, with networking. **Mid 1990s:** loadable modules; ports to Alpha, SPARC and MIPS; version 2.0 (1996) uses several processors.
- **2000s:** servers, paid company developers, embedded devices. **2008:** Android phones ship with a Linux kernel (section 2.11). **Today:** a release every nine or ten weeks, thousands of developers, about twenty processor families.

Why Linux succeeded

- **Free software under the GPL:** anyone may run, study, change and share the code. Whoever distributes it, changed or not, must pass on the same freedoms and make the source available. This rule is **copyleft**: improvements can never be locked away. "Free" means freedom, not zero price.
- **The GNU tools** supplied every layer except the kernel, so Linux completed a usable system (hence "GNU/Linux" for the whole system; strictly, Linux is only the kernel).
- Collaboration over the Internet, and a modular, portable design.

GPL cases. Allowed: reading the source, changing it privately without distributing it, selling copies (with the source). Not allowed: shipping a device with a changed kernel while hiding the changes, or putting GPL code into a program under a license that forbids sharing.

Monolithic, microkernel and Linux

A **monolithic kernel** runs all OS services (scheduling, memory, file systems, drivers, networking) as one big program in kernel mode, in one address space, and its parts call each other directly. A **microkernel** keeps only message passing, address spaces and basic scheduling in the kernel and runs file systems and drivers as user-mode server processes. **Linux is monolithic, but structured as a collection of loadable modules.** Modules do not make it a microkernel: a loaded module shares the kernel's address space and privileges, so a buggy one can crash the system (an "oops" may kill only the current process; a "kernel panic" stops everything).

	Monolithic	Microkernel	Linux
Request speed	Fast: direct calls	Slower: messages	Fast: direct calls
A driver bug	Can crash everything	Contained; server restarted	Can crash everything
Adding a driver	Rebuild and restart	Start a server process	Load a module, no restart

Worked example. Reading a file in a monolithic kernel (or Linux) costs 2 user/kernel crossings (the system call in, the return out) and 0 messages. In a microkernel where the app asks a file server, which asks a disk-driver server, there are 2 requests + 2 replies = **4 messages**, each going into and out of the kernel: **8 crossings**.

Loadable modules

A **loadable module** is a relatively independent block of kernel code that usually does one job: a file system (vfat), a device driver (usb_storage) or a network protocol (bluetooth). It can be loaded and unloaded while

the system runs, and it is **not a separate process**: its code runs in kernel mode on behalf of the current process, for example during that process's system call. Two key properties:

1. **Dynamic linking.** A module can be loaded and linked into the kernel while the kernel is in memory and running, and unlinked and removed at any time. Loading: read the .ko file from disk → copy its code and data into kernel memory → give each name it calls but does not contain its real address from the kernel **symbol table** → enter it at the front of the module table with its dependencies → run its init function. Unloading runs its exit function, removes its symbols, unlinks its entry and frees its memory.
2. **Stackable modules.** Modules form a hierarchy: a lower module is a library for the client modules above it (usb_storage and xhci_hcd use usbcore; uas uses usb_storage; vfat uses fat), and the kernel counts these references. Shared code therefore exists only once, and the kernel can load prerequisites first and refuse to remove a module that others still need.

The module table

The kernel keeps its loaded modules in a **linked list**, newest first. Each entry records:

Field	What it records
next	Address of the next entry (NULL at the end of the list)
name	The module's name; rmmod and lsmod find entries by it
size	Kernel memory its code and data occupy, freed on unload
usecount	Operations using it right now: +1 when one starts, -1 when it ends
flags	Its state: loading (init still running), live, or going (being removed)
symbol table (syms, nsyms)	The names it exports, each with its address
dependencies (deps, ndeps)	The modules it relies on; the entry also lists the modules that rely on it (refs)

- **rmmod is refused** if the usecount is above 0 **or** another loaded module depends on the target. Usecount 0 alone is not enough: with usb_storage loaded, `rmmod usbcore` fails ("in use by: usb_storage").
- Usecount example: start at 0, five operations begin and two end, so the usecount is $0 + 5 - 2 = 3$.
- `insmod` loads exactly one module and fails with "Unknown symbol" if a module it needs is missing. `modprobe` loads prerequisites first (for uas: usbcore, usb_storage, uas); the system runs it automatically when you plug in a device.
- Modern kernels merge the usecount and the references from other modules into one counter (the "Used by" number in lsmod); the rules are the same.

Kernel components

Component	Job
System calls	The entry points through which processes ask the kernel for services
Signals	Notify a process of an event (Ctrl+C sends SIGINT)
Processes and scheduler	Create and end processes and threads; choose which runs next
Virtual memory	Give each process its own address space, mapping its pages onto page frames
File systems	Turn disk blocks into files and folders (ext4, vfat and more)
Network protocols	Carry out IP, TCP and other protocols for sockets
Character device drivers	Byte-stream devices: keyboards, terminals, mice
Block device drivers	Fixed-size-block devices: disks, SSDs, USB sticks
Network device drivers	Move packets to and from the network interface controller
Traps and faults	Handle exceptions the processor raises itself (page fault, divide by zero)
Physical memory	Track and hand out RAM's page frames (fixed-size slots, typically 4 KB)
Interrupts	Run the handler when a device raises an interrupt

Above the kernel sit user-level processes and threads; below it the hardware: CPU, system memory, terminal, disk and network interface controller (NIC). File systems, network protocols and device drivers are the parts most often loaded as modules.

- **Key press:** interrupt → character (terminal) driver buffers the byte → scheduler wakes the shell → its read() returns. Ctrl+C instead sends a SIGINT signal.
- **Packet arrives:** NIC copies it to memory by DMA and interrupts → network device driver takes it first → IP and TCP → socket buffer → scheduler wakes the browser → recv() returns.
- **File read:** read() → file system (is it cached?) → a page frame is set aside → block driver asks the disk → another process runs meanwhile → disk interrupt → a page fault may bring in the buffer's page → read() returns.

Key terms

Linux

A free, open-source, UNIX-like operating-system kernel started by Linus Torvalds in 1991. The name is also used loosely for whole systems built around that kernel.

Open source

Software whose source code is published so that anyone may read it, change it and share it, under the terms of its license.

GNU General Public License (GPL)

The free-software license Linux uses. Anyone may run, study, change and share the code, but whoever passes a copy on, changed or not, must pass on the same freedoms and make the source code available.

Copyleft

A license rule that uses copyright to keep software free: any version you distribute must carry the same license, so nobody can turn shared code into a closed product.

Free Software Foundation (FSF)

A nonprofit founded in 1985 by Richard Stallman to promote software that users are free to run, study, change and share. It supports the GNU project and wrote the GPL.

GNU project

An effort begun in 1983 to build a complete, free, UNIX-like operating system. By 1991 it had a compiler, a shell, a C library, editors and utilities, but no finished kernel. GNU stands for "GNU's Not Unix".

Monolithic kernel

A kernel that holds almost the whole operating system (scheduling, memory management, file systems, networking, device drivers) as one large program that runs in kernel mode in one shared address space, so its parts can call each other directly.

Microkernel

A small kernel that keeps only the essential core (address spaces, interprocess communication, basic scheduling) and runs every other OS service as a server process in user mode.

Loadable module

A relatively independent block of kernel code, usually doing one job such as a device driver, a file system or a network protocol, that can be loaded into and removed from the kernel while the system runs. It is not a separate process: it runs in kernel mode on behalf of the current process.

Dynamic linking

Connecting a module to the kernel while the kernel is in memory and running: every name the module uses is looked up and replaced by its real address. The module can later be unlinked and removed, all without a reboot.

Stackable modules

Modules arranged in a hierarchy in which a lower module acts as a library for the client modules above it. The kernel counts these references, so it can load prerequisites first and refuse to remove a module that others still need.

Module table

The kernel's linked list of loaded modules. Each entry records the module's name, size, usecount, flags, the symbols it exports and the modules it depends on.

Usecount (use count)

A counter in a module's table entry. It goes up when an operation that uses the module's functions starts and down when that operation ends. The module cannot be unloaded while it is above zero.

Symbol table (exported symbols)

The list of names (functions and variables) that the core kernel and loaded modules make available to other modules, each with its address in memory. A newly loaded module is linked against it.

Signal

A short software notice the kernel delivers to a process to report an event, such as Ctrl+C being pressed (SIGINT) or a child process ending. The process may handle it, ignore it, or be stopped by it.

Traps and faults

Events the processor raises by itself while running an instruction, such as a page fault, a divide by zero or an illegal instruction. Like an interrupt, each one switches the processor into kernel mode to run a handler.

Character device

A device that sends or receives data as a stream of bytes, one after another, such as a keyboard, a terminal or a mouse. A character device driver operates it.

Block device

A device, such as a hard disk, an SSD or a USB flash drive, that stores data in fixed-size numbered blocks. A block device driver operates it.

Network interface controller (NIC)

The hardware that connects a computer to a network, such as an Ethernet port or a Wi-Fi chip. It sends and receives packets and raises an interrupt when one arrives.

System call

A request from a running program to the kernel for a service, such as reading a file. A special instruction switches the processor into kernel mode and enters the kernel at a fixed, pre-arranged point.

Review questions with answers

1. Linux was first written for which kind of computer?

- A. A DEC PDP-7 minicomputer
- B. A Cray supercomputer
- C. A PC built around Intel's 80386 processor
- D. An ARM-based smartphone

Answer: C. A PC built around Intel's 80386 processor

In 1991 Linus Torvalds, a computer science student in Finland, wrote a UNIX-like kernel for his own IBM-compatible PC with an Intel 80386 processor and shared it on the Internet.

2. In the early 1990s, what did the GNU project and Linux each contribute to a complete free operating system?

- A. Linux had the tools; GNU supplied the kernel
- B. GNU had the compiler, shell, C library and utilities; Linux supplied the missing kernel
- C. GNU wrote the GPL specially for Linux once Linux was finished
- D. Nothing: Linux was a complete system on its own from day one

Answer: B. GNU had the compiler, shell, C library and utilities; Linux supplied the missing kernel

The FSF's GNU project had built almost every layer of a UNIX-like system except a finished kernel. Linux filled that gap, which is a big part of why it succeeded.

3. Under the GNU GPL, a company that sells a device running a modified Linux kernel must make the source code of its kernel changes available to the people who buy the device.

Answer: True

Distributing GPL software, changed or not, brings the copyleft duty: recipients must be able to get the source code, including the changes, under the same license.

4. Which of these is **not** stored in a module's entry in the Linux module table?

- A. The module's usecount
- B. The symbols the module exports, with their addresses
- C. The modules it depends on
- D. The ID of the process in which the module runs

Answer: D. The ID of the process in which the module runs

A module is not a process, so it has no process ID: its code runs in kernel mode on behalf of whichever process calls it. An entry holds the name, size, usecount, flags, exported symbols and dependencies, plus a pointer to the next entry.

5. Which statements about a Linux loadable module are true? Select all that apply.

- A. It runs in kernel mode
- B. It can be loaded and unloaded while the system is running
- C. It runs as a separate process with its own address space
- D. It usually implements one function, such as a driver, a file system or a network protocol
- E. It communicates with the rest of the kernel only by sending messages

Answer: A, B, D

A module is linked into the kernel and runs in kernel mode on behalf of the current process. It is not a process of its own, and it uses ordinary function calls, not messages. That is why modules do not turn Linux into a microkernel.

6. Put the life of a Linux loadable module into the correct order, from loading to unloading.

Correct order:

1. The module file (.ko) is read from disk
2. Its code and data are copied into kernel memory
3. Each unresolved name gets its real address from the kernel symbol table
4. Its init function runs and registers the module
5. Operations call its functions, so its usecount rises above 0
6. Its usecount falls back to 0, and `rmmod` unloads it

Each step needs the one before it: code must be in memory before it can be linked, linked (every call pointing at a real address) before init can safely run, registered before anyone can use it, and idle before it can be removed.

7. In the Linux module table, a module whose usecount is 0 can always be unloaded with `rmmod`.

Answer: False

The usecount only counts operations using the module right now. The kernel also checks the modules stacked on it: with `usb_storage` loaded, `rmmod usbcore` is refused ("in use by: `usb_storage`") even at usecount 0, because `usb_storage` calls `usbcore`'s functions.

8. A module's usecount is 0. Five operations that use its functions start, and then two of them finish. What is its usecount now?

Answer: 3

The usecount rises by 1 when an operation using the module starts and falls by 1 when it ends: $0 + 5 - 2 = 3$. The module cannot be unloaded until the count is back to 0.

9. In a microkernel system, an app sends a read request to a user-mode file server, which sends its own request to a user-mode disk-driver server. Each request gets a reply. How many messages are passed in total?

Answer: 4

Two requests (app → file server, file server → driver) plus two replies (driver → file server, file server → app) make 4 messages, each a trip into and out of the kernel. A monolithic kernel such as Linux does the same work with direct function calls.

10. Match each Linux kernel component to its job.

Signals	Tell a process that an event happened, such as Ctrl+C
Processes and scheduler	Decide which ready process runs next
Virtual memory	Map each process's addresses onto real page frames
Traps and faults	Handle errors the processor itself detects, such as a page fault
Block device drivers	Read and write fixed-size blocks on disks
Network protocols	Carry out rules such as IP and TCP

Each component owns one kind of work. Signals and traps both deal with events, but signals are notices sent to processes, while traps are raised by the processor itself.

11. Which kernel design does each statement describe?

Monolithic / Linux	Microkernel
All services share one address space A buggy driver can crash everything Parts call each other directly	Drivers run as user-mode processes A request needs several messages A crashed driver can be restarted

Monolithic kernels, Linux included, trade isolation for speed: one address space and direct calls.

Microkernels trade speed for isolation: services are separate processes that talk by messages.

12. A network packet arrives and the network card raises an interrupt. Which kernel component takes the packet from the card first?

- A. Network device drivers
- B. Network protocols
- C. File systems
- D. Signals

Answer: A. Network device drivers

The interrupt handler belongs to the card's network device driver, which collects the packet and passes it up to the network protocols. Only then is the waiting process woken.

2.11 Android

You should be able to

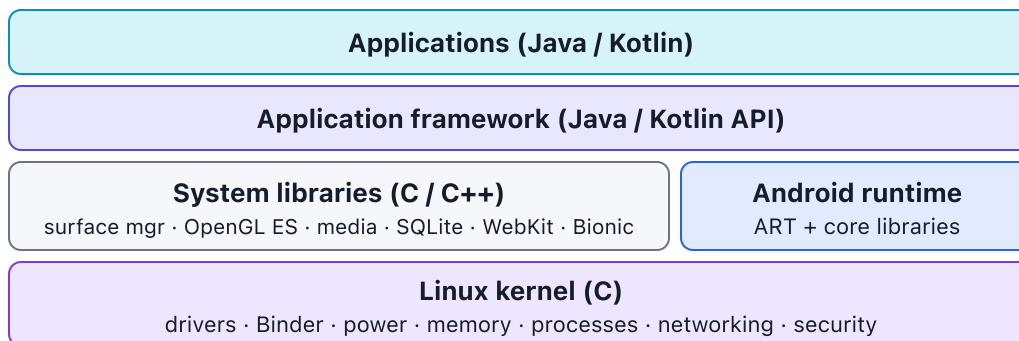
- Describe what Android is, where it came from, and the range of devices it runs on today.
- Name the five layers of the Android software stack (applications, application framework, system libraries, Android runtime, Linux kernel) and say what each provides.
- Trace a request from an app through Binder IPC, the Android system services and the hardware abstraction layer down to a Linux device driver.
- Compare the Dalvik virtual machine with ART, explaining just-in-time versus ahead-of-time compilation and their trade-offs.
- Explain activities and the back stack, and how alarms and wakelocks let Android sleep to save battery without missing work.

What Android is

Android is an operating system built on the **Linux kernel**, first designed for touchscreen mobile devices such as phones and tablets. It is the world's most widely used mobile OS. It began at a small start-up, **Android Inc.**, which **Google bought in 2005**. In 2007 Google and the **Open Handset Alliance** (phone makers, chip makers, carriers and software firms) announced it as an open platform; in 2008 the first Android phone went on sale and the **open-source** code was released. The same core now also runs TVs, cars, watches (Wear OS) and embedded or Internet-of-Things devices.

The software stack: five layers

This is the classic layout Android was designed with. Two of its parts belonged to early Android only: the XMPP service (removed early on) and the WebKit browser engine (later replaced by a Chromium-based web view).



Layer	What it provides
Applications	The programs users open: home screen, dialer, contacts, SMS, email, calendar, maps, browser and everything installed later. Each app runs in its own Linux process under its own user ID (a sandbox). Most built-in apps can be replaced.
Application framework	The API developers program against. Activity manager : activities and the back stack. Window manager : arranges windows. Package manager : installs apps, records permissions. Telephony manager : calls, network, SIM. Content providers : share one app's data with others. Resource manager : strings and layouts for the language and screen. View system : buttons, lists, text boxes. Location manager : device position. Notification manager : status-bar alerts. XMPP service (Extensible Messaging and Presence Protocol; early Android only): pushed server messages to apps over one shared connection.
System libraries	Native C/C++ code: surface manager (combines all apps' surfaces into the screen image), OpenGL ES (graphics), media framework (codecs), SQLite (in-app SQL database stored in one file), WebKit (browser engine in early Android; later replaced by a Chromium-based web view), Bionic libc (Android's small C library).
Android runtime	Executes app code: ART (formerly the Dalvik VM) plus the core Java libraries . Each app has its own runtime instance.
Linux kernel	Owns the hardware: device drivers, the Binder IPC driver , power management (with Android's alarms and wakelocks), memory, process management, networking and security (per-app user IDs).

Rule of thumb for placing a component: things users open are applications; services apps call through the API are framework; native C/C++ helpers are libraries; whatever executes app code is the runtime; anything that touches hardware or needs kernel mode is in the kernel.

System architecture: how a request travels

Seen as running processes, Android has five tiers: **applications and framework** → **Binder IPC** → **Android system services** → **hardware abstraction layer (HAL)** → **Linux kernel**.

- **Binder IPC** is Android's main interprocess communication mechanism. Framework objects inside an app are often only **proxies**; calling them sends a Binder transaction that the kernel's Binder driver delivers to another process, stamped with the caller's identity (Linux user ID).
- **Android system services** are trusted processes that own shared resources: mainly the **system server** (activity, window, power, package, location and other manager services) and the **media server** (camera, audio and media services). They check permissions and share each device among apps.
- The **HAL** is a set of standard interfaces (camera, audio, GNSS/GPS, graphics, sensors) that each vendor implements for its own chips, so the services above are identical on every device.

Example, taking a photo: Camera app → framework Camera API (proxy) → Binder → camera service (checks the camera permission) → camera HAL (vendor module) → camera driver in the kernel → image sensor; the image returns up the same path. Music and location requests work the same way. Apps never touch drivers directly.

From source code to machine code: Dalvik and ART

Developers write Java or Kotlin. A compiler produces **.class** bytecode, and a dex tool converts it into **.dex (Dalvik executable)** format, packed into the app package (APK). One .dex file holds all of an app's classes and shares their repeated strings and constants, so it is compact and quick to load. **Bytecode** is portable code that no processor runs directly: an **interpreter** performs it one instruction at a time on every run (slow), while a **compiler** translates it into machine code once. The runtime must still turn .dex bytecode into machine code; the question is **when**.

- **Dalvik VM (Android 1.0 to 4.4)**: interprets the bytecode, and from Android 2.2 a **just-in-time (JIT)** compiler turns busy code into machine code, work that is repeated on every launch.
- **ART with ahead-of-time (AOT) compilation (optional in 4.4, standard in 5.0 and 6.0)**: the whole app is compiled to native machine code at install time and stored. Launches are faster and each run needs less processor work, so **battery life improves**. The costs: **longer installs, more storage**, and every app had to be recompiled after a system update.
- **ART hybrid (Android 7.0 on)**: early runs use ART's faster interpreter and JIT while it records the hot code (a profile); that code is compiled ahead of time while the phone idles on the charger. Installs stay quick and there is no long recompile after updates.

	Dalvik	ART (AOT)	ART (hybrid)
Install time	short	long	short
Storage for code	small	large	medium
Launch speed	slower	fast	fast once warm
Battery per launch	higher	low	low once warm
After an OS update	quick	recompile all apps	quick

Worked example (illustrative units of processor work on battery): suppose Dalvik costs 1 unit to install and 5 per launch, while AOT costs 12 to install and 1 per launch. After n launches Dalvik has used $1 + 5n$ and AOT $12 + n$. AOT is cheaper once $12 + n < 1 + 5n$, that is $n > 2.75$, so from the **third launch** on; after 10 launches the totals are 51 versus 22.

Activities and the back stack

An **activity** is a single visual user-interface component of an app: one screen with one focused job, such as an email inbox, a message reader or a compose screen. An app usually has several. The **activity manager** keeps the activities a user opens for one job (a **task**) in a **back stack** (last in, first out): starting an activity **pushes** it on top, and pressing **Back pops** the top one so the one beneath returns. Home keeps the stack in the background. One stack can hold activities from **different apps**: an email app's compose screen can launch the Gallery app's photo picker, which sits on top of it. **Example**: inbox, message, compose and picker are pushed (4); two presses of Back leave 2 (message on top of inbox). Activity states and how Android manages their processes are covered in Section 4.7.

Power management: alarms and wakelocks

A sleeping phone turns off the screen and suspends the processor, drawing almost no power. Android's goal is to sleep as much as possible without missing work. It adds two features to Linux's power management:

- **Alarms:** an app asks the framework's **AlarmManager** for work at a set time. The alarm is implemented in the Linux kernel using a hardware timer that keeps running during sleep, so it can **wake the device**; the app does its work and the device sleeps again.
- **Wakelocks:** they **prevent the system from sleeping**. An app that must keep working requests a wakelock through the framework's power manager; the device may sleep only when no wakelock is held. The kernel exposes wakelocks to user space as files (`/sys/power/wake_lock` and `wake_unlock`).

Wakelock type	Processor	Screen	Keyboard light
Partial	on	off	off
Screen dim	on	dim	off
Screen bright	on	bright	off
Full	on	bright	bright

Background work that nobody watches (a download, music playback) needs only a **partial** wakelock; the screen types are for things the user is looking at (newer Android versions steer apps toward a keep-screen-on setting for that). The classic bug is **forgetting to release** a wakelock, which keeps the phone awake doing nothing. With no wakelock, a background job freezes while the phone sleeps.

Worked example: asleep a phone uses 1% of its battery per hour; awake under a partial wakelock, 8% per hour. A wakelock left held for 6 hours overnight costs $6 \times 8 = 48$ points instead of $6 \times 1 = 6$, an extra **42 percentage points**.

Key terms

Android

A Linux-based operating system first built for touchscreen phones and tablets and now also used in TVs, cars, watches and embedded devices. It began at a start-up, Android Inc., which Google bought in 2005.

Open Handset Alliance (OHA)

A group of phone makers, chip makers, mobile carriers and software companies, led by Google, formed in 2007 to create open standards for mobile devices. It launched Android as its shared, open-source platform.

Application framework

The layer of Android that app developers program against: a set of Java/Kotlin services and classes (activity manager, window manager, content providers, notification manager and more) that give every app the same building blocks.

Activity manager

The framework service that starts and stops activities, keeps each task's back stack, and decides what the user sees when they switch apps or press Back.

Content provider

A framework component through which one app shares some of its data (such as contacts or calendar events) with other apps, using a standard query interface with permission checks.

SQLite

A small but complete SQL database engine that runs as a library inside the app that uses it and keeps each database in one ordinary file.

Surface manager

The system library that combines the drawing surfaces (windows) of every visible app, the status bar and the keyboard into the one image shown on the display.

Bionic libc

Android's own standard C library (functions such as malloc, printf and open). It is smaller and quicker to load than the GNU C library used on desktop Linux.

Dalvik virtual machine

Android's original virtual machine for app code. It ran .dex bytecode by interpreting it and, from 2010, compiling busy parts just in time. ART replaced it in Android 5.0 (2014).

Dalvik executable (.dex)

The compact bytecode format Android apps ship in. Build tools convert compiled Java or Kotlin classes into .dex files, which both Dalvik and ART can run.

Android runtime (ART)

The runtime that replaced Dalvik. It turns an app's .dex bytecode into native machine code: at first all of it at install time; since Android 7.0, by JIT while the app runs plus ahead-of-time compiling of its most-used code while the phone is idle.

Just-in-time compilation (JIT)

Translating bytecode into machine code while the program runs, usually only for the parts that run often. Unless the results are saved, the same work is redone every time the program runs.

Ahead-of-time compilation (AOT)

Translating bytecode into machine code before the program runs (on Android, when the app is installed or while the phone is idle), so no translation is needed when it launches.

Binder (Binder IPC)

Android's main interprocess communication mechanism: a Linux kernel driver plus libraries that let code in one process call a method on an object in another process as if it were a local call, with the caller's identity attached.

Android system services

Long-running Android processes that own the device's shared resources, mainly the system server (activity, window, power, location and other managers) and the media server (camera, audio, video). Apps reach them through Binder.

Android HAL

Android's hardware abstraction layer: standard interfaces (camera, audio, GPS, sensors and more) that each hardware vendor implements for its own chips, so the layers above never depend on which parts are inside a device. Unlike the Windows HAL (section 2.7), which hides a platform's wiring inside the kernel, it sits above the kernel's drivers.

Activity

One screen of an Android app with a single focused job, such as an inbox list or a compose-message screen. An app is usually made of several activities.

Back stack

The last-in, first-out list of activities the user has opened in one task. Starting an activity pushes it on top; pressing Back removes the top one and shows the one beneath.

Wakelock (wake lock)

A request by an app to keep the device from going to sleep. While any wakelock is held the processor (and, for some types, the screen) stays on; the device may sleep only when none is held.

Alarm (AlarmManager)

An Android addition to the Linux kernel that lets an app schedule work for a set time; apps request one through the AlarmManager class. The kernel programs a hardware timer that keeps running during sleep, so an alarm can wake a sleeping device.

Review questions with answers

1. Where did Android come from?

- A. Google designed it from scratch in 2007 as part of Chrome OS
- B. A start-up, Android Inc., created it, and Google bought the company in 2005
- C. The Open Handset Alliance bought it from a phone maker in 2008
- D. It is a phone edition of Windows NT that Google licensed

Answer: B. A start-up, Android Inc., created it, and Google bought the company in 2005

Android Inc. was a small start-up. Google bought it in 2005, and in 2007 Google and the Open Handset Alliance unveiled Android as an open-source, Linux-based platform.

2. A camera app takes a photo. Put the layers the request passes through in order, starting from the app.

Correct order:

- 1. App and application framework
- 2. Binder IPC
- 3. Android system service (camera service)
- 4. Hardware abstraction layer (camera HAL)
- 5. Linux kernel (camera driver)

The framework proxy in the app sends the call through Binder to the camera service, which uses the vendor's camera HAL module, which drives the camera driver in the kernel.

3. Which layer of the Android software stack provides each component?

Framework	Libraries	Runtime	Kernel
Activity manager	Surface manager SQLite	ART	Camera driver Wakelocks

Services apps call through the Java/Kotlin API are framework; native C/C++ helpers such as SQLite and the surface manager are system libraries; whatever executes app code (ART, with the core Java libraries) is the runtime; drivers and power management run in the kernel.

4. Match each application-framework component to its job.

Activity manager	Starts app screens and keeps the back stack
Content providers	Share one app's data with other apps
Package manager	Installs apps and records their permissions
Resource manager	Supplies strings and layouts for the device's language and screen
Location manager	Reports where the device is
Notification manager	Shows alerts in the status bar

Each manager owns one job and offers it to every app through the same API, which is why all apps behave consistently.

5. What is the key difference between the Dalvik virtual machine and ART?

- A. ART runs Java source code directly, while Dalvik ran machine code
- B. ART compiles an app's .dex bytecode into machine code ahead of time (originally at install), while Dalvik translated it while the app ran
- C. Dalvik compiled apps at install time, while ART compiles only while the app runs
- D. ART dropped the .dex format, so apps now arrive from the store as machine code

Answer: B. ART compiles an app's .dex bytecode into machine code ahead of time (originally at install), while Dalvik translated it while the app ran

Dalvik repeated its translation work (interpreting plus JIT) every time an app ran. ART does the translation ahead of time and stores the machine code, so launches are faster and each run costs less battery.

6. Compared with Dalvik, which were costs of ART's original compile-everything-at-install approach?

Select all that apply.

- A. Longer app installs
- B. More storage used for compiled code
- C. Slower app launches
- D. Every app had to be recompiled after a system update
- E. More battery used each time the app runs

Answer: A, B, D

Compiling everything up front takes time and space and must be redone when the OS changes. In exchange, launches are faster and each run needs less processor work, which saves battery. Later ART versions mix JIT and AOT to cut those costs.

7. A wakelock is the Android feature that wakes a sleeping phone at a scheduled time.

Answer: False

That is an alarm. A wakelock does the opposite job: while an app holds one, the phone is not allowed to go to sleep.

8. A podcast app must keep downloading after the screen turns off, and nobody is looking at the phone.

Which wakelock fits best?

- A. A full wakelock
- B. A partial wakelock
- C. A screen bright wakelock
- D. No wakelock: downloads continue while the phone sleeps

Answer: B. A partial wakelock

A partial wakelock keeps only the processor running and lets the screen turn off, exactly what background work needs. The app should release it as soon as the download finishes.

9. A user opens the Email app's inbox, opens a message, taps Reply, then taps Attach photo, which opens the Gallery app's photo picker. They then press Back twice. How many activities are left on the back stack?

Answer: 2 activities

Four activities were pushed: inbox, message, compose, picker. Each Back pops one, so $4 - 2 = 2$ remain: the message on top of the inbox.

10. A phone uses 1% of its battery per hour while asleep, but 8% per hour while a partial wakelock keeps the processor awake. An app forgets to release its wakelock for 6 hours overnight. How many extra percentage points of battery does the bug cost?

Answer: 42 points

Awake: $6 \times 8 = 48$ points. Asleep it would have used $6 \times 1 = 6$ points. The bug costs $48 - 6 = 42$ percentage points.

11. Because Android runs on Linux, an ordinary app can open the camera's device driver directly to take a picture.

Answer: False

Apps never talk to drivers. The request goes through the framework and Binder to the camera service, which uses the camera HAL, which talks to the driver. That path lets the service check permissions and share the camera among apps.

12. Why does Android put a hardware abstraction layer (HAL) between its system services and the kernel drivers?

- A. So apps can reach hardware faster by skipping the kernel
- B. So each vendor can implement standard interfaces for its own chips, keeping the services above identical on every device
- C. Because the Linux kernel cannot run device drivers
- D. To translate .dex bytecode into machine code

Answer: B. So each vendor can implement standard interfaces for its own chips, keeping the services above identical on every device

Phone makers use different chips. By writing HAL modules for standard interfaces, they plug their hardware in without changing the services, and Android can be updated without rewriting their code.

Chapter 3 · Process Description and Control

The process is the idea everything else is built on. Here is how the OS describes and controls every one.

A program on disk is just a file. Running it creates a process: a program in execution that the OS can start, pause, resume and protect. This chapter models a process's life as states and transitions, shows the tables and the process control block the OS keeps for each process, and explains how processes are created and switched. It ends with where the OS's own code runs and how UNIX SVR4 puts it all together.

Learning objectives

- Define a process and explain how it relates to its process control block.
- Explain the two-state, five-state and suspend process models and their transitions.
- Describe the tables and data structures the OS uses to manage processes.
- Explain process creation, mode switching and process switching.
- Compare the ways the OS's own code can be executed.
- Describe process management in UNIX SVR4.

How Chapter 3 fits together

Chapter 2 showed that multiprogramming and time sharing need a way to keep many programs in progress at once. The answer is the **process**, and this chapter describes it from four angles: what it is, what states it passes through, how the OS records it, and how the OS controls it.

What a process is (3.1)

A process is a program in execution. The OS tracks each one with a **process control block (PCB)** holding its identifier, state, priority, program counter, memory pointers, saved register values (context), I/O status and accounting information. Saving the program counter and context data in the PCB is what lets the OS interrupt a process and later resume it.

Its life story (3.2)

From the processor's point of view, running processes produce interleaved **traces** of instructions, with a small **dispatcher** switching between them. Process models grow step by step. The **two-state** model (Running, Not Running) cannot tell waiting processes from ready ones, so the **five-state** model adds New, Ready, Blocked and Exit, with transitions such as admit, dispatch, timeout, event wait, event occurs and release. When memory fills up, the OS may **swap** processes to disk, giving the **suspend** states: Ready/Suspend and Blocked/Suspend.

How the OS records it (3.3)

The OS keeps memory, I/O, file and process tables. Each process has a **process image**: program code, data, stack and PCB. The PCB's contents fall into three groups: process identification, processor state information and process control information.

How the OS controls it (3.4)

The processor has a **user mode** and a **kernel mode**. Creating a process takes five steps, from assigning an identifier to building the other data structures. The OS can regain control through an interrupt, a trap or a system call. A **mode switch** is cheap; a full **process switch** must save one process's context in its PCB, update queues and restore another's.

Where the OS itself runs (3.5) and a real example (3.6)

The OS kernel may run outside all processes, inside each user process, or as a set of separate system processes. UNIX SVR4 mostly runs OS code inside user processes, refines the seven-state model into nine process states, splits each process image into user-level, register and system-level context, and creates processes with the **fork** call.

Why it matters next

Chapter 4 splits the process in two: the resource owner (still the process) and the unit that runs (the thread).

3.1 What Is a Process?

You should be able to

- Explain why an OS stands between applications and hardware, and state its three process-level duties.
- Define a process in four complementary ways and tell a process apart from a program.
- Name the two essential elements of a running program and the eight elements of a process control block.
- Trace how the process control block lets the OS interrupt a process and resume it exactly where it stopped.
- Pick the PCB element the OS would consult to answer a given question about a process.

Why the OS thinks in processes

A computer platform is a collection of **hardware resources**: the processor, main memory, I/O modules, timers, disk drives and so on. **Applications** are programs written to do useful tasks for people. Writing each application directly for the bare hardware would be **inefficient** (every application would need its own code for every device) and **unsafe** (nothing would stop one program from damaging another). So the operating system sits in between. It gives applications a **convenient, rich, safe and consistent** interface and an **abstract representation** of each resource: named files instead of disk blocks, a private memory region for each process instead of raw memory cells, and processes that take fair turns instead of the bare processor.

Because many applications run at once, the OS must also manage how they share those resources. It does that bookkeeping per **process**.

The three fundamental requirements, stated in terms of processes

1. **Interleave** the execution of multiple processes to maximize processor utilization while providing reasonable response time.
2. **Allocate resources** to processes according to a specific policy (for example, higher priority is served first) while **avoiding deadlock**.
3. Support **interprocess communication (IPC)** and **user creation of processes**.

Processor utilization = busy time ÷ total time. **Response time** is how long a user waits for a result. The two goals can clash: switching more often answers people sooner, but every switch costs the processor some work.

Worked example: one at a time vs interleaved

Three processes arrive together. A needs 6 units of processor work. B needs 2 units, then waits 4 units for the disk, then needs 2 more. C needs 1 unit (echo a keystroke).

- **One at a time** (A, then B, then C): the processor idles while B waits for the disk. Total time 15, busy time $6 + 4 + 1 = 11$, so utilization = $11 \div 15 = 73.3\%$. C gets its answer at time 15.
- **Interleaved**, time slice 2, switching whenever a process blocks: A 0–2, B 2–4 (then waits for the disk until 8), C 4–5, A 5–9, B 9–11. Total 11, busy 11, utilization **100%**. C gets its answer at time 5.

Shorter slices answer short jobs sooner, but every real switch costs a little processor time, so real systems pick a moderate slice. Very long slices can bring idle time back (slice 6: utilization $11 \div 14 \approx 78.6\%$).

Allocation and deadlock

If P1 holds the printer and waits for the scanner while P2 holds the scanner and waits for the printer, neither can ever continue: that is **deadlock**. A policy such as "serve requests by priority, and always request the printer before the scanner" prevents the circle of waiting: a process that must wait for the printer holds nothing anyone else needs. Deadlock is not mere slowness; deadlocked processes never finish unless the OS intervenes.

Communication and creation

Each process's memory is private, so cooperating processes need an OS-provided channel (messages, pipes, shared memory) to exchange data: that is IPC. Users and programs must also be able to ask the OS to create new processes, for example by double-clicking an icon or typing a command; each new process gets a new identifier, its own memory and its own record.

What is a process?

Four standard definitions, all correct, each from a different angle:

- **A program in execution**: execution is what turns a program into a process.
- **An instance of a program running on a computer**: one program can have many instances at once.
- **The entity that can be assigned to and executed on a processor**: the OS's point of view; it is what the OS picks to run next.
- **A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources**: its **trace** (the sequence of instruction addresses it executes), where it stands now, and what it holds (memory, open files, devices).

Program vs process. A program is passive: a set of instructions stored in a file. A process is active: that program executing, with its own data, state, resources and PCB. Two processes can run the same program at the same time.

Two essential elements

While a program executes, its process has two essential elements: the **program code**, which may be **shared** with other processes running the same program (running code is only read, never changed), and a **set of data** associated with that code. Launch the same program twice and the OS loads the code once; each process gets its own data area, its own PCB and its own program counter, so the two can be at different places in the same code.

The process control block (PCB)

For every process the OS **creates and manages** a process control block. Processes themselves cannot read or change it. The PCB does not contain the code or data; its memory pointers say where they are. Its elements:

Element	What it holds	The OS uses it to answer...
Identifier	A unique number for this process (PID)	Which of two copies of a program should get this request?
State	Where it stands now, e.g. Running, Ready, Blocked	May this process run right now?
Priority	Its importance relative to other processes	Two are ready: who goes first?
Program counter	Address of the next instruction to execute	Where should it continue?
Memory pointers	Where its code and data are, plus memory blocks shared with other processes	Which memory can be freed? What does it share?
Context data	The processor register contents while it runs (saved when it stops)	What did R2 hold when it was interrupted?
I/O status information	Outstanding I/O requests, assigned I/O devices, files in use	Which files must be closed? Which request is it waiting on?
Accounting information	Processor time and clock time used, time limits, account numbers	How much should be billed? Is it over its limit?

While a process runs, its live program counter and register values sit in the processor; the PCB holds the copy saved when it last stopped. Processor time grows only while the process runs; clock time keeps growing while it waits. Example: 15 ms of processor time over 60 ms of clock time means the process was running for $15 \div 60 = 25\%$ of its lifetime.

Kitchen analogy: the PCB is the order ticket: order number, status, rush flag, next recipe step, which shelf and pans, what is in the pans now, waiting on the oven, time in the kitchen and who pays.

Interrupt and resume: why the PCB matters

The PCB holds enough information to **interrupt a running process and later resume it as if the interruption had not occurred**. The sequence:

1. Process A is running on the processor.
2. An interrupt (for example, the timer ending A's time slice) stops A between two instructions: the processor sets A's PC aside and jumps into the OS.
3. The OS saves A's program counter **and** context data into A's PCB and updates A's state.
4. The OS loads another process's saved values from its PCB, and that process runs.
5. Later the OS restores A's saved values, and A continues at the saved instruction with its own register values.

Worked example. A executes $R1 = 17$ and $R2 = 40$ and is interrupted with $PC = 0x1208$. B, restored with $R1 = 5$, $R2 = 9$, doubles $R2$ to 18. When A is restored ($PC\ 0x1208$, $R1\ 17$, $R2\ 40$), $R1 = R1 + R2$ gives **57**. If the OS saved only the program counter, B would start with A's leftover registers ($40 \times 2 = 80$), and A would resume with $R2 = 80$ and compute **97**: the right instruction with the wrong data. That is why the PCB keeps the context data as well as the program counter.

Without PCBs, an OS could not keep many processes in progress at once. In one line: **process = program code + associated data + PCB.**



Coming next: section 3.2 follows the State element as a process moves between states, and section 3.3 describes everything else the OS records about a process.

Key terms

Process

A program in execution: one running instance of a program, together with its data, its current state and the resources the OS has given it. It is the unit the OS schedules and manages.

Program

A passive set of instructions stored in a file. It does nothing by itself until the OS loads it and starts a process to run it.

Process control block (PCB)

The record the OS creates and maintains for each process. It holds the identifier, state, priority, program counter, memory pointers, context data, I/O status information and accounting information.

Process identifier (PID)

A unique number the OS gives each process so it can tell it apart from every other process, even ones running the same program.

Program counter (PC)

The processor register, and the PCB field that saves it, holding the address of the next instruction the process will execute.

Context data

The values in the processor registers while a process runs. The OS copies them into the PCB when the process stops, so they can be put back later.

Memory pointers

PCB entries that record where the process's program code and data sit in memory, plus any memory blocks it shares with other processes.

I/O status information

The PCB part that lists a process's outstanding I/O requests, the I/O devices assigned to it and the files it has in use.

Accounting information

The PCB part that records how much processor time and clock time a process has used, its time limits, its account numbers and similar bookkeeping.

Priority

A level that says how important a process is compared with the others. The OS uses it to decide who gets the processor or a resource first.

Processor utilization

The fraction of time the processor spends doing useful work instead of sitting idle, usually given as a percentage.

Response time

How long a user waits between making a request (a click, a keystroke) and seeing the result.

Time slice

A short, fixed amount of processor time a process may use before the OS switches the processor to another process.

Interleaving

Running pieces of several processes one after another on one processor, switching between them so that all of them make progress.

Deadlock

A standstill in which two or more processes each hold a resource another one needs, so every one of them waits forever.

Interprocess communication (IPC)

Any OS-provided way for processes to exchange data or signals, such as messages, pipes or a shared block of memory.

Abstraction

A simplified, tidy view of something complicated. The OS shows applications files instead of disk blocks, and processes instead of raw processor time.

Shared code

Program instructions kept in memory once and used by several processes at the same time. It is safe because running code only reads its instructions and never changes them.

Interrupt

A signal that makes the processor stop the current program between two instructions and run an OS routine, for example when a timer expires or a device finishes.

Process state

The condition a process is in right now, such as Running, Ready or Blocked. The PCB records it so the OS knows what the process can do next.

Review questions with answers

1. What is the key difference between a program and a process?

- A. A program is stored on disk; a process is that same file copied into main memory, unchanged.
- B. A program is passive instructions; a process is that program executing, with its own data, state and resources.
- C. A program is written by its developers; a process is written by the operating system at run time.
- D. There is no real difference: "process" is simply the operating system's word for a program.

Answer: B. A program is passive instructions; a process is that program executing, with its own data, state and resources.

A program is passive code. A process is an active instance of that code: executing, with its own data, state, resources and PCB.

2. While a program is executing, which two essential elements make it up?

- A. Program code and a set of data associated with that code
- B. A process control block and an interrupt handler
- C. A disk file and a timer
- D. A priority and an identifier

Answer: A. Program code and a set of data associated with that code

A running program consists of its code (which several processes may share) and the data that code works on. The OS adds a PCB to manage it.

3. Which of these are elements of a process control block? Select all that apply.

- A. Program counter
- B. The full source code of the program
- C. Accounting information
- D. I/O status information
- E. A copy of every other process's registers
- F. Priority

Answer: A, C, D, F

A PCB holds the identifier, state, priority, program counter, memory pointers, context data, I/O status information and accounting information. It points to the code in memory rather than storing it, and it holds only this process's own register values.

4. Match each PCB element to what it holds.

Program counter	Address of the next instruction to execute
Context data	Register contents while the process is running
Memory pointers	Where the code, data and shared blocks are in memory
Accounting information	Processor time used, time limits, account numbers
Identifier	A unique number for this process

Each element answers a different question the OS may ask about a process: where to continue, what the registers held, where its memory is, how much time it used, and which process it is.

5. After an interrupt, a process resumes at exactly the right instruction but computes a wrong result. Which PCB element did the OS most likely fail to save and restore?

- A. Program counter
- B. Context data
- C. Priority
- D. Identifier

Answer: B. Context data

Context data is the saved copy of the processor registers. If it is not restored, the process continues with another process's leftover register values.

6. Which of these is **not** one of the fundamental OS requirements expressed in terms of processes?

- A. Interleave the execution of multiple processes to maximize processor utilization while giving reasonable response time
- B. Allocate resources to processes according to a specific policy while avoiding deadlock
- C. Support interprocess communication and user creation of processes
- D. Give every process an equal, fixed share of main memory

Answer: D. Give every process an equal, fixed share of main memory

The three requirements are interleaving for utilization and response time, policy-based allocation without deadlock, and IPC plus process creation. Allocation follows a policy such as priorities; nothing demands equal, fixed shares.

7. Over a 15-unit stretch, a processor does useful work for 11 units and sits idle for the rest. What is its processor utilization, in percent? (Give one decimal place.)

Answer: 73.3 %

Utilization = busy time ÷ total time = $11 \div 15 \approx 0.733$, which is 73.3%.

8. Put the steps of interrupting and later resuming process A in order.

Correct order:

1. Process A is running on the processor
2. An interrupt stops A between two instructions
3. The OS saves A's program counter and registers in A's PCB
4. The OS loads process B's saved values, and B runs
5. The OS restores A's saved values, and A continues where it stopped

The OS saves first, then switches, and later restores. Because A's PC and context data were kept in its PCB, A resumes as if nothing had happened.

9. A process creates its own process control block and updates it as it runs.

Answer: False

The PCB is created and managed by the operating system. Ordinary processes cannot read or change it; that protects the OS's bookkeeping.

10. Raw hardware, or an abstraction the OS offers? Sort each item.

Hardware resource	OS abstraction
Disk drive I/O module	File Process A private memory region for each process

The OS hides raw devices behind convenient abstractions: files instead of disk blocks, processes instead of raw processor time, and protected memory regions instead of one shared array of bytes.

11. A PCB's accounting information shows 15 ms of processor time and 60 ms of clock time since the process was created. For what percentage of its lifetime was the process actually running on the processor?

Answer: 25 %

$15 \div 60 = 0.25 = 25\%$. Clock time keeps growing while a process waits; processor time grows only while it runs.

12. Which definition of a process describes it from the point of view of the OS deciding what to run next?

- A. A program in execution
- B. An instance of a program running on a computer
- C. The entity that can be assigned to and executed on a processor
- D. A passive file of machine instructions

Answer: C. The entity that can be assigned to and executed on a processor

The OS repeatedly picks something to give the processor to. The entity it assigns to the processor is a process.

3.2 Process States

You should be able to

- Read an interleaved instruction trace and explain what the dispatcher does between processes.
- Draw the two-state and five-state models and name the event behind every transition.
- List the reasons a process is created or terminated, and explain parent/child spawning.
- Explain why the OS keeps ready and blocked queues, and why one queue per event helps.
- Explain why processes are swapped out, why a single Suspend state falls short, and use the seven-state model with Ready/Suspend and Blocked/Suspend.

Why an OS tracks process states

A machine holds many processes but has few processors, so most processes are not running at any instant. The OS records each process's **state** in its PCB. A **state model** lists the states and the events that move a process between them (think of a one-doctor clinic: waiting room, exam room, lab tests, discharge).

Traces and the dispatcher

A process's **trace** is the ordered list of instruction addresses it executes (the values its program counter takes). The **dispatcher** is a small piece of OS code that switches the processor from one process to another.

Example. A, B and C are loaded at 5000, 8000 and 12000; the dispatcher is at 100 and is 6 instructions long (100–105). A process may run at most 6 instruction cycles before a timer interrupt ends its turn (a **timeout**). B's 4th instruction (8003) requests I/O. Each process sees only its own trace; the processor sees them interleaved:

Cycles	Running	Addresses	How the turn ends
1–6	A	5000–5005	timeout; A back to Ready
7–12	Dispatcher	100–105	chooses B
13–16	B	8000–8003	I/O request; B becomes Blocked
17–22	Dispatcher	100–105	chooses C (B is not eligible)
23–28	C	12000–12005	timeout
29–34	Dispatcher	100–105	chooses A
35–40	A	5006–5011	timeout
41–46	Dispatcher	100–105	chooses C
47–52	C	12006–12011	timeout

Worked numbers. In 52 cycles A ran 12, B 4, C 12 and the dispatcher $4 \times 6 = 24$, so overhead = $24 \div 52 \approx 46\%$. It is high only because the slices are tiny: a real slice lasts milliseconds, which is millions of instructions. C first runs at cycle 23 ($6 + 6 + 4 + 6 = 22$ cycles come before it). A resumed process continues exactly where it stopped (A at 5006).

The two-state model

Each process is **Running** or **Not Running**; not-running processes wait in one queue. **Enter**: a new process joins the back. **Dispatch**: a queued process gets the processor. **Pause**: the running process is interrupted and rejoins the back. **Exit**: the running process finishes.

The flaw: some queued processes are waiting for I/O and cannot run, so the dispatcher must search past them instead of taking the front. Fix: split Not Running into **Ready** and **Blocked**.

Why processes are created

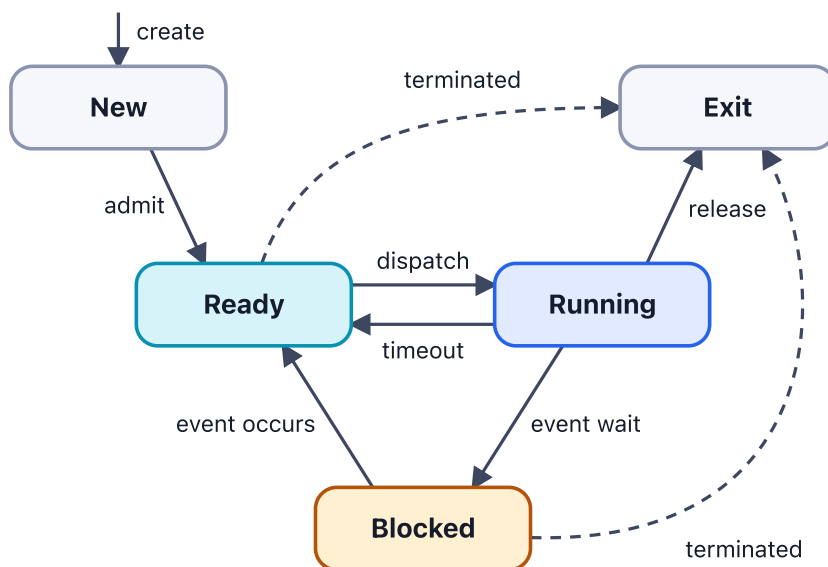
1. **New batch job**: a previously submitted job is picked up.
2. **Interactive log-on**: a user signs in at a terminal.
3. **Created by the OS to provide a service**: e.g. to manage printing.
4. **Spawned by an existing process**: a program asks the OS for another process.

Process spawning: the creator is the **parent process**, the new one the **child process**; children can spawn children, forming a tree.

Why processes are terminated (14 reasons)

Reason	Meaning
Normal completion	Finished; told the OS it is done.
Time limit exceeded	Ran longer in total than allowed (wall-clock time, processor time, or time since an interactive user last typed).
Memory unavailable	Needs more memory than the system can give.
Bounds violation	Reached a memory location it may not use.
Protection error	Used a resource in a forbidden way (writing a read-only file).
Arithmetic error	Division by zero, overflow.
Time overrun	<i>Waited</i> for an event longer than the set maximum.
I/O failure	An I/O operation failed (e.g. file not found).
Invalid instruction	Executed something that is not an instruction.
Privileged instruction	Executed an instruction reserved for the OS.
Data misuse	Wrong type of data, or data never initialised.
Operator or OS intervention	Stopped by an operator or the OS (e.g. to break a deadlock).
Parent termination	The parent ended, and the OS ends its children too.
Parent request	The parent asked the OS to end this child.

The five-state model



- **New**: PCB built but not yet admitted, and usually not yet in main memory (lets the OS cap how many processes compete).
- **Ready**: could run now; waiting only for the processor.

- **Running**: executing; at most one per processor.
- **Blocked** (Waiting): cannot run until an event occurs.
- **Exit**: finished or aborted; records may be kept briefly, then deleted.

Transition	Typical trigger
Null → New	A process is created.
New → Ready (admit)	The OS is prepared to take on another process.
Ready → Running (dispatch)	The dispatcher chooses it.
Running → Exit (release)	It finishes or is aborted.
Running → Ready	Timeout , or preemption by a more important process.
Running → Blocked (event wait)	It requests I/O or another event.
Blocked → Ready (event occurs)	The awaited event happens.
Ready / Blocked → Exit	Killed from outside, e.g. by its parent.

No Blocked → Running arrow: a blocked process is still waiting, so the processor would be wasted on it; after its event it becomes Ready first. Only a Running process can time out, block or release.

Queues

The dispatcher takes processes from the **ready queue**; waiting processes sit in **blocked queues**.

- **One blocked queue**: on every event the OS scans all blocked processes to find the ones waiting for it.
- **One queue per event**: the whole matching queue moves to Ready at once. Example: 8 blocked, 4 waiting for the disk: a disk completion costs 8 checks with one queue, 4 moves with per-event queues.
- **Priority ready queues**: one per priority level; the dispatcher serves the highest non-empty level. With preemption, a newly ready higher-priority process takes the processor. Low-priority work may wait a long time.

Swapping and the single Suspend state

I/O is far slower than the processor, so every process in memory may be Blocked at once: the processor idles and there is no room to admit more. More memory helps only briefly, since programs grow to fill it.

Swapping moves all or part of a process from main memory to disk so the space can hold a process that can run. Swapping is itself I/O, but the disk is usually the fastest I/O device, so it normally pays off.

First attempt: one Suspend state. Add Suspend to the five-state model with two arrows: Blocked → Suspend (suspend) when memory is needed, and Suspend → Ready (activate) to bring a process back.

The flaw: every suspended process was Blocked when it left, and its event may or may not have happened since. One state cannot record which, so the OS may spend a disk transfer bringing back a process that still cannot run.

The seven-state model

Track two independent facts, **waiting or not** and **in memory or on disk**: Ready, Blocked, **Ready/Suspend** (on disk, not waiting) and **Blocked/Suspend** (on disk, still waiting). With New, Running and Exit, that makes

seven states.

New transition	When it happens
Blocked → Blocked/Suspend	Usual choice when memory is needed: a blocked process cannot use its memory anyway.
Blocked/Suspend → Ready/Suspend	The event occurs while on disk (not Ready yet).
Ready/Suspend → Ready (activate)	No Ready process in memory, or it has higher priority.
Ready → Ready/Suspend	Only way to free enough memory, or it is low priority.
New → Ready/Suspend or Ready	Admit to disk when memory is tight, into memory when there is room.
Blocked/Suspend → Blocked (activate)	Unusual: it is important and its event is due soon.
Running → Ready/Suspend	Preempted and swapped out for a more important process.
Any state → Exit	Killed, e.g. by its parent or the OS.

Only processes in main memory can be dispatched. The OS prefers to swap out Blocked processes and to swap in Ready/Suspend ones.

Characteristics of a suspended process

1. It is not immediately available for execution.
2. It may or may not be waiting for an event; if it is, the event occurring does not by itself let it run.
3. An agent put it there to stop it running: itself, its parent, or the OS.
4. It stays suspended until that agent orders its removal.

Reasons for suspending a process

- **Swapping:** free memory for a process that is ready to run.
- **Other OS reason:** a background process, or one suspected of causing a problem.
- **Interactive user request:** e.g. pausing a program to debug it.
- **Timing:** a periodic process waits for its next interval.
- **Parent process request:** to examine, modify or coordinate a child.

Key terms

Trace

The list of instruction addresses a process executes, in the order it executes them. Comparing traces shows how the processor moves between processes.

Dispatcher

A small piece of OS code that switches the processor from one process to another: it records where the old process stopped and starts the next chosen process where that one left off.

Timeout

The OS takes the processor from a running process because it has used up its allowed slice of time. The process goes back to Ready.

Two-state model

The simplest process model: each process is either Running or Not Running, and all not-running processes wait in one queue.

Five-state model

A process model with the states New, Ready, Running, Blocked and Exit. It separates processes that could run from those waiting for an event.

New state

The OS has just created the process (it has an identifier and a control block) but has not yet admitted it into the group of processes allowed to run. Usually its program is not yet loaded into main memory.

Ready state

The process could run right now; it is only waiting for its turn on the processor.

Running state

The process is executing on the processor at this moment. On a single processor at most one process is Running.

Blocked state

The process cannot run until some event happens, such as an I/O operation finishing. Also called the Waiting state.

Exit state

The process has finished or been aborted. It will never run again, although the OS may keep its records for a short while.

Preemption

Taking the processor away from a running process that did not ask to stop, for example because a more important process has become ready.

Process spawning

One process asking the OS to create another process.

Parent process

A process that has created (spawned) another process.

Child process

A process that was created by another process, called its parent.

Ready queue

The list of processes in the Ready state. The dispatcher picks the next process to run from it.

Blocked queue

A list of processes in the Blocked state. An OS may keep one list for every blocked process, or a separate list for each kind of event.

Swapping

Moving all or part of a process from main memory out to disk (and later back in) so the memory can be used by other processes.

Suspended process

A process that has been set aside, typically by swapping it out to disk, and is not available to run until something explicitly brings it back. The simplest design uses one Suspend state; a better one splits it

into Ready/Suspend and Blocked/Suspend.

Ready/Suspend state

The process is swapped out to disk but is not waiting for any event: it could run as soon as it is brought back into main memory.

Blocked/Suspend state

The process is swapped out to disk and is also waiting for an event, so two things stand between it and the processor.

Review questions with answers

1. In the five-state model, which transition can **never** happen?

- A. Ready → Running
- B. Blocked → Running
- C. Running → Blocked
- D. Blocked → Ready

Answer: B. Blocked → Running

A blocked process is still waiting for its event, so handing it the processor would waste it. When the event occurs it moves to Ready and waits its turn like everyone else.

2. Processes A (at 5000), B (at 8000) and C (at 12000) share one processor with a 6-instruction dispatcher at address 100. Each process may run at most 6 instruction cycles before a timeout, and B requests I/O on its 4th instruction. A runs first, then B, then C. At which cycle number does C execute its first instruction?

Answer: 23

A uses cycles 1–6, the dispatcher 7–12, B 13–16 (then it blocks for I/O), the dispatcher 17–22. So C starts at cycle 23, at address 12000.

3. Three processes share one processor. A 6-instruction dispatcher runs between turns, each turn lasts at most 6 instruction cycles, and one process blocks for I/O after 4. Over the first 52 instruction cycles the dispatcher runs 4 times. What percentage of those 52 cycles is dispatcher overhead? Round to the nearest whole percent.

Answer: 46 %

$4 \times 6 = 24$ dispatcher cycles, and $24 \div 52 \approx 0.46$, so about 46%. It is this high only because 6-instruction turns are tiny; a real slice lasts milliseconds, which is millions of instructions, so real dispatch overhead is a small fraction.

4. Match each five-state transition to the event that typically causes it.

New → Ready	The OS is prepared to take on another process
Ready → Running	The dispatcher chooses the process
Running → Ready	The time slice runs out
Running → Blocked	The process requests I/O
Blocked → Ready	The awaited event occurs

Admit, dispatch, timeout, event wait and event occurs: each arrow in the model is the OS reacting to one kind of event.

5. Is each event a reason a process is **created** or a reason one is **terminated**?

Creation	Termination
A user logs on at a terminal A running program asks the OS to start a helper process A batch job reaches the front of the job queue	The process divides by zero The process tries to run an instruction reserved for the OS The process waits longer than allowed for an event

The four creation reasons are a new batch job, an interactive log-on, the OS providing a service, and spawning. Dividing by zero (arithmetic error), running a privileged instruction, and waiting too long for an event (time overrun) are all termination reasons.

6. A process that is only allowed to *read* a file tries to *write* to it, and the OS ends the process. Which termination reason is this?

- A. Bounds violation
- B. Protection error
- C. Data misuse
- D. Privileged instruction

Answer: B. Protection error

Using a resource, such as a file, in a way the process is not permitted to is a protection error.

7. Match each reason for suspending a process to the situation it describes.

Swapping	Main memory must be freed to bring in a process that is ready to run
Other OS reason	The OS sets aside a background utility, or a process it suspects of causing a problem
Interactive user request	A person pauses a program in order to debug it
Timing	A monitoring job that runs once an hour waits for its next run
Parent process request	A process wants to examine or change one of the processes it created

Swapping (memory pressure) is the most common reason, but the OS, a user, the clock or a parent can also set a process aside. Whatever the reason, it stays suspended until whoever suspended it releases it.

8. Which statements describe a **suspended** process?

- A. It is not immediately available for execution
- B. It may or may not be waiting for an event
- C. An agent put it there: itself, its parent, or the OS
- D. It leaves the state only when that agent orders it
- E. It is always waiting for I/O
- F. It has been terminated and its PCB deleted

Answer: A, B, C, D

Those four are the defining characteristics. Suspension is independent of waiting (so “always waiting for I/O” is wrong), and a suspended process still exists: it can be brought back.

9. A process is swapped out to disk while it waits for a disk read, and the read completes while it is still on disk. Put its moves in order.

Correct order:

1. Dispatched for the first time (Running)
2. Requests a disk read (Blocked)
3. Swapped out to disk (Blocked/Suspend)
4. The disk read completes (Ready/Suspend)
5. Swapped back into main memory (Ready)
6. Dispatched again (Running)

Running → Blocked → Blocked/Suspend → Ready/Suspend → Ready → Running. The read completes while the process is on disk, so it becomes Ready/Suspend, not Ready; and it must reach Ready in main memory before it can run, because no arrow leads from a Blocked state straight to Running.

10. Why do many operating systems keep a separate blocked queue for each kind of event?

- A. So a blocked process can be dispatched directly from its event queue
- B. So that when an event occurs, the OS can move the whole matching queue to Ready without scanning every blocked process
- C. So each device gets its own processor
- D. Because a single queue cannot hold more than a few processes

Answer: B. So that when an event occurs, the OS can move the whole matching queue to Ready without scanning every blocked process

With one queue per event, every process in the matching queue is waiting for exactly that event, so the OS can wake them all at once without searching.

11. Every process in main memory is Blocked waiting for I/O, and the processor sits idle. What does the OS typically do?

- A. Dispatch one of the blocked processes anyway
- B. Swap a blocked process out to disk (Blocked/Suspend) to make room for a process that can run
- C. Terminate all of the blocked processes
- D. Wait for one of the I/O operations to finish and leave memory as it is

Answer: B. Swap a blocked process out to disk (Blocked/Suspend) to make room for a process that can run

Swapping a blocked process out frees memory, so the OS can admit a new process or bring back a Ready/Suspend one and keep the processor busy.

12. In the two-state model, processes waiting for I/O share one Not Running queue with processes that could run. Even so, the dispatcher can always simply take the process at the front of that queue.

Answer: False

The front process may be waiting for I/O and unable to run, so the dispatcher must search past it. That flaw is why Not Running is split into Ready and Blocked.

3.3 Process Description

You should be able to

- Describe the four kinds of tables the OS keeps (memory, I/O, file and process) and explain why they must point at one another.
- Name the four parts of a process image and explain how paging lets some of it sit on disk.
- Sort PCB contents into process identification, processor state information and process control information.
- Explain how condition codes in the program status word (x86 EFLAGS) are set, and why they are saved with the process.
- Explain why the PCB is the most important OS data structure, how PCBs form queues, and how the OS protects them.

The OS as the manager of resources

Many processes (P1 ... Pn) compete for a few processors, a fixed amount of main memory and some I/O devices. The OS hands these out and takes them back, so it must always know **where each process is, what it holds and what it is waiting for**. Example: P1 runs and owns the keyboard; P2 is blocked while the disk reads for it; P3 is swapped out (only its PCB stays in main memory); P4 is ready, holds the printer and waits for the processor.

OS control structures: four families of tables

Table	What it records
Memory tables	Main memory allocated to each process; secondary memory (disk) allocated to each process; protection attributes of regions, e.g. who may read or write a shared region; information needed to manage virtual memory.
I/O tables	Each I/O device and channel (a small processor dedicated to I/O): available or assigned (to which process); status of the operation in progress; the main-memory location used as source or destination of the transfer.
File tables	Which files exist; their location on secondary memory; current status (e.g. open for writing by process 3); attributes (owner, permissions, size). May be kept by a separate file management system.
Process tables	One entry per process; each entry points to that process's image (and so its PCB).

The tables are cross-referenced. Memory, I/O and file tables name processes by ID, and I/O entries point into memory that the memory tables allocate. Example with 4 KB **frames** (fixed-size slots of main memory; frame 4 = 0x4000–0x4FFF in hexadecimal): "Disk 0 reads for process 2 into 0x4200" matches a memory row "frame 4 belongs to process 2" and a file row "song.mp3 open for reading by process 2". When a process ends, every table that mentions it must be updated. The tables themselves live in main memory, so memory management covers them too.

Configuration comes first. Before building tables, the OS must learn its environment (how much main memory, which I/O devices) from firmware, hardware probing or an administrator's settings.

The process image

- **User program:** the instructions to be executed.
- **User data:** the modifiable part of user space: program data, a user stack area, and programs that may be modified.
- **Stack:** one or more last-in, first-out system stacks holding parameters and return addresses of procedure and system calls.
- **Process control block (PCB):** the attributes the OS needs to control the process.



Location. The image may be one **contiguous** block (one pointer to its start, but it needs one big enough gap and must be wholly loaded to run) or, with **paging**, cut into equal pages placed in any free frames, with pages not needed now left on disk. The OS can manage a process only if at least a small part of its image (the PCB) is in main memory, and the parts in use must be there for it to run. So the OS must know where every part of every image is: the process table points to each image, and the page tables map every page.

What a PCB holds: three groups

1. Process identification

Identifiers of this process (often its index into the process table), of its parent (creator), and of the user. Two runs of one program = two processes, two IDs.

2. Processor state information

The register contents: in the processor while the process runs, saved in the PCB when it is interrupted, restored when it resumes. **User-visible registers** (often 8 to 32); **control and status registers**: the program counter, the **condition codes** (sign, zero, carry, equal, overflow) and status information (interrupts enabled/disabled, execution mode); and **stack pointers**. Condition codes and status bits usually share one register, the **program status word (PSW)**; on x86 it is the 32-bit **EFLAGS** register.

3. Process control information

- **Scheduling and state**: state, priority, scheduling data (e.g. time waiting), the event awaited.
- **Data structuring**: pointers to other PCBs (queues, parent-child links).
- **Interprocess communication**: flags, signals, messages.
- **Process privileges**: memory it may access, instruction types it may execute, system services it may use.
- **Memory management**: pointers to its segment and/or page tables.
- **Resource ownership and utilization**: resources such as open files, and processor-use history. (Processor time used is control information, not processor state.)

The PSW in action: x86 EFLAGS

Key bits: CF carry (0), PF parity (2), AF auxiliary carry (4), ZF zero (6), SF sign (7), TF trap (8), IF interrupt enable (9), DF direction (10), OF overflow (11), IOPL I/O privilege level (12–13), NT nested task (14); bits 16–21 are more control bits, the rest reserved.

Negative numbers on x86: two's complement. The teaching machine in 1.3 used sign-magnitude (8005 hex = -5), but x86 stores negative numbers in **two's complement**. For 8 bits: if the top bit is 0 the value reads the same signed or unsigned (0 to 127); if the top bit is 1, the signed value is the unsigned value minus 256 (200 → -56, 255 → -1, 128 → -128). The same bit pattern can therefore be read two ways, which is why there are two "did not fit" flags: **CF** means the *unsigned* result did not fit in 0...255, and **OF** means the *signed* result did not fit in -128...127. The lab concentrates on CF, ZF, SF, OF (and IF); the other bits (NT, IOPL, DF, TF, AF, PF) have jobs this section does not need.

Worked example (8 bits). $200 + 100 = 300$ needs 9 bits, so the register keeps $300 - 256 = 44$ and **CF = 1**; **ZF = 0**, **SF = 0**, and as signed values $-56 + 100 = 44$ fits, so **OF = 0**. For $100 + 50 = 150$ (1001 0110): **CF = 0** and **ZF = 0**, but **SF = 1** and **OF = 1**: two positives gave a signed sum above 127 (the register reads -106). After SUB, **ZF = 1** means the values were **equal**; $3 - 5$ needs a borrow (**CF = 1**) and leaves 254 (-2 signed, **SF = 1**).

The flags describe the running process's own last result, so the OS saves EFLAGS in the PCB when stopping a process; otherwise a pending "jump if equal" could take the wrong branch.

The role of the PCB

The PCB is the **most important OS data structure**: virtually every module reads or modifies it (scheduling, resource allocation, interrupt processing, performance monitoring). The ready queue and blocked queues are **linked lists of PCBs**; moving a process between queues changes a few pointers, never copies the PCB.

Protection problem: (1) a bug in one routine, such as an interrupt handler, can damage PCBs, e.g. a garbage *next* pointer makes every later PCB in the queue unreachable; (2) a change to the PCB's structure affects every module that uses it. **Solution:** all routines go through a single **handler routine whose only job is to protect PCBs**, the sole arbiter of reads and writes. It checks requests, and a layout change means rewriting only the handler. **Trade-off:** a small performance cost on every access, and it helps only as far as the rest of the OS can be trusted to use it.

Key terms

Memory table

An OS table that records which parts of main memory and of secondary memory (disk) are allocated to each process, the protection settings of each region (including shared ones), and whatever is needed to manage virtual memory.

I/O table

An OS table that records every I/O device and channel (a small processor dedicated to running I/O transfers): whether it is free or assigned (and to whom), the status of any operation in progress, and the main-memory address used as the source or destination of the transfer.

File table

An OS table that records which files exist, where each one is stored on secondary memory, its current status (for example open for writing) and its attributes. Often kept by a separate file management system.

Process table

The OS table with one entry per process. Each entry holds a pointer that leads to that process's image, and so to its process control block.

Process image

The complete set of things that make up one process: its user program, its user data, its stack(s) and its process control block.

Process control block (PCB)

The record the OS creates and maintains for each process, holding the attributes it needs to control it. Its elements (identifier, state, priority, program counter and the rest) fall into three groups: process identification, processor state information and process control information.

User data

The part of a process image the program itself may change: its variables and working data, a user stack area, and any code that modifies itself.

Frame (page frame)

A fixed-size slot of main memory (4 KB in this section's examples, so frame 4 covers addresses 0x4000–0x4FFF). With paging, a process is cut into pieces of the same size, called pages, and each

page can be placed in any free frame.

System stack

A last-in, first-out area in the process image that remembers calls in progress: the parameters and return addresses of procedure calls and system calls. (Section 3.5 also meets the separate system stack that a nonprocess kernel keeps for itself, outside every process image.)

Process identification

The identifier part of the PCB: the ID of this process, the ID of the parent that created it, and the ID of the user it runs for.

Processor state information

The part of the PCB that holds a copy of the processor registers for the process: user-visible registers, control and status registers, and stack pointers. It is saved when the process is interrupted and loaded back into the processor when it resumes.

Process control information

The part of the PCB the OS uses to manage and coordinate the process: scheduling and state details, links to other PCBs, interprocess communication, privileges, memory-management pointers and resource use.

User-visible register

A processor register that a program's own machine instructions can name and use, such as a general-purpose data register.

Control and status registers

Processor registers that steer execution and report on it, such as the program counter, the condition codes and the status bits (interrupts on or off, user or kernel mode).

Condition codes

Bits the processor sets after an arithmetic or logic instruction to describe the result (sign, zero, carry, equal, overflow). Later instructions test them to decide whether to jump.

EFLAGS

The 32-bit flags register of x86 processors, their version of the program status word. It holds the condition codes (carry, zero, sign, overflow and more) and control bits such as interrupt enable.

Interprocess communication (IPC)

The ways separate processes exchange information, such as signals, messages and shared flags. The PCB records what is pending for each process.

Process privileges

PCB entries that say what a process is allowed to do: which memory it may touch, which kinds of instructions it may execute and which system services it may use.

Program status word (PSW)

A processor register (or set of registers) holding status about the running program: its condition codes, whether interrupts are enabled, and whether the processor is in user or kernel mode.

Linked list

A chain of records in which each record holds a pointer to the next one. The OS builds its process queues by linking PCBs together this way.

Review questions with answers

1. The OS must decide whether the printer can be given to a process right now. Which table does it consult?

- A. A memory table
- B. An I/O table
- C. A file table
- D. The process table

Answer: B. An I/O table

I/O tables record every device and channel: whether it is free or assigned (and to which process), the status of any operation, and the memory address used for the transfer.

2. Match each OS table to something it records.

Memory table	Protection of a shared memory region
I/O table	Memory address a disk transfer writes into
File table	Where a file is stored on disk
Process table	One entry per process, leading to its image

Memory tables hold allocation and protection; I/O tables hold device status and transfer addresses; file tables hold existence, location, status and attributes of files; the process table is the list of processes.

3. Which of these are parts of a process image?

- A. The user program
- B. The user data
- C. The stack
- D. The process control block
- E. The file table
- F. The ready queue

Answer: A, B, C, D

A process image is the user program, user data, stack(s) and PCB. The file table and the ready queue are OS-wide structures, not part of any one process image.

4. Sort each PCB field into its group of attributes.

Identification	Processor state	Process control
ID of the parent process	Program counter Condition codes	Priority List of open files

Identifiers name the process; register contents (including the PSW bits) are processor state; everything the OS uses to schedule, link, protect and account for the process is process control information.

5. With paging, every page of a process image must stay in main memory for as long as the process exists.

Answer: False

Pages can be spread over any free frames, and pages not needed right now can wait on disk. The memory tables record where each one is.

6. Process 7 is interrupted just after a compare instruction and before its "jump if equal". Where are its condition codes while other processes run?

- A. In its PCB, as part of the processor state information
- B. In its user data, next to its variables
- C. In the I/O table entry for the interrupting device
- D. Nowhere: the flags are recomputed when process 7 resumes

Answer: A. In its PCB, as part of the processor state information

The PSW (EFLAGS on x86) is saved in the PCB when a process stops and restored before it runs again, so its pending jump sees its own result.

7. An 8-bit register computes $200 + 100$. What value does the 8-bit result register hold (as an unsigned number)?

Answer: 44

The true sum 300 needs 9 bits. Only the low 8 bits fit: $300 - 256 = 44$, and the lost carry is recorded in the carry flag ($CF = 1$).

8. An 8-bit ADD computes $100 + 50$. Which of the flags CF, ZF, SF and OF are set afterwards?

- A. CF (carry)
- B. ZF (zero)
- C. SF (sign)
- D. OF (overflow)

Answer: C, D

150 fits in 8 unsigned bits, so $CF = 0$, and it is not 0, so $ZF = 0$. But 150 is 1001 0110: the top bit is 1 ($SF = 1$), and as signed numbers $100 + 50$ exceeds 127, so $OF = 1$.

9. Put in order the lookups the OS follows to find where process 8's stack page is right now.

Correct order:

1. Use process ID 8 to find its entry in the process table
2. Follow that entry's pointer to the process image and its PCB
3. Read the memory-management pointer in the PCB (its page table)
4. Look up the stack page to find its frame, or its place on disk

The process table leads to the image and PCB; the PCB's memory-management information points to the page table; the page table says where each page is. This chain is the cross-referencing between tables in action.

10. Moving a process from the ready queue to a blocked queue normally means copying its whole PCB into the other queue.

Answer: False

Queues are linked lists of PCBs. The OS only changes a few pointer fields; the PCB itself stays where it is.

11. Why do many operating systems make every routine go through a single handler to read or write PCBs?

- A. It makes each PCB access faster
- B. A buggy routine cannot damage PCBs unchecked, and a change to the PCB layout only affects the handler
- C. PCBs are stored on disk and need a special reader
- D. It lets user programs edit their own PCBs safely

Answer: B. A buggy routine cannot damage PCBs unchecked, and a change to the PCB layout only affects the handler

Nearly every OS module uses PCBs. A single guard routine catches bad requests in one place and hides the PCB layout. The trade-off: a small detour on each access, and it only helps if every other routine can be trusted to go through it.

12. An OS keeps a running total of the processor time each process has used, for scheduling and accounting. Where in the PCB does it belong?

- A. Process identification
- B. Processor state information, since it is about the processor
- C. Process control information (resource ownership and utilization)
- D. Inside the saved program status word, next to the condition codes

Answer: C. Process control information (resource ownership and utilization)

Resource ownership and utilization is part of process control information: it lists resources the process controls (such as open files) and a history of its use of the processor and other resources, which the scheduler and accounting rely on.

3.4 Process Control

You should be able to

- Explain what user mode and kernel mode are, why the processor has both, and how the mode bit changes.
- Name the main jobs of an OS kernel in process, memory and I/O management plus its support functions.
- Walk through the five steps the OS takes to create a new process.
- Classify an event as an interrupt, a trap or a supervisor call, and predict whether it leads to a process switch.
- Tell a mode switch from a process switch, and put the seven steps of a process switch in order.

Process control: how the OS stays in charge

While a program runs, the processor executes its instructions, not the OS's. The OS keeps control through hardware **privilege** (two processor modes), careful **creation** of every process, and the power to take the processor back and **switch** it to another process.

1. Modes of execution

- **User mode** (less privileged): ordinary programs run here. They cannot execute **privileged instructions** (disable interrupts, set the timer, start I/O, halt, change the mode) or touch **protected memory** such as the OS's tables.
- **Kernel mode** (also **system mode** or **control mode**, more privileged): the OS kernel runs here and may execute any instruction and reach any memory. The OS uses privileged instructions itself: it sets the timer before dispatching a process, and halts the processor only when nothing is Ready.
- **Why two modes?** The OS's data, such as every PCB and the process table, must be protected. If any program could rewrite them or switch off the clock, one bug or one attacker could take over the machine.
- **How the processor knows:** a **mode bit** in the **program status word (PSW)**, checked before every privileged instruction and protected access. A violation causes a **trap**, and the OS usually ends the offending process.
- **How the mode changes:** user → kernel only on an **interrupt**, a **trap** or a **system call**, each entering the OS at an address fixed in advance. (The processor checks for a pending interrupt at the end of every instruction.) Kernel → user when the OS executes a return instruction that restores the saved PSW. A system call (supervisor call) is the legal way to get privileged work done: the program asks and the OS does the work.

2. What the kernel does

The **kernel** is the core of the OS that stays in main memory and runs in kernel mode. Its typical functions:

Family	Typical functions
Process management	creation and termination; scheduling and dispatching; process switching; synchronization and inter-process communication; managing PCBs
Memory management	allocating address space; swapping; page and segment management
I/O management	buffer management; allocating I/O channels and devices to processes
Support functions	interrupt handling; accounting; monitoring

3. Creating a process: five steps, in order

1. **Assign a unique process identifier (PID)** and add an entry to the primary process table.
2. **Allocate space for the process:** every element of the process image (user address space for code and data, the stack(s), the PCB). Sizes come from defaults for that kind of program or from the creator's request. If memory is full the image may start in the swap area on disk.
3. **Initialize the PCB.** Identification: PID, parent's PID, user. Processor state: mostly zeros, except the program counter (program entry point) and stack pointers (new stacks). Process control: state = **Ready** (or **Ready/Suspend** if it starts on disk), default priority, no resources owned yet (unless requested or inherited from the parent).
4. **Set the appropriate linkages**, e.g. link the PCB onto the Ready (or Ready/Suspend) list.
5. **Create or expand other data structures**, e.g. an accounting record.

Why this order: no space without an identity, no PCB before its space exists, and no queue entry while the PCB is blank.

4. When can the OS switch processes?

Mechanism	Cause	Examples
Interrupt	an event outside the current instruction	Clock interrupt: time slice used up → process to Ready, another runs. I/O interrupt: waiting processes move Blocked → Ready; the OS resumes the current process or preempts it for a more urgent one. Memory fault: a valid address not in main memory; the OS brings it in and blocks the process meanwhile.
Trap	an error or exception in the current instruction	Divide by zero, privileged instruction in user mode. Fatal (e.g. the program has no handler for it): process → Exit, another is dispatched. Recoverable: the OS recovers or informs the process, which may continue.
Supervisor call	an explicit request by the program	Open a file, read data, ask the time. If the service must wait (e.g. a disk read) the process is Blocked; otherwise it continues.

5. Mode switch vs process switch

On an interrupt, trap or system call the hardware saves the program counter and PSW, sets the mode bit to kernel and jumps to the handler: a **mode switch**. The same process is still Running. If it can continue, the OS returns: PC and PSW are restored and the mode goes back to user. **Every interrupt causes a mode switch; only some cause a process switch.** A mode switch is cheap. A **process switch** also rewrites two PCBs, moves queue entries, runs the scheduler and changes the memory map, and the new process then runs slowly for a while because the caches hold the old process's data.

6. The seven steps of a process switch

1. **Save the context of the processor** (PC, PSW, stack pointer, other registers) into the running process's PCB. First, because the OS's own code will overwrite the registers.
2. **Update the PCB of the running process:** new state (Ready, Blocked, Ready/Suspend or Exit), the reason, accounting such as time used.
3. **Move that PCB to the appropriate queue:** Ready, Blocked on event i , or Ready/Suspend.
4. **Select another process** to run.
5. **Update the PCB of the selected process:** state = Running.
6. **Update memory-management data structures** so addresses lead to the new process's memory.
7. **Restore the context of the selected process** (mode bit back to user). Last, because it hands over the processor.

A bad order breaks things: restoring before saving destroys the old registers; queueing a PCB that still says Running leaves contradictory records; restoring before the memory map is switched lets the new process reach the old one's memory.

Worked example

The timer interrupts every 2 ms, the slice is 10 ms, and every process uses its whole slice. A slice lasts $10 \div 2 = 5$ ticks, so of every 100 clock interrupts, $100 \div 5 = 20$ cause a process switch and **80** cause only a mode switch. With illustrative costs of 3 units for a mode-switch-only tick and 13 for a slice-ending tick (13 already includes entering the kernel), OS work = $20 \times 13 + 80 \times 3 = 500$ units per 100 ticks; with 1 tick per slice it would be $100 \times 13 = 1,300$. Shorter slices mean more switching overhead; longer slices mean longer

waits for everyone else. With a 1 ms tick and a 20 ms slice: $1,000 \div 20 = 50$ switches per second, 950 mode-switch-only interrupts.

Key terms

User mode

The less-privileged processor mode in which ordinary programs run. Privileged instructions and protected memory, such as the OS's own tables, are off limits.

Kernel mode

The more-privileged processor mode in which the OS kernel runs: every instruction may be executed and every part of memory reached. Also called system mode or control mode.

Privileged instruction

An instruction the hardware accepts only in kernel mode, such as disabling interrupts, setting the timer, starting I/O on a device or halting the processor. Tried in user mode, it causes a trap.

Kernel

The core of the operating system. It stays in main memory, runs in kernel mode and does the essential work: managing processes, memory and I/O, and handling interrupts.

Program status word (PSW)

A processor register describing the running program's status: condition codes, whether interrupts are enabled, and the mode bit that says user or kernel mode.

Mode bit

A bit in the program status word recording whether the processor is in user mode or kernel mode. The hardware consults it before every privileged instruction.

System call (supervisor call)

A deliberate request from a running program for an OS service, such as opening a file. A special instruction switches the processor to kernel mode and enters the OS at a fixed, pre-arranged address.

Interrupt

A signal caused by something outside the instruction being executed (the clock, an I/O device, the memory system) that makes the processor pause the running program and run an OS handler.

Trap

An entry into the OS caused by an error or exception in the instruction just executed, such as dividing by zero or trying a privileged instruction in user mode.

Interrupt handler

The OS routine that runs when an interrupt or trap is accepted. It works out what happened and deals with it, then returns or asks for a process switch.

Clock interrupt

An interrupt from the system timer, fired at regular intervals. It lets the OS check whether the running process has used up its time slice.

Time slice

A short, fixed amount of processor time a process may use before the OS switches the processor to another process. Also called a quantum.

Memory fault

An interrupt raised when the running program refers to a valid address whose contents are not in main memory right now. The OS must fetch that part from disk first. In a paging system it is called a page fault.

Mode switch

A change of the processor between user mode and kernel mode, for example on an interrupt and again on return. The same process stays Running, so it is cheap.

Process switch

Taking the processor away from the running process and giving it to another: save one context, update PCBs and queues, change the memory map, restore the other context. Also called a context switch.

Context (processor state)

Everything the processor needs to resume a program exactly where it stopped: the program counter, the PSW, the stack pointer and the other registers.

Process identifier (PID)

A number the OS assigns to a process that no other current process has. Every table and queue refers to the process by it.

Process control block (PCB)

The OS's record for one process: identification, the saved processor state, and control information such as state, priority, queue links and resources owned.

Process image

All the pieces that make up one process: its program code, its data, its stacks and its PCB.

Process table

The OS's master list of processes, one entry per process, each leading to that process's image and PCB.

Review questions with answers

1. Which of these can a program running in **user mode** do on its own, without trapping to the OS?

- A. Add two numbers held in its own registers
- B. Disable interrupts
- C. Set the mode bit to kernel
- D. Write into another process's PCB

Answer: A. Add two numbers held in its own registers

Arithmetic on a program's own registers harms nobody, so user mode allows it. Anything that could take control away from the OS or reach its tables is privileged or protected.

2. The processor keeps track of whether it is in user mode or kernel mode with a bit in the program status word (PSW).

Answer: True

The mode bit lives in the PSW. The hardware checks it before every privileged instruction and every access to protected memory.

3. Which of these switch the processor from user mode to kernel mode?

- A. A clock interrupt
- B. A system call
- C. A divide-by-zero trap
- D. A return-from-interrupt instruction
- E. Adding two registers

Answer: A, B, C

Interrupts, traps and system calls all enter the kernel at a pre-arranged address. The return instruction goes the other way, kernel to user, and ordinary arithmetic changes nothing.

4. Sort these kernel functions into their family: **process**, **memory** or **I/O** management, or **support** functions.

Process	Memory	I/O	Support
Scheduling and dispatching	Swapping	Buffer management Assigning devices to processes	Interrupt handling

Process management runs the life of processes; memory management decides what lives where in memory; I/O management shares devices and moves data; support functions such as interrupt handling and accounting serve the rest.

5. Put the steps the OS takes to **create a process** in order.

Correct order:

1. Assign a unique process identifier
2. Allocate space for the process image
3. Initialize the process control block
4. Set the linkages, such as putting it in the Ready list
5. Create or expand other data structures, such as accounting records

Identity first (PID and process-table entry), then space for the image, then the PCB inside it is filled in, then the process is linked into a queue where the scheduler can find it, and finally extra bookkeeping.

6. When the OS initializes a brand-new process's PCB, which set of values is typical?

- A. State Ready, program counter at the program's entry point, no resources owned yet
- B. State Running, registers copied from the parent, all of the parent's open files
- C. State Blocked until the user types something, program counter at 0
- D. State Exit, so the scheduler ignores it until the linkages are set

Answer: A. State Ready, program counter at the program's entry point, no resources owned yet

Most processor-state fields start at zero, the program counter points to the entry point and the stack pointer to the new stack. The state is Ready (or Ready/Suspend), since the OS admits the process as it creates it; the priority is the default, and it owns no resources unless it asked for some or inherits them from its parent.

7. Match each event to what hands control to the OS.

The timer signals that a time slice is over	Clock interrupt
A program asks the OS to open a file	Supervisor call
An instruction tries to divide by zero	Trap
A program refers to code that is not yet in main memory	Memory fault
A disk controller reports a finished transfer	I/O interrupt

Interrupts (clock, I/O, memory fault) come from outside the current instruction, a trap comes from an error in it, and a supervisor call is a deliberate request.

8. Every interrupt causes a process switch.

Answer: False

Every interrupt causes a mode switch into the kernel. If the interrupted process can carry on, for example because its time slice is not used up, the OS simply returns to it.

9. Put the seven steps of a full **process switch** in order.

Correct order:

1. Save the context of the processor
2. Update the PCB of the running process
3. Move that PCB to the appropriate queue
4. Select another process to run
5. Update the PCB of the selected process
6. Update memory-management data structures
7. Restore the context of the selected process

Save first so nothing is overwritten, finish the old process's paperwork, choose, prepare the new process and its memory map, and restore its registers last because that hands over the processor.

10. Why is a mode switch much cheaper than a process switch?

- A. Only a little processor state is saved and the same process continues, so no process states, queues or memory maps change
- B. A mode switch does not involve the kernel at all
- C. A mode switch runs on a separate processor
- D. The hardware saves nothing at all during a mode switch

Answer: A. Only a little processor state is saved and the same process continues, so no process states, queues or memory maps change

A mode switch saves and restores a small amount of state. A process switch also rewrites two PCBs, moves queue entries, runs the scheduler, changes the memory map, and leaves the caches full of the old process's data.

11. A program executes an instruction that divides by zero and the OS treats it as fatal. What happens to the process?

- A. It moves to Exit and another process is dispatched
- B. It moves to Blocked until the error clears
- C. It keeps running in kernel mode
- D. It is moved to the Ready queue to try again later

Answer: A. It moves to Exit and another process is dispatched

A trap enters the OS. If the error is fatal the OS terminates the process (Exit) and switches to another process; a recoverable trap may let it continue.

12. The timer interrupts every 2 ms, the time slice is 10 ms, and every process always uses its full slice. Out of every 100 clock interrupts, how many cause **only** a mode switch (no process switch)?

Answer: 80 interrupts

A slice lasts $10 \div 2 = 5$ ticks, so every 5th interrupt ends a slice and triggers a process switch: $100 \div 5 = 20$. The other $100 - 20 = 80$ are handled with a mode switch into the kernel and straight back.

3.5 Execution of the Operating System

You should be able to

- Explain why “is the operating system a process?” is a real design question, and name the three standard answers.
- Describe how a nonprocess kernel handles an interrupt or supervisor call, and which memory and stack it uses.
- Identify every part of a process image when the OS executes within user processes, including the kernel stack and the shared address space.
- Explain why a system call in that design costs a mode switch rather than a process switch, and estimate the switching overhead of each design.
- Weigh the modularity and multiprocessor benefits of a process-based OS against its extra switching cost.

The puzzle: is the operating system a process?

The OS is software: the processor fetches and executes its instructions just as it does a user program's. Yet the OS also manages processes: it creates, schedules and switches them. So where does OS code run, and is the OS itself a process? Three standard designs give different answers:

1. **Nonprocess kernel:** the kernel runs outside every process.
2. **Execution within user processes:** OS routines run inside whichever user process needs them.
3. **Process-based OS:** the OS itself is built from system processes.

Analogy (an apartment building): the staff work from a basement office (1), work inside your apartment from a locked service closet that every apartment has (2), or are specialists in offices of their own that you send requests to (3).

Design 1: the nonprocess kernel

The traditional approach, used by many older operating systems. The kernel runs **outside of any process**, as a separate entity in privileged (kernel) mode, with its **own region of memory** and its **own system stack** for procedure calls and returns. The idea of a process applies **only to user programs**: the kernel has no PCB, no state and is never scheduled.

On an interrupt, a trap or a supervisor call, the processor switches to kernel mode (a mode switch), the running process's context is saved, and control passes to the kernel. The process is still in the Running state: a mode switch changes no process's state. Then:

- **Service finishes at once** (data already in memory): the kernel does the work and restores the same process. Cost: 2 mode switches, 0 process switches.
- **Process must wait** (data must come from disk): the kernel starts the I/O, marks the process Blocked, and its dispatcher loads the context of a Ready process. Cost: 2 mode switches, 1 process switch.

Because one system stack serves every request, the kernel cannot leave a waiting request's half-done work on it. That progress is recorded in kernel tables instead (the process table, the device's request list).

Design 2: execution within user processes

Common on smaller machines such as PCs and workstations. Virtually all OS software runs **in the context of a user process**: the OS is a collection of routines that programs call, and each routine runs within the calling process, in kernel mode. Each process image therefore contains:

Part	What it holds, and when it is used
Process control block	Identification, processor state, process control information. Touched only by OS code in kernel mode.
User stack	Frames of the program's own calls, in user mode.
Private user address space	The process's own program and data; no other process can reach it.
Kernel stack	Frames of OS calls made in kernel mode, kept apart from the user stack so the program cannot touch them. A system call that must wait leaves its unfinished work here.
Shared address space	OS code and data, present in every address space but mapped to one physical copy. Usable only in kernel mode, so user programs cannot tamper with it.

An interrupt, trap or supervisor call causes a **mode switch, not a process switch**: the same process stays Running, now executing OS code. If the OS then lets the process continue, another mode switch resumes it (quick request: 2 mode, 0 process switches). If a process switch is needed, control passes to a small process-switching routine. It may or may not start in the current process, but while it hands the processor over it is best viewed as running **outside all processes** (request that must wait: 2 mode, 1 process switch).

Why a mode switch is cheap

A mode switch flips the mode bit, saves a few registers and changes stacks; no process changes state. A process switch saves a whole context, updates PCBs and queues and changes the memory map, and the new process starts with caches full of the old one's data.

Cost model: switching time per second = requests per second × (switches per request × time per switch).

Worked example (illustrative prices): 50,000 requests/s, mode switch 0.2 μs, process switch 3 μs.

- Designs 1 and 2: $50,000 \times (2 \times 0.2) \mu\text{s} = 20,000 \mu\text{s} = \mathbf{20 \text{ ms per second}}$ (2% of one processor).
- Design 3: $50,000 \times (2 \times 0.2 + 2 \times 3) \mu\text{s} = 320,000 \mu\text{s} = \mathbf{320 \text{ ms per second}}$ (32%).

Designs 1 and 2 tie on switch count; they differ in where a waiting request's progress lives (kernel tables versus the caller's kernel stack).

Design 3: the process-based operating system

The OS is a **collection of system processes**. Major kernel functions (file system, memory manager, I/O manager and so on) are separate processes that run in kernel mode and are scheduled by the dispatcher like any other process. Only a **small amount of process-switching code** runs outside all processes.

Advantages:

1. **Modular design:** each function lives in its own process, with minimal, clean interfaces to the rest.
2. **Noncritical functions** fit neatly as processes, e.g. a monitor that records how busy the processor, memory and I/O channels are and how fast user processes progress. No process requests a service from it, so as a process it runs at its own priority, interleaved with other work by the dispatcher.
3. **Multiprocessor and multicomputer fit:** some OS services can be moved to dedicated processors (or machines), improving performance.

The price: reaching a service in another process costs process switches. A quick request costs 2 mode + 2 process switches; a request that must wait also costs 2 before another user process runs, and more when the I/O completes.

Side by side

	Nonprocess kernel	Within user processes	Process-based OS
Where OS code runs	Outside every process	Inside the calling process	In system processes
Stacks the OS uses	One system stack	A kernel stack per process	Each system process's own stack
Quick request	2 mode, 0 process	2 mode, 0 process	2 mode, 2 process
Request that must wait	2 mode, 1 process	2 mode, 1 process	2 mode, 2 process
Waiting request's progress	Kernel tables	Caller's kernel stack	Inside the service process
Modularity	Up to the designers	Up to the designers	Enforced by process boundaries
Protecting OS code	Separate kernel memory, kernel mode only	Shared region, locked outside kernel mode	Kernel mode; parts in separate address spaces
Many processors	Runs on whichever processor entered it	Runs on the caller's processor	Services can get dedicated processors

No design wins every row: switch overhead trades against modularity and flexibility. Real systems mix designs; UNIX (next section) runs most OS code inside user processes and also keeps a few system processes for background work.

Key terms

Nonprocess kernel

A design in which the kernel runs outside every process. It has its own region of memory and its own system stack, and the idea of a process applies only to user programs.

Execution within user processes

A design in which almost all OS code runs inside whichever user process was interrupted or asked for a service, in kernel mode, on that process's kernel stack.

Process-based operating system

A design in which the major OS functions are themselves separate system processes, scheduled like other processes, with only a small amount of switching code running outside all processes.

System process

A process that carries out an operating-system function, such as file handling or memory management, rather than a user's program. It runs in kernel mode.

System stack of a nonprocess kernel

The stack a nonprocess kernel keeps in its own memory region for its procedure calls and returns while it works outside every process. Unlike the stacks in a process image (section 3.3), it belongs to no process; compare the kernel stack that each process has when the OS runs inside user processes.

Kernel stack

A second stack inside each process image, used only while that process is running OS code in kernel mode. It holds the return addresses, parameters and local variables of kernel procedure calls, out of reach of the user program.

User stack

The stack a process uses for the procedure calls of its own program while it runs in user mode.

Private user address space

The part of a process's address space that holds its own program and data. No other process can reach it.

Shared address space

The part of every process's address space that holds the OS code and data. It leads to the same physical memory in every process and can be used only in kernel mode.

Mode switch

A change of the processor between user mode and kernel mode, for example on an interrupt or system call and again on the return. No process changes state and the memory map stays the same, so it is far cheaper than a process switch.

System call (supervisor call)

A deliberate request from a running program for an OS service, such as reading a file. A special instruction switches the processor to kernel mode and enters the OS at a fixed, pre-arranged address.

Trap

An entry into the OS caused by an error or exception in the instruction just executed, such as dividing by zero or trying a privileged instruction in user mode.

Context (processor state)

Everything the processor needs to resume a program exactly where it stopped: the program counter, the status word, the stack pointer and the other registers.

Dispatcher

A small piece of OS code that takes the processor away from one process and hands it to the next process chosen to run.

Modularity

Building a system from separate parts that each do one job and talk to each other only through small, well-defined interfaces, so each part can be understood, tested and replaced on its own.

Multiprocessor

A computer with two or more processors (or cores) that can execute instructions at the same moment.

Multicomputer

A system made of several complete computers, each with its own processor and memory, linked by a network and working together on one job.

Review questions with answers

1. In a **nonprocess kernel**, what happens when the running process issues a supervisor call?

- A. The OS creates a new process to handle the call, and the dispatcher schedules that process.
- B. The kernel's code runs inside the calling process, on that process's own kernel stack.
- C. The process's context is saved and the kernel takes over, outside any process, on its system stack.
- D. The call waits in the Ready queue until the kernel's turn on the processor comes round.

Answer: C. The process's context is saved and the kernel takes over, outside any process, on its system stack.

In the nonprocess design the OS is a separate entity running in kernel mode, with its own memory region and system stack. The idea of a process applies only to user programs.

2. When the OS executes within user processes, why does each process image contain its own **kernel stack**, separate from its user stack?

- A. It holds that process's private copy of the OS code, so that processes cannot interfere with each other's OS.
- B. It gives kernel-mode calls a stack the user program cannot touch, and a system call that must wait can leave its half-finished kernel work there while other processes run.
- C. It stores the process control block while the process is running, and the user stack stores it while the process is waiting.
- D. It lets a system call run without any mode switch, because the kernel stack is already in kernel mode.

Answer: B. It gives kernel-mode calls a stack the user program cannot touch, and a system call that must wait can leave its half-finished kernel work there while other processes run.

OS routines run inside the process, so their calls need a stack. Keeping it apart from the user stack protects the kernel's frames, and because every process has one, the OS can pause a system call halfway (the process is Blocked) and resume it later from exactly that point.

3. When the OS executes within user processes, which of these belong to each process image? Select all that apply.

- A. Process control block
- B. User stack
- C. Private user address space (program and data)
- D. Kernel stack
- E. Shared address space holding OS code and data
- F. A private copy of all the OS code, stored separately for that process

Answer: A, B, C, D, E

Each image has a PCB, a user stack, its private program and data, a kernel stack for kernel-mode calls, and the shared address space. That shared region leads to one physical copy of the OS, so the OS code is never duplicated.

4. Sort each feature into the design it describes.

Separate kernel	Inside user processes	OS as processes
Only user programs are processes	A kernel stack in every process OS code mapped into every process's address space	Kernel functions are processes A usage monitor as a process

Design 1 keeps the kernel outside every process. Design 2 runs OS routines inside each process, so each image carries a kernel stack and the shared region. Design 3 turns OS functions into processes, which also suits noncritical jobs like monitoring.

5. A process-based OS handles 40,000 service requests per second. Each request costs 2 mode switches and 2 process switches. A mode switch takes $0.5\ \mu\text{s}$ and a process switch takes $4\ \mu\text{s}$. How many **milliseconds** of processor time per second go to switching?

Answer: 360 ms

Per request: $2 \times 0.5 + 2 \times 4 = 9\ \mu\text{s}$. Then $40,000 \times 9\ \mu\text{s} = 360,000\ \mu\text{s} = 360\ \text{ms}$, over a third of one processor.

6. An OS that executes within user processes handles 40,000 quick service requests per second. Each costs 2 mode switches of $0.5\ \mu\text{s}$ and no process switch. How many **milliseconds** of processor time per second go to switching?

Answer: 40 ms

Per request: $2 \times 0.5 = 1\ \mu\text{s}$. Then $40,000 \times 1\ \mu\text{s} = 40,000\ \mu\text{s} = 40\ \text{ms}$, about 4% of one processor.

7. Which of these is **not** an advantage of building the OS as a collection of system processes?
- A. It encourages a modular design with small, clean interfaces between parts.
 - B. Noncritical functions, such as a resource-usage monitor, can run as processes at their own priority.
 - C. Some OS services can be moved onto dedicated processors of a multiprocessor.
 - D. Each request for an OS service becomes cheaper, because no process switch is needed to reach it.

Answer: D. Each request for an OS service becomes cheaper, because no process switch is needed to reach it.

The opposite is true. Reaching a service that lives in another process costs process switches, which is the main overhead of this design.

8. Put these events in order for a quick system call when the OS executes within user processes.

Correct order:

1. P1 runs its program in user mode
2. P1 executes the system-call instruction
3. The processor switches to kernel mode (a mode switch)
4. The OS routine runs inside P1, using P1's kernel stack
5. The processor returns to user mode and P1 continues

The call enters kernel mode, the OS routine runs in the context of P1 on its kernel stack, and a second mode switch returns to P1's program. No process switch occurs.

9. Match each stack to the moment it is in use.

User stack	A process is running its own program in user mode
Kernel stack	A process is running an OS routine in kernel mode
The kernel's own system stack	A nonprocess kernel is working outside every process
A system process's stack	An OS function packaged as its own process is running

Design 1 keeps one system stack for the kernel. Design 2 gives every process a user stack and a kernel stack. In design 3 each system process has a stack of its own, like any process.

10. When the OS executes within user processes, a user program can change the OS code, because that code is mapped into its address space.

Answer: False

The shared address space can be used only in kernel mode. In user mode the hardware blocks every access, so the OS is protected even though it appears in every address space.

11. When the OS executes within user processes and a process switch turns out to be necessary, the small routine that performs the switch may run outside all processes.

Answer: True

Depending on the system, the switching routine may or may not start in the current process. But while it takes the processor from one process and gives it to another it belongs to neither, so it is most natural to view it as running outside all processes.

12. Which statements about a mode switch are true? Select all that apply.

- A. It changes the processor between user mode and kernel mode.
- B. The same process remains the running process.
- C. It requires switching the memory map to another process's memory.
- D. It costs much less than a process switch.

Answer: A, B, D

A mode switch flips the processor's mode and saves a little state. Changing the memory map belongs to a process switch, which is why a process switch costs so much more.

3.6 UNIX SVR4 Process Management

You should be able to

- Explain how UNIX SVR4 runs most kernel code inside user processes, and tell system processes from user processes.
- Name the nine UNIX process states, map each onto the seven-state model, and explain why Preempted is a separate state.
- Trace a process through the UNIX transition diagram, including sleeping, swapping, preemption and becoming a zombie.
- Describe the process image as user-level, register and system-level context, and say which fields live in the process table entry and which in the U area.
- List the six things the kernel does during fork(), name its three choices afterwards, and predict what a program that calls fork() prints.

1. Where the kernel runs in UNIX SVR4

In SVR4 **most of the OS executes inside the environment of a user process**. On a system call, exception or interrupt the **same process** switches to kernel mode and runs kernel code on its own kernel stack: a cheap mode switch, not a process switch. Interrupts are handled inside whichever process is running.

- **System processes** run **only in kernel mode**, executing OS housekeeping code such as memory allocation and swapping (e.g. process 0, the swapper).
- **User processes** run programs and utilities in **user mode**, and switch to **kernel mode** to execute kernel instructions on a system call, exception or interrupt.

2. The nine UNIX process states

UNIX state	Meaning	Seven-state
User Running	Executing its own program in user mode.	Running
Kernel Running	The same process executing kernel code in kernel mode.	
Ready to Run, in Memory	Ready to run as soon as the kernel schedules it.	Ready
Preempted	Was returning from kernel mode to user mode, but the kernel preempted it and switched to another process.	
Asleep in Memory	Cannot run until an event occurs; image in main memory.	Blocked
Ready to Run, Swapped	Ready, but the swapper must bring it into main memory first.	Ready/Suspend
Sleeping, Swapped	Waiting for an event and swapped out to disk.	Blocked/Suspend
Created	Newly created, not yet ready to run.	New
Zombie	No longer exists, but leaves a record (exit status, usage times) for its parent to collect.	Exit

Running is split by mode; Ready is split into Ready to Run, in Memory and Preempted (below, “Ready in Memory” and “Ready, Swapped” are short for the two Ready to Run states). Those two are essentially the same state (one queue for the scheduler), but Preempted is kept apart to stress that preemption happens **only as a process is about to return from kernel mode to user mode**. The traditional kernel is **nonpreemptible**: it never **forces** a process off the processor in the middle of kernel code (involuntary preemption waits until the process is about to return to user mode), which makes it unsuitable for real-time work. This does not mean no switch can happen in kernel mode: a process that **sleeps** inside the kernel to wait for an event gives up the processor **voluntarily**, and the kernel switches to another process right there.

3. Transitions and a typical life

- **fork** → Created; **enough memory** → Ready in Memory; **not enough memory** (swapping systems) → Ready, Swapped.
- **reschedule**: Ready in Memory → Kernel Running. **return to user**: Kernel Running → User Running. **system call / interrupt**: User Running → Kernel Running; an interrupt in the kernel returns to Kernel Running.
- **sleep**: Kernel Running → Asleep in Memory. **wakeup**: Asleep in Memory → Ready in Memory; Sleeping, Swapped → Ready, Swapped.
- **swap out**: Asleep → Sleeping, Swapped; Ready in Memory → Ready, Swapped. **swap in**: Ready, Swapped → Ready in Memory.
- **preempt**: Kernel Running → Preempted; **return to user**: Preempted → User Running. **exit**: Kernel Running → Zombie.

A process is always switched out, and always resumes, inside the kernel (Kernel Running).

4. Process 0 and process 1

Process 0, the **swapper**, is built by the kernel at boot. It spawns **process 1**, **init**, the ancestor of every other process. **init** creates a user process for each interactive user who logs in (the user's shell). Every process except process 0 is created by `fork()`, which needs an existing process to copy.

5. The process image: three kinds of context

- **User-level context:** process text (executable machine instructions), process data, user stack (arguments, locals, return addresses in user mode), shared memory (one physical copy, mapped by every sharer).
- **Register context:** program counter, processor status register (hardware status, including mode), stack pointer (user or kernel stack), general-purpose registers. Copied to memory when the process enters the kernel or is switched out.
- **System-level context:** process table entry (always accessible to the kernel), U area (needed only while the process runs), per-process region table (virtual-to-physical mapping plus permission field: read-only, read-write, read-execute), kernel stack (frames of kernel functions called in kernel mode).

6. Process table entry versus U area

Process table entry: process status; pointers to the U area and memory; process size; user IDs (real, effective); process IDs (own, parent); event descriptor (event a sleeper awaits); priority; signal (sent, not yet handled); timers (execution time, resource use, alarm); **P_link** (next in the ready queue); memory status (in memory or swapped; locked).

U area: process table pointer; user IDs; timers (user- and kernel-mode time of the process and its children); signal-handler array (exit, ignore or run a function); control terminal; error field; return value (of a system call); I/O parameters; file parameters (current directory and root); user file descriptor table (open files); limit fields; permission modes fields.

User IDs and timers appear in both. **Rule of thumb:** needed even when the process is not running (to schedule, wake, signal or swap it)? Process table entry, which always stays in memory. Needed only while it runs its own system calls? U area, which can be swapped out.

7. Process creation with fork()

A process is created by the system call `pid = fork();`. In kernel mode, inside the parent, the kernel:

1. allocates a slot in the process table for the new process;
2. assigns a unique process ID to the child;
3. copies the parent's process image, except any shared memory (shared, not copied);
4. increments the counters of files the parent has open (the child owns them too);
5. assigns the child to the Ready to Run state;
6. returns the ID of the child to the parent, and 0 to the child.

Then the kernel chooses one of three options: **stay in the parent** (it returns to user mode at the point of the fork call); **switch to the child** (it starts at the return from `fork()`, not at the top of its program); or **switch to another process** (parent and child both stay Ready to Run). The choice changes the order of

output, never the amount (with output to a terminal; see section 8). The return value tells them apart (0 = child; the child's ID = parent; -1 = failure). Typically the child calls `exec` to load a new program (same PID), and the parent's `wait()` sleeps until the child exits, then collects its zombie record. Real kernels defer copying (copy-on-write) with the same visible effect.

```
pid = fork();                /* clone the caller */
if (pid == 0) execlp("ls","ls",NULL); /* child: become ls */
else wait(NULL);             /* parent: wait      */
```

8. Counting processes and printed lines

After a fork **both** processes run every later line, including later forks, so n forks in a row give 2^n processes (three forks, then `printf`: 8 lines). `fork(); printf("A\n"); fork(); printf("B\n");` prints 2 A + 4 B = 6 lines. `if (fork() == 0) fork(); printf("X\n");` prints 3 lines (parent, child, grandchild). Line order may vary between runs; with output to a terminal, the count cannot.

Why the count can change: output buffering

These counts assume the output goes to a **terminal**. There the C library writes each line as soon as `printf` prints its `\n` (line buffering). If the output is **redirected to a file or a pipe**, the library switches to full buffering: printed text waits in a buffer in the process's own memory until the buffer fills or the process exits. Because `fork()` copies that memory, it also copies any text still waiting. In the A/B program each of the two processes still holds an unwritten "A\n" at the second fork, so all four processes end up writing A and B: 4 A + 4 B = **8** lines. Calling `fflush(stdout)` just before `fork()` empties the buffer first and restores the count of 6. (For the same reason, a buffered line printed just before `exec` can vanish: `exec` replaces the memory that held it.)

Key terms

UNIX System V Release 4 (SVR4)

An influential release of AT&T's UNIX, from the late 1980s, that combined features of several UNIX families. Its process design is the classic UNIX model.

System call

A request a running program makes to the kernel for a service it may not perform itself, such as reading a file. It switches the processor into kernel mode.

System process

In UNIX, a process that runs only in kernel mode and carries out operating-system housekeeping, such as allocating memory or swapping processes in and out.

User process

In UNIX, a process that runs a user's program in user mode and switches into kernel mode, still as the same process, whenever it makes a system call, causes an exception (fault) or an interrupt arrives.

Nonpreemptible kernel

A kernel that never forces a process off the processor while that process is executing kernel code; forced switches wait for a safe point, such as the return to user mode. A process may still give up the processor voluntarily in the kernel, by sleeping.

Preempted state

The UNIX state of a process that was about to return from kernel mode to user mode when the kernel switched to a more important process instead. It waits just like a ready process.

Zombie

A process that has finished. Its memory is gone, but a small record (its exit status and usage totals) stays in the process table until its parent collects it.

Swapper (process 0)

The first process, built by the kernel itself when the system boots. It is a system process whose job is to move process images between main memory and disk.

init (process 1)

The second process, created by process 0 at boot. Every other process is its descendant; it starts a process for each user who logs in.

User-level context

The part of a UNIX process image the program itself sees: its machine instructions (text), its data, its user stack and any shared memory.

Register context

The processor register values that belong to a process: program counter, processor status register, stack pointer and general-purpose registers. Saved when the process stops running.

System-level context

The part of a UNIX process image only the kernel uses: the process table entry, the U area, the per-process region table and the kernel stack.

Process table entry

The kernel's always-reachable record for one process: its state, IDs, priority, pending signals, the event it sleeps on and where its image is.

U area (user area)

A per-process kernel record holding information the kernel needs only while that process is running, such as its open-file table and system-call parameters.

Per-process region table

A kernel table describing the memory regions (text, data, stack) of one process: how its virtual addresses map to physical memory and whether each region is read-only or writable.

Kernel stack

A second, private stack a process uses while it executes kernel code, holding the kernel functions' local variables and return addresses.

Shared memory

A region of memory that two or more processes can all read and write, used to pass data between them quickly.

Signal

A short software notification the kernel delivers to a process, such as "your child has finished" or "please terminate".

fork()

The UNIX system call that creates a new process as an almost exact copy of the caller. It returns the child's process ID to the parent and 0 to the child.

exec

A family of UNIX system calls that replaces the calling process's program with a new program loaded from a file. The process ID stays the same.

Review questions with answers

1. In UNIX SVR4, what happens when a user process makes a system call?

- A. The same process switches into kernel mode and executes the kernel's code itself
- B. The request is passed to a separate kernel process, which runs it and replies
- C. The process is swapped out to disk until the kernel has finished
- D. The swapper (process 0) executes the call on the process's behalf

Answer: A. The same process switches into kernel mode and executes the kernel's code itself

SVR4 runs most of the OS within the environment of the user process: a system call is just a switch to kernel mode inside the same process, with no process switch needed.

2. A UNIX system process runs in user mode most of the time and switches into kernel mode only for system calls.

Answer: False

That describes a user process. A system process runs only in kernel mode and executes OS code for housekeeping, such as memory allocation and swapping.

3. Match each UNIX process state to its meaning.

Asleep in Memory	Waiting for an event; image in main memory
Sleeping, Swapped	Waiting for an event; image on disk
Ready to Run, Swapped	Could run, but must be swapped in first
Created	Just made by fork(); not yet ready to run
Zombie	Finished; leaves a record for its parent
Preempted	Was returning to user mode when the kernel switched away

Two questions sort most states: can it run (ready or waiting for an event)? And where is its image (main memory or disk)? Created, Zombie and Preempted mark birth, death and the preemption point.

4. Why does UNIX show Preempted as a state separate from Ready to Run, in Memory?

- A. To stress that preemption happens only when a process is about to return from kernel mode to user mode
- B. Because a preempted process is kept on disk until it runs again
- C. Because a preempted process jumps ahead of every other ready process
- D. Because a preempted process must restart its system call from the beginning

Answer: A. To stress that preemption happens only when a process is about to return from kernel mode to user mode

The two states are essentially the same kind of waiting. Keeping Preempted separate shows that the traditional kernel is nonpreemptible: it never forces a process off the processor in the middle of kernel code, only at the return to user mode (a process may still give it up voluntarily by sleeping in the kernel). That also makes it unsuitable for real-time work.

5. Which of these correctly map a UNIX state onto the seven-state model?

- A. User Running and Kernel Running → Running
- B. Preempted → Blocked
- C. Sleeping, Swapped → Blocked/Suspend
- D. Created → New
- E. Zombie → Ready/Suspend

Answer: A, C, D

Running splits by mode into User Running and Kernel Running. Preempted is a kind of Ready (not Blocked), and Zombie corresponds to Exit.

6. Where does the kernel keep each field: in the process table entry or in the U area?

Process table entry	U area
Event descriptor Signal (sent but not yet handled) Memory status	Signal-handler array User file descriptor table

The process table entry holds what the kernel needs even when the process is not running (to schedule, wake or swap it). The U area holds what is needed only while the process runs its own system calls.

7. Which part of the UNIX process image does each item belong to?

User-level	Register	System-level
Process text	Program counter Processor status register	Kernel stack Per-process region table

User-level context is what the program itself sees, register context is the processor's registers for this process, and system-level context is the kernel's private bookkeeping about it.

8. Put the kernel's work during `fork()` in order.

Correct order:

1. Allocate a slot in the process table
2. Assign a unique process ID to the child
3. Copy the parent's process image, except shared memory
4. Increment the counters of files the parent has open
5. Put the child in the Ready to Run state
6. Return the child's ID to the parent and 0 to the child

The kernel first reserves a slot and an ID, then builds the child as a copy of the parent (sharing, not copying, shared memory), accounts for the inherited open files, makes the child ready, and finally returns two different values.

9. How many lines does this program print in total?

```
if (fork() == 0)    // fork; did it return 0?
    fork();        // runs only where it returned 0
printf("X\n");     // print one line
```

Answer: 3 lines

`fork()` returns the new child's ID (never 0) in the parent and 0 in the child. So the parent skips the inner `fork`, while the child runs it and makes a grandchild. Parent, child and grandchild each print X once: 3 lines.

10. A UNIX kernel has just completed all of its work for a `fork()` call. Which of these may it do next?

- A. Return to the parent in user mode, at the point of the `fork` call
- B. Switch to the child, which starts executing at the return from `fork()`
- C. Switch to some other process, leaving both parent and child Ready to Run
- D. Start the child at the first line of its program's main function
- E. Hold the child in the Created state until the parent calls `wait()`

Answer: A, B, C

The kernel may stay in the parent, switch to the child, or run another process. The child never starts from the top of its program: it resumes at the return from `fork()`, and it is already Ready to Run, not Created.

11. How many lines (counting both A and B) does this program print in total? Assume the output goes to a terminal, so each `printf` that ends in `\n` is written out at once.

```
fork();           // ask for a child
printf("A\n");    // print A
fork();           // ask for another child
printf("B\n");    // print B
```

Answer: 6 lines

Every process alive runs each later `fork`, so each `fork` doubles the count. Two processes exist when A is printed (2 lines). Both then `fork`, making 4 processes, and all 4 print B (4 lines): $2 + 4 = 6$. **Why the count can change:** sent to a file or pipe, the output is buffered inside each process, the second `fork` copies the unwritten "A", and 8 lines appear; `fflush(stdout)` before `fork()` prevents that.

12. Which process does the kernel build by hand when the system boots, rather than creating it with `fork()`?

- A. Process 0, the swapper
- B. Process 1, init
- C. The first login shell
- D. The first program a user runs

Answer: A. Process 0, the swapper

Process 0 (the swapper) is hand-made at boot. It creates process 1 (init), from which every other process descends; init starts a process for each user who logs in.

Chapter 4 · Threads

One process, several paths of execution: why that helps, and how real systems build it.

A process bundles two separate ideas: owning resources and being scheduled to run. Threads pull them apart, so one process can run several independent paths of execution that share its memory and files. This chapter compares threads managed by a library inside the program (user-level) with threads managed by the kernel (kernel-level), uses a short formula, Amdahl's law, to measure what multicore chips can really gain, and tours thread management in Windows, Solaris, Linux, Android and Mac OS X (today's macOS).

Learning objectives

- Distinguish the process (the unit that owns resources) from the thread (the unit that runs).
- Describe thread states, thread operations and why threads must synchronize.
- Compare user-level threads, kernel-level threads and combined approaches.
- Use Amdahl's law to estimate the speedup multithreaded software can get from more cores.
- Describe thread management in Windows, Solaris, Linux, Android and Mac OS X (Grand Central Dispatch).

How Chapter 4 fits together

Chapter 3 treated the process as one thing. This chapter splits it into two ideas: the process as the owner of resources (address space, files, devices) and the **thread** as the unit that is scheduled and runs.

The idea (4.1)

In a multithreaded process, all threads share the code, data, heap, open files and address space. Each thread keeps its own program counter, registers, stack, execution state and a small **thread control block**. Compared with separate processes, threads are quicker to create, end and switch between, and they communicate through shared memory without calling the kernel. Threads have states (Running, Ready, Blocked) and operations (spawn, block, unblock, finish). Because they share data, they must **synchronize**, which leads straight into Chapter 5.

Who manages threads (4.2)

User-level threads live in a library inside the process. Switching them needs no kernel help, but one blocking system call can stall them all and they cannot run on two processors at once. **Kernel-level threads** are known to the OS, so they can block independently and run in parallel, at the cost of a mode switch per thread switch. **Combined** designs map many user threads onto a smaller or equal number of kernel threads.

What more cores can buy (4.3)

Amdahl's law limits the speedup: if a fraction f of a program can run in parallel on N processors, speedup = $1 / ((1 - f) + f / N)$. Even a small serial part caps the gain. Worked example: with $f = 0.9$ and $N = 8$, speedup = $1 / (0.1 + 0.1125) \approx 4.7$, far short of 8.

Real systems (4.4 to 4.8)

Each case study answers the same questions differently. **Windows** builds processes and threads as objects and gives threads a larger set of states. **Solaris** layers user threads, lightweight processes and kernel threads. **Linux** treats both as tasks created by clone, with flags choosing what is shared. **Android** organises apps into components and processes with a life cycle. **Mac OS X Grand Central Dispatch** lets programmers submit blocks of work to queues and leaves thread management to the system.

Why it matters next

Threads and processes that share data can interfere with one another. Chapter 5 shows how to keep them correct.

4.1 Processes and Threads

You should be able to

- Explain the two characteristics of a process (resource ownership and scheduling/execution) and why separating them gives us threads.
- Match the four process/thread models to example systems, and say what a process owns versus what each thread owns.
- State the four performance benefits of threads and recognise the four ways threads are used in a single-user system.
- Trace thread states and the Spawn, Block, Unblock and Finish operations, including interleaving on one processor and an RPC example.
- Explain why threads that share one address space must synchronize their work.

1. Two characteristics of a process

- **Resource ownership.** A virtual address space holding the process image (code, data, stacks and the attributes kept in the PCB), plus control of resources such as main memory, files and I/O devices. The OS protects these from other processes.
- **Scheduling and execution.** An execution path (trace) through one or more programs, with an execution state (Running, Ready, Blocked...) and a dispatching priority. This is the part the OS schedules and dispatches.

The two are independent, so an OS can separate them. The unit of dispatching is the **thread** (or **lightweight process**); the unit of resource ownership is the **process** (or **task**). **Multithreading** is the ability of an OS to support several concurrent paths of execution within one process.

2. Four process/thread models

Model	Example
One process, one thread	MS-DOS
One process, many threads	A Java runtime environment (one virtual machine process)
Many processes, one thread each	Traditional UNIX
Many processes, many threads each	Windows, Solaris, modern UNIX versions

3. What the process has, what each thread has

The process: a virtual address space holding the process image, and protected access to processors, other processes (IPC), files and I/O resources.

Each thread: an execution state; a saved thread context when not running (in effect, its own program counter); an execution stack; per-thread static storage for local variables (thread-local storage); and access to the memory and resources of its process, shared with its siblings.

Threads share one address space, so each can see the data the others write, and a file opened by one thread can be used by all. Seeing another thread's update **reliably and in the right order** is not automatic: it takes synchronization (a lock or an atomic operation), because processors and compilers may delay or reorder memory writes (section 5.1 covers memory order). A thread's stack is its own but lives in the shared address space: the OS does not protect threads of one process from each other.

4. Single-threaded vs multithreaded process models

Structure	Single-threaded	Multithreaded
Process control block (PCB)	1, also holds the registers and state	1, process-wide facts only
User address space	1	1, shared by all threads
Thread control block (TCB): registers, priority, state	none	1 per thread
User stack and kernel stack	1 each	1 each per thread

Each thread needs its own kernel stack because the kernel may be working for several threads at once.

Process-wide actions affect every thread: suspending a process swaps its one address space out of main memory, so all its threads are suspended together; terminating a process terminates all of its threads.

5. Four performance benefits of threads

1. **Create:** far quicker than creating a process (no new address space, image, file table or protection).
2. **Terminate:** quicker than ending a process (the address space and files remain).
3. **Switch:** switching between threads of one process is quicker than a process switch (the memory map and cached data stay).

4. **Communicate:** threads share memory, so they exchange data without invoking the kernel; separate processes need kernel-mediated IPC.

Worked example (illustrative costs): a file server creates and ends a helper for every request, 500 μ s for a process or 50 μ s for a thread. Overhead = requests per second $\times$ cost. At 800 requests/s: $800 \times 500 \mu\text{s} = 0.4$ s of every second, 40% of one processor, versus 4% with threads. At 1,200/s: 60% versus 6%. At 2,000/s the process version uses 100%. Busy servers often go further and reuse a pool of threads.

6. Uses of threads in a single-user system

- **Foreground and background work:** one spreadsheet thread shows menus and reads input while another runs commands and updates the sheet.
- **Asynchronous processing:** work set off by a timer or outside event, e.g. saving a word processor's buffer to disk once a minute.
- **Speed of execution:** compute one batch while another thread reads the next; on a multiprocessor, threads of one process run at the same time.
- **Modular program structure:** programs with many activities or many I/O sources and destinations are easier to design as threads.

7. Thread states and operations

States: **Running, Ready, Blocked**. There is no thread-level Suspended state: suspension swaps out the shared address space, a process-level act.

- **Spawn:** creating a process spawns its first thread; a thread may spawn another in the same process, supplying a start address and arguments. The new thread gets its own register context and stacks and joins the ready list.
- **Block:** the thread waits for an event; its registers, program counter and stack pointer are saved and the processor runs another ready thread.
- **Unblock:** the event occurs and the thread returns to the ready list.
- **Finish:** its register context and stacks are deallocated.

Whether one blocked thread blocks its whole process depends on how threads are implemented (Section 4.2). On a **uniprocessor** only one thread runs at a time, so threads of several processes are interleaved. The processor passes on when the running thread blocks, finishes, uses up its time quantum, or is preempted by a higher-priority thread. A switch between threads of one process keeps the memory map; a switch to another process changes it.

8. Example: remote procedure calls

An RPC runs a procedure on another computer; the caller is blocked until the reply arrives. Suppose sending a request and handling a reply each take 1 ms of processor time, and the replies arrive a and b ms after their sends.

- **One thread:** the waits happen one after the other: total = $4 + a + b$ ($a = b = 5$ gives 14 ms; $a = b = 6$ gives 16 ms).
- **One thread per server, one processor:** T1 sends at 0–1, T2 at 1–2, and then both wait together. Replies arrive at $1 + a$ and $2 + b$ and are handled one at a time. With $a = b = 5$ they are handled at 6–7

and 7–8, so the job ends at 8 ms (1.75× faster); with $a = b = 6$ it ends at 9 ms.

The gain comes from overlapping the waits, not from running at the same instant.

9. Why threads must synchronize

Threads of a process share its address space and resources, so their activities must be coordinated or they can interfere and corrupt shared data. `count = count + 1` is three machine steps: LOAD (copy count into a private register), ADD, STORE (write it back). If T1 and T2 both LOAD 10 before either STOREs, both store 11 and one increment is lost: a **lost update**, a kind of race condition. If one thread stores before the other loads, count ends at 12. It can never end at 10 or 13. A lock that lets only one thread at a time run the three steps always gives 12; Chapter 5 builds such tools.

Key terms

Resource ownership

The side of a process that holds things: a virtual address space containing the process image, plus open files, I/O devices and other resources, all protected by the OS from other processes.

Scheduling and execution

The side of a process that runs: a path through program code, with its own execution state (Running, Ready, Blocked...) and a dispatching priority the scheduler uses.

Unit of dispatching

Whatever the scheduler picks to run on a processor. In a multithreaded operating system this is the thread, not the whole process.

Task

Another name for a process when we mean the unit that owns resources (address space, files, devices) rather than the thing that runs.

Lightweight process

Another name for a thread, the unit that gets dispatched. It is "lightweight" because it carries only execution state, not a whole set of resources. (Some systems, such as Solaris, also use the name for one specific kernel structure.)

Multithreading

The ability of an operating system to support several concurrent paths of execution (threads) inside a single process.

Virtual address space

The range of memory addresses a process may use, which the OS maps onto real memory. It holds the process image, and every thread of the process works inside this one space.

Thread context

The register values, above all the program counter and stack pointer, that a thread needs to resume exactly where it stopped. It is saved in the thread control block whenever the thread is not running.

User stack

The stack a thread uses while running ordinary program code in user mode: the return addresses, parameters and local variables of its function calls.

Kernel stack

A separate stack used while the kernel runs on behalf of a thread (during a system call or an interrupt), so kernel work never mixes with the thread's user stack.

Thread-local storage (TLS)

A small amount of static storage that belongs to one thread only, such as its own copy of an error code or a counter. Every thread has its own copy.

Interprocess communication (IPC)

Any mechanism that lets separate processes exchange data or signals, such as pipes, messages or shared memory. It normally needs the kernel's help, because processes are protected from each other.

Dispatching priority

A number the scheduler uses to decide which ready unit of execution gets the processor first. A higher-priority thread is served before a lower-priority one.

Spawn

The thread operation that creates a new thread: it gets its own register context and stack and is placed on the ready list. Creating a process also spawns its first thread.

Remote procedure call (RPC)

A call to a procedure that runs on another computer. The caller sends a request over the network and waits (is blocked) until the reply comes back.

Time quantum

The longest stretch a thread may run before the scheduler can take the processor away and give another ready thread a turn. Also called a time slice.

Preemption

Taking the processor away from a running thread before it chooses to stop, for example because its time quantum expired or a higher-priority thread became ready.

Uniprocessor

A computer with a single processor (one core). Only one thread can execute at any instant, so threads take turns.

Thread synchronization

Coordinating threads that share data so their actions happen in a safe order and cannot corrupt shared data structures.

Lost update

A race-condition outcome: two threads read the same old value, each writes back its own result, and the second write silently wipes out the first.

Review questions with answers

1. In an operating system that supports threads, what is the **unit of dispatching**, the thing the scheduler picks to run?

- A. The process
- B. The thread
- C. The process control block
- D. The virtual address space

Answer: B. The thread

Threads separate the two characteristics of a process: the thread (lightweight process) is dispatched, while the process (task) owns the resources.

2. Which is the standard example of the model with **a single process that contains multiple threads** (rather than many processes)?

- A. MS-DOS
- B. A Java runtime environment
- C. Traditional UNIX
- D. Windows

Answer: B. A Java runtime environment

A Java runtime environment is a single process (the virtual machine) in which the program can run many threads.

3. In a multithreaded process, does the OS keep one of these **per process** or **per thread**?

One per process	One per thread
Virtual address space holding the process image Open files Process control block	Thread control block Execution state (Running, Ready, Blocked) Saved register context User stack and kernel stack

Resources (address space, files, the PCB) exist once for the whole process. Everything that describes a path of execution (TCB, state, registers, stacks) exists once per thread.

4. Which of these are performance benefits of threads compared with separate processes?

- A. A thread takes less time to create than a process
- B. A thread takes less time to terminate than a process
- C. Switching between two threads of the same process is quicker than switching between processes
- D. The OS protects threads of one process from each other
- E. Threads of one process can exchange data without involving the kernel

Answer: A, B, C, E

The four classic benefits are faster create, faster terminate, faster switch, and kernel-free communication through shared memory. Threads of one process are **not** protected from each other; they share one address space.

5. Match each situation to the use of threads it illustrates.

A word processor saves a backup copy of its buffer every minute	Asynchronous processing
A spreadsheet keeps reading input while another thread recalculates the sheet	Foreground and background work
One thread computes on a batch of data while another reads the next batch	Speed of execution
A program with many I/O sources is written as one thread per source	Modular program structure

Timed or periodic jobs are asynchronous; keeping the user side responsive is foreground/background; overlapping computing with I/O is speed; organising many activities is modularity.

6. In the general thread model, one thread of a process can be swapped out to disk (the Suspended state) while its sibling threads keep running.

Answer: False

Suspension swaps the address space out of main memory, and every thread shares that address space. It is therefore a process-level state: suspending a process suspends all of its threads. (Windows can also "suspend" one thread, as section 4.4 shows, but that only stops it being scheduled; nothing is swapped out.)

7. Terminating a process terminates every thread in that process.

Answer: True

Threads live inside the process and use its resources. When the process ends, its address space and resources are released, so all of its threads end too.

8. Put the life of a thread that makes one remote procedure call in order.

Correct order:

1. Spawn: the thread is created and placed on the ready list
2. It is dispatched and runs, sending its request
3. Block: it waits for the reply
4. Unblock: the reply arrives and the thread becomes Ready
5. It runs again and handles the reply
6. Finish: its register context and stacks are freed

Spawn puts a new thread in Ready; running leads to Block while it waits; Unblock returns it to Ready; after running again it Finishes and its resources are freed.

9. One thread makes two remote procedure calls, one after the other, to two different servers. Sending a request takes 1 ms of processor time, handling a reply takes 1 ms, and each server's reply arrives 6 ms after the request was sent. How many ms does the whole job take?

Answer: 16 ms

Four pieces of processing (send A, handle A, send B, handle B) take 4 ms, and the two 6 ms waits happen one after the other: $4 + 6 + 6 = 16$ ms.

10. Now two threads share one processor, one thread per server. T1 sends its request from 0 to 1 ms; T2 sends its request from 1 to 2 ms. Each reply arrives 6 ms after its send finishes, and handling a reply takes 1 ms of processor time. At what time (ms) is all the work finished?

Answer: 9 ms

Reply A arrives at $1 + 6 = 7$ ms and is handled from 7 to 8. Reply B arrives at $2 + 6 = 8$ ms and is handled from 8 to 9. The waits overlap, so the job ends at 9 ms instead of 16.

11. A file server creates a helper for every request and ends it afterwards. Creating plus ending a **process** costs 500 μ s. At 1,200 requests per second, what percentage of one processor is spent on that overhead?

Answer: 60 %

$1,200 \times 500 \mu\text{s} = 600,000 \mu\text{s} = 0.6$ s of every second, which is 60%. With threads at 50 μ s each it would be only 6%.

12. Two threads of one process both run `count = count + 1` on a shared variable that starts at 10, with no synchronization. Which final values are possible?

- A. 10
- B. 11
- C. 12
- D. 13

Answer: B, C

If the increments do not overlap, count ends at 12. If both threads read 10 before either writes, both write 11 and one update is lost. It can never stay 10 or reach 13. Preventing the lost update is the job of thread synchronization.

4.2 Types of Threads

You should be able to

- Explain who manages threads in the user-level (ULT) and kernel-level (KLT) approaches, and what the kernel can and cannot see in each.
- Trace how ULT states relate to process states when a thread blocks, when the time slice ends, and when one thread waits for another.
- Weigh the advantages and disadvantages of ULTs and KLTs, including the workarounds, using measured latency numbers.
- Describe the combined approach and the 1:1, M:1, 1:M and M:N thread-to-process arrangements, with an example of each.

Two ways to manage threads

Every thread needs a manager: something must create it, decide when it runs, and save and restore its place when it pauses. With **user-level threads (ULTs)** a thread library inside the application does that job and the kernel never learns the threads exist. With **kernel-level threads (KLTs)**, also called kernel-supported threads or lightweight processes (the general name for a thread from section 4.1; Solaris gives it a narrower meaning, see 4.5), the kernel manages every thread itself. A **combined approach** mixes the two.

User-level threads

The application manages its own threads through a **thread library**: ordinary routines linked into the program that run in user mode. Its four jobs: **create and destroy threads**, **pass messages and data between threads**, **schedule** which thread runs next, and **save and restore thread contexts** (program counter, stack pointer and other registers). The kernel knows none of this: it schedules the **process as a single unit** with one execution state (Running, Ready or Blocked).

How ULT states relate to the process state

Start: process B has two ULTs; the library says Thread 2 Running, Thread 1 Ready; the kernel says B Running.

Scenario	Kernel's record for B	Library's table
a) Thread 2 makes a blocking system call (e.g. read)	B → Blocked while the I/O runs; then Ready, later Running.	Unchanged: Thread 2 still "Running", Thread 1 Ready but unable to run. B resumes inside Thread 2.
b) Clock interrupt: B's time slice is used up	B → Ready; later Running.	Unchanged: Thread 2 still "Running".
c) Thread 2 needs some action by Thread 1	B stays Running the whole time.	Thread 2 → Blocked, Thread 1 → Running (a library thread switch). Later Thread 2 → Ready.

In a) and b) the library's "Running" is only a belief: none of B's code runs, so nothing updates the table.

Advantages of ULTs

1. **Cheap switches.** Every thread record lives in the process's own memory, so switching threads never needs kernel-mode privileges and skips the two mode switches (user → kernel and kernel → user).
2. **Custom scheduling.** Each application can use the policy that suits it (say, favour its UI thread) without touching the kernel's scheduler.
3. **Runs on any OS.** The library is ordinary application code, so the kernel needs no changes and no thread support.

Disadvantages of ULTs

1. **A blocking system call blocks every thread of the process**, because the kernel blocks the only thing it knows: the process.
2. **No true multiprocessing.** The kernel assigns a process to one processor at a time, so only one thread of the process can execute at any moment.

Example: 3 threads × 4 ticks of CPU, and Thread 2 reads the disk for 4 ticks. On one core ULTs take 16 ticks (the process sits blocked for 4) and KLTs 12; on two cores KLTs take 8 (6 with no read), while ULTs still take 16 and never use core 2.

Working around the limits

- **Write the application as multiple processes** instead of threads. This fixes both limits, but every switch becomes a costly process switch.
- **Jacketing** (fixes blocking only): turn a blocking system call into a nonblocking one. The "jacket" first asks the kernel, without waiting, whether the call would block. If so, the library marks the thread

waiting, runs another thread and checks again later; the real call is made only when it will return at once.

```
int jacket_read(int fd, char *buf, int n) { // replaces read()
    while (!io_ready(fd)) { // would read block? (quick check)
        mark_waiting(me, fd); // yes: this thread waits
        thread_yield();       // run another thread meanwhile
    }                         // resumed later: check again
    return read(fd, buf, n); // data ready: cannot block
}
```

Kernel-level threads

With KLTs the application contains no thread-management code; it asks the kernel for threads through system calls. The kernel keeps one record for the process **and** a saved context for each of its threads, and its scheduler chooses among **threads**, not whole processes. Windows and Linux work this way.

Advantages

1. Several threads of one process can run on several processors at the same time.
2. If one thread of a process blocks, the kernel can schedule another thread of the same process.
3. Kernel routines themselves can be multithreaded.

Disadvantage

Handing the processor from one thread to another **of the same process** has to go through the kernel: a mode switch in, and another back out. KLTs win on blocking and parallelism, not on switch speed.

What it costs: measured latency

Two classic benchmarks, both measured on the same older single-processor machine. **Null fork** measures pure creation overhead: make a thread or process whose body is empty, schedule it, let it run and finish. **Signal-wait** measures synchronization overhead: one thread or process wakes a waiting partner and then waits itself.

Operation (µs)	ULT	KLT	Process
Null fork	34	948	11,300
Signal-wait	37	441	1,840

Worked examples: $11,300 \div 34 \approx 332$, so one process creation costs about 332 ULT creations; $948 \div 34 \approx 27.9$ and $11,300 \div 948 \approx 11.9$. For signal-wait, $441 \div 37 \approx 11.9$ and $1,840 \div 441 \approx 4.2$. Creating 1,000 threads takes $1,000 \times 34 \mu s = 34 \text{ ms}$ with ULTs, 948 ms with KLTs and 11.3 s with processes; at 1,000 creations every second that is 3.4% of a processor, 94.8%, or more than the machine has.

Why the gaps: a ULT operation is a library call; a KLT operation must enter the kernel (mode switches plus bookkeeping); a process operation also builds or switches a whole address space. Today's numbers are smaller but the pattern holds, and if most thread switches need kernel services anyway, the ULT advantage shrinks.

Combined approach

Some systems mix both designs; older Solaris is the classic example (see 4.5). The application creates its threads with library calls, and the library also does most of their **scheduling and synchronization**, all in user space. Underneath, the library runs the application's M user-level threads on a **smaller or equal number N** of kernel-level threads, and the programmer can tune N for the application and the machine. Done well, most thread operations stay cheap, threads can run in parallel on several processors, and one blocking call need not stop the whole process. With N = 1 it behaves like pure ULTs (often called **many-to-one**); with one KLT per ULT it behaves like pure KLTs (**one-to-one**); in between it is **many-to-many**.

Other arrangements: threads per process

Threads : processes	Meaning	Examples
1 : 1 (1 thread : 1 process)	Every path of execution is its own process, with a private address space and resources.	Traditional UNIX
M : 1 (M threads : 1 process)	One process owns the address space and resources; many threads run inside it and share them.	Windows NT, Solaris, Linux, OS X, iOS
1 : M (1 thread : M processes)	A thread can migrate out of one process environment into another, even on another computer.	Ra (Clouds), Emerald
M : N (M threads : N processes)	Both at once: many threads per process, and threads can migrate.	TRIX

M : 1 (many threads in one process) is the everyday case; 1 : M and M : N come from distributed-systems research. Do not confuse these threads : processes ratios with the one-to-one / many-to-one / many-to-many names for mapping ULTs onto KLTs; they answer different questions. A Linux process running 50 threads is "M threads : 1 process", yet each of those threads has its own kernel-level thread, so its ULT-to-KLT mapping is one-to-one, not many-to-one. Each ratio describes one process, whereas the four models of section 4.1 describe a whole system: Windows, for example, runs many processes, and each of them is M : 1.

Key terms

User-level thread (ULT)

A thread that is created, scheduled and switched entirely by a thread library inside the application. The kernel does not know it exists; it only sees the process that contains it.

Kernel-level thread (KLT)

A thread that the kernel itself creates, keeps records for and schedules. Also called a kernel-supported thread or a lightweight process.

Lightweight process (LWP)

Another name for a thread (section 4.1), the unit that gets dispatched; when threads are sorted into user-level and kernel-level, the name usually means a kernel-level thread. Solaris uses it for one specific kernel-side carrier that user-level threads run on (section 4.5).

Thread library

A package of ordinary user-mode routines, linked into a program, that creates and destroys threads, passes data between them, schedules them, and saves and restores their contexts.

Thread context

Everything a thread needs to resume exactly where it stopped: its program counter, stack pointer and other register values.

Kernel mode

The privileged processor mode in which operating-system code runs; it may execute any instruction and touch any memory. Ordinary programs run in the restricted user mode.

Mode switch

A change of the processor between user mode and kernel mode: user → kernel when a program makes a system call or an interrupt arrives, and kernel → user when the kernel returns. Each one costs time.

System call

A request from a program to the kernel for a service, such as reading a file. It enters the kernel through a controlled gate, which means a mode switch.

Blocking system call

A system call that cannot finish right away (for example, a read whose data is still on the disk), so the caller is put to sleep until the event it needs happens.

Time slice

The longest stretch of processor time the scheduler hands out in one go before it may switch to someone else. Also called a quantum.

Clock interrupt

A regular signal from a hardware timer that hands control to the kernel, so it can check whether the running process has used up its time slice.

Jacketing

A thread-library trick that wraps a possibly blocking system call in code that first asks, without blocking, whether the call would block. If it would, the library runs another thread and tries again later.

Multiprocessor

A computer with two or more processors (or cores) that can execute instructions at the same moment.

Latency

How long one operation takes from start to finish. Here it is measured in microseconds (μs): millionths of a second.

Null fork

A benchmark that measures the pure overhead of creating, scheduling, running and finishing a thread or process whose body does nothing at all.

Signal-wait

A benchmark that measures the overhead of one thread or process signalling a waiting partner and then waiting itself: the basic cost of synchronizing two of them.

Combined approach (hybrid threading)

A design in which threads are created and mostly managed by a user-level library, and the many user-level threads of an application are mapped onto a smaller or equal number of kernel-level threads.

Thread migration

Moving an executing thread out of one process environment (address space and resources) into another, possibly on a different machine.

Address space

The range of memory addresses a process is allowed to use. All threads of one process share the same address space.

Review questions with answers

1. A program uses pure user-level threads. Thread 2 calls `read()` on a file whose data is not in memory yet. What happens?

- A. Only Thread 2 blocks; the library immediately runs Thread 1
- B. The whole process blocks, and the library's table still shows Thread 2 as Running
- C. The kernel blocks Thread 2 and schedules Thread 1 of the same process
- D. The call fails, because user-level threads may not make system calls

Answer: B. The whole process blocks, and the library's table still shows Thread 2 as Running

With ULTs the kernel sees only the process, so a blocking system call blocks the process as a whole. No library code runs while the process is blocked, so the library still records Thread 2 as Running; when the I/O completes and the process runs again, Thread 2 simply continues.

2. Switching between two user-level threads of the same process requires a mode switch into the kernel.

Answer: False

A ULT switch is done by library code in user mode, so it avoids the two mode switches (into the kernel and back out) that a KLT switch needs.

3. Which of these are advantages of kernel-level threads over pure user-level threads? Select all that apply.

- A. Threads of the same process can run at the same moment on different processors
- B. If one thread blocks, another thread of the same process can still run
- C. Kernel routines themselves can be multithreaded
- D. Switching between threads of the same process needs no mode switch
- E. Each application can pick its own thread-scheduling policy without kernel changes

Answer: A, B, C

The first three are the classic KLT advantages. The last two describe ULTs: their switches stay in user mode, and their library can use any scheduling policy the application likes.

4. Creating an empty ULT (null fork) took 34 μ s; creating an empty process took 11,300 μ s. How many ULT creations fit in the time of one process creation? Round to the nearest whole number.

Answer: 332 ULT creations

$11,300 \div 34 \approx 332.4$, so about 332 ULT creations cost as much as creating one process.

5. Signal-wait took 37 μ s with user-level threads and 441 μ s with kernel-level threads. How many times slower is the KLT version? Give one decimal place.

Answer: 11.9 x

$441 \div 37 \approx 11.9$. Most of the extra cost is the trip into the kernel and back that every KLT synchronization needs.

6. Does each statement describe user-level threads or kernel-level threads?

User-level threads	Kernel-level threads
Can run on an operating system that knows nothing about threads One blocking system call stops every thread of the process The scheduling policy can be tailored to the application	The kernel keeps a context record for every thread A thread switch needs a mode switch into the kernel Two threads of one process can use two cores at the same moment

ULTs live entirely in a user-space library: portable, cheap and customizable, but invisible to the kernel. KLTs are known to the kernel: parallel and independently blockable, but every switch goes through the kernel.

7. A process uses ULTs, and Thread 2 makes a blocking system call. Put the events in order.

Correct order:

1. Thread 2 makes the system call and the CPU enters the kernel
2. The kernel starts the I/O and moves the process to Blocked
3. The kernel runs another process; the library still lists Thread 2 as Running
4. The I/O completes and the kernel moves the process to Ready
5. The process is dispatched and execution resumes inside Thread 2

Every state change here happens in the kernel, at the level of the whole process. The library is frozen along with the process, so Thread 2's entry in the library's table never changes.

8. Match each threads : processes ratio (threads per process, not the ULT-to-KLT mapping) with its meaning and an example system.

1 : 1	Each thread is its own process (UNIX)
M : 1	Threads share one process (Windows NT)
1 : M	One thread migrates (Emerald, Ra)
M : N	Many threads that migrate (TRIX)

The ratio is threads : processes. 1 : 1 gives every thread its own process (traditional UNIX); M : 1 packs many threads into one process (Windows NT, Solaris, Linux, OS X, iOS); 1 : M lets a single thread move between process environments (Emerald, Ra on Clouds); M : N combines the last two (TRIX). This is a different idea from the many-to-one ULT-to-KLT mapping: a Linux process is M : 1 here, yet each of its threads has its own kernel-level thread.

9. What does jacketing do?

- A. Converts a blocking system call into a nonblocking check, so the thread library can run another thread instead of letting the process block
- B. Wraps each ULT in its own process so the kernel can see it
- C. Lets the kernel read the thread library's table so it can schedule individual ULTs
- D. Lengthens the process's time slice so a blocked thread has time to finish

Answer: A. Converts a blocking system call into a nonblocking check, so the thread library can run another thread instead of letting the process block

The jacket first asks the kernel, without waiting, whether the call would block. If it would, the library parks the thread and runs another one, retrying later, so the process as a whole never blocks.

10. Which statement best describes the combined (hybrid) approach, as used by older versions of Solaris?

- A. Threads are created and mostly scheduled in user space, and M user-level threads are mapped onto $N \leq M$ kernel-level threads
- B. Every kernel-level thread is split into several processes so the kernel can schedule them
- C. The kernel creates every thread and the library only gives them names
- D. Each ULT always gets exactly one KLT, and the programmer cannot change the number

Answer: A. Threads are created and mostly scheduled in user space, and M user-level threads are mapped onto $N \leq M$ kernel-level threads

The library does most thread work cheaply in user space, while the N kernel-level threads let the process use several processors and keep one blocking call from stopping everything.

11. When a clock interrupt ends the time slice of a process that uses ULTs, the thread library changes the running thread's state to Ready.

Answer: False

The kernel moves the process to Ready. The library does not run at all during this, so its table still shows the thread as Running, and that thread simply continues when the process is dispatched again.

12. Process B uses user-level threads. Thread 2 is running and must wait until Thread 1 fills a buffer, so it calls the library's wait routine. Which states result?

- A. Library: Thread 2 Blocked, Thread 1 Running. Kernel: B stays Running
- B. Library: Thread 2 Blocked, Thread 1 Running. Kernel: B becomes Blocked
- C. Library: Thread 2 still Running. Kernel: B becomes Blocked
- D. Library: Thread 2 Blocked, Thread 1 Ready. Kernel: B becomes Ready

Answer: A. Library: Thread 2 Blocked, Thread 1 Running. Kernel: B stays Running

Waiting for another thread is a library matter. The library blocks Thread 2 and switches to Thread 1 in user mode, while the kernel keeps seeing process B executing instructions, so B stays Running.

4.3 Multicore and Multithreading

You should be able to

- Calculate the speedup of a program on N cores with Amdahl's law, and explain why the serial fraction puts a ceiling on it.
- Explain why real software can stop improving, or even slow down, as cores are added (communication, work distribution, cache coherence).
- Classify software into the four kinds that benefit from multicore: multithreaded native, multiprocess, Java and multi-instance applications.
- Compare coarse-grained, fine-grained and hybrid threading, using the way the Valve game engine was rebuilt for multicore.

More cores, but not proportionally more speed

A multicore processor puts several independent cores on one chip, so several threads can run at the same instant. Doubling the cores rarely doubles the speed of a program, because some of every program's work cannot be shared out. **Speedup** measures the gain: the time a job takes on one processor divided by the time it takes on N processors.

Analogy: friends can chop vegetables for a lasagna in parallel, but it still bakes for the same time in your one oven. The oven time is the serial part of the job.

Amdahl's law

Split the one-processor running time T into two parts. A fraction **f** (the parallel fraction) can be divided among any number of processors perfectly, with no scheduling overhead. The rest, **1 – f** (the serial fraction), must run one step at a time and does not shrink however many processors you have.

- Time on one processor: $T_1 = (1 - f)T + fT = T$
- Time on N processors: $T_N = (1 - f)T + fT / N$ (only the parallel part is divided)
- Speedup = $T_1 / T_N = 1 / ((1 - f) + f / N)$ (T cancels out)
- Ceiling: as $N \rightarrow \infty$, $f / N \rightarrow 0$, so speedup $\rightarrow 1 / (1 - f)$
- Efficiency = speedup / N: how much of each core does useful work

Cores N (f = 0.9, T = 100 ms)	Time on N cores	Speedup	Efficiency
1	10 + 90 = 100 ms	1×	100%
2	10 + 45 = 55 ms	1.82×	91%
4	10 + 22.5 = 32.5 ms	3.08×	77%
8	10 + 11.25 = 21.25 ms	4.71×	59%
64	10 + 1.41 = 11.41 ms	8.77×	14%
∞	10 ms	10× (the ceiling)	$\rightarrow 0\%$

Two lessons. (1) When f is small, extra processors barely help. (2) Even when f is large, each extra core adds less than the one before (diminishing returns) and the speedup flattens at $1 / (1 - f)$. To go faster, shrink the serial part: halving it doubles the ceiling.

Worked examples

- $f = 0.95$, $N = 16$: $1 / (0.05 + 0.059375) = 1 / 0.109375 \approx \mathbf{9.14\times}$
- $f = 0.9$, $N = 8$: $1 / (0.1 + 0.1125) \approx \mathbf{4.71\times}$; the ceiling is $10\times$, and 1000 cores give about $9.91\times$
- $f = 0.5$, $N = 1000$: $1 / 0.5005 \approx \mathbf{2.00\times}$ (ceiling $2\times$)
- $f = 0.99$, $N = 100$: $1 / 0.0199 \approx \mathbf{50.25\times}$, half of the ideal $100\times$
- $f = 0.75$, $N = 4$: $1 / 0.4375 \approx \mathbf{2.29\times}$ (efficiency 57%)
- $f = 0.8$, unlimited cores: $1 / 0.2 = \mathbf{5\times}$
- In seconds: 200 s on one core with 30 s serial, on 8 cores: $30 + 170 / 8 = \mathbf{51.25\ s}$. Likewise 100 s with 20 s serial on 4 cores: $20 + 80 / 4 = \mathbf{40\ s}$.
- Working backwards: $8\times$ on 16 cores needs $(1 - f) + f / 16 = 0.125$, so $f \times 15/16 = 0.875$ and $f = \mathbf{0.933}$ (93.3% parallel).

Real software: overhead can make it slower

Amdahl's law is a best case, because it assumes that splitting work is free. Real programs pay a **parallel overhead** that grows with the number of cores:

- **Communication**: threads exchange results and wait for each other at synchronization points.
- **Distribution of work**: the job must be cut into pieces and handed out; more cores mean more pieces.
- **Cache coherence**: when cores share data, the hardware must keep every core's cached copy consistent, which costs extra memory traffic.

So real performance often rises, **peaks**, and then **falls** as cores are added: past the peak each new core costs more than it contributes. A simple illustrative model: $\text{time} = (1 - f) + f / N + \text{overhead} \times (N - 1)$. With $f = 0.9$ and 1% overhead per extra core, the best is about $3.57\times$ at 9 or 10 cores, and 64 cores manage only $1.34\times$. Amdahl's law on its own never predicts a slowdown (its speedup always rises with N); only overhead can.

Not all software hits this wall. **Database management systems** and **Java applications** are examples that can scale well on multicore: they serve many independent requests at once, so almost all of the work is parallel (f close to 1) and each extra core simply takes on more requests.

Four kinds of applications that benefit from multicore

Kind	Shape	Examples
Multithreaded native applications	A few processes, each with many threads; compiled to run directly on the hardware	Lotus Domino, Siebel CRM
Multiprocess applications	Many processes, each single-threaded; the OS runs them on different cores	Oracle database, SAP, PeopleSoft
Java applications	Java supports threads directly and the Java virtual machine is itself multithreaded (garbage collection, compiling); Java EE application servers serve many requests in parallel	Java EE application servers, the JVM
Multi-instance applications	Several copies of the same program run at once, often each in its own virtual machine for isolation; helps even if one copy cannot use many cores	Copies running in parallel virtual machines

In every case the OS ends up with many runnable threads to place on different cores; the difference is who created them.

Game engine example: re-threading Valve's Source engine

Valve rebuilt its Source game engine to use multicore chips and weighed three threading strategies:

- **Coarse-grained threading:** each module (rendering, AI, physics and so on) is its own single thread on its own processor, and a **timeline thread** keeps the module threads in step each frame. Simple, but a frame cannot finish before its biggest module does. Valve saw up to 2× on two processors only in contrived cases; in real gameplay the gain was about **1.2×**.
- **Fine-grained threading:** many similar or identical tasks, such as the iterations of a loop over an array, are spread across all processors. Cores stay busy, but every split adds coordination overhead, and Valve found it hard to use well: each unit of work takes a variable time, and keeping outcomes and their consequences in order made the code complex.
- **Hybrid threading:** fine-grained threading for the systems worth splitting, single threading for the others. Valve found this the **most promising**, expected to scale best to 8 or 16 processors. Sound mixing (little user interaction, its own data, not tied to the frame) stays on one processor; scene rendering is spread over many threads.

Toy frame: 24 ms of work, 4 cores, 0.5 ms sync per split	Frame time
One thread	24 ms
Coarse-grained (best arrangement; the 12 ms renderer is the floor)	12 ms (2×)
Fine-grained (every system split; 3 ms of the frame is sync overhead)	9 ms
Hybrid (split only rendering and physics)	7.5 ms (3.2×)

Inside the re-threaded renderer

1. **Many readers, one writer:** threads read the shared world data about 95% of the time and write it at most 5%. Locking the whole world for each thread was far too slow, so a **single-writer, multiple-readers lock** lets any number of readers in together; only a writer needs exclusive access.

2. **Scene lists** (the visible objects) for several views are built in parallel: the world from the main camera, its reflection in water, a shadow view.
3. **Per-object work**, such as the bone transformations of every character in every scene, is spread over all cores, and graphics simulation (such as particles) is overlapped with it.
4. Several threads "draw" in parallel, appending drawing commands to a **lock-free queue** (an atomic compare_and_swap per append, retried on a clash), so no thread sleeps on a lock.
5. A **single render thread** owns the graphics API (which took commands from one thread at a time), drains the queue and submits to the GPU in order.

Goals: avoid locking, share data that is mostly read, keep every thread busy (workers pull the next task the moment they are free), and give the graphics API one owner. In the step-through model, the 12 ms of rendering work that one core needs (the same Rendering block as the four-core lab) finishes in about 3.3 ms on four cores, close to the ideal $12 \div 4 = 3$ ms, with the cores about 91% busy instead of 25%.

Key terms

Multicore processor

A single chip that holds two or more independent processing units (cores). Each core can run its own thread at the same moment as the others.

Speedup

How many times faster a job finishes on N processors than on one: the time on one processor divided by the time on N processors.

Amdahl's law

A formula for the best speedup N processors can give when only part of a program can run in parallel: $\text{speedup} = 1 / ((1 - f) + f / N)$.

Parallel fraction (f)

The share of a program's one-processor running time that can be split perfectly across any number of processors. Amdahl's law assumes this part costs nothing extra to split.

Serial fraction

The share of the running time, $1 - f$, that must run one step after another on a single processor. Adding cores does not shorten it at all.

Parallel efficiency

Speedup divided by the number of processors: the average share of each core's time spent on useful work. 100% would mean no core ever waits or wastes effort.

Diminishing returns

The pattern in which each extra core adds less speedup than the one before, because the serial part never shrinks.

Scalability

How well a program keeps getting faster as processors are added. Software that scales well gains close to N times on N cores.

Parallel overhead

Extra work that exists only because a job was split up: threads communicating, work being divided and handed out, threads waiting for each other, and caches being kept coherent.

Multithreaded native application

Software made of a small number of processes, each running many threads, compiled to the machine's own instructions (native code). Examples: Lotus Domino, Siebel CRM.

Multiprocess application

Software built from many separate single-threaded processes that the OS can place on different cores. Examples: Oracle database, SAP, PeopleSoft.

Java virtual machine (JVM)

The program that runs Java code and manages its memory and threads. It is itself multithreaded: garbage collection (freeing unused memory), just-in-time compilation (turning Java code into machine instructions while it runs) and the application's threads can all run on different cores.

Multi-instance application

Running several copies (instances) of the same program at once, often each in its own virtual machine, so the cores stay busy even if one copy could not use them.

Coarse-grained threading

Giving each whole module of a program (for example rendering, AI or physics) its own thread, so each module runs on its own core. Each module stays single-threaded inside.

Timeline thread

In a coarse-grained game engine, the thread that keeps the module threads in step, synchronizing them once per frame so that, for example, rendering uses the positions physics has just computed.

Fine-grained threading

Splitting many similar or identical pieces of work, such as the iterations of one loop over an array, into small tasks spread across all the cores.

Hybrid threading (game engines)

A game-engine strategy: use fine-grained threading only for the systems that benefit from it and leave the other systems single-threaded. (Not the same as the combined user-level/kernel-level thread model.)

Single-writer, multiple-readers lock

A lock with two modes: any number of threads may hold it at once to read, but a thread that wants to write must hold it alone. It suits data that is read far more often than it is changed.

Lock-free data structure

A shared structure (such as a queue) that threads update with an atomic instruction such as `compare_and_swap`, which changes a memory word only if it still holds the value the thread expects. If another thread got there first, the update simply retries. No lock is ever taken, so no thread sleeps waiting for another to release one.

Scene list

In a game renderer, the list of objects visible from one viewpoint (the main camera, a reflection, a shadow) that must be drawn this frame.

Review questions with answers

1. In Amdahl's law, $\text{speedup} = 1 / ((1 - f) + f / N)$. What does **f** stand for?

- A. The fraction of the running time that must run on a single processor
- B. The fraction of the running time that can be split perfectly across any number of processors
- C. The fraction of the processors that are busy at any moment
- D. The clock frequency of each core

Answer: B. The fraction of the running time that can be split perfectly across any number of processors

f is the parallel fraction: the part of the one-processor running time that can be divided among N processors with no scheduling overhead. The rest, $1 - f$, is inherently serial.

2. A program is 90% parallelizable ($f = 0.9$). According to Amdahl's law, what speedup does it get on **8** processors? (Two decimal places.)

Answer: 4.71 x

$1 / (0.1 + 0.9 / 8) = 1 / (0.1 + 0.1125) = 1 / 0.2125 \approx 4.71$. Eight processors, less than five times faster.

3. A game engine uses **hybrid** threading. Which system is the best candidate to leave **single-threaded**, running whole on one core?

- A. Scene rendering, the biggest system in each frame
- B. Computing the bone transformations of every character in every scene
- C. Building the scene lists for the main view, a reflection and the shadows
- D. Sound mixing, a small system with little user interaction that works on its own data

Answer: D. Sound mixing, a small system with little user interaction that works on its own data

Hybrid threading splits only the systems that gain from splitting. A small, self-contained system such as sound mixing runs well whole on one core, while big systems such as rendering are spread over many threads.

4. A program is 80% parallel ($f = 0.8$). With an **unlimited** number of processors, what is the largest speedup it can ever reach?

Answer: 5 x

As N grows, f / N shrinks to 0, so speedup approaches $1 / (1 - f) = 1 / 0.2 = 5$. The serial 20% sets the ceiling.

5. A job takes 100 s on one core, and 20 s of it is inherently serial. The rest parallelizes perfectly. How many seconds does it take on **4** cores?

Answer: 40 s

The serial 20 s stays; the parallel 80 s is split four ways into 20 s. Total $20 + 20 = 40$ s, a speedup of $2.5\times$.

6. If 90% of a program can run in parallel, 1000 processors will make it roughly 900 times faster.

Answer: False

The serial 10% caps the speedup at $1 / (1 - 0.9) = 10\times$. With 1000 processors Amdahl's law gives only about $9.9\times$.

7. Which of these are sources of overhead that can make a real multithreaded program **slower** as more cores are added? Select all that apply.

- A. Threads communicating and waiting for each other
- B. Cutting the job into pieces and handing the pieces out to the threads
- C. Keeping the cores' cached copies of shared data coherent
- D. Each core having a smaller share of the parallel work to do
- E. The serial part shrinking as cores are added

Answer: A, B, C

Communication, distributing the work and cache coherence all cost more as cores are added. A smaller share of parallel work per core is the benefit, not a cost, and the serial part never shrinks at all.

8. Sort each application into the kind of multicore-friendly software it is.

Multithreaded native	Multiprocess	Java	Multi-instance
Lotus Domino Siebel CRM	Oracle database SAP PeopleSoft	A Java EE application server	Several copies of one program, each in its own virtual machine

A few processes with many threads each: multithreaded native. Many single-threaded processes: multiprocess. The multithreaded JVM and Java EE servers: Java. Running extra copies side by side: multi-instance.

9. Match each threading strategy used in game engines with its description.

Coarse-grained threading	Each whole module, such as rendering or AI, gets its own thread
Fine-grained threading	Many similar tasks, such as the iterations of one loop, are spread over all cores
Hybrid threading	Fine-grained threading for some systems, single threads for the others

Coarse splits by module, fine splits one kind of work into many small pieces, and hybrid mixes the two, choosing per system.

10. A game frame on a 4-core chip has a 12 ms rendering module and 12 ms of other, smaller modules. Using **coarse-grained** threading only (a module can never be split) and assuming modules never wait for each other, what is the shortest possible frame time?

- A. 3 ms
- B. 6 ms
- C. 12 ms
- D. 24 ms

Answer: C. 12 ms

With coarse threading a frame cannot finish before its biggest module does, so 12 ms is the floor, however the other modules are arranged.

11. Threads in a game engine read the shared world data about 95% of the time and write it at most 5% of the time. Which way of protecting that data suits this pattern best?

- A. A single-writer, multiple-readers lock: readers hold it together, a writer holds it alone
- B. One ordinary lock around the whole world that every thread must take, whether it reads or writes
- C. A private copy of the whole world for every thread, merged at the end of each frame
- D. No protection at all, because writes are rare

Answer: A. A single-writer, multiple-readers lock: readers hold it together, a writer holds it alone

Reads do not interfere with each other, so letting all readers in together removes almost all waiting. Only the rare writer needs exclusive access.

12. Put the steps for building one frame in a renderer re-threaded for multicore in order.

Correct order:

1. Scene lists are built in parallel, one for each view (main camera, reflection, shadows)
2. Per-object work, such as bone transformations, is spread over all the cores
3. Workers append their drawing commands to a lock-free queue
4. The single render thread drains the queue and submits the commands to the GPU

First find what is visible in each view, then do the work each of those objects needs, queue the resulting drawing commands without locks, and let the one thread that owns the graphics API feed the GPU.

4.4 Windows Process and Thread Management

You should be able to

- Describe what a Windows process and a Windows thread each hold, and name the extra building blocks: job objects, thread pools, fibers and user-mode scheduling.
- Explain how Windows 8 and later suspend, terminate and restore background apps, and what app developers must do about it.
- Identify the resources of a Windows process (access token, virtual address descriptors, handle table, threads) and sort the attributes of process and thread objects.
- Explain why a multithreaded process is an efficient server, and trace a thread through the six Windows thread states.
- Explain how environment subsystems build their own process features on top of the executive's generic services.

Windows process and thread management

Windows separates **owning resources** (the process) from **being scheduled** (the thread). Both are kernel **objects** with built-in synchronization: another thread can wait on either one.

1. The basic building blocks

An **application** consists of one or more **processes**. A process is a running instance of a program together with everything it owns:

- a private **virtual address space** and the **executable code** loaded into it;
- **open handles** to system objects (files, events, other processes and threads);

- a **security context** (the access token: user, groups, privileges) and a **unique process ID**;
- **environment variables**, a **priority class**, and a **minimum and maximum working-set size**;
- **at least one thread**; the first is called the primary thread.

A **thread** is the schedulable entity inside a process. All threads of a process share its address space and resources. Each thread also keeps its own **exception handlers**, **scheduling priority**, **thread-local storage** (a variable with a separate value per thread), **unique thread ID** and **saved context** (register values kept while it is not running).

2. Four extra building blocks

Tool	What it is	Who decides what runs
Job object	A group of processes managed as one unit: shared limits (memory, processor time, number of processes), shared accounting, one call ends them all.	Normal scheduling; the job only groups and limits.
Thread pool	System-kept worker threads that run asynchronous callbacks the application submits (timers, I/O completions, small tasks).	The kernel schedules the workers.
Fiber	A unit of execution scheduled manually by the application. Fibers run inside a thread and take on its identity (its ID, its thread-local storage). The kernel does not know they exist, so one blocking fiber stalls its whole thread, exactly like the user-level threads of section 4.2. Useful for porting programs that schedule their own tasks.	The application, by explicit switches.
User-mode scheduling (UMS)	Lets an application schedule its own real threads, switching in user mode without the system scheduler. Each UMS thread has its own context, and the app regains control when one blocks in the kernel. More efficient than a pool for short work items that make few system calls. Introduced in 64-bit Windows 7; deprecated and no longer supported starting with Windows 11.	The application, for real threads.

3. Background apps from Windows 8 onward

Classic desktop programs keep running in the background. Modern (Store) apps are managed by **process lifetime management (PLM)**:

- Only the **foreground** app is active.
- A few seconds after an app leaves the foreground, it is **suspended**: its threads get no processor time, but its memory is kept, so switching back is instant.
- If memory runs low, Windows may **terminate** a suspended app, with no warning.
- So the developer must save **app state data** on suspension and restore it on relaunch; done right, the user never notices.
- An app can register **background tasks** that run on a trigger with limited processor time and memory while the app itself stays suspended.

Why: on a tablet or laptop, a hidden app should use neither memory nor battery.

4. Inside a Windows process

Each process is a **process object** made of:

- an **access token**: whom the process acts for. Windows checks it whenever the process opens a protected object (a file only group HR may open is refused if the token lacks HR).
- **virtual address descriptors (VADs)**: one record per range of the address space in use (program image, heap, each thread's stack, private data, mapped views), kept by the memory manager.
- a **handle table**: one entry per object it has opened (files, sections, threads...). A handle is the entry's number, private to this process; the object itself lives in system memory, and the program never gets a direct pointer to it. A freed entry is reused. Closing a thread's handle does not stop the thread.
- **one or more threads**, each represented by a thread object.

Process object attributes	Thread object attributes
Process ID	Thread ID
Security descriptor (who may open or control it)	Thread context (saved registers)
Base priority (starting point for its threads)	Dynamic priority (current value, may be boosted) and base priority (its floor)
Default processor affinity	Thread processor affinity (all or part of the process's set)
Quota limits (memory, paging file, processor time)	Thread execution time (user and kernel mode)
Execution time (all threads added together)	Alert status (may it run an asynchronous procedure call while waiting?)
I/O counters and VM operation counters	Suspension count (suspends minus resumes; runs only at 0)
Exception/debugging ports (messages to the subsystem or a debugger)	Impersonation token (act on behalf of another user) and termination port
Exit status (object signaled at exit)	Thread exit status (object signaled at exit)

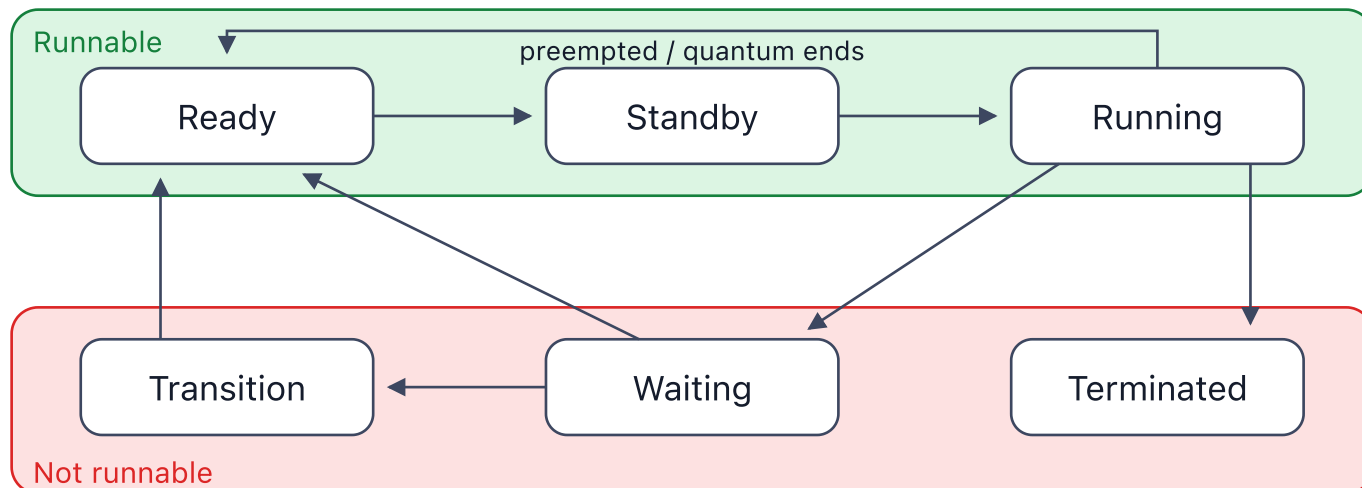
Pattern: the process holds what applies to the whole container; the thread holds what belongs to one path of execution. ID, base priority, execution time and exit status appear on **both**. Example: a thread suspended 3 times and resumed once has a suspension count of 2 and cannot run.

5. Multithreading

- Threads in **different processes** may execute concurrently (on one processor they interleave and only appear to run together).
- Threads of the **same process** can also be placed on different processors and run at the same instant, so one program can use several processors.
- Threads of one process exchange information through their shared address space; threads in different processes need **shared memory set up between the processes** (a section object mapped into both) or messages.
- A multithreaded process is an efficient **server**: one copy of code and data, one thread per client request.

Worked example. Four requests arrive at time 0; each needs 2 ms of processing, a 3 ms disk read (reads may overlap) and 1 ms to reply. A single-threaded server takes $4 \times (2 + 3 + 1) = 24$ ms, and the processor is idle half the time. With one thread per request, disk waits overlap other requests' processing: all four finish after **12 ms** on one processor, **8 ms** on two and **6 ms** on four.

6. The six thread states



- **Ready:** may be scheduled; the dispatcher considers ready threads in priority order.
- **Standby:** selected to run next on a particular processor; waits until that processor is free. If its priority is high enough, the thread running there is preempted.
- **Running:** executing until it is preempted by a higher-priority thread, uses up its quantum (both send it back to Ready), blocks or terminates.
- **Waiting:** blocked on an event (such as I/O), waiting voluntarily to synchronize, or suspended by its environment subsystem. (Suspending one Windows thread only stops it being scheduled; its memory stays in place. It is not the swapped-out, process-wide Suspended state of section 4.1.)
- **Transition:** ready to run but a resource is missing, typically its kernel stack paged out of memory.
- **Terminated:** it ended itself, was ended by another thread, or the process it belongs to ended. After clean-up it is removed, or its object is kept for reuse.

Legal moves: Ready → Standby → Running; Running → Ready, Waiting or Terminated; Waiting → Ready (resources in memory) or Transition (one is paged out); Transition → Ready once it is back. Example: Ready → Standby → Running → Waiting (disk read) → Transition (stack paged out) → Ready.

7. Support for OS subsystems

Each **environment subsystem** (Win32 today; older Windows also had POSIX) is responsible for the process and thread features of its own personality. The **executive** provides generic services to create process and thread objects and returns **handles**; the subsystem **interprets** those handles by its own rules. For CreateProcess, the Win32 subsystem has the executive create the process object and, because every Win32 process starts with one thread, its first thread; it then records the process and gives the program handles and IDs. The subsystem names the **calling program** as parent, so the new process inherits the caller's access token, quota limits, base priority and default processor affinity (not the subsystem server's). For fork, a POSIX subsystem uses the same services but copies the parent's address space and keeps the parent-child link itself: the executive records who created a process but enforces no hierarchy.

Key terms

Job object

A Windows kernel object that groups several processes so they can be managed as one unit: one set of limits (such as total memory or processor time), one set of accounting totals, and one call to end them all.

Thread pool

A set of worker threads that the system keeps ready. The application hands it small pieces of work (callbacks) and an idle worker runs each one, so the program does not create a thread per task.

Fiber

A unit of execution that the application itself schedules by explicitly switching from one fiber to another. Fibers run inside an ordinary thread, and the kernel does not know they exist.

User-mode scheduling (UMS)

A lightweight Windows mechanism that lets an application run its own scheduler for its own real threads, switching between them in user mode and regaining control whenever one of them blocks in the kernel. Introduced in 64-bit Windows 7; no longer supported starting with Windows 11.

Process lifetime management (PLM)

The part of Windows 8 and later that decides when a Modern (Store) app runs, is suspended in the background, or is terminated to free memory.

Background task

A small, separately registered piece of an app that Windows runs on a trigger (a timer, a push message, a network change) with a tight budget of processor time and memory while the app itself stays suspended.

Access token

The kernel object that records whom a process runs as: the user's security identity, groups and privileges. Windows checks it every time the process tries to open a protected object.

Virtual address descriptor (VAD)

One record in a tree the memory manager keeps for each process, describing a range of the process's virtual addresses that is in use (program image, heap, a thread's stack, a mapped file) and how it may be accessed.

Handle table

The per-process table that lists every object the process has opened. Each entry points to one object, and the entry's number is the handle the program uses.

Handle

A small number, private to one process, that stands for an object the process has opened. It indexes the process's handle table; programs never get a direct pointer to the object.

Working set

The pages of a process that are currently in physical memory. Windows lets a process have a minimum and a maximum working-set size.

Thread-local storage (TLS)

A small amount of storage that belongs to one thread only: one variable name has a separate value in every thread, so each thread can keep private data (such as its own error code) even though all threads share one address space.

Processor affinity

The set of processors a thread is allowed to run on. A process holds a default set for its threads, and each thread may be limited to part of it.

Base priority

The starting point of a thread's scheduling priority. A process's priority class sets the base priority for its threads, and a thread's current priority never drops below its own base priority.

Dynamic priority

A thread's scheduling priority at this moment. Windows may raise it temporarily above the base priority (for example after a wait ends) and lets it fall back over time.

Quantum

The amount of processor time a thread may use before the scheduler checks whether another ready thread of the same priority should get a turn. Also called a time slice.

Standby state

The Windows thread state of a thread that has been chosen to run next on one particular processor and is waiting for that processor to become free.

Transition state

The Windows thread state of a thread that is ready to run except that something it needs, typically its kernel stack, has been paged out of memory.

Section object

A Windows object that represents a block of memory which can be mapped into the address space of one or more processes; it is how two processes share memory.

Environment subsystem

A user-mode server process together with its DLLs that gives programs one operating-system "personality", meaning one set of API calls. Win32 is the main subsystem in Windows.

Review questions with answers

1. A Windows program opens a file and gets back the number 0x04. What is that number?
- A. A handle: an index into the process's own handle table, whose entry points to the file object
 - B. A direct pointer to the file object in system memory, which the program follows to read the file
 - C. A system-wide file ID that any other process can also use to reach the same open file
 - D. A count of how many of the program's threads are currently allowed to use the file

Answer: A. A handle: an index into the process's own handle table, whose entry points to the file object

A handle is a small number private to one process. It indexes that process's handle table, and the table entry points to the object, which lives in system memory outside the process.

2. Which of these does each Windows thread keep for itself instead of sharing with the other threads of its process? Select all that apply.

- A. Thread-local storage
- B. Exception handlers
- C. Virtual address space
- D. Saved context (register values)
- E. The process's access token
- F. Environment variables

Answer: A, B, D

Each thread has its own ID, saved context, scheduling priority, thread-local storage and exception handlers. The address space, security context and environment variables belong to the process and are shared.

3. Match each Windows building block to what it does.

Job object	Manages a group of processes as one unit
Thread pool	Worker threads that run asynchronous callbacks for the app
Fiber	A unit of execution the app switches to by hand inside one thread
User-mode scheduling (UMS)	Lets an app schedule its own real threads in user mode

Jobs group processes; pools run callbacks on system-managed workers; fibers are scheduled manually and are invisible to the kernel; UMS lets an application schedule real kernel threads itself.

4. When memory runs low, Windows warns a suspended Modern app just before terminating it, so the app can save its state at that moment.

Answer: False

A suspended app is terminated without warning. That is why it must save its app state data when it is suspended, and restore that data if it is relaunched.

5. One Windows thread hosts four fibers. One fiber makes a blocking file read that must wait for the disk. What happens to the other three fibers until the read completes?

- A. None of them runs: the kernel blocks the host thread, and all four fibers live inside it
- B. The kernel switches to another fiber of the same thread
- C. The application's own scheduler is told about the block and runs another fiber
- D. They keep running, because the kernel schedules each fiber separately

Answer: A. None of them runs: the kernel blocks the host thread, and all four fibers live inside it

Fibers are switched by explicit calls inside one thread and are invisible to the kernel, so when one fiber blocks, the kernel blocks its whole host thread. UMS avoids this: its threads are real kernel threads, and the app's scheduler regains control when one blocks.

6. Sort each attribute onto the Windows object that holds it.

Process object	Thread object
Quota limits VM operation counters Exception/debugging ports	Suspension count Impersonation token Thread context

The process object holds what belongs to the whole container: quota limits, counters and the exception/debugging ports. The thread object holds what belongs to one path of execution: its context, suspension count and impersonation token.

7. Every Windows process has an access token. What is it for?

- A. Recording which ranges of the address space are in use
- B. Identifying whom the process acts for, so Windows can decide whether it may open protected objects
- C. Numbering the objects the process has opened
- D. Storing the saved registers of the primary thread

Answer: B. Identifying whom the process acts for, so Windows can decide whether it may open protected objects

The access token holds the user identity, groups and privileges. Each attempt to open a protected object is checked against it.

8. A ready thread is picked for a processor, runs, blocks on a disk read, and while it waits its kernel stack is paged out. Put the states it visits in order, up to the moment its read has completed.

Correct order:

- 1. Ready
- 2. Standby
- 3. Running
- 4. Waiting
- 5. Transition

Ready → Standby (picked for one processor) → Running → Waiting (blocked on the read) → Transition (the read is done, but the kernel stack must come back into memory). Only then does it return to Ready.

9. What does the Standby state mean for a Windows thread?

- A. It has been selected to run next on a particular processor and is waiting for that processor
- B. It is blocked until an I/O operation finishes
- C. It is ready to run, but its kernel stack is paged out of memory
- D. It has finished and is waiting to be cleaned up

Answer: A. It has been selected to run next on a particular processor and is waiting for that processor

Standby sits between Ready and Running: the dispatcher has picked the thread for one processor. If its priority is high enough, the thread now running on that processor may be preempted.

10. A single-threaded server handles 4 requests strictly one after another. Each request needs 2 ms of processing, then a 3 ms disk read, then 1 ms to reply, and the thread does nothing else while it waits. How many milliseconds pass until all four are answered?

Answer: 24 ms

Each request takes $2 + 3 + 1 = 6$ ms and nothing overlaps: $4 \times 6 = 24$ ms. With one thread per request, one thread's disk wait overlaps another's processing, so the same work finishes far sooner (12 ms on one processor).

11. A Windows thread is suspended 3 times and then resumed once. What is its suspension count now?

Answer: 2

Each suspend adds 1 and each resume subtracts 1: $3 - 1 = 2$. The thread may run again only when its suspension count is back to 0.

12. When a program creates a new process, what does its environment subsystem do that the executive does not?

- A. Allocate the physical memory for the new process
- B. Interpret the returned handles and apply its own personality's rules, such as Win32's first thread or POSIX's parent-child link
- C. Schedule the new process's threads on processors
- D. Build the process object itself

Answer: B. Interpret the returned handles and apply its own personality's rules, such as Win32's first thread or POSIX's parent-child link

The executive offers generic process and thread services and returns handles. Each subsystem gives those objects the meaning its own personality requires; it also names the calling program as the parent, so the new process inherits that program's access token, quotas, base priority and affinity.

4.5 Solaris Thread and SMP Management

You should be able to

- Describe the four thread-related entities in Solaris (process, user-level threads, lightweight processes and kernel threads) and trace how work flows from one to the next and onto a processor.
- Explain why Solaris uses three levels of thread and what that gives both the application and the operating system.
- Compare the traditional UNIX process structure with the Solaris one, and list what each LWP data structure holds.
- Trace a kernel thread through the RUN, ONPROC, SLEEP, STOP, ZOMBIE and FREE states, naming the event behind each move.
- Explain how Solaris turns interrupts into interrupt threads, what pinning means, and why this costs less than blocking interrupts.

1. Four thread-related concepts in Solaris

Solaris, a UNIX system built for multiprocessors, splits running a thread across four layers:

- **Process:** the ordinary UNIX process, with the user's address space, a stack and a process control block (PCB). It is the container whose memory, files and signal settings every thread shares.
- **User-level threads (ULTs):** made by a thread library in the process's address space; invisible to the OS. They are the program's interface for parallelism, and creating or switching them needs no kernel call.
- **Lightweight processes (LWPs):** a mapping between ULTs and kernel threads. (In Solaris the name means this specific structure; section 4.1 used "lightweight process" loosely for any thread.) Each LWP supports one or more ULTs (running one at a time) and maps to exactly one kernel thread. The kernel schedules LWPs independently, so they may execute in parallel on a multiprocessor.
- **Kernel threads:** the fundamental entities that are scheduled and dispatched onto one of the system's processors.

Rule to remember: there is **always exactly one kernel thread per LWP**. The reverse does not hold: some kernel threads have no LWP. The kernel creates, runs and destroys them to carry out its own system functions, such as handling interrupts.

ULTs (in the library) → LWPs – exactly one each – kernel threads → processors
(+ kernel threads with no LWP, e.g. interrupts)

The simplest case is one ULT on one LWP: a single flow of execution, which behaves like a traditional UNIX process. A program that wants concurrency uses several ULTs and LWPs. **Two versions:** Solaris 2 to 8 let many ULTs share a smaller set of LWPs (many-to-many, as in the lab); from Solaris 9 on, the library gives every ULT its own LWP (one-to-one). The layers and the one-kernel-thread-per-LWP rule are the same in both.

2. Why three levels of thread?

- **A clean interface for the application:** the program uses a standard thread library. In the many-to-many design, the library does cheap user-space bookkeeping: creating and switching ULTs and choosing which ULT runs on an LWP, with no kernel call.
- **Control for the OS:** each LWP is bound to one kernel thread with matching execution states, so concurrency and execution are managed at the kernel-thread level. The kernel picks which kernel thread runs on which processor, blocks one LWP without stalling the rest, and runs a process's LWPs in parallel.
- **Cheaper kernel work:** running system functions as kernel threads (with no LWP) rather than as kernel processes means switching among them inside the kernel is a thread switch, not a more expensive process switch.

3. How the mapping behaves

- An LWP runs one ULT at a time; ULTs sharing an LWP take turns under the library's control. (In the many-to-many design a ULT could be bound to its own LWP; unbound ULTs could run on any free LWP of their process.)
- At most one ULT per LWP, and at most one kernel thread per processor, can run at once. Example: 5 ULTs on 4 LWPs with 2 processors → at most **2** ULTs execute at the same instant. 6 ULTs on 3 LWPs → exactly **3** kernel threads support them.

- Only a ULT that is actually executing (its LWP's kernel thread is ONPROC) can make a system call. A blocking call puts only that LWP's kernel thread to sleep (ONPROC → SLEEP); other LWPs keep running. ULTs waiting on the same LWP are stuck too. If a process has a single LWP, one blocking call stalls the whole process, just like pure ULTs.
- With more LWPs than processors, runnable LWPs take turns: the extras wait in RUN.

4. Process structure: traditional UNIX vs Solaris

Traditional UNIX process	Solaris process
Process ID, user IDs, signal dispatch table, memory map, file descriptors	The same five shared items
One processor state : priority, signal mask, registers, stack	A list of LWP structures , one per LWP, each holding its own per-execution state

Each **LWP data structure** holds: (1) an LWP identifier; (2) the priority of this LWP, and hence of the kernel thread that supports it; (3) a signal mask telling the kernel which signals will be accepted; (4) saved values of user-level registers while the LWP is not running; (5) the kernel stack for this LWP, with system call arguments, results and error codes for each call level; (6) resource usage and profiling data; (7) a pointer to the corresponding kernel thread; (8) a pointer to the process structure.

The principle: the process keeps what the whole program shares; each LWP keeps what one flow of execution needs. That is what lets one process run in several places at once.

5. Kernel-thread states (simplified)

State	Meaning
RUN	Runnable: ready to execute, waiting for a processor.
ONPROC	Executing on a processor.
SLEEP	Blocked, waiting for an event.
STOP	Stopped (its process was stopped, e.g. by a debugger).
ZOMBIE	Terminated; leftovers not yet collected.
FREE	Resources released; awaiting removal from the OS thread data structure.

Transitions: RUN → ONPROC on **dispatch**. ONPROC → RUN on **preemption** by a higher-priority thread, **end of quantum** (time slicing) or **yield**. ONPROC → SLEEP on a **blocking system call**; SLEEP → RUN on **wakeup** (it must still wait for a processor). ONPROC → STOP on **stop** (a thread stops itself, so a stop request takes effect when it next runs); STOP → RUN on **continue**. ONPROC → ZOMBIE on **exit**; ZOMBIE → FREE when the thread is **reaped**. When an interrupt arrives, the running thread (and its LWP) may be **pinned** until the interrupt is handled; this is a temporary hold, not one of the six states.

6. Interrupts as threads

- Solaris handles interrupts with a set of **interrupt threads**, kernel threads that each have their own identifier, priority, context and stack.

- Interrupt threads get higher priorities than all other kernel threads; only a higher-priority interrupt thread can preempt one.
- The kernel controls access to shared data and synchronizes interrupt threads with mutual exclusion primitives (mutex locks), just as for other threads. An interrupt thread that needs a locked structure waits for it; its top priority does not let it skip the lock.

Sequence: (1) the interrupt is delivered to a particular processor; (2) the thread running there is **pinned**: its context is saved, it is suspended and it cannot move to another processor; (3) the processor starts an interrupt thread taken from a **pool of deactivated interrupt threads**, so no thread is created; (4) the interrupt thread handles the interrupt, locking shared data as needed; (5) it returns to the pool; (6) the pinned thread is unpinned and resumes exactly where it stopped.

Going deeper: in the real kernel, if an interrupt thread has to go to sleep (not just wait briefly for a lock), Solaris turns it into a full kernel thread and releases the pinned thread early, so the interrupted work is not held up.

Why: a traditional kernel protects data shared with interrupt handlers by raising the interrupt priority level before each access and lowering it after, although most interrupts never touch that data. That costs time on every critical section, and on a multiprocessor the kernel may have to block interrupts on all processors (and needs locks anyway). With interrupt threads the cost appears only when an interrupt happens, and interrupts are far rarer than critical sections (stretches of code that use shared data). Illustration with made-up unit costs: blocking costs 40 critical sections/ms $\times$ 4 processors $\times$ 1 unit = 160 units/ms; interrupt threads cost 2 interrupts/ms $\times$ 6 units = 12 units/ms, about 13.3 times less. Common mistake: Solaris does not create a thread per interrupt; it reuses pooled ones.

Key terms

User-level thread (ULT)

A thread that is created, scheduled and switched entirely by a thread library inside the application. The kernel does not know it exists; it sees only the process that contains it (in Solaris, the LWPs the thread runs on).

Solaris lightweight process (LWP)

In Solaris, a kernel-supported carrier between user-level threads and the kernel. The thread library treats each LWP like a virtual processor that runs one ULT at a time; every LWP is backed by exactly one kernel thread, and the kernel schedules each LWP independently, so several can run in parallel on different processors.

Kernel thread

The basic unit the Solaris kernel schedules and dispatches onto a processor. Every LWP has exactly one; other kernel threads work only inside the kernel, for example to handle interrupts.

Blocking system call

A system call that cannot finish right away (for example, a read whose data is still on the disk), so the caller is put to sleep until the event it needs happens.

Process identifier (PID)

A unique number the OS gives each process so it can tell it apart from every other process, even ones running the same program.

User ID

A number that says which user a process is working for. The kernel checks it (and related group IDs) to decide which files and actions the process is allowed.

Signal dispatch table

A per-process table that records, for each kind of signal (a short notice such as Ctrl+C), what should happen when it arrives: ignore it, take the default action, or run a handler function the program installed.

Signal mask

A set of bits telling the kernel which signals will be accepted right now. A signal that is masked (blocked) waits until the mask changes. A traditional process has one mask; in Solaris each LWP has its own.

File descriptor

A small whole number a process uses to name one of its open files, pipes or network connections. By convention 0, 1 and 2 are standard input, output and error.

Memory map

The kernel's record of which regions of a process's address space are in use (code, data, heap, stacks, shared libraries) and what memory backs each one.

Quantum

The slice of processor time a thread may use before the scheduler may hand the processor to another ready thread. Also called a time slice.

ONPROC

The Solaris kernel-thread state meaning "on a processor": the thread is executing right now.

Zombie thread

A thread that has called exit and finished, but whose leftover records have not been collected yet. Solaris calls this state ZOMBIE.

Reaping (reap)

Collecting a finished (zombie) thread's leftovers so its resources can be released. The thread is then FREE: it only waits to be removed from the kernel's thread data structure.

Pinned thread (pinning)

A thread that was running when an interrupt arrived on its processor. It is held on that processor, suspended with its context saved, until the interrupt has been handled; it cannot move to another processor meanwhile.

Interrupt thread

A Solaris kernel thread that handles an interrupt. It has its own ID, priority, context and stack, runs at a higher priority than every other kernel thread, and waits in a pool of deactivated threads until an interrupt needs it.

Critical section

A stretch of code that uses shared data (or another shared resource) and must not run at the same time as other code that uses the same data.

Mutex (mutex lock)

A lock that only one thread can hold at a time. A thread that wants it while it is held must wait, and only the thread that locked it may unlock it.

Interrupt priority level (IPL)

A processor setting that decides which interrupts may be delivered right now. Raising it holds back lower-level interrupts; lowering it lets them through again.

Symmetric multiprocessor (SMP)

One computer with two or more similar processors that share main memory and the I/O devices, where any processor can run any work, including the kernel itself.

Review questions with answers

1. In Solaris, which entity does the kernel actually schedule and dispatch onto a processor?

- A. A user-level thread
- B. A kernel thread
- C. The thread library
- D. The process structure

Answer: B. A kernel thread

Kernel threads are the fundamental entities that can be scheduled and dispatched onto a processor. Each LWP is backed by one, and some exist with no LWP at all.

2. What is the main reason Solaris uses three levels of thread (user-level threads, LWPs and kernel threads)?

- A. Let the OS manage what really runs, while programs keep a simple, flexible threads interface
- B. Let the kernel schedule every ULT directly, so no thread library is needed
- C. Give every thread its own private address space, so threads cannot corrupt each other
- D. Make kernel threads unnecessary on machines with a single processor

Answer: A. Let the OS manage what really runs, while programs keep a simple, flexible threads interface

The levels split the work: a thread library gives the application a clean, cheap interface (ULTs), and each LWP is tied to one kernel thread so the OS controls scheduling, blocking and parallel execution. Kernel threads with no LWP let the kernel run its own jobs with cheap thread switches instead of process switches.

3. Every LWP is supported by exactly one kernel thread, and every kernel thread belongs to exactly one LWP.

Answer: False

The first half is true, the second is not. Some kernel threads have no LWP: they run kernel functions such as interrupt handling.

4. A Solaris process has 6 user-level threads mapped onto 3 LWPs. How many kernel threads support this process's LWPs?

Answer: 3 kernel threads

There is always exactly one kernel thread per LWP, so 3 LWPs means 3 kernel threads. The number of ULTs does not matter; the kernel never sees them.

5. A process spreads 5 ULTs over 4 LWPs. The machine has 2 processors and no LWP is blocked. At most how many of the process's ULTs can execute at the same instant?

Answer: 2 ULTs

Each LWP runs one ULT at a time, and each LWP runs only when the kernel puts its kernel thread on a processor. With 2 processors, at most 2 kernel threads, so 2 ULTs, run at once.

6. A process has 3 LWPs, each carrying ULTs. A ULT on LWP 2 calls `read()` and must wait for the disk. What happens?

- A. The whole process blocks until the read finishes
- B. LWP 2 and its kernel thread sleep; LWPs 1 and 3 keep running
- C. The kernel moves the waiting ULT onto another LWP
- D. The library runs another ULT on LWP 2 while the read waits

Answer: B. LWP 2 and its kernel thread sleep; LWPs 1 and 3 keep running

The kernel schedules each LWP independently. A blocking system call puts that one LWP's kernel thread to SLEEP; the process's other LWPs are unaffected.

7. Where does Solaris keep each item?

Process structure	Each LWP structure
Signal dispatch table Memory map File descriptors	Signal mask Kernel stack Saved user-level registers

The process keeps what every thread shares (identity, permissions, signal actions, memory map, open files). Each LWP keeps what one flow of execution needs, plus links to its kernel thread and process. A traditional UNIX process held that per-execution part only once, as a single processor state.

8. Match each kernel-thread state to its meaning.

RUN	Runnable, waiting for a processor
ONPROC	Executing on a processor
SLEEP	Blocked, waiting for an event
STOP	Stopped, for example by a debugger
ZOMBIE	Terminated, leftovers not yet collected
FREE	Resources released, awaiting removal

These six states describe a Solaris kernel thread from ready to fully gone. FREE comes after ZOMBIE, once the thread has been reaped.

9. A thread in the ONPROC state uses up its quantum. Which state does it move to?

- A. SLEEP
- B. RUN
- C. STOP
- D. ZOMBIE

Answer: B. RUN

Time slicing takes the processor away, but the thread is still ready to run, so it goes back to RUN. Preemption and yield lead there too.

10. Put the handling of an interrupt in Solaris in order.

Correct order:

1. The interrupt is delivered to one particular processor
2. The thread running there is pinned and its context saved
3. An interrupt thread is taken from the pool of deactivated threads
4. The interrupt thread runs the handler, taking mutex locks as needed
5. The interrupt thread goes back to the pool
6. The pinned thread is unpinned and resumes

Deliver, pin, activate a pooled interrupt thread, handle (with ordinary locking), return it to the pool, and resume the pinned thread where it stopped.

11. Which statements about Solaris interrupt threads are true?

- A. Each has its own identifier, priority, context and stack
- B. They run at a higher priority than all other kernel threads
- C. A new thread is created each time an interrupt arrives
- D. They synchronize with other kernel threads using mutual exclusion primitives
- E. A pinned thread may move to another processor while it waits

Answer: A, B, D

Interrupt threads are full kernel threads with top priority, kept deactivated in a pool so none is created on demand, and they use ordinary locks. A pinned thread stays on its processor until the interrupt is handled.

12. Why does Solaris handle interrupts with interrupt threads instead of blocking interrupts around shared kernel data?

- A. Blocking interrupts is impossible on a single processor
- B. The extra cost is paid only when an interrupt occurs, not on every entry to shared kernel data
- C. Interrupt threads never need locks, because they run at the highest priority
- D. User-level threads can then handle hardware interrupts directly, without entering the kernel

Answer: B. The extra cost is paid only when an interrupt occurs, not on every entry to shared kernel data

Critical sections are entered far more often than interrupts arrive, so paying per interrupt is cheaper than raising and lowering the interrupt level on every access. On a multiprocessor the old way is even worse, since interrupts may have to be blocked on every processor.

4.6 Linux Process and Thread Management

You should be able to

- List the kinds of information Linux keeps in a `task_struct` and sort real fields into those categories.
- Name the five Linux task states, say which events move a task between them, and explain how Interruptible and Uninterruptible sleep differ.
- Explain why Linux has no separate thread structure: a thread is a task created by `clone()` that shares memory, files and other resources and belongs to the same thread group.
- Choose `clone()` flags that give `fork()`-like, `vfork()`-like or thread-like behaviour, predict what parent and child share, and say what `CLONE_NEWPID`, `CLONE_PARENT`, `CLONE_SYSVSEM`, `CLONE_SETTLS` and `CLONE_VFORK` are for.
- Describe the six namespaces and control groups, and explain how together they build containers.

1. One kind of task

Linux keeps one kind of record. Every process and every thread is a **task** described by a **`task_struct`** (Linux's process control block). Two tasks count as two processes or as two threads of one process only by **how much they share**: processes have their own copies of memory, open files and so on; threads point to the same ones. (This Linux meaning of "task" is not the older one from section 4.1, where "task" was another name for the resource-owning process. Because each thread has its own `task_struct`, the `task_struct` also plays the role of the thread control block.)

2. `task_struct`: the kernel's record of a task

Well over a hundred fields, in nine groups:

Group	What it holds	Example fields
State	Which of the five states the task is in.	state
Scheduling information	Normal or real-time class (real-time runs first), priority, and a counter of the processor time it may still use.	policy, prio, time_slice
Identifiers	Unique process ID; user and group IDs used for access rights.	pid, tgid, uid, gid
Interprocess communication	Pending signals, signal handlers, System V semaphores, message queues, shared memory.	pending, sighand, sysvsem
Links	Pointers to parent, siblings and children.	real_parent, children, sibling
Times and timers	Creation time, processor time used (user and kernel mode), interval timers that signal the task when they expire.	start_time, utime, stime
File system	Pointers to the open-file table, the current directory and the root directory.	files, fs
Address space	Pointer to the description of the virtual address space.	mm
Processor-specific context	Saved registers and stack information so the task can resume; depends on the processor type.	thread, stack

Many fields are **pointers**, so two tasks can point to the same table: that is how sharing works.

3. The five Linux task states

State	ps shows	Meaning
Running	R	Executing now, or ready in the run queue (TASK_RUNNING for both).
Interruptible	S	Blocked until an event: I/O done, a resource free, or a signal. A signal wakes it.
Uninterruptible	D	Blocked directly on a hardware condition (e.g. a disk transfer); handles no signals until it is met. Newer kernels add a killable variant that SIGKILL alone can end.
Stopped	T	Halted; resumes only when another process acts (e.g. SIGCONT). A program held by a debugger is stopped too (ps shows t, traced); the debugger resumes it.
Zombie	Z	Terminated, but its task_struct stays in the process table until the parent's wait().

Transitions. Creation puts a task in Running (ready); scheduling moves it between ready and executing. Waiting sends it to Interruptible or Uninterruptible; the event (or, for Interruptible only, a signal) returns it to ready. A stop signal sends it to Stopped; SIGCONT returns it to ready. Termination makes it a Zombie; the parent's wait() removes it.

A task acts on signals only when it runs, so a signal sent to an Uninterruptible task, even SIGKILL, stays pending until the hardware wait ends: a process stuck in plain D sleep cannot be killed. A typical life: created (ready) → executing → waits for a key (Interruptible) → key press (ready) → exit() (Zombie) → parent's wait() frees the task_struct.

4. Linux threads are tasks that share

Traditional UNIX gave each process one thread. Linux keeps no separate structure for threads: each user-level thread is mapped onto its own kernel-level task (one-to-one), and the tasks of one program share one **thread group ID (TGID)**. getpid() returns the TGID; gettid() returns the task's own ID. ps shows one line per process (PID = TGID); ps -L shows one line per task (LWP column).

Because tasks of a group share memory and files, switching between them skips the expensive part of a context switch: the memory map stays loaded. Switching to another group's task must load a new map.

Worked example. A web server's main thread creates 7 workers with pthread_create(). The kernel keeps 1 + 7 = **8 task_structs**, all in one thread group with one TGID.

5. clone() and its flags

A new task starts as a copy of the current one. Linux creates it with **clone()**, whose flags choose, resource by resource, sharing or copying. **fork() is clone() with all sharing flags cleared.** Two tasks sharing one virtual memory act as threads of one process, with no separate thread structure.

Flag	Effect
CLONE_VM	Share the address space (memory).
CLONE_FILES	Share the open-file table.
CLONE_FS	Share current directory, root directory and umask (default permissions for new files).
CLONE_SIGHAND	Share the table of signal handlers.
CLONE_THREAD	Join the caller's thread group (same TGID).
CLONE_NEWPID	Start the new task in a new PID namespace, where it is PID 1 (container runtimes; needs administrator rights).
CLONE_PARENT	The new task gets the caller's parent, so it is the caller's sibling and its exit is reported to that parent.
CLONE_SYSVSEM	Share one list of semaphore undo entries (changes the kernel reverses when a task exits); undone once, when the last sharer exits.
CLONE_SETTLS	Point the new task at its own thread-local storage block (per-thread variables such as errno).
CLONE_VFORK	Caller sleeps until the child calls exec() or exits; with CLONE_VM the child borrows the parent's memory, no copying (vfork(), posix_spawn()).

fork() = no sharing flags (plus SIGCHLD, the signal the parent gets when the child ends); vfork() = CLONE_VM + CLONE_VFORK; pthread_create() = CLONE_VM, FS, FILES, SIGHAND, THREAD, SYSVSEM, SETTLS. Without CLONE_VM a child's write changes only its own copy; with it, both tasks use the same memory, so the parent can read the new value (reliably, and in order, only with synchronization such as a lock or an atomic operation; section 5.1).

Invalid combinations (errno = EINVAL, no task is made). For the flags above: CLONE_SIGHAND without CLONE_VM (handlers are code addresses, so sharing them needs shared memory); CLONE_THREAD without CLONE_SIGHAND (a thread group shares one set of handlers); and CLONE_THREAD with CLONE_NEWPID (the threads of a group share one queue of waiting signals, so they must share one PID numbering). **A documentation discrepancy:** the clone() manual also lists CLONE_PARENT with CLONE_NEWPID as EINVAL, and kernels before 3.13 refused it, but current kernels accept the pair (the new namespace's PID 1 then becomes a child of the caller's parent); the lab follows current kernels and shows a portability note, because portable code still avoids the pair. The manual also forbids CLONE_FS with CLONE_NEWNS (a new mount namespace needs its own root and current directory), CLONE_SYSVSEM with CLONE_NEWIPC (the shared undo entries would refer to semaphores the new IPC namespace cannot reach), and CLONE_NEWUSER with CLONE_THREAD or CLONE_FS.

6. Namespaces

A **namespace** gives a process, or processes sharing it, a different view of the system from other processes; namespaces are the basis of containers. The six classic ones (newer kernels add cgroup and time namespaces):

Namespace	Its processes get their own...	clone() flag
Mount	mount points, so a different file-system tree and root	CLONE_NEWNS
UTS	hostname and domain name (what uname() reports)	CLONE_NEWUTS
IPC	semaphores, message queues, shared-memory segments	CLONE_NEWIPC
PID	process ID numbering; the first process inside is PID 1	CLONE_NEWPID
Network	network devices, IP addresses, routing table, port numbers	CLONE_NEWNET
User	user and group ID mapping	CLONE_NEWUSER

They are made with clone() flags or unshare() (for PID, unshare() places only the caller's future children in the new namespace), and joined with setns(). In a PID namespace a process can be PID 1 inside and have another PID on the host. In a user namespace it can be root (UID 0) inside yet map to an ordinary UID such as 100000 outside, with no power over the host.

7. Control groups and containers

A **control group (cgroup)** is a group of tasks in a tree of directories under /sys/fs/cgroup. The kernel can **limit** the group's CPU time, memory, disk I/O and network (files such as cpu.max, memory.max), **prioritize** it (cpu.weight), **account** for its use, and **control** it as a unit (freeze or kill all its tasks). Namespaces decide what a process can **see**; cgroups, how much it can **use**.

Worked example. cpu.max = "50000 100000" means a quota of 50,000 μ s of CPU time in every 100,000 μ s period: half a processor. One second holds 10 periods, so the group can use at most $10 \times 50 \text{ ms} = \mathbf{500 \text{ ms}}$ of CPU time per second; once the quota is spent its threads are throttled until the next period. Without cgroups, 8 busy threads in one container and 1 in another on 4 cores get $4/9 \approx 0.44$ core each, so the lone thread runs 2.25 times slower. A group over its memory.max loses a process to the out-of-memory killer inside that group only.

A **container** = namespaces + cgroups + its own root file system, as ordinary tasks on the host's one kernel (Docker, LXC). A virtual machine runs its own guest kernel on a hypervisor and must boot a whole OS.

Key terms

task_struct

The data structure the Linux kernel keeps for every task, that is, for every process and every thread. It records the task's state, scheduling data, identifiers, family links, times, open files, address space and saved processor context.

Linux task

The unit Linux creates and schedules. Every process and every thread is a task with its own task_struct; tasks count as "threads of one process" only because they share resources. (Not the older use of "task" in section 4.1 as another name for the resource-owning process.)

Running (Linux state)

The Linux state that covers two situations: the task is executing on a processor right now, or it is ready and waiting for the scheduler to pick it. The ps tool shows it as R.

Interruptible (Linux state)

A blocked state. The task sleeps until an event happens, such as I/O finishing, a resource becoming free or a signal arriving; a signal wakes it. ps shows it as S.

Uninterruptible (Linux state)

A blocked state in which the task waits directly on a hardware condition and does not react to signals until that condition is met. ps shows it as D.

Stopped (Linux state)

The task has been halted, for example by the SIGSTOP signal or by a debugger, and resumes only when another process acts on it, for example by sending SIGCONT. ps shows it as T.

Zombie (Linux state)

The task has terminated, but its task_struct must stay in the process table until its parent collects the exit status with wait(). ps shows it as Z.

clone()

The Linux system call that creates a new task. Its flag bits decide, resource by resource, whether the new task shares with its creator or gets its own copy.

Clone flags

The bits passed to clone(), such as CLONE_VM (share memory) or CLONE_FILES (share the open-file table). With none of the sharing flags set, clone() behaves like fork().

Thread group

The set of tasks created with CLONE_THREAD from one original task. They share one thread group ID (TGID), which is the process ID that getpid() reports for every one of them.

Signal handler

A function a program registers to run when a particular signal arrives, for example to save its work before quitting.

Namespace

A Linux feature that gives a task, or a group of tasks, its own private view of one kind of system resource, such as process IDs, network devices or the hostname.

Mount namespace

Gives its tasks their own set of mount points, so they can see a different file-system tree, with a different root directory, from other tasks.

UTS namespace

Gives its tasks their own hostname and domain name: the values that the uname() call and the hostname command report.

IPC namespace

Gives its tasks their own set of interprocess-communication objects: semaphores, message queues and shared-memory segments.

PID namespace

Gives its tasks their own process-ID numbering. The first task inside gets PID 1, and every task also has its own PID in the parent namespace, so the host can still see and manage it.

Network namespace

Gives its tasks their own network stack: network devices, IP addresses, routing table and port numbers.

User namespace

Gives its tasks their own mapping of user and group IDs, so a task can be root (UID 0) inside the namespace while being an ordinary, unprivileged user outside it.

Control group (cgroup)

A kernel mechanism that places tasks in a tree of groups and limits, prioritizes, accounts for and controls (for example, freezes) the resources each group uses: processor time, memory, disk I/O and network.

Container

A group of ordinary processes that the kernel isolates with namespaces and limits with cgroups, so it looks like its own small machine while sharing the host's one kernel. Docker and LXC build containers this way.

Review questions with answers

1. A Linux task is blocked while a disk controller finishes a transfer, and it will not react to any signal until the hardware is done. Which state is it in?

- A. Interruptible
- B. Uninterruptible
- C. Stopped
- D. Zombie

Answer: B. Uninterruptible

Uninterruptible (ps shows D) is the blocked state for waits directly on hardware. Signals sent meanwhile stay pending and are acted on only after the wait ends.

2. Linux keeps a separate, smaller data structure for each thread, alongside the task_struct of the process that owns it.

Answer: False

Linux has no separate thread structure. Every thread is a task with its own full task_struct; the threads of one process simply point to the same memory, open files and other resources.

3. Sort each piece of a task_struct into its category.

Identifiers	Links	File system	Address space
user and group IDs	list of the task's children	pointer to the open-file table current and root directory	description of the virtual memory

Identifiers say who the task is and who it runs for; links hold its family tree; the file-system part points to open files and directories; the address-space part describes its virtual memory.

4. Match each clone() flag to what the new task shares with its creator.

CLONE_VM	the address space (memory)
CLONE_FILES	the open-file table
CLONE_FS	current directory, root directory and umask
CLONE_SIGHAND	the table of signal handlers
CLONE_THREAD	membership of the same thread group

Each flag shares one resource instead of copying it. Setting all five (as pthread_create() does) makes the new task a thread of the same process; setting none of them is exactly fork().

5. Match each Linux namespace to what it gives its processes their own copy of.

Mount	the file-system tree (mount points)
UTS	hostname and domain name
IPC	semaphores, message queues and shared memory
PID	process ID numbers
Network	network devices, IP addresses, routes and ports
User	user and group ID numbers

Each namespace isolates one kind of resource. Together the six give a container its own view of the system.

6. A web server's main thread starts 7 worker threads with pthread_create(). How many task_structs does the Linux kernel keep for this one process?

Answer: 8

Every thread is its own task: 1 main thread + 7 workers = 8 task_structs. They form one thread group and share one TGID, which is the process ID that getpid() reports.

7. A container's cgroup has cpu.max set to "50000 100000": a quota of 50,000 µs of CPU time in every 100,000 µs period. At most how many milliseconds of CPU time can the container use in one second?

Answer: 500 ms

50,000 µs per 100,000 µs is half a processor. One second holds 10 periods, so the most it can use is 10 × 50 ms = 500 ms; after that its threads are throttled until the next period.

8. Put the life of a Linux task in order.

Correct order:

1. clone() or fork() creates it: Running, ready to run
2. The scheduler dispatches it and it executes
3. It waits for a key press and becomes Interruptible
4. The key press wakes it: Running (ready) again
5. It calls exit() and becomes a Zombie
6. Its parent calls wait() and the task_struct is freed

Created ready, dispatched, blocked on an event, woken back to ready, terminated into a zombie, and finally removed when the parent collects its exit status.

9. A shell is about to start a new program. Its child will call exec() at once, so the shell wants the child to **borrow** the shell's memory instead of getting its own copy, and the shell wants to sleep until that exec() happens. Which clone() flags ask for exactly this?

- A. CLONE_VM | CLONE_VFORK
- B. No flags at all, as fork() uses
- C. CLONE_VM | CLONE_SIGHAND | CLONE_THREAD
- D. CLONE_VFORK on its own

Answer: A. CLONE_VM | CLONE_VFORK

CLONE_VM lends the parent's memory to the child with no copying, and CLONE_VFORK keeps the parent asleep until the child calls exec() or exits, so the two never use that memory at once. This pair is what vfork() asks for; fork(), by contrast, is clone() with none of the sharing flags.

10. Which statements about Linux containers are true?

- A. All containers on a host share the host's one kernel
- B. Namespaces decide what a container's processes can see
- C. cgroups limit and account for the resources a container uses
- D. Each container boots its own guest kernel, like a virtual machine
- E. A process can be PID 1 inside its container and have a different PID on the host

Answer: A, B, C, E

A container is a set of ordinary tasks isolated by namespaces and limited by cgroups on the host's kernel. Its PID namespace numbers processes from 1, while the host numbers the same processes its own way. Only a virtual machine runs a separate kernel.

11. Inside its user namespace a container process runs as UID 0 (root), and that ID maps to UID 100000 on the host. What may it do to a host file owned by the host's real root?

- A. Anything, because it is root
- B. Only what an ordinary user with UID 100000 could do
- C. Nothing with any file anywhere
- D. Read it freely but not write it

Answer: B. Only what an ordinary user with UID 100000 could do

A user namespace maps IDs. Root powers apply to things owned by the process's own namespaces; on the host it is treated as the unprivileged user it maps to.

12. A debugger has paused a program. Which Linux state is the program in, and what brings it back?

- A. Stopped: it runs again only when another process acts on it, here the debugger telling it to continue
- B. Interruptible: it wakes by itself when its I/O completes
- C. Zombie: its parent must call wait()
- D. Running: the scheduler resumes it when its time slice refills

Answer: A. Stopped: it runs again only when another process acts on it, here the debugger telling it to continue

Stopped (ps shows T, or a lowercase t while a debugger traces it) means halted and not runnable. Only a positive action by another process lets it run again: SIGCONT for an ordinary stop, or the debugger's continue command for a traced program.

4.7 Android Process and Thread Management

You should be able to

- Name Android's four kinds of application component and choose the right one for a given job.
- Trace the callbacks an activity receives as the user launches, covers, leaves, rotates and returns to it, and name its three nested lifetimes.
- Explain why an app starts with one process and one main (UI) thread, and why slow work belongs on a worker thread.
- Rank processes by Android's five-level importance hierarchy and predict which one is killed first when memory runs short.

Why Android needs its own rules

A phone has far less memory than a laptop, and its user switches apps constantly. A desktop keeps a program alive until the user quits it (parking idle parts on disk if needed); Android does not wait. When a new app needs memory, Android **kills whole processes**, starting with the ones the user would miss least. This is safe because Android drives every app component through callbacks, so an app can save its state (a bookmark) before it may be killed.

The four kinds of app component

Component	What it is	Activated by	Examples
Activity	One screen the user sees and touches; most apps have several.	an intent	inbox, settings page
Service	Long-running work with no screen; keeps going after the user switches apps.	an intent	music playback, photo upload
Content provider	A standard doorway (query, insert, update, delete) to app data kept in files, an SQLite database or on the web; used by the app and, with permission, by other apps.	a query to its <code>content://</code> name, not an intent	contacts, calendar events
Broadcast receiver	Sleeps until a system-wide announcement arrives, reacts briefly, then finishes; longer follow-up work goes to a service.	a broadcast intent	battery low, screen off

An **intent** is a small message asking Android to activate a component. An **explicit** intent names the exact component (open the player screen); an **implicit** one describes an action and lets Android choose an app ("share this link"). Because components are started through Android rather than called directly, Android can first start the process, check permissions or pick the app.

Choosing: the user looks at it → activity; long work, no screen → service; shares stored data → content provider; reacts to an announcement → broadcast receiver.

One app, one sandbox

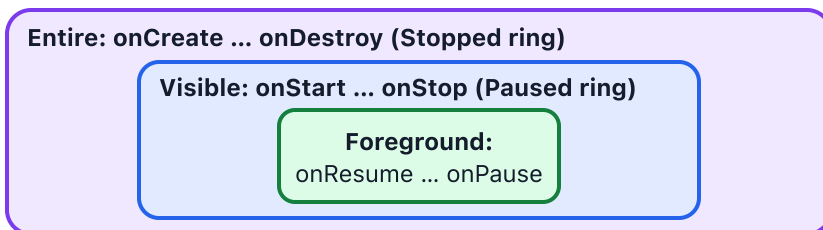
- By default each app runs in **its own Linux process under its own user ID**. This **application sandbox** stops one app reading or damaging another app's memory or files.
- Each process runs **its own instance of the virtual machine** (Dalvik on early phones, ART today). This is a language runtime like the Java virtual machine of section 4.3, not a simulated computer with its own operating system like the virtual machines compared with containers in section 4.6. To start quickly, new app processes are forked from Zygote, a parent process that already has the runtime loaded.
- All of an app's components share that one process (the manifest can request extra processes, but that is not the default). Android creates it when the first component is needed and, to reclaim memory, kills it with every component inside.

The activity lifecycle: seven callbacks

Callback	Called when	A well-written app uses it to
<code>onCreate()</code>	being created: first launch, return after a kill, or rotation	build the layout, set up data, restore saved state
<code>onStart()</code>	about to become visible	start what the screen needs while seen
<code>onResume()</code>	about to come to the front and get input	restart animations, video, camera, sensors
<code>onPause()</code>	another activity comes in front, even partly (e.g. a permission window; an ordinary in-app dialog does not count), or the user starts to leave	pause animations, release the camera; must be quick
<code>onStop()</code>	no longer visible at all	save the user's work, release what is not needed
<code>onRestart()</code>	a Stopped activity is about to be shown again	anything special on return; <code>onStart</code> and <code>onResume</code> follow
<code>onDestroy()</code>	finishing (<code>Back</code> , <code>finish()</code>) or being rebuilt (rotation)	final clean-up. Not guaranteed to run.

States. Resumed (running): in front, has the user's focus. **Paused**: at least partly visible but without focus (an activity being left also passes briefly through Paused). **Stopped**: completely hidden, still in memory with its data.

Three nested lifetimes. Entire: `onCreate()` to `onDestroy()`. **Visible**: `onStart()` to `onStop()`, possibly entered many times. **Foreground**: `onResume()` to `onPause()`, entered and left very often, so its code must be light.



Killed for memory. Android may kill a process whose activities are Stopped (or, only when desperate, Paused). **No callback runs**, not even `onDestroy()`. What survives is the small bundle of state the activity saved in `onSaveInstanceState()` as it stopped, held by Android outside the process. If the user returns, a new process starts and `onCreate()` gets that bundle, so the screen looks as it was. Common mistake: saving work only in `onDestroy()`; use `onPause()` or `onStop()`.

Lifecycle traces worth knowing (activities A = list, B = editor)

What the user does	Callbacks, in order
Launch (no process yet)	new process; A.onCreate, A.onStart, A.onResume
A permission window covers part of A, then closes	A.onPause only; later A.onResume
A opens B full-screen	A.onPause; B.onCreate, B.onStart, B.onResume; then A.onStop
Back from B	B.onPause; A.onRestart, A.onStart, A.onResume; B.onStop, B.onDestroy
Home, then return	onPause, onStop; later onRestart, onStart, onResume
Return after a low-memory kill	new process; onCreate (with saved state), onStart, onResume
Rotate the phone	onPause, onStop, onDestroy, then onCreate, onStart, onResume on a new instance (six)
Back on the last activity	onPause, onStop, onDestroy; the process stays as an empty process , so the next launch is a fast "warm" start (Android 12+ usually just stops it instead)

The **back stack** lists the activities the user has opened, newest on top. Opening B pushes it on top of A; Back finishes the top one and brings back the one underneath. Android keeps the back stack and saved state even when it kills the process.

One process, one main thread

- By default Android starts each app as **one process with one thread** for its code, the **main thread** (UI thread). It runs every component's lifecycle callbacks, draws every frame (about 60 per second) and handles every tap. (Exception: other apps' queries to a content provider run on helper threads.)
- If the main thread is stuck in slow work (resizing photos, a big query), nothing is redrawn and taps queue up. When input has waited about **5 seconds**, Android shows the **Application Not Responding (ANR)** dialog: wait, or close the app (its process is killed, unfinished work lost).
- The fix: do slow work on a **worker thread** the app creates and hand the result back (e.g. with `runOnUiThread`). Only the main thread may touch the screen, because the drawing code is not built for two threads at once.
- Common mistake: a service does **not** get its own thread; slow work in a service freezes the screen too.

Example. An 8-second photo resize runs in a tap handler on the main thread; the user taps Like at 1 s. At 6 s the tap has waited 5 s, so the ANR dialog appears; the tap is handled only at 8 s. With a worker thread it is handled at 1 s.

The importance hierarchy: who is killed first

Level	The process holds	Example	If killed, the user loses
1. Foreground	what the user is interacting with now (a Resumed activity, or a receiver handling an event)	the chat you are typing in	the app in their hands
2. Visible	an activity that can be seen but is not in front (Paused)	an app behind a permission window; a picture-in-picture video	something on screen
3. Service	a started service, nothing visible	an episode download, a cloud upload	work they asked for
4. Background	only Stopped activities	apps you switched away from	little: rebuilt from saved state
5. Empty	no active components; kept only as a cache	an app you left with Back	a little launch speed

A service the user is plainly aware of (music playing with a notification shown) is promoted to the visible level.

Rules (applied by Android's **low-memory killer**): kill from the lowest level upward; within a level, kill the **least recently used** process first; stop as soon as enough memory is free. Level always beats recency: an empty process used 10 minutes ago dies before a background process unused for hours.

Worked example. 250 MB free; a new app needs 700 MB. Kill Weather (empty, used 3 h ago, 90 MB): 340 MB free. Kill Calculator (empty, 1 h, 60 MB): 400 MB. No empties left, so kill the oldest background app, Game (2 h, 600 MB): 1,000 MB, enough. Every other process survives.

Key terms

Activity

One screen of an Android app that the user sees and touches, such as a message list or a compose screen. An app usually has several.

Service

An Android app component with no screen that carries out long-running work in the background, such as playing music or downloading a file, and can keep going after the user switches to another app.

Content provider

An Android app component that offers a standard interface to a set of app data (stored in files, an SQLite database or on the web) so that the owning app and, with permission, other apps can read or change it. The contacts list is the classic example.

Broadcast receiver

An Android app component that sleeps until the system (or another app) announces an event to everyone, such as "battery low", "screen turned off" or "download finished", and then reacts briefly.

Intent

A small message object that asks Android to start an activity or a service, or that carries a broadcast announcement. An explicit intent names the exact component wanted; an implicit intent names only the kind of action wanted and lets Android pick an app.

Application sandbox

Android's isolation of apps: by default each app runs in its own Linux process, under its own user ID, with its own instance of the virtual machine, so one app cannot read or damage another app's memory or files.

Back stack

The list of activities the user has opened, in the order they were opened, with the newest on top.

Pressing Back finishes the top activity and brings back the one underneath; Android keeps the list even if it kills the app's process.

Lifecycle callback

A method (onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy) that Android calls on an activity each time the activity moves from one state to another, giving the app a chance to set up, save or release things.

Paused state

The state of an activity that is still at least partly visible but no longer has the user's focus, because another activity has appeared in front of part of it. An activity that is being left also passes briefly through Paused on its way to Stopped.

Stopped state

The state of an activity that is completely hidden. It stays in memory with its data, but it is not drawn and should do no work.

Entire lifetime

The span of an activity between onCreate() and onDestroy(): everything the activity owns is set up at the start and released at the end.

Visible lifetime

The span of an activity between onStart() and onStop(), during which it is on screen, even if only partly.

Foreground lifetime

The span of an activity between onResume() and onPause(), during which it is in front of every other activity and receives the user's input.

Main thread (UI thread)

The first thread of an app's process: it handles the user's input and is the only thread that draws the screen, so it must never be kept busy for long or the app freezes. On Android it also runs the lifecycle callbacks of all the app's components by default.

Application Not Responding (ANR)

The error dialog Android shows when an app's main thread has not responded to an input event, such as a tap, within about five seconds. It offers to wait or to close the app.

Importance hierarchy

Android's ranking of processes, from most to least important: foreground, visible, service (running a started service, nothing visible), background, empty. When memory is short, Android's low-memory killer ends the least important processes first.

Foreground process

A process doing what the user is focused on right now, for example running the activity the user is touching or a broadcast receiver that is handling an event. It is killed only as a last resort.

Visible process

A process whose activity can still be seen but is not in the foreground (it is Paused), for example an app partly hidden behind a permission window or shown in a small picture-in-picture window. A service the user is plainly aware of, such as music playing with a notification on screen, also ranks here.

Background process

A process holding only activities that are Stopped (not visible). Android keeps these in least-recently-used order and kills the oldest first when it needs memory.

Empty process

A process that holds no active app components at all. Android keeps it only as a cache so the app starts faster next time, and it is the first to be killed.

Review questions with answers

1. A podcast app must keep playing an episode after the user switches to the web browser. Which kind of Android component should do the playing?

- A. An activity
- B. A service
- C. A broadcast receiver
- D. A content provider

Answer: B. A service

Long-running work that needs no screen and must keep going when the user moves to another app is exactly the job of a service.

2. Match each Android component to its job.

Activity	One screen the user sees and touches
Service	Long-running work with no user interface
Content provider	A standard doorway through which apps read or change a set of stored data
Broadcast receiver	Wakes up briefly when an event such as "battery low" is announced

Screens are activities, background jobs are services, shared data goes through a content provider, and reactions to system-wide announcements are handled by broadcast receivers.

3. Which statements about how Android runs an app are true **by default**? Select all that apply.

- A. Each app runs in its own Linux process, under its own user ID
- B. Each app has its own instance of the virtual machine
- C. The lifecycle callbacks of an app's activities, services and broadcast receivers all run on one main thread unless the app creates more threads
- D. A service automatically gets a worker thread of its own
- E. Every activity of an app gets a separate process
- F. Content providers are started by intents, just like activities

Answer: A, B, C

The default is one process, one virtual-machine instance and one main thread per app, shared by all its components. A service runs on that main thread unless the app starts a worker, and a content provider is reached by a query, not an intent.

4. The user launches an activity, uses it, then presses Back to leave it for good. Put the callbacks in the order Android calls them.

Correct order:

1. onCreate()
2. onStart()
3. onResume()
4. onPause()
5. onStop()
6. onDestroy()

On the way in, the activity is created, becomes visible, then comes to the front. Leaving for good reverses the path: it loses the focus, becomes hidden, then is destroyed.

5. Match each lifetime of an activity to the pair of callbacks that begins and ends it.

Entire lifetime	onCreate() ... onDestroy()
Visible lifetime	onStart() ... onStop()
Foreground lifetime	onResume() ... onPause()

The three lifetimes are nested: the foreground lifetime sits inside the visible lifetime, which sits inside the entire lifetime.

6. Which state is activity X in, in each situation?

Resumed	Paused	Stopped
The user is typing a message into X The user just came back to X and its onResume() has finished	A system "Allow access to the microphone?" permission window (a separate activity) covers part of X The user tapped Home; X's onPause() has run but onStop() has not yet been called	The user pressed Home and X can no longer be seen at all X opened a second full-screen activity, which now completely covers X

Resumed means in front with the user's focus. Paused means at least partly visible (or on the way out) without the focus. Stopped means completely hidden but still in memory.

7. A notes app has two activities. The user opens the note list, taps a note to open the editor, then presses Back. What happens?

- A. The editor is paused and stays on the back stack, and the list is created again from scratch.
- B. The editor is finished and removed from the back stack, and the list, which was Stopped underneath it, restarts and resumes.
- C. Both activities are destroyed and the user lands on the home screen.
- D. Android kills the app's process to free the memory the editor was using.

Answer: B. The editor is finished and removed from the back stack, and the list, which was Stopped underneath it, restarts and resumes.

The back stack holds the list with the editor on top. Back pops the editor (onPause, onStop, onDestroy) and brings the list back with onStart, onResume.

8. When Android kills an app's process to reclaim memory, it first calls onDestroy() on each of the app's activities, so saving the user's work in onDestroy() is safe.

Answer: False

A low-memory kill removes the whole process at once and no callback runs at all. Work must be saved earlier, in onPause() or onStop().

9. An app resizes 300 photos inside a button's tap handler, which runs on the main thread. The work takes 8 seconds. What does the user experience?

- A. Nothing unusual: Android notices the slow work and moves it to a worker thread.
- B. The screen freezes and ignores taps, and after about 5 seconds of unanswered input Android offers to close the app (an ANR dialog).
- C. The resizing is cancelled after 5 seconds, but the app keeps running normally.
- D. Android immediately kills the process because long work is forbidden in a tap handler.

Answer: B. The screen freezes and ignores taps, and after about 5 seconds of unanswered input Android offers to close the app (an ANR dialog).

The main thread both runs the handler and draws the screen, so while it is busy nothing is redrawn and taps pile up. Slow work belongs on a worker thread, which hands its result back to the main thread.

10. Order these processes from MOST important (killed last) to LEAST important (killed first).

Correct order:

1. The app the user is typing into right now
2. An app still partly visible behind a permission window that has the focus
3. An app with no screen showing that is downloading podcast episodes through a started service
4. An app whose only activity is Stopped, last used an hour ago
5. An app with no active components, kept only as a cache

The importance hierarchy is foreground, visible, service, background, empty. Android reclaims memory starting from the bottom of that list.

11. Memory is short and Android must kill exactly one process. Which one goes first?

- A. A background process last used 3 hours ago
- B. An empty process last used 10 minutes ago
- C. A service process that is uploading photos
- D. A background process last used 5 minutes ago

Answer: B. An empty process last used 10 minutes ago

An empty process holds no running component, so killing it costs the user nothing but a slower next launch. The level decides first; recency decides only within a level.

12. A phone has 300 MB free and a new app needs 1,000 MB. The other processes are: empty E1 (150 MB, used 1 h ago), empty E2 (100 MB, used 3 h ago), background B1 (400 MB, used 2 h ago), background B2 (200 MB, used 10 min ago) and service S (500 MB). Android kills in importance order and stops as soon as at least 1,000 MB is free. How many MB are free when it stops?

Answer: 1150 MB

Empty processes go first, oldest first: E2 (now 400 MB free), then E1 (550). Next come background processes, oldest first: B1 (950), still short, then B2 (1,150). The service process S survives.

4.8 Mac OS X Grand Central Dispatch

You should be able to

- Explain what problem Grand Central Dispatch solves and how its OS-managed thread pool differs from creating threads by hand.
- Read and write simple blocks, and predict which values a block captures.
- Tell serial, concurrent, main and global dispatch queues apart and predict the orders in which their blocks can run and finish.
- Use the `dispatch_async` pattern to move slow work off the main thread and hand the result back to the main queue.
- Describe what a dispatch source does and name events it can watch.

1. The problem GCD solves

Extra cores help only when a program splits its work into pieces that can run at the same time, and something must create threads, hand them the pieces and clean up. **Grand Central Dispatch (GCD)**, first shipped in **Mac OS X 10.6 (Snow Leopard)**, moves that job into the OS. The programmer describes **tasks** (units of work, such as analysing one document) and GCD maps them onto a **thread pool** it manages: worker threads created once and reused. The pool is sized to the **degree of concurrency** of the hardware (roughly the number of cores), plus a few extra workers while some are blocked on I/O.

- **Your job:** find work that can run in parallel, write each piece as a **block**, and put it on a **dispatch queue**.
- **GCD's job:** decide how many threads exist, give each waiting block to a free thread, reuse threads, grow the pool when workers are blocked.
- **The kernel's job:** schedule the pool's threads onto the cores like any other threads.

Layers: your code (blocks) → dispatch queues → thread pool → cores. Because the pool follows the hardware, the same program uses 2 cores on a 2-core laptop and 10 on a 10-core desktop.

What is new? Thread pools are old: servers and Windows (whose thread pool appears in section 4.4) have had them for years. GCD adds a **language extension (blocks)** for writing a task right where it belongs in the code, plus **queues** that keep tasks in order, so a unit of work can be split off without losing the order and dependencies between its parts.

2. Threads by hand versus tasks for GCD

Three ways to run many small, independent tasks:

Strategy	Threads	Weakness
One thread per task	as many as tasks	Creating and destroying threads costs CPU; each thread reserves stack memory; more threads than cores must take turns (oversubscription): extra switching, no speed.
Hand-made pool	a number the author guessed	Too few leaves cores idle on a big machine; too many oversubscribes a small one. The author also writes the queue, locking and shutdown code.
GCD	about one per core, chosen by the system	None: the OS sizes the pool to the machine. Queueing a block is almost free: a dispatch queue is a lightweight data structure in the app's own memory.

Worked example (toy cost model): 32 tasks of 0.1 ms each on 8 cores; creating plus destroying a thread costs 0.04 ms; each thread reserves 0.5 MB of stack.

- One thread per task: 32 threads, $32 \times 0.5 = 16$ MB of stack, 4 threads per core. CPU time = $32 \times 0.1 + 32 \times 0.04 = 4.48$ ms, of which 1.28 ms (about 29%) is bookkeeping. Over 8 cores: 0.56 ms.
- Hand-made pool of 4: 4 threads, 2 MB. CPU time = $3.2 + 0.16 = 3.36$ ms, but only 4 of the 8 cores work: 0.84 ms.
- GCD: 8 threads, 4 MB. CPU time = $3.2 + 0.32 = 3.52$ ms over 8 cores: **0.44 ms**, the fastest.

Common mistake: more threads is not more speed; once every core is busy, extra threads only take turns.

3. Blocks

A **block** is an extension to C, Objective-C and C++ for writing an unnamed piece of code inline (an **anonymous function**). A caret and braces make one: `^{ printf("hello world\n"); }`. A block can be stored in a variable, passed to a function such as `dispatch_async`, and run later, even on another thread.

```
int (^twice)(int) = ^(int n) { return n * 2; }; // a block variable holding a doubling block
int r = twice(21);                          // call it like a function: r is 42
```

- `int` is the return type; `(^twice)` declares a variable that holds a block (the caret plays the role * plays for pointers); `(int)` lists parameter types.
- `^(int n) { ... }` is the block literal: the caret starts it, `(int n)` names the parameter, the braces hold the body.

- Writing a block does **not** run it. The body runs only when the block is called, or when GCD takes it off a queue.

Capture: a block remembers the outside variables it uses. Code packaged with the values it uses is a **closure**. By default an ordinary local variable is captured **by value** at the moment the block is created:

```
int x = 5;                // ordinary local variable
void (^greet)(void) = ^{ printf("x is %d\n", x); }; // block copies x (5) now
x = 9;                    // change x after the block exists
greet();                  // prints "x is 5"
```

If the variable is declared `__block int x = 5;` the block keeps a shared link to x instead of a copy, so the same code prints "x is 9". Copying is the safe default: by the time a pool thread runs the block, the function that made it may have returned and its locals be gone.

4. Dispatch queues and dispatch sources

A **dispatch queue** is a FIFO line of blocks; GCD takes blocks from the front and runs them on pool threads.

- **Serial queue:** one block at a time; the next starts only after the previous finishes. It may use different pool threads, never two at once. Routing every use of some shared data through one serial queue replaces a lock.
- **Concurrent queue:** blocks **start** in FIFO order but the next is handed out as soon as a thread is free, so blocks overlap and may **finish** in any order.
- **Main queue** (`dispatch_get_main_queue()`): serial, runs on the **main thread**, which handles events and drawing. UI updates go here; slow work never does.
- **Global queues** (`dispatch_get_global_queue(priority, 0)`): concurrent, system-provided, at **high, default and low** priority (plus **background** since 10.7). Newer GCD code usually asks for a **quality-of-service (QoS) class** instead of one of these priorities: user-interactive, user-initiated, utility or background, from most to least urgent.
- **Private queues:** `dispatch_queue_create(name, DISPATCH_QUEUE_SERIAL)` . In 10.6 every app-made queue was serial; private concurrent queues (`DISPATCH_QUEUE_CONCURRENT`) came in 10.7.

`dispatch_async(q, block)` adds the block and returns at once; `dispatch_sync` waits until it has run.

A **dispatch source** watches for a system event (a timer, a signal, a readable file descriptor or socket, a process exiting) and, when it happens, submits a handler block to a chosen queue. No thread of yours sits waiting. Events that arrive before the handler starts are **merged** into one run, and a source never runs its handler twice at once.

5. Timing on serial and concurrent queues

Serial: blocks of 300, 100 and 200 ms take $300 + 100 + 200 = 600$ ms, even with 4 cores free.

Concurrent, 2 threads: blocks of 300, 100, 200, 100 ms. At 0: thread 1 takes 300 (to 300), thread 2 takes 100 (to 100). At 100: thread 2 takes 200 (to 300). At 300: the last 100 ms block runs to **400 ms**. Speed-up = total work ÷ finish time = $700 \div 400 = 1.75\times$. The 100 ms block finished before the 300 ms block that

started earlier. If two overlapping blocks change the same data, the result is a race condition; putting them on one serial queue removes the overlap.

6. Which output orders are possible?

An order is impossible only if it breaks a guarantee: (1) blocks in the same serial queue run one at a time in queued order; (2) lines on one thread run in program order; (3) a main-queue block waits until the main thread is free. Durations make an order likely, never certain.

- Same **serial** queue: output order = submission order, whatever the durations.
- **Concurrent** queue: any finish order is possible.
- **Two queues**: A then B on serial s, C on global g: A precedes B and C may land anywhere (A B C, A C B, C A B).
- **Round trip from the main thread**: print 1, dispatch_async to g (print 2, then dispatch_async to main to print 3), print 4. Possible: 1 4 2 3 and 1 2 4 3. Impossible: 1 2 3 4, because 3 needs the main thread, busy with the handler until after 4.

7. The classic pattern

```
dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0), ^{
    Stats st = analyze(doc);                // slow work on a pool thread
    dispatch_async(dispatch_get_main_queue(), ^{
        showStats(st);                      // UI update on the main thread
    });
});
```

1. The handler (main thread) puts the work block on a global queue and returns at once, so the main thread keeps handling events.
2. A pool thread runs the slow analysis.
3. That block queues a second block, carrying the captured result, on the main queue.
4. The main thread runs it and updates the window.

Common mistake: doing slow work directly in an event handler freezes the app (spinning wait cursor) and leaves the other cores idle.

Key terms

Grand Central Dispatch (GCD)

Apple's system for running work in parallel, first shipped in Mac OS X 10.6 (Snow Leopard). A program describes units of work as blocks placed on dispatch queues, and the system runs them on a pool of threads it manages.

Thread pool

A group of worker threads that are created once and reused. Work is handed to the pool and an idle worker runs it, so no thread has to be created and destroyed for each piece of work.

Degree of concurrency

How many pieces of work the hardware can truly run at the same moment. On an ordinary machine it is about the number of cores.

Task (GCD)

In GCD, a unit of work: a self-contained piece of a job that can be done on its own, such as analysing one document. (Not the same meaning as the process-as-task of section 4.1 or the Linux task of section 4.6.)

Block (GCD)

An extension to C, Objective-C and C++ for writing an unnamed piece of code inline, marked by a caret, as in `^{ ... }`. A block can use variables from the code around it and can be stored or passed around like a value. GCD uses blocks to describe tasks.

Anonymous function

A function written inline without a name, usually so it can be handed straight to another function.

Closure

A piece of code packaged together with the variables it uses from the surrounding code, so it can run later, somewhere else, and still see those values.

Capture

What a block does with an outside variable it uses: it keeps its own copy of the value (or, for a variable marked `__block`, a shared link to it) for when the block runs later.

Dispatch queue

A first-in, first-out waiting line of blocks, kept as a lightweight data structure in the app's own memory, so adding a block is far cheaper than creating a thread. GCD takes blocks from the front and runs them on threads from its pool.

FIFO (first in, first out)

An ordering rule: items leave a line in the same order in which they joined it.

Serial queue

A dispatch queue that runs one block at a time: the next block starts only after the previous one has finished. It is often used instead of a lock to protect shared data.

Concurrent queue

A dispatch queue that starts its blocks in first-in, first-out order but lets several of them run at the same time on different threads, so they may finish in any order.

Main queue

The serial dispatch queue whose blocks run on the application's main thread. Work that updates the user interface is sent here.

Main thread

The first thread of an application (also called the UI thread): it runs the event loop that handles clicks, key presses and drawing, and it is the thread that updates the window. While it is busy, the app looks frozen.

Global queue

One of the concurrent dispatch queues that the system provides to every application, at several priority levels: high, default and low (plus background, added in Mac OS X 10.7). Newer code usually picks one by quality-of-service class instead.

dispatch_async

The GCD function that adds a block to a queue and returns immediately, without waiting for the block to run.

Dispatch source

A GCD object that watches for a system event, such as a timer firing, a signal arriving or a file descriptor becoming readable, and submits a handler block to a queue when the event happens. Events that arrive before the handler has run are merged into one run.

Oversubscription

Having more runnable threads than cores, so the threads must take turns; the extra switching adds cost without adding speed.

Review questions with answers

1. When a program uses Grand Central Dispatch, what does the programmer hand to the system?

- A. A number of threads to create, one for each core
- B. Units of work written as blocks, placed on dispatch queues
- C. A list of cores on which each thread must run
- D. Finished threads that GCD only has to start

Answer: B. Units of work written as blocks, placed on dispatch queues

With GCD you name the work, not the threads. Each task is written as a block and put on a queue; GCD maps those blocks onto a pool of threads it manages, about one per core.

2. A serial dispatch queue always runs all of its blocks on the same thread.

Answer: False

A serial queue promises only that its blocks run one at a time, in order. Each block can be run by whichever pool thread is free, so different blocks may use different threads; they just never overlap.

3. Blocks A, B and C are added, in that order, to a **concurrent** dispatch queue on a 4-core machine. What does the queue guarantee?

- A. They finish in the order A, B, C
- B. They run one at a time, A then B then C
- C. They are started (taken off the queue) in the order A, B, C, but may run at the same time and finish in any order
- D. Nothing at all: they may even be started in any order

Answer: C. They are started (taken off the queue) in the order A, B, C, but may run at the same time and finish in any order

Every dispatch queue hands blocks out first in, first out. A concurrent queue does not wait for one block to finish before handing out the next, so blocks overlap and can finish in any order.

4. Which statements about the **main queue** are true? Select all that apply.

- A. It is a serial queue
- B. Its blocks run on the application's main thread
- C. It is the right place to update the user interface
- D. It is the best place for slow work, because it is always available
- E. It is one of the concurrent global queues

Answer: A, B, C

The main queue is serial and served by the main thread, the thread that handles events and draws the window. UI updates belong there; slow work does not, because while a block runs on it the app cannot respond. The global queues are a separate, concurrent set.

5. Match each GCD object to what it does.

Main queue	Serial; runs on the main thread
Global queue	Concurrent; shared, by priority
Private serial queue	App-made; can stand in for a lock
Dispatch source	Turns events into queued blocks

The main queue serves the user interface, the global queues run independent background work at a chosen priority (high, default, low, and background since 10.7), a private serial queue can replace a lock, and a dispatch source turns events such as timers or signals into queued blocks.

6. A button handler on the main thread uses GCD to analyse a document without freezing the app. Put the events in order.

Correct order:

1. The user clicks Analyze and the main thread starts running the handler
2. The handler calls `dispatch_async` to put an analysis block on a global queue
3. A pool thread takes the block off the queue and runs the slow analysis
4. The analysis block calls `dispatch_async` to put a UI block on the main queue
5. The main thread runs the UI block and shows the results

Each event causes the next. `dispatch_async` returns at once, so the handler finishes right away and the main thread stays free while a pool thread does the slow part. The result travels back as a second block on the main queue, because window code must run on the main thread.

7. Three blocks that take 300 ms, 100 ms and 200 ms are added, in that order, to a **serial** queue on an otherwise idle 4-core machine. How many milliseconds after the first block starts does the last block finish?

Answer: 600 ms

A serial queue runs one block at a time no matter how many cores are free: $300 + 100 + 200 = 600$ ms. The other three cores do nothing for this queue.

8. Four blocks that take 300, 100, 200 and 100 ms are added, in that order, to a **concurrent** queue served by 2 pool threads. Each time a thread becomes free it takes the block at the front of the queue. When does the last block finish (in ms)?

Answer: 400 ms

At 0, thread 1 takes the 300 ms block and thread 2 the 100 ms block. At 100 thread 2 is free and takes the 200 ms block (busy until 300). At 300 both are free; one takes the last 100 ms block, finishing at 400 ms.

9. What does this code print?

```
int x = 5;           // ordinary local variable
void (^show)(void) = ^{ // create a block
    printf("%d\n", x); // the block uses x
};                  // end of the block
x = 9;              // change x after the block exists
show();             // run the block
```

- A. 5
- B. 9
- C. 14
- D. Nothing: the block was never queued

Answer: A. 5

An ordinary variable used inside a block is captured by value at the moment the block is created. The block's copy is 5; changing x afterwards does not affect it.

10. With GCD, who is responsible for each job?

The programmer	GCD and the OS
Finding pieces of work that can run at the same time Writing each piece of work as a block Choosing which queue receives each block	Deciding how many worker threads exist Assigning a waiting block to a free thread Adding workers when some are blocked on I/O

The programmer identifies and describes the work and chooses queues. GCD owns the thread pool: how many threads, which thread runs which block, and when to grow or shrink the pool.

11. This code runs on the main thread inside a button handler. `g` is a global queue and `main` is the main queue. Which output is **impossible**?

```
print("1");           // right away, on the main thread
dispatch_async(g, ^{   // queue a block on g; do not wait
    print("2");        // later, on a pool thread
    dispatch_async(main, ^{ print("3"); }); // queue a block for the main thread
});                  // end of the g block
print("4");           // the handler carries on at once
```

- A. 1 4 2 3
- B. 1 2 4 3
- C. 1 2 3 4
- D. All three are possible

Answer: C. 1 2 3 4

The block that prints 3 sits on the main queue, which only the main thread serves. The main thread is busy running this handler, so 3 cannot print until the handler has printed 4 and returned.

12. To react to a timer or a signal with a dispatch source, the program must dedicate one of its own threads to waiting for the event.

Answer: False

The point of a dispatch source is that nobody waits: the system watches for the event and, when it happens, puts the handler block on the queue you chose, where a pool thread runs it. Events that arrive before the handler runs are merged into one run.

Chapter 5 · Concurrency: Mutual Exclusion and Synchronization

When threads share data, timing decides the result. Here is how to make them take turns safely.

When several processes or threads are in progress at the same time (concurrency) and share data, the result can depend on the exact order in which their instructions happen to interleave. This chapter first tries to make two processes take turns using nothing but ordinary reads and writes, then steps back to see how timing produces a race condition and what any correct solution must guarantee. It then builds stronger tools for mutual exclusion and synchronization: special hardware instructions, semaphores, monitors and message passing, ending with the classic readers/writers problem.

Learning objectives

- Explain race conditions, the OS concerns they raise and the requirements for mutual exclusion.
- Trace software solutions to mutual exclusion, including Dekker's and Peterson's algorithms.
- Explain hardware support: disabling interrupts and special atomic instructions.
- Use semaphores, monitors and message passing to synchronize processes.
- Solve the producer/consumer and readers/writers problems.

How Chapter 5 fits together

Chapters 3 and 4 gave us many processes and threads in progress at once. This chapter deals with the price: when they share data, the result can depend on timing. Concurrency arises in three contexts: **multiple applications** sharing a machine, **structured applications** built as cooperating parts, and the **operating system's own structure**. It appears whether processes are interleaved on one processor (**multiprogramming**), overlapped on several (**multiprocessing**) or spread across machines (**distributed processing**).

The vocabulary

A **race condition** happens when the outcome depends on who gets to shared data first. The code that touches shared data is a **critical section**, and **mutual exclusion** means at most one process is inside it at a time. **Atomic operations** are the indivisible building blocks used to enforce it. A careless solution can cause **deadlock** (everyone waits for everyone), **livelock** (everyone stays busy but nobody progresses) or **starvation** (one process is passed over forever).

Getting it right in software (5.1) and why it is hard (5.2)

Section 5.1 builds mutual exclusion from ordinary instructions through a series of flawed attempts, ending with **Dekker's** and **Peterson's** algorithms. Section 5.2 steps back to the principles: a simple example of a race, the OS concerns, the three ways processes interact (competition, cooperation by sharing, cooperation by communication) and the six requirements any mutual exclusion solution must meet.

Help from the hardware (5.3)

On one processor, disabling interrupts protects a critical section. On multiprocessors, special atomic instructions such as **compare_and_swap** and **exchange** do the job, at the cost of busy waiting and a risk of starvation.

Higher-level tools (5.4 to 5.6)

Semaphores (5.4) are counters with atomic **semWait** and **semSignal** operations that block instead of spinning; they solve mutual exclusion and the **producer/consumer** problem. **Monitors** (5.5) package shared data with its procedures and condition variables so mutual exclusion is automatic. **Message passing** (5.6) replaces shared data with send and receive, and works across machines.

A classic test (5.7)

The **readers/writers** problem lets many readers share data at once but gives writers exclusive access, and shows how the choice of priority can starve one side.

5.1 Mutual Exclusion: Software Approaches

You should be able to

- Describe the setting for software-only mutual exclusion: two processes, shared variables, a memory arbiter, and no special hardware or OS help.
- Trace each of the four flawed attempts and find the interleaving that breaks it: strict alternation, violated mutual exclusion, deadlock, livelock.
- Explain how Dekker's algorithm uses a turn variable to settle ties, and why it is correct.
- Explain why Peterson's algorithm guarantees mutual exclusion with no deadlock and no starvation, and how its idea extends to n processes.
- Estimate the cost of busy waiting, and explain why these algorithms break on hardware that reorders memory operations unless a memory barrier is added.

The setting: only reads and writes

Two processes, **P0** and **P1**, share memory. Each loops forever through four parts: an **entry protocol** (asks for permission), its **critical section** (uses the shared data), an **exit protocol** (announces it has left) and its **remainder section** (everything else). The goal is **mutual exclusion**: never both inside their critical sections at once. Without it, overlapping updates can corrupt shared data, a **race condition**.

- **Allowed**: ordinary reads and writes. A **memory arbiter** lets one access at a time touch a memory location, so each single read or write is **atomic** (two writes to **turn** land one after the other, in an unpredictable order, never blended). We also assume each process's reads and writes reach memory in program order.
- **Not allowed**: special instructions (Section 5.3) or OS help. A waiting process can only loop and re-test a variable: **busy waiting** (spinning).
- **No speed assumptions**: a process may be paused after any step. One order of steps is an **interleaving**; a solution must work for all of them.

We want mutual exclusion with no **deadlock** (all frozen for good), no **livelock** (all busy, none progressing), no **starvation**, no **strict alternation**, and survival of a halt in the remainder section. Section 5.2 lists the full requirements.

Attempt 1: one turn variable

```
int turn = 0;
while (turn != i) ;    /* wait for my turn */
/* critical section */
turn = j;              /* hand the turn over */
```

Mutual exclusion holds: only the process named by `turn` can leave its loop. **Flaw 1, strict alternation:** visits must go P0, P1, P0, P1, so the pace is set by the slower process. Example: P0 needs 1 tick of other work, P1 needs 5. Over 20 ticks P0 enters only 4 times and spins for 12 ticks, although on its own it could enter 10 times. **Flaw 2:** if P1 halts, even in its remainder section, P0 gets one more visit, sets `turn = 1`, and then waits forever.

Attempt 2: look, then raise your flag

```
boolean flag[2] = {false, false};
while (flag[j]) ;    /* wait while the other is inside */
flag[i] = true;      /* announce, then go in */
/* critical section */
flag[i] = false;
```

An uninterested or halted process keeps its flag down and blocks no one. But **mutual exclusion fails:** P0 tests (false), P1 tests (false), P0 raises its flag and enters, P1 does the same. Looking and announcing are separate steps.

Attempt 3: raise your flag, then look

```
flag[i] = true;      /* announce first */
while (flag[j]) ;    /* then wait if needed */
/* critical section */
flag[i] = false;
```

Mutual exclusion holds: each raises its flag before looking, so of two contenders the one that looks second sees the other's flag and waits. But **deadlock** is possible: both raise their flags, then each waits forever for the other's to drop.

Attempt 4: flag, polite back-off, retry

```
flag[i] = true;
while (flag[j]) {
    flag[i] = false;      /* step back      */
    /* delay a moment */
    flag[i] = true;      /* then ask again   */
}
/* critical section */
flag[i] = false;
```

Safe and deadlock-free, but **livelock** is possible: in perfect step both raise, both see the other's flag, both lower, pause and raise again, forever. Not deadlock: any change in relative speed lets one process find the other's flag down and enter.

Dekker's algorithm: flags plus a tiebreaker

```
boolean flag[2] = {false, false};
int turn = 1;
flag[i] = true;
while (flag[j]) {          /* both want in?    */
    if (turn == j) {       /* tie is the other's */
        flag[i] = false;  /* step back      */
        while (turn == j) ; /* wait for the turn */
        flag[i] = true;   /* ask again      */
    }
}
/* critical section */
turn = j;              /* next tie: other   */
flag[i] = false;
```

The first known correct software-only solution for two processes. Flags keep it safe; the **turn variable** settles ties. With `turn = 1` : both raise flags; P0 finds `turn == 1` , lowers its flag and waits on `turn` ; P1 keeps checking, sees P0's flag down and enters. On exit P1 sets `turn = 0` and lowers its flag, so P0 raises its flag again and enters.

- **Mutual exclusion:** a process enters only after seeing the other's flag down while its own is up.
- **No deadlock:** with both flags up, `turn` is 0 or 1, so exactly one process backs off.
- **No livelock:** only the process without the turn backs off, and it waits for the turn rather than retrying blindly.
- **No starvation:** each exit hands the next tie to the other process.
- `turn` is not "who is inside": if the other process is not interested, you enter at once whatever `turn` says.

Peterson's algorithm

```
boolean flag[2] = {false, false};
int turn = 0;
flag[i] = true;           /* I want in          */
turn = j;                 /* but you go first  */
while (flag[j] && turn == j) ; /* you want in AND it is your turn */
/* critical section */
flag[i] = false;
```

The later writer of `turn` waits. Each process writes the *other's* number, and `turn` holds one value, so it ends up naming the rival of whoever wrote last. Example: P0 writes `turn = 1`, then P1 writes `turn = 0`: P0's test `flag[1] && turn == 1` is false, so P0 enters; P1's test is true, so P1 waits. This holds in all six orders of the four entry writes.

- **Mutual exclusion:** if both want in, the later writer of `turn` finds the other's flag already up (each process raises its flag before writing `turn`) and `turn` naming the other, so it waits until the other lowers its flag.
- **No deadlock:** both waiting would need `turn == 0` and `turn == 1` at once.
- **No starvation:** a winner that rushes back sets `turn` to the waiter's number, so it waits itself; nobody waits more than one turn.
- Simpler than Dekker's and easier to prove.

Scorecard

Algorithm	Mutual exclusion	No deadlock	No livelock	No forced alternation	Survives halt in remainder	No busy waiting
Attempt 1 (turn only)	✓	✓	✓	×	×	×
Attempt 2 (look, then flag)	×	✓	✓	✓	✓	×
Attempt 3 (flag, then look)	✓	×	✓	✓	✓	×
Attempt 4 (flag, back off)	✓	✓	×	✓	✓	×
Dekker	✓	✓	✓	✓	✓	×
Peterson	✓	✓	✓	✓	✓	×

Going deeper (optional): the filter algorithm, Peterson for n processes

```
for (L = 1; L < n; L++) {      /* climb levels 1 .. n-1 */
    level[i] = L;              /* like flag                */
    victim[L] = i;             /* like turn: I came last */
    while (victim[L] == i && some other Pk has level[k] >= L) ;
}
/* critical section */
level[i] = 0;
```

Each level is a Peterson-style contest: the **latest arrival** (`victim[L]`) waits while anyone else is at that level or above; a newer arrival releases it. So at most $n - L$ processes get past level L . Example with $n = 5$: at most 2 get past level 3, and at most 1 past level 4, the last door before the critical section. With $n = 2$ it is exactly Peterson's algorithm. It is free of deadlock and starvation, but a process may wait at every level.

Going deeper (optional): costs and hidden assumptions

- **Busy waiting wastes CPU time:** tests wasted = time spinning $\div$ time per test. On **two CPUs** the spinner waits only until the holder leaves: $3\ \mu\text{s}$ in the critical section at $6\ \text{ns}$ per test = $3,000 \div 6 = 500$ useless tests. On **one CPU** it is far worse: if the holder was switched out inside its critical section, it cannot run while the spinner spins, so the spinner burns its whole time slice ($10\ \text{ms}$ at $6\ \text{ns}$ per test ≈ 1.67 million tests). Short spins are acceptable on multiprocessors; longer waits should block (sleep), as semaphores do (Section 5.4).
- **In-order memory is assumed.** Real CPUs park writes in a **store buffer** and let later reads go first; compilers may also reorder. Then in Peterson both processes can write their flags into their buffers, read the other's flag from memory as false, and both enter. The fix is a **memory barrier** (fence) between the writes and the test, which forces earlier writes out before any later read, or atomic variables that include one.
- **A halt inside the critical section** (or with a flag up) blocks the other process in every algorithm here; no software rule can prevent that.

Key terms

Critical section

A stretch of code in which a process uses a shared resource, such as shared variables, that must not be used by another process at the same moment.

Mutual exclusion

The guarantee that while one process is inside its critical section, no other process is inside its critical section for the same shared resource.

Entry protocol

The code a process runs just before its critical section to get permission to go in.

Exit protocol

The code a process runs just after its critical section to announce that it has left, so another process may enter.

Remainder section

Everything else a process does: code that does not touch the shared resource and needs no permission.

Memory arbiter

Hardware that lets only one access touch a given memory location at a time, so two reads or writes of the same word happen one after the other, never blended together.

Atomic

Describes an operation that happens as one indivisible step: no other process can see it half-done or slip in between its parts.

Interleaving

Running several processes on one processor by switching between them, so their individual steps take turns. Each possible order of those steps is called an interleaving, and a correct solution must work for every one.

Race condition

A bug in which the result depends on the exact timing or order in which processes access shared data.

Busy waiting

Waiting by testing a condition over and over in a loop. The waiting process keeps using the CPU while doing no useful work. Also called spinning.

Turn variable

A shared variable that holds the number of the process that has priority, that is, whose turn it is.

Flag variable

A shared true/false variable that a process sets to announce that it wants to enter (or is inside) its critical section.

Strict alternation

A rule that forces processes into their critical sections in a fixed order, P0, P1, P0, P1, and so on, even when one of them does not need to go in.

Deadlock

A situation in which each process in a group waits for something that only another waiting process can do, so none of them can ever continue.

Livelock

A situation in which processes keep running and reacting to each other, yet none makes progress. Unlike deadlock, a change in their relative timing can end it.

Starvation

A situation in which a process that wants to enter waits indefinitely because others are always allowed to go ahead of it.

Dekker's algorithm

The first known correct software-only solution to mutual exclusion for two processes. It combines one flag per process with a turn variable that decides who backs off when both want in.

Peterson's algorithm

A shorter correct two-process solution: each process raises its flag, then gives the turn to the other, and waits only while the other wants in and holds the turn.

Filter algorithm

Peterson's idea extended to n processes. Each process climbs $n - 1$ waiting levels; at each level the latest arrival waits while any other process is at that level or higher, so at most one process gets past the top level.

Memory barrier

An instruction that makes a processor finish all of its earlier memory reads and writes, so other processors can see them, before it performs any later one. Also called a fence.

Review questions with answers

1. Attempt 1 uses only `int turn`. P0 has just left its critical section (setting `turn = 1`) and wants to go straight back in, while P1 is busy in a long remainder section. What happens?

- A. P0 re-enters at once, because the critical section is empty
- B. P0 must wait until P1 enters and leaves its critical section
- C. P0 and P1 both end up inside their critical sections
- D. P0 and P1 deadlock

Answer: B. P0 must wait until P1 enters and leaves its critical section

Attempt 1 forces strict alternation. After P0's visit only P1 can hand the turn back, so the fast process is held to the slow one's pace.

2. Attempt 2 runs `while (flag[j]) ; then flag[i] = true;` then the critical section. Which interleaving puts both processes inside at once?

- A. P0 tests, P1 tests, P0 raises its flag, P1 raises its flag
- B. P0 tests, P0 raises its flag, P1 tests, P1 raises its flag
- C. P1 tests, P1 raises its flag, P0 tests, P0 raises its flag
- D. No interleaving can do it

Answer: A. P0 tests, P1 tests, P0 raises its flag, P1 raises its flag

If both test before either raises its flag, both see `false` and both walk in.

3. **Going deeper (optional):** on a multiprocessor whose CPUs let a later read go ahead of an earlier write that is still waiting in a store buffer, Peterson's algorithm with no memory barrier can let both processes into their critical sections at once.

Answer: True

Each process's write of its own flag can still sit in its store buffer when it reads the other's flag from memory, so both read false and both enter. A memory barrier between the writes and the test forces the writes out to memory first.

4. **Going deeper (optional):** five processes use Peterson's idea extended to n processes (the filter algorithm, with levels 1 to 4). At most how many of them can be past level 3 at the same moment?

Answer: 2 processes

At most $n - L$ processes get past level L , so $5 - 3 = 2$. Past the top level, 4, at most $5 - 4 = 1$ gets through: that one is in its critical section.

5. Match each attempt with its flaw.

Attempt 1 (turn only)	Forces strict alternation
Attempt 2 (look, then flag)	Both can be inside at once
Attempt 3 (flag, then look)	Can deadlock with both flags up
Attempt 4 (flag, then back off)	Can livelock in perfect step

Each fix exposed a new problem: alternation, then broken mutual exclusion, then deadlock, then livelock. Dekker's tiebreaker cures all four.

6. Put the parts of Peterson's algorithm for process P_i in order.

Correct order:

1. `flag[i] = true;`
2. `turn = j;`
3. `while (flag[j] && turn == j) ;`
4. critical section
5. `flag[i] = false;`
6. remainder section

Announce interest, give the turn away, wait only while the other wants in and holds the turn, use the resource, withdraw interest, then do other work.

7. Both processes run Peterson's entry code at about the same time, and both flags are up. P_0 executes `turn = 1` first; P_1 executes `turn = 0` a moment later. Which process enters its critical section first?

- A. P_0
- B. P_1
- C. Neither: they deadlock
- D. Both at once

Answer: A. P_0

The later writer of `turn` waits. P_1 wrote last, leaving `turn = 0`, so P_0 's test `flag[1] && turn == 1` is false and P_0 goes in.

8. Which statements about Dekker's algorithm are true?

- A. It uses a flag for each process plus a turn variable
- B. When both flags are up, `turn` decides which process backs off
- C. It needs a special test-and-set instruction
- D. A process that must wait still busy-waits
- E. The process that backs off just pauses and retries, as in Attempt 4

Answer: A, B, D

Dekker combines flags with a turn variable as a tiebreaker and needs only ordinary reads and writes. The process that backs off waits for the turn rather than retrying blindly, but waiting is still a busy loop.

9. Which failure does each scenario describe?

Both inside	Deadlock	Livelock
Both pass their checks, then both flag	Both flags up; each waits for the other No schedule can ever let anyone in	Flags go down and up in step, forever A small speed change would end it

Both inside means mutual exclusion is violated. Deadlock means stuck with no possible progress. Livelock means busy but going nowhere, and a change in timing can break it.

10. P0 and P1 run on two different CPUs. P0 busy-waits on a flag while P1 spends 3 more microseconds in its critical section. One test of the flag takes 6 nanoseconds. About how many times does P0 test the flag before P1 leaves?

Answer: 500 tests

$3\ \mu\text{s} = 3,000\ \text{ns}$, and $3,000\ \text{ns} \div 6\ \text{ns per test} = 500\ \text{tests}$, every one of them wasted CPU work. On a single CPU it could be far worse: the spinner would burn its whole time slice while P1 could not run at all.

11. Apart from memory operations happening in program order, what must the hardware guarantee for the software algorithms in this section to work?

- A. Accesses to the same memory location happen one at a time
- B. An instruction that reads and writes a variable in one indivisible step
- C. The OS can put a waiting process to sleep
- D. Both processes run at the same speed

Answer: A. Accesses to the same memory location happen one at a time

The memory arbiter serializes accesses to one location, so each single read or write is atomic. Everything else is built from that, plus the assumption that reads and writes happen in program order.

12. With Attempt 1, a process that halts in its remainder section (outside its critical section) can leave the other process blocked forever.

Answer: True

The halted process never hands the turn back, so once the survivor gives the turn away it waits forever. With flags (Attempts 2 to 4, Dekker, Peterson), a process halted in its remainder section leaves its flag down and blocks no one.

5.2 Principles of Concurrency

You should be able to

- Explain why the relative speed of concurrent processes cannot be predicted, and the three difficulties this creates.
- Trace the echo example and other race conditions, and work out every final value an interleaving can produce.
- List the four concerns concurrency raises for the OS, and classify process interaction by degree of awareness.
- Recognise mutual exclusion, deadlock, starvation and data-coherence problems in competing and cooperating processes.
- State the six requirements for mutual exclusion and identify which ones a proposed mechanism breaks.

Why concurrency is hard

On one processor the OS **interleaves** processes (a slice of one, then another). On a multiprocessor they are interleaved and also **overlapped** (truly running at the same instant). Both raise the same problems, because a process's **relative speed** cannot be predicted: it depends on what other processes do, how the OS handles interrupts, and the OS's scheduling policy. Three difficulties follow:

1. **Sharing global resources is risky:** if two processes read and write the same global variable, the order of their steps decides the result.
2. **Optimal allocation is hard:** if P1 is granted an I/O channel and suspended before using it, locking the channel for P1 leaves it idle while others wait (and can lead to deadlock).
3. **Errors are hard to locate:** results are nondeterministic, so a failure may not repeat when you rerun the program.

The echo example

The assumption that makes the race possible: to save memory, the OS loads echo **and its global variables** once, into a region of memory shared by every application, so all callers use the same chin and chout. (The same holds if the callers are threads of one process.) Sharing code alone would not be enough: separate processes running the same program normally get private copies of its globals.

```
char chin, chout;    // in shared memory: one copy for all callers
void echo() {        // any process may call it
    chin = getchar(); // 1. read a key into chin
    chout = chin;     // 2. copy it into chout
    putchar(chout);   // 3. display chout
}                    // return to the caller
```

One processor: P1 reads *x* into chin and is interrupted. P2 runs echo completely, reading and displaying *y*. P1 resumes, copies chin (now *y*) and displays *y*: *x* is lost, *y* appears twice. **Two processors:** no interrupt is needed. P1 reads *x*, P2 reads *y*, both copy chin (*y*) into chout, and both display *y*. **Fix:** let only one process

be inside echo at a time; a second caller is blocked at the entrance until the first returns. The cause is shared data, not the number of processors.

Race conditions

A **race condition** occurs when several processes or threads read and write shared data and the final result depends on the relative timing of their execution. When writes collide, the “loser” of the race (the last writer) decides the value.

- Shared `a` : P1 runs `a = 1` , P2 runs `a = 2` . Whichever runs last leaves its value.
- Shared `b = 1, c = 2`: P3 runs `b = b + c` , P4 runs `c = b + c` . P3 then P4: `b = 3, c = 5`. P4 then P3: `b = 4, c = 3`. If both load `b` and `c` before either stores its sum, `b = 3` and `c = 3`.

A correct program must give the right answer for **every** possible interleaving.

Four OS concerns

1. **Keep track of the processes**, using a process control block for each (state, priority, resources held, what it waits for).
2. **Allocate and deallocate resources**: processor time, memory, files and I/O devices.
3. **Protect** each process’s data and physical resources against unintended interference by others (address spaces, file permissions, access to devices only through the OS).
4. **Make results independent of speed**: a process’s function and output must not depend on how fast it runs relative to other processes. This is the subject of the rest of the chapter.

How processes interact: degree of awareness

Degree of awareness	Relationship	Influence on each other	Control problems
Unaware of each other	Competition	Results independent of the others; timing may suffer	Mutual exclusion, deadlock (renewable resource), starvation
Indirectly aware (shared object)	Cooperation by sharing	Results may depend on information from the others; timing may suffer	Mutual exclusion, deadlock (renewable resource), starvation, data coherence
Directly aware (named, messages)	Cooperation by communication	Results may depend on information from the others; timing may suffer	Deadlock (consumable resource), starvation

A **renewable** (reusable) resource, such as a processor, memory or an I/O channel, is released and reused.

A **consumable** resource, such as a message, is destroyed when a process receives it.

Competition among processes

A **critical resource** can be used by only one process at a time (a printer). The code that uses it is the process’s **critical section**. **Mutual exclusion**: at most one process at a time is in its critical section for that resource. Competing processes exchange no information, but one may be delayed while another uses the resource.

```

void P(int i) {           // same shape for every process
    while (true) {        // loop forever
        entercritical(R); // wait here while R is busy
        use(R);           // critical section
        exitcritical(R);  // free R, let one waiter in
        other_work();      // remainder: needs no R
    }                     // around again
}                         // end of P(i)

```

A process that finds R busy waits in `entercritical`; `exitcritical` lets a waiter in. Without them, two processes can print at once and their pages mix. Enforcing mutual exclusion creates two new problems:

- **Deadlock:** P1 and P2 both need R1 and R2. P1 holds R1 and waits for R2; P2 holds R2 and waits for R1. Neither can ever continue, though each program is fine alone. Asking for resources in the same order prevents this circle.
- **Starvation:** P1, P2 and P3 all want R repeatedly. If the OS keeps handing R to P1 and P3 in turn, P2 is passed over indefinitely even though there is no deadlock and work continues. A fair policy such as first come, first served avoids it.

Cooperation by sharing

Processes share variables, files or databases without knowing each other's identity. Many may read at once, but a writer needs exclusive access, so mutual exclusion, deadlock and starvation apply. The new demand is **data coherence**: every required relationship (an **invariant**) must still hold after concurrent updates.

Example: $a = b = 1$ and the rule $a = b$. P1 runs `a = a + 1; b = b + 1;` and P2 runs `b = 2 * b; a = 2 * a;`. Each keeps $a = b$ when run alone (P1 then P2 gives 4, 4; P2 then P1 gives 3, 3). Every one of the four mixed orders, such as $a = a + 1, b = 2 * b, b = b + 1, a = 2 * a$, gives **$a = 4, b = 3$** . Protecting each variable separately is not enough: the **whole sequence** of updates in each process must be one critical section.

Cooperation by communication

Processes know each other by name and exchange messages using **communication primitives** (send, receive) supplied by the OS or the language. Nothing is shared, so mutual exclusion is not a control requirement. **Deadlock** is still possible (P1 waits to receive from P2 while P2 waits to receive from P1), and so is **starvation** (P2 and P3 both want to talk to P1, but P1 keeps exchanging messages with P2, so P3 is never served).

Six requirements for mutual exclusion

1. Mutual exclusion must be enforced: among all processes with critical sections for the same resource or shared object, only one at a time may be inside.
2. A process that halts in its non-critical section must do so without interfering with other processes.
3. A process requesting entry must not be delayed indefinitely: no deadlock and no starvation.
4. When no process is in a critical section, a process that requests entry must be admitted without delay.
5. No assumptions are made about relative process speeds or the number of processors.

6. A process remains inside its critical section for a finite time only.

Key terms

Relative speed

How fast one process runs compared with the others. It changes from run to run because it depends on other processes, interrupts and the scheduler.

Nondeterministic

Describes a program whose result can differ from one run to the next, even with identical input, because the timing of its steps differs. Its bugs are hard to reproduce.

Global variable

A variable declared outside every procedure, so all code in the program uses the same copy. Threads of one process share it; separate processes each get their own copy unless it is placed in memory they share.

I/O channel

A piece of I/O hardware, in effect a small special-purpose processor, that moves data between main memory and devices with little help from the CPU. The OS can grant a process the use of a channel for its I/O.

Race condition

Several processes or threads read and write shared data, and the final result depends on the relative timing of their steps. When two writes collide, the last one to write decides the value.

Critical resource

A resource that cannot be shared: only one process may use it at a time. A printer is the classic example.

Critical section

The part of a program that uses a critical resource or shared data. At most one process may be inside its critical section for a given resource at a time.

Mutual exclusion

The guarantee that while one process is inside a critical section for a resource, no other process is inside a critical section for that same resource.

Deadlock

Each process in a group waits for something that only another member of the group can give, so none of them can ever continue.

Starvation

A process that is ready to go is passed over again and again, indefinitely, while other processes keep getting the resource it needs.

Degree of awareness

How much processes know about each other: not at all, indirectly through a shared object, or directly by name through messages.

Competition

The relationship between processes that are unaware of each other but need the same resources, so the OS must referee who gets what.

Cooperation by sharing

Processes that know about each other only indirectly, because they read and write the same shared object, such as a variable, file or buffer.

Cooperation by communication

Processes that know each other by name and work together by sending and receiving messages, instead of sharing memory.

Communication primitive

A basic operation, such as send or receive, provided by the OS or programming language so that processes can exchange messages.

Data coherence

Keeping shared data consistent: every relationship that is supposed to hold between data items, such as a being equal to b , still holds after concurrent updates.

Invariant

A condition that must be true whenever no update is in progress, for example $a = b$. An update may break it briefly but must restore it before it finishes.

Renewable resource

A resource that is not used up: when one process releases it, another can use it. Processors, memory, files and I/O channels are examples. Also called a reusable resource.

Consumable resource

A resource that is created and then destroyed when a process takes it, such as a message or a signal. Once received, it is gone.

Review questions with answers

1. The globals `chin` and `chout` sit in memory shared by two processes, which call the echo procedure at the same time, each on **its own processor**. The steps happen in this order: P1 reads x into `chin`; P2 reads y into `chin`; P1 copies `chin` into `chout`; P2 copies `chin` into `chout`; P1 displays `chout`; P2 displays `chout`. What appears on the screen?

- A. x , then y
- B. y , then y
- C. x , then x
- D. y , then x

Answer: B. y , then y

A second processor does not help. P2 overwrote the shared `chin` with y before P1 copied it, so both processes put y into `chout` and both displayed y . The x was lost exactly as on one processor, but no interrupt was needed.

2. Put the classic echo failure (one processor; chin and chout in memory both processes share) in the order it happens.

Correct order:

1. P1 reads x into chin
2. P1 is interrupted before copying chin
3. P2 runs echo completely and displays y
4. P1 resumes and copies chin (now y) into chout
5. P1 displays y; the x is lost

The damage happens in the gap between P1 reading chin and P1 copying it: P2 overwrote the shared variable in that gap.

3. In a race condition, the process that writes to the shared variable first decides its final value.

Answer: False

The first value is simply overwritten. The last process to write (the loser of the race) decides the final value.

4. Shared variables start at $b = 1$ and $c = 2$. P3 executes $b = b + c;$ and P4 executes $c = b + c;$. If P4 runs completely and then P3 runs, what is the final value of b?

Answer: 4

P4 first: $c = 1 + 2 = 3$. Then P3: $b = 1 + 3 = 4$. (In the opposite order you get $b = 3$, $c = 5$.)

5. Which of these are concerns the operating system must handle because processes run concurrently?

- A. Keep track of every process
- B. Allocate and deallocate resources
- C. Protect each process's data and resources from the others
- D. Make a process's results independent of its relative speed
- E. Make every process run at the same speed
- F. Remove all resources that processes share

Answer: A, B, C, D

The four concerns are tracking, allocation, protection and speed-independent results. The OS cannot equalise speeds, and sharing resources is unavoidable.

6. Match each degree of awareness to the relationship it produces.

Unaware of each other	Competition
Indirectly aware, through a shared object	Cooperation by sharing
Directly aware, by name	Cooperation by communication

Unaware processes only compete for resources. Processes linked by shared data cooperate by sharing. Processes that name each other and exchange messages cooperate by communication.

7. Sort each situation by how the processes interact.

Competition	Cooperation by sharing	Cooperation by communication
Two unrelated batch jobs both need the one tape drive Programs from two users both print on the office printer	Threads of a server all update one shared counter Several processes update records in one shared file	A client sends a request message to a named server Pipeline stages pass records with send and receive

No shared data and no messages means competition. A shared variable or file means cooperation by sharing. Named processes exchanging messages means cooperation by communication.

8. The part of a program that uses a resource which only one process may use at a time is called the...

- A. critical section
- B. critical resource
- C. remainder section
- D. global variable

Answer: A. critical section

The resource is the critical resource; the code that uses it is the critical section. Mutual exclusion allows at most one process in its critical section for that resource.

9. Which statements about deadlock and starvation are true?

- A. In deadlock, each process in the group waits for something only another member of the group can provide
- B. During starvation the rest of the system can keep doing useful work
- C. Enforcing mutual exclusion can lead to either problem
- D. Starvation requires a circle of processes waiting for each other

Answer: A, B, C

Deadlock is a closed circle of waiting, so nobody in it moves. Starvation needs no circle: the system runs, but one process is passed over indefinitely. Both can arise from how mutual exclusion is enforced.

10. Shared $a = b = 1$ and the rule $a = b$ must hold. P1 runs $a = a + 1; b = b + 1;$ and P2 runs $b = 2 * b; a = 2 * a;$. Each process keeps its own two statements in order. Of the six possible interleavings of the four statements, how many finish with $a \neq b$?

Answer: 4

Only the two orders where one process runs completely first keep $a = b$ (giving 4, 4 or 3, 3). All four mixed orders finish with $a = 4, b = 3$.

11. A proposed lock is correct only if the process holding it always runs faster than the processes waiting for it. Which requirement for mutual exclusion does it break?

- A. Only one process may be in its critical section at a time
- B. No assumptions may be made about relative speeds or the number of processors
- C. A process must leave its critical section after a finite time
- D. A process asking to enter a free critical section must get in without delay

Answer: B. No assumptions may be made about relative speeds or the number of processors

Relative speeds depend on the scheduler, interrupts and other processes, so they cannot be predicted. A correct scheme must work at any speed.

12. Processes that cooperate only by sending and receiving messages share no data. Which control problem therefore does **not** arise for them?

- A. Mutual exclusion
- B. Deadlock
- C. Starvation
- D. All three still arise

Answer: A. Mutual exclusion

With nothing shared there is nothing to lock. Deadlock (both waiting in receive) and starvation (one process never answered) still can occur. Messages can also serve as a tool: in Section 5.6 a single token message enforces mutual exclusion on a resource that is shared.

5.3 Mutual Exclusion: Hardware Support

You should be able to

- Explain why disabling interrupts guarantees mutual exclusion on a uniprocessor, and name its two costs.
- Trace `compare_and_swap` and `exchange` step by step, and use each one to build a spinlock.
- Explain why the read and the write of a lock must happen as one atomic instruction.
- State and check the exchange invariant: `bolt` plus the sum of all keys equals `n`.
- List the advantages and disadvantages of the machine-instruction approach, including busy waiting, starvation and the priority deadlock.

5.3 Mutual Exclusion: Hardware Support

Every lock comes down to two moves on a shared **lock variable** (here `bolt` : 0 = free, 1 = taken): **look** (is it 0?) and **lock** (set it to 1). If another process reaches `bolt` in the gap between them, it also sees 0 and both enter their critical sections. The software approaches of 5.1 cope using ordinary reads and writes, but the code is subtle. Hardware helps in two ways: (1) switch off interrupts so nothing else can run (one processor only), or (2) special machine instructions that look and lock in one indivisible step, so there is no gap.

Route 1: interrupt disabling (uniprocessor)

On a **uniprocessor**, processes never truly overlap; they only **interleave**. A running process keeps the processor until it either calls an OS service or an **interrupt** arrives and the OS decides to switch. So a process that switches interrupts off before its critical section, and makes no OS calls inside it, cannot be switched out. No other process can run, so none can enter, and no lock variable is needed.

```
while (true) {
    disable interrupts; /* no switch can happen from here on */
    /* critical section */ /* use the shared data */
    enable interrupts; /* switching allowed again */
    /* remainder */ /* other work */
}
```

Interrupts that arrive while they are off are not lost: the hardware keeps them **pending** and delivers them as soon as interrupts are switched back on.

Two costs.

- **Efficiency.** While interrupts are off, the processor cannot interleave other processes or serve devices; a finished disk read must wait. Worked example: interrupts go off at $t = 10\ \mu\text{s}$ for a $20\ \mu\text{s}$ critical section (until $30\ \mu\text{s}$); a device interrupt arriving at $18\ \mu\text{s}$ waits $30 - 18 = 12\ \mu\text{s}$. Longer critical sections delay more events, for longer.
- **It fails on a multiprocessor.** Switching interrupts off affects only the processor that does it. Processes on other processors are already running and can reach the shared data at the same instant, so mutual exclusion is not guaranteed.

Switching interrupts off is a **privileged instruction** (kernel only), and even the kernel keeps such stretches very short.

Route 2: special machine instructions

The hardware fact underneath: memory serves one access to a given location at a time; other accesses to the same location wait. Processor designers built instructions that perform **two actions on one word** (such as a read and a write, or a read and a test) within a single instruction cycle. Because they cannot be interrupted part-way and no other access to that word can come between the two actions, they are **atomic**.

Common mistake: `if (bolt == 0) bolt = 1;` (or `while (bolt == 1) ; bolt = 1;`) looks like one action but is two separate memory accesses, a read and then a write. If P1 reads 0, then P2 reads 0, then both write 1, both are inside. An atomic instruction closes exactly this gap.

compare_and_swap (compare and exchange)

```
int compare_and_swap(int *word, int testval, int newval) {
    int oldval;
    oldval = *word; /* 1. read the word */
    if (oldval == testval) /* 2. compare with the expected value */
        *word = newval; /* 3. equal? write the new value */
    return oldval; /* 4. always return the OLD value */
} /* all one atomic instruction */
```

It always returns the old value, whether or not it wrote. The caller compares the result with testval: equal means the write happened; anything else means memory was left untouched. Examples: word = 5, `compare_and_swap(&word, 5, 9)` returns 5 and leaves 9 in word. Word = 3, `compare_and_swap(&word, 0, 1)` returns 3 and leaves word at 3.

The spinlock:

```

int bolt = 0; /* shared: 0 free, 1 taken */
void P(int i) {
    while (true) {
        while (compare_and_swap(&bolt, 0, 1) == 1)
            ; /* got 1: taken, test again */
        /* critical section */ /* got 0: bolt is now 1, mine */
        bolt = 0; /* unlock */
        /* remainder */
    }
}

```

Only a process that finds `bolt = 0` leaves the loop, and finding it sets `bolt` to 1 in the same atomic step, so every other process sees 1 and keeps waiting. Getting 1 back does not mean success: it means someone else holds the lock.

The exchange instruction

```

void exchange(int *reg, int *mem) { /* a register, a memory word */
    int temp;
    temp = *mem; /* copy the memory word aside */
    *mem = *reg; /* register value goes to memory */
    *reg = temp; /* old memory value to register */
} /* all one atomic instruction */

```

Each process keeps a private **key** and swaps it with the shared `bolt` until it pulls out the 0:

```

int bolt = 0;
void P(int i) {
    while (true) {
        int keyi = 1; /* fresh key for each attempt */
        do exchange(&keyi, &bolt); /* swap my key with bolt, atomically */
        while (keyi != 0); /* no 0 pulled out? swap again */
        /* critical section */ /* I hold the only 0 */
        bolt = 0; /* put the 0 back: unlock */
        /* remainder */
    }
}

```

The invariant (a fact that stays true after every step). With n processes, **$\text{bolt} + \text{key}_1 + \dots + \text{key}_n = n$** .

Exchanges only move values around, so the sum never changes and there is exactly one 0 in the whole system. (A process outside the loop counts as key 1, the value its next attempt starts with; a waiter always holds 1.) If `bolt = 0`, no process is in its critical section. If `bolt = 1`, exactly one process is inside: the one whose key is 0. Worked example: 6 processes, one inside and five waiting, so `bolt = 1` and the keys add up to $6 - 1 = 5$. A bug that writes `bolt = 0` while the lock is held creates a second 0 (the sum drops), and then two processes can get in.

Busy waiting and starvation

Busy waiting (spin waiting): a process that finds the lock taken keeps executing the test in a loop, using processor time and doing no useful work. On a multiprocessor, each spinner wastes its own processor. Worked example: 4 processes on 4 processors; P1 holds the lock for 40 μ s while the other 3 spin the whole time: $3 \times 40 = 120 \mu$ s of processor time wasted. On a uniprocessor it is worse: the spinner uses up its turn while the holder, the only process that can release the lock, is not running. The waste grows with the number of waiters and the length of the critical section.

Starvation: when the lock is released, all spinners race and whichever instruction reaches memory first wins. The hardware keeps no queue and no record of who waited longest, so the choice is arbitrary and some process could lose indefinitely. With fewer rivals a process wins more often, but there is still no guarantee (a releaser that loops straight back can beat it again). Starvation differs from deadlock: the system keeps making progress; only the starved process is stuck.

Deadlock with priorities

On one processor with **priority scheduling** (always run the highest-priority Ready process): low-priority P_{low} takes the lock; high-priority P_{high} becomes Ready (say, its disk read finishes) and preempts P_{low} inside its critical section; P_{high} then tries the same lock and spins. A spinning process is still Ready, so at every timer interrupt the dispatcher picks P_{high} again. P_{high} waits for P_{low} to release bolt; P_{low} waits for P_{high} to release the processor: a **deadlock**. If the waiter slept (was Blocked) instead of spinning, P_{low} would run, unlock, and P_{high} would be woken. That is the idea behind semaphores (5.4).

Scorecard: machine-instruction approach

Advantages	Disadvantages
Works for any number of processes	Busy waiting: waiting processes consume processor time
Works on a uniprocessor or on multiple processors sharing main memory	Starvation is possible: the next process to enter is chosen arbitrarily
Simple, so easy to verify	Deadlock is possible: a high-priority spinner can shut out a low-priority holder
Supports multiple critical sections, each with its own lock variable	

The two routes compared: interrupt disabling works only on a uniprocessor, only in the kernel, and stalls everything else; atomic instructions work wherever memory is shared, at the price of busy waiting, possible starvation and the priority deadlock.

Key terms

Uniprocessor

A computer with exactly one processor. Its processes can take turns (interleave) but never run at the same instant.

Multiprocessor

A computer with two or more processors that share one main memory. Processes on different processors really do run at the same instant.

Interleaving

Running pieces of several processes one after another on a processor, switching between them, so they appear to run together.

Overlapping (true parallelism)

Two processes executing at literally the same moment, which is only possible when there is more than one processor.

Interrupt

A signal from hardware (a timer, a disk, a network card) that makes the processor pause the running program and jump to operating-system code. The OS may then switch to another process.

Interrupt disabling

Getting mutual exclusion on one processor by switching interrupts off just before a critical section and back on just after it, so nothing can take the processor away in between.

Pending interrupt

An interrupt that arrived while interrupts were switched off. The hardware remembers it and delivers it as soon as interrupts are switched back on.

Privileged instruction

An instruction that only the operating system kernel may execute. Switching interrupts off is one, so ordinary programs cannot use it.

Critical section

A stretch of code that uses a shared resource. While one process is inside it, no other process may be inside a critical section for that same resource.

Mutual exclusion

The guarantee that while one process is inside its critical section for a resource, no other process is inside a critical section for that same resource.

Atomic instruction

A machine instruction whose parts (for example, read a memory word and then write it) happen as one indivisible action. No other process or processor can get in between the parts.

compare_and_swap (compare and exchange)

An atomic instruction that reads a memory word, compares it with a test value, writes a new value only if the two match, and returns the old value. Often abbreviated CAS.

Exchange instruction

An atomic instruction that swaps the contents of a register and a memory word in one indivisible step.

Lock variable (bolt)

A shared memory word that says whether the lock is free (0) or taken (1). In this section it is called bolt.

Invariant

A condition on a program's variables that must be true whenever no update is in progress. In the exchange lock every update is one atomic swap, so the invariant holds after every step, which makes it a tool for proving the lock correct.

Busy waiting (spin waiting)

Waiting by running a loop that tests a condition again and again. The waiting process keeps using processor time while doing no useful work.

Spinlock

A lock where a process that finds it taken busy-waits, testing it over and over, until it becomes free.

Starvation

A situation in which a process that wants to enter waits indefinitely because others are always chosen ahead of it.

Deadlock

A permanent standstill: each process in a group waits for something that only another waiting process in the group can provide, so none can ever continue.

Priority scheduling

A dispatching rule that always gives the processor to the highest-priority process that is ready to run.

Review questions with answers

1. On a computer with a single processor, why does switching interrupts off before a critical section give mutual exclusion?

- A. Without interrupts the running process cannot be switched out, so no other process runs until it switches them back on
- B. It locks the shared variables in memory so that other processes cannot read them
- C. The hardware throws away every interrupt that arrives, so the operating system never runs
- D. It tells every other processor to pause until interrupts are switched back on

Answer: A. Without interrupts the running process cannot be switched out, so no other process runs until it switches them back on

On one processor, a process loses the processor only through an OS call or an interrupt. With interrupts off (and no OS calls inside), nothing can switch it out, so nothing else can run and get into the critical section.

2. Which statements about using interrupt disabling for mutual exclusion are true? Select all that apply.

- A. While interrupts are off, the processor cannot interleave other processes, so efficiency can suffer
- B. It does not work on a multiprocessor
- C. Device interrupts that arrive while interrupts are off are lost for good
- D. Switching interrupts off is a privileged instruction, available only to the kernel
- E. It forces processes on other processors to wait

Answer: A, B, D

The two costs are lost efficiency (no interleaving, devices wait) and failure on multiprocessors, since the other processors are not affected. Arriving interrupts are held as pending, not lost, and ordinary programs are not allowed to switch interrupts off.

3. The line `if (bolt == 0) bolt = 1;` is a safe lock, because it is only one line of code.

Answer: False

One line of source code can be several machine actions: here a read of bolt and then a separate write. Another process can run between them, see bolt = 0 too, and both get in. Only an atomic instruction closes that gap.

4. The shared word holds 5. A process executes `old = compare_and_swap(&word, 5, 9);`. What happens?

- A. old is 5, and word now holds 9
- B. old is 9, and word now holds 9
- C. old is 5, and word still holds 5
- D. old is 1 (meaning success), and word now holds 9

Answer: A. old is 5, and word now holds 9

`compare_and_swap` reads the word (5), compares it with testval (5), and because they match, writes newval (9). It always returns the old value, so `old = 5`.

5. The shared word holds 3. A process executes `compare_and_swap(&word, 0, 1)`. What value does the instruction return?

Answer: 3

The old value 3 does not equal testval 0, so nothing is written and the word stays 3. The instruction still returns the old value: 3.

6. In the `compare_and_swap` spinlock, a process that gets 1 back from `compare_and_swap` has just entered its critical section.

Answer: False

Getting 1 back means the word already held 1: someone else holds the lock, and nothing was written. The process loops and tests again. It enters only when it gets 0 back, which means it just changed bolt from 0 to 1 itself.

7. Put the actions of a single `compare_and_swap(word, testval, newval)` instruction in order.

Correct order:

1. Read the memory word into oldval
2. Compare oldval with testval
3. If they are equal, write newval into the word
4. Return oldval to the caller

Read, compare, conditionally write, return the old value. All four happen inside one atomic instruction, so no other access to the word can come between them.

8. Six processes share a lock built with the exchange instruction (bolt starts at 0, and each process starts every attempt with its private key set to 1). Right now one process is inside its critical section and the other five are waiting in their exchange loops. What is $key_1 + key_2 + \dots + key_6$?

Answer: 5

The invariant says $bolt + \sum key_i = n = 6$. With a process inside, $bolt = 1$, so the keys add to 5: the holder's key is 0, and each waiter keeps swapping its 1 for bolt's 1, so the other five keys are 1.

9. Match each idea with its description.

Interrupt disabling	Prevents switches on one processor
compare_and_swap	Conditional write; returns old value
exchange	Swaps a register with a memory word
Busy waiting	Looping on a test, wasting CPU time
Starvation	A waiter passed over indefinitely

Interrupt disabling blocks switches on one processor. compare_and_swap and exchange are the two atomic instructions. Busy waiting and starvation are two of the costs of spinlocks built from them.

10. Four processes each run on their own processor of a shared-memory multiprocessor. P1 holds a spinlock for 40 μ s, and during that entire time the other three processes spin on it. How much processor time is spent spinning, in microseconds?

Answer: 120 μ s

Three processors each spin for the full 40 μ s: $3 \times 40 = 120 \mu$ s of processor time that does no useful work. That waste is the cost of busy waiting.

11. One processor, priority scheduling. Low-priority P1 holds a compare_and_swap lock and is preempted inside its critical section by high-priority P2, which then spins on the same lock. What happens?

- A. P2 spins forever and P1 never runs again: a deadlock
- B. A timer interrupt soon gives P1 a turn, so P2 only waits a little longer
- C. compare_and_swap notices that the holder is paused and hands the lock to P2
- D. Both get into their critical sections, so mutual exclusion is broken

Answer: A. P2 spins forever and P1 never runs again: a deadlock

P2 needs P1 to release bolt, and P1 needs P2 to give up the processor. Spinning keeps P2 Ready, so under priority scheduling P1 never runs. Each waits for the other forever. P2 keeps running, yet this is deadlock, not livelock (5.1): no change in timing can end it.

12. Machine-instruction locks: advantage or disadvantage?

Advantage	Disadvantage
Works for any number of processes Works on a multiprocessor with shared memory Simple, so easy to verify Supports many critical sections, one variable each	Busy waiting uses processor time Starvation is possible Deadlock is possible under priority scheduling

Advantages: any number of processes, one or many processors sharing memory, simple to verify, many critical sections. Disadvantages: busy waiting, possible starvation (the next winner is arbitrary), possible deadlock (the priority scenario).

5.4 Semaphores

You should be able to

- Define a semaphore and trace exactly what `semWait` and `semSignal` do to its value and its queue.
- Read a semaphore's value, and tell counting semaphores, binary semaphores, mutexes, and strong versus weak semaphores apart.
- Use one semaphore to enforce mutual exclusion and others to pass signals between producers and consumers.
- Solve the producer/consumer problem for infinite and bounded buffers, and explain why the order of `semWait` calls matters.
- Explain why `semWait` and `semSignal` must be atomic and how `compare_and_swap` or disabling interrupts makes them so.

The core idea: stop here until you get a signal

Processes can cooperate using simple signals: a process can be forced to halt at a chosen point in its code until a particular signal arrives. The variable that carries signals is a **semaphore**: `semSignal(s)` sends one, `semWait(s)` receives one, and if no signal has been sent yet the receiver is suspended until it is.

The concurrency toolbox

Mechanism	What it is
Semaphore	Integer changed only by <code>initialize</code> , <code>semWait</code> , <code>semSignal</code> .
Binary semaphore	Semaphore holding only 0 or 1.
Mutex	Binary lock; the process that locks it (sets 0) must be the one to unlock it (set 1).
Condition variable	Queue a process joins to wait until some condition holds (5.5).
Monitor	Language construct bundling shared data and its procedures; one process active inside at a time.
Event flags	Memory word, one bit per event; wait for one, any or all flags.
Mailboxes / messages	Messages wait in a mailbox until received; waiting for one also synchronizes.
Spinlock	Waiter busy-waits in a loop until the lock is free.

Definition: one integer, three operations

1. **Initialize** the semaphore to a nonnegative integer.
2. **semWait(s)** decrements the value; if it becomes negative, the caller is blocked in the semaphore's queue, otherwise it continues.
3. **semSignal(s)** increments the value; if it is then ≤ 0 , one blocked process is moved to Ready. The caller never blocks.

```
semWait(s):  s.count--; if (s.count < 0) { add caller to s.queue; block it; }
semSignal(s): s.count++; if (s.count <= 0) { remove P from s.queue; make P ready; }
```

Three consequences

- A process cannot know in advance whether its semWait will block.
- After a semSignal wakes a waiter, both continue concurrently in an unknown order.
- A signalling process does not learn whether anyone was waiting (it released one process or none).

Reading the value

Positive: how many more processes can call semWait and continue. **Zero:** no units left, nobody waiting.

Negative: its magnitude is the number of blocked processes (−3 means three are asleep). Blocked processes use no processor time, unlike busy waiting.

Kinds of semaphore

Kind	Values	Behaviour
Counting (general)	any integer	As defined above.
Binary	0 or 1	semWaitB: if 1, set to 0 and continue, else block. semSignalB: if nobody waits, set to 1, else release one waiter (value stays 0). A signal while already 1 is lost.
Mutex	locked / unlocked	Binary lock with ownership: only the locker may unlock.

Blocked processes wait in a **queue**. A **strong semaphore** releases them first-in-first-out, so none can starve. A **weak semaphore** leaves the order unspecified, so a process may be passed over again and again (starvation).

Worked trace: one producer, three consumers

A, B and C each need a result produced by D. D announces each result with semSignal; a consumer claims one with semWait. s starts at 1 (one result waiting); strong semaphore.

Operation	s	Queue	Operation	s	Queue
1 A wait	0	–	6 B wait	−3	C A B
2 B wait	−1	B	7 D signal (C freed)	−2	A B
3 D signal (B freed)	0	–	8 D signal (A freed)	−1	B
4 C wait	−1	C	9 D signal (B freed)	0	–
5 A wait	−2	C A	10 D signal (saved)	1	–

Mutual exclusion with one semaphore

```
semaphore s = 1;
void P(int i) { while (true) { semWait(s); /* critical section */; semSignal(s); /* remainder */; } }
void main() { parbegin (P(1), P(2), P(3)); }
```

The start value is how many may be inside at once, so it must be 1 (2 would let two in). If A, B, C call `semWait` in turn, `s` goes $1 \rightarrow 0$ (A enters) $\rightarrow -1$ (B blocks) $\rightarrow -2$ (C blocks). A's `semSignal` gives -1 and releases B into the critical section; B's gives 0 and releases C; C's returns `s` to 1.

Producer/consumer with an infinite buffer

The producer appends at `in`, the consumer takes at `out`; never take from an empty buffer, never touch the buffer at the same time.

Flawed solution with binary semaphores

```
int n = 0; binary_semaphore s = 1, delay = 0;
producer: produce(); semWaitB(s); append(); n++; if (n == 1) semSignalB(delay); semSignalB(s);
consumer: semWaitB(delay); loop { semWaitB(s); take(); n--; semSignalB(s); consume(); if (n == 0) semWaitB(delay); }
```

The bug: the consumer tests `n` **after** releasing `s`. If the producer runs in that gap, it adds an item and, since `n == 1`, signals `delay` although nobody waits (`delay = 1`). The consumer skips waiting, consumes that item, finds `n == 0` and calls `semWaitB(delay)`, which the stale 1 lets through. It then consumes an item that does not exist, and `n = -1`.

Fix: inside the critical section copy `n` into a private variable (`m = n`) and test `if (m == 0)` afterwards.

Correct solution with counting semaphores

```
semaphore n = 0, s = 1;
producer: produce(); semWait(s); append(); semSignal(s); semSignal(n);
consumer: semWait(n); semWait(s); take(); semSignal(s); consume();
```

`n` counts items, so the count and the wake-up can never disagree. Swapping the producer's two `semSignal` calls is harmless.

Bounded (circular) buffer

The buffer has a fixed size and is used in a circle: `in = (in + 1) % size`, `out = (out + 1) % size`.

Three semaphores:

- **s = 1:** mutual exclusion on the buffer.
- **n = 0:** number of items ready (the consumer waits on it when the buffer is empty).
- **e = size:** number of empty slots (the producer waits on it when the buffer is full).

```
producer: produce(); semWait(e); semWait(s); append(); semSignal(s); semSignal(n);
consumer: semWait(n); semWait(s); take(); semSignal(s); semSignal(e); consume();
```

Order matters. If the consumer called `semWait(s)` before `semWait(n)` with the buffer empty, it would block on `n` while holding `s`; the producer would block on `s`, and neither could continue: a **deadlock**. The bug only strikes when the buffer is empty, so it can hide in testing.

Implementing semaphores: the operations must be atomic

`semWait` and `semSignal` update shared data (count and queue). Run concurrently, both could read `count = 1`, both store `0`, and both enter. Only one process at a time may be inside either operation on a semaphore.

- **Hardware (multiprocessors):** guard a `s.flag` with `while (compare_and_swap(s.flag, 0, 1) == 1) ;` at the start and `s.flag = 0` at the end (a blocking process frees the flag as it blocks). The busy waiting lasts only for the few instructions of the operation, never a whole critical section.
- **Disabling interrupts (one processor only):** inhibit interrupts at the start, allow them at the end (or as the caller blocks). With one CPU a process loses the processor only through an interrupt, so the operation becomes indivisible; another core on a multiprocessor is not stopped.

Key terms

Semaphore

A shared integer variable that processes may touch only through three operations: initialize, `semWait` and `semSignal`. Processes use it to wait for, and to send, simple signals.

semWait

The semaphore operation that subtracts 1 from the value. If the result is negative, the calling process is blocked and placed in the semaphore's queue.

semSignal

The semaphore operation that adds 1 to the value. If the result is zero or negative, one process waiting in the semaphore's queue is unblocked.

Binary semaphore

A semaphore that can only hold 0 or 1. Its operations are usually written `semWaitB` and `semSignalB`.

Counting semaphore (general semaphore)

A semaphore whose value can be any integer, so it can count available units of a resource and, when negative, how many processes are waiting.

Mutex

A lock that behaves like a binary semaphore with one extra rule: the process that locked it (set it to 0) is the only one allowed to unlock it (set it to 1).

Strong semaphore

A semaphore that releases blocked processes in first-in, first-out order, so the longest waiter goes first and no waiter can starve.

Weak semaphore

A semaphore that does not say which blocked process is released next, so an unlucky process could be passed over again and again.

Starvation

A situation in which a process is overlooked indefinitely, even though other processes keep making progress.

Critical section

A piece of code that uses a shared resource. While one process is inside it, no other process may be inside a critical section for that same resource.

Mutual exclusion

The guarantee that while one process is inside a critical section for a resource, no other process is inside a critical section for that same resource.

Producer/consumer problem

A classic coordination problem: producers put items into a shared buffer and consumers take them out; the buffer must never be corrupted, over-filled, or read when empty.

Bounded buffer

A buffer with a fixed number of slots, usually reused in a circle; a producer must wait when every slot is full.

Deadlock

A permanent standstill in which every process in a group is waiting (blocked or spinning) for something that only another waiting member of the group can provide.

Atomic operation

An operation that happens as one indivisible unit: no other process can run in the middle of it or see it half-done.

Busy waiting

Waiting by running a loop that keeps testing a condition, which burns processor time instead of sleeping.

Spinlock

A lock where a process that finds it taken busy-waits (spins) in a loop until the lock becomes free.

Condition variable

A named queue that a process joins to wait until some condition about shared data becomes true; another process signals it when that happens.

Monitor

A programming-language construct that packages shared data with the only procedures allowed to use it, and lets just one process be active inside at a time.

Message passing (mailbox)

Coordination by sending and receiving messages. A mailbox is a shared place where messages wait until a receiver collects them.

Review questions with answers

1. A counting semaphore `s` has the value 0. Process P calls `semWait(s)`. What happens?

- A. `s` becomes `-1` and P blocks.
- B. `s` stays 0 and P blocks.
- C. `s` becomes `-1` and P continues.
- D. P busy-waits until `s` becomes positive.

Answer: A. `s` becomes `-1` and P blocks.

`semWait` always subtracts 1 first. A negative result means no unit was available, so the caller joins the queue and is blocked.

2. A counting semaphore `s` counts the results produced by process D and starts at 1. Consumers A, B and C each call `semWait(s)` once, in that order, and then D calls `semSignal(s)` once. What is the value of `s` now?

Answer: `-1`

$1 - 3 = -2$ after the three waits: A passed, B and C are blocked. D's signal gives $-2 + 1 = -1$ and releases B, the front waiter; the `-1` says C is still waiting.

3. Three processes share a critical section guarded by a semaphore initialized to 1. Each calls `semWait` in turn, and none has left the critical section yet. What is the semaphore's value?

Answer: `-2`

$1 \rightarrow 0$ (the first enters) $\rightarrow -1$ (the second blocks) $\rightarrow -2$ (the third blocks). The magnitude 2 is the number of waiting processes.

4. A binary semaphore has the value 1 and no process is waiting on it. After one more `semSignalB`, its value is 2.

Answer: False

A binary semaphore can only hold 0 or 1. With nobody waiting, `semSignalB` just sets the value to 1, so a second signal in a row is lost. Counting extra signals needs a counting semaphore.

5. Which statements follow directly from the definition of a semaphore?

- A. A process cannot know in advance whether its `semWait` will block.
- B. After a `semSignal` that wakes a waiter, the order in which the two processes continue is not specified.
- C. A process calling `semSignal` cannot tell whether any process was waiting.
- D. `semSignal` releases every waiting process at once.
- E. A semaphore's value can never be negative.

Answer: A, B, C

The first three are the standard consequences. `semSignal` releases at most one process, and a negative value is normal: it counts the waiters.

6. Which kind of semaphore guarantees that a blocked process cannot be passed over forever?

- A. A strong semaphore, which releases waiters first in, first out
- B. A weak semaphore, because it chooses at random
- C. A binary semaphore
- D. Any semaphore, because semSignal wakes the newest waiter

Answer: A. A strong semaphore, which releases waiters first in, first out

A strong semaphore serves its queue in arrival order, so every waiter eventually reaches the front. A weak semaphore makes no such promise.

7. Match each concurrency mechanism to its description.

Semaphore	An integer changed only by initialize, semWait and semSignal
Mutex	Only the process that locked it may unlock it
Spinlock	A waiter loops, testing the lock until it frees up
Event flags	Each bit of a memory word stands for one event
Mailbox	Holds messages until a receiver collects them
Monitor	Bundles shared data with the only procedures allowed to use it

These are the common tools. Semaphores and mutexes block the waiter; a spinlock makes it busy-wait; event flags and mailboxes signal events and data; a monitor is a language-level construct.

8. Put the bounded-buffer producer's loop body in order.

Correct order:

1. produce()
2. semWait(e)
3. semWait(s)
4. append()
5. semSignal(s)
6. semSignal(n)

Make the item, claim an empty slot (e), lock the buffer (s), add the item, unlock, then announce the new item (n).

9. Bounded buffer: sort each fact to the semaphore (s, n or e) it describes.

s	n	e
Initialized to 1 Keeps append() and take() from overlapping	Initialized to 0 The consumer waits on it before taking an item	Initialized to the buffer size The producer waits on it when every slot is full

s is the lock (starts at 1), n counts items (starts at 0), and e counts empty slots (starts at the buffer size).

- 10.** In the bounded-buffer consumer, what can happen if `semWait(s)` is called before `semWait(n)` ?
- A. Deadlock: with an empty buffer the consumer sleeps on `n` while holding `s`, so the producer can never add an item
 - B. Nothing: the order of the two `semWait` calls never matters
 - C. The consumer may take an item that does not exist
 - D. The producer may overwrite an item that has not been consumed

Answer: A. Deadlock: with an empty buffer the consumer sleeps on `n` while holding `s`, so the producer can never add an item

The consumer grabs `s`, then blocks on `n`. The producer finishes an item but blocks on `s`, which the sleeping consumer holds. Each waits for the other forever.

- 11.** In the flawed binary-semaphore producer/consumer with an infinite buffer (variables `n` and `delay`), why can the consumer end up consuming an item that does not exist?
- A. It tests `n` outside the critical section, so the producer can change `n` in between and leave a stale signal in `delay`
 - B. The producer forgets to lock the buffer before appending
 - C. `delay` is initialized to 1 instead of 0
 - D. Binary semaphores cannot provide mutual exclusion

Answer: A. It tests `n` outside the critical section, so the producer can change `n` in between and leave a stale signal in `delay`

The unmatched signal stored in `delay` later lets the consumer pass `semWaitB(delay)` when the buffer is empty. Saving `n` into a private `m` inside the critical section, or using a counting semaphore, fixes it.

- 12.** On a single-processor machine, what is a simple way to make `semWait` and `semSignal` atomic?
- A. Disable interrupts at the start of the operation and re-enable them at the end
 - B. Protect the semaphore with a second semaphore
 - C. Give every process the same priority
 - D. Busy-wait for the whole critical section

Answer: A. Disable interrupts at the start of the operation and re-enable them at the end

On one processor a process loses the CPU only through an interrupt, so blocking interrupts makes those few statements indivisible. Multiprocessors instead use an instruction such as `compare_and_swap`.

5.5 Monitors

You should be able to

- Explain why monitors were invented as a safer, language-level alternative to semaphore calls scattered through a program.
- Describe the parts of a monitor (local data, procedures, initialization) and its entrance, condition and urgent queues.
- Use `cwait` and `csignal` on condition variables, and explain why a signal sent when nobody waits is lost.
- Trace the bounded-buffer producer/consumer solution written as a monitor, line by line.
- Compare Hoare monitors (`csignal`) with Lampson/Redell Mesa monitors (`cnotify`, `cbroadcast`, watchdog timers), including why Mesa code re-tests with `while`.

Why monitors?

Semaphores can solve any synchronization problem, but every process must call `semWait` and `semSignal` in exactly the right places. For a bounded buffer shared by 2 producers and 2 consumers that means 16 calls spread over 4 processes (4 each; semaphore `s` guards the buffer, `n` counts items, `e` counts empty slots). Swap one pair, say a consumer taking `s` before `n`, and an empty buffer causes **deadlock**: the consumer sleeps on `n` while holding `s`, so no producer can get in. A **monitor** is a programming-language construct with the same power that is far easier to control: the shared data and all code touching it live in one module, and the language enforces mutual exclusion. Monitors are built into Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3 and Java.

Structure of a monitor

A software module made of **local data**, one or more **procedures**, and an **initialization sequence** that runs once when the monitor is created. Three characteristics define it:

1. **Private data**: only the monitor's own procedures can access the local data.
 2. **One way in**: a process enters only by calling one of its procedures.
 3. **One at a time**: only one process may be executing inside; other callers wait. Mutual exclusion comes for free.
- **Entrance queue**: callers blocked because another process is inside.
 - **Condition queues**: one per condition variable, holding processes suspended by `cwait`. They do not count as being inside.
 - **Urgent queue** (Hoare): signalers that still have work to do, parked after handing the monitor to the process they woke. When the monitor frees up, it is served before the entrance queue.

`cwait`, `csignal` and the lost signal

Condition variables are named waiting lines that exist only inside the monitor. `cwait(c)` always suspends the caller on `c` and releases the monitor. It tests nothing, so code checks its own data first: `if (count == 0) cwait(notempty);`. `csignal(c)` resumes one process waiting on `c`; if none is waiting, nothing happens

and the signal is **lost**. A semaphore instead remembers signals in its count. Example: with nobody waiting, 3 `csignal(c)` calls followed by 4 `cwait(c)` calls leave all 4 suspended; with a semaphore starting at 0, the 3 signals are saved (count = 3) and only 1 of 4 `semWait` calls blocks. In correct monitor code a lost signal is harmless, because a later arrival checks the variables itself.

The bounded buffer as a monitor

N slots used as a ring: `nextin` is the next slot to fill, `nextout` the next to empty, `count` the number of full slots. Producers wait on `notfull`, consumers on `notempty`.

```
monitor boundedbuffer;
char buffer[N]; int nextin, nextout, count;
cond notfull, notempty;
void append(char x) {
    if (count == N) cwait(notfull); /* full: sleep */
    buffer[nextin] = x;             /* store item */
    nextin = (nextin + 1) % N;      /* advance, wrap */
    count++;                        /* one more item */
    csignal(notempty);             /* wake a consumer */
}
void take(char &x) {
    if (count == 0) cwait(notempty); /* empty: sleep */
    x = buffer[nextout];            /* oldest item */
    nextout = (nextout + 1) % N;    /* advance, wrap */
    count--;                        /* one fewer item */
    csignal(notfull);              /* wake a producer */
}
{ nextin = 0; nextout = 0; count = 0; } /* init */
```

Producers loop `produce(x); append(x);` and consumers loop `take(x); consume(x);` with no synchronization of their own. **Ring index:** with $N = 5$ and `nextin = 4`, the next append stores into slot 4 and sets `nextin = (4 + 1) % 5 = 0`. **Traced run** ($N = 3$, Hoare): P1-P3 fill slots 0-2 (their signals are lost; `nextin` wraps to 0); P4 waits on `notfull`; C1 takes a and signals `notfull`, so P4 takes over at once while C1 waits in the urgent queue.

Hoare monitors and their drawbacks

Hoare's `csignal` hands the monitor **immediately** to the woken process, so its condition is guaranteed to hold and `if` is enough. The signaler must either leave at once (possible when `csignal` is its last statement) or wait in the urgent queue. Two drawbacks:

1. **Extra process switches:** if the signaler has not finished, the signal costs **two** extra switches, one to suspend it and one to resume it, however much work is left. Example: with switches of 0.5 ticks, identical work ends at 11.5 under Hoare versus 10.5 under Mesa.
2. **Perfectly reliable scheduling:** the woken process must be the next one inside. If another gets in first, the condition may be false again.

Lampson/Redell (Mesa) monitors

Mesa replaces `csignal` with **`cnotify(x)`**: a waiter on `x` becomes ready but the notifier keeps running. The waiter resumes at some later, convenient time, and others may enter first. Example: C1 waits on an empty buffer; P1 stores `a`, notifies and leaves; C2, already at the entrance, gets in first and takes `a`. With `if`, C1 then takes from an empty buffer (`count = -1`). The fix costs only a cheap re-test:

```
while (count == 0) cwait(notempty);    /* consumer */
while (count == N) cwait(notfull);     /* producer */
```

`cbroadcast(x)` makes every waiter on `x` ready, and each re-tests. Use it when one change may satisfy several waiters and the signaler cannot tell which. Example: 5 KB free, requests for 40, 10 and 25 KB wait; a release of 30 KB leaves 35 KB. `cnotify` wakes only the 40 KB request, which fails, so 35 KB sits idle; `cbroadcast` lets the 10 and 25 KB requests succeed. Broadcast is wasteful when only one waiter can benefit. POSIX threads (`pthread_cond_signal` , `pthread_cond_broadcast`) and Java (`notify` , `notifyAll`) follow Mesa rules.

Watchdog timer: a wait may carry a time limit; when it expires the waiter is made ready anyway and re-tests. Example: C waits from `t = 1`; a producer stores an item at `t = 8` but forgets `cnotify`. No timer: C sleeps forever. Limit 2: C wakes at 3, 5 and 7 (`count = 0`, back to sleep) and takes the item at 9, 1 tick late. With `if`, the wake-up at 3 would take from an empty buffer, so timers are safe only with `while`.

Advantages of Mesa: fewer process switches; less error-prone (early, extra or stray wake-ups are harmless because every waiter re-tests); more modular (a notifier need not know who waits or why).

Summary: Hoare vs Mesa

	Hoare (<code>csignal</code>)	Mesa (<code>cnotify</code> , <code>cbroadcast</code>)
After a signal	Woken process runs at once; an unfinished signaler waits (urgent queue)	Signaler keeps running; waiter resumes later
Waiter's test	<code>if</code> is enough	<code>while</code> is required
Cost	2 extra switches if the signaler is not done	No extra switches, a cheap re-test
Scheduling	Must be perfectly reliable	Tolerant of extra or early wake-ups
Extras	None	<code>cbroadcast</code> ; optional watchdog timer

Key terms

Monitor

A programming-language construct that bundles shared data with the only procedures allowed to touch it, and guarantees that at most one process is executing inside those procedures at any moment.

Condition variable

A named waiting line inside a monitor. A process that cannot continue waits on it with `cwait`; another process wakes it with `csignal` (or `cnotify` / `cbroadcast`). It stores no count and no value.

`cwait`

cwait(c): the calling process always suspends itself on condition c and gives up the monitor, so another process may enter.

csignal

csignal(c): resume one process that is waiting on condition c. If no process is waiting, nothing happens: the signal is lost.

Entrance queue

The line of processes that have called a monitor procedure but must wait outside because another process is already inside.

Condition queue

The line of processes suspended on one particular condition variable, waiting to be signaled.

Urgent queue

In a Hoare monitor, the place where a process that issued csignal, and still has work to do inside, waits after handing the monitor to the process it woke. It gets back in before any newcomer from the entrance queue.

Hoare monitor

The original monitor rules: csignal hands the monitor at once to the woken process, so the condition it waited for is guaranteed to still be true when it resumes.

Mesa monitor

The Lampson and Redell rules, first used in the Mesa language: cnotify only makes a waiter ready, the notifier keeps running, and the waiter must re-test its condition with a while loop.

cnotify

cnotify(x): tell one process waiting on condition x that its condition may now hold. It resumes at some convenient later time while the notifier carries on.

cbroadcast

cbroadcast(x): make every process waiting on condition x ready. Each one re-tests its own condition when it gets the monitor.

Watchdog timer

A time limit on a wait. If a waiting process is not notified in time, it is made ready anyway and re-tests its condition, so a missing notify cannot strand it forever.

Lost signal

A csignal sent when no process is waiting on that condition. The monitor keeps no record of it, whereas a semaphore remembers a signal by increasing its count.

Bounded buffer

A shared store with a fixed number of slots, filled by producers and emptied by consumers. Producers must wait when it is full, consumers when it is empty.

Semaphore

An integer used for signaling between processes that is changed only by the atomic operations semWait (decrement, maybe block) and semSignal (increment, maybe unblock).

Mutual exclusion

The guarantee that while one process is using a shared resource or critical section, no other process can be using it.

Deadlock

A permanent standstill: each process in a group is waiting (blocked or spinning) for something that only another waiting member of the group can provide.

Process switch

Taking the processor away from one process and giving it to another. It costs time to save one process's state and restore the other's.

Review questions with answers

1. Which guarantee does a monitor give automatically, without the programmer writing any lock calls?

- A. At most one process is executing inside the monitor at any moment
- B. Processes enter the monitor in priority order
- C. A signal sent while nobody waits is saved for later
- D. Waiting processes never need to re-check their condition

Answer: A. At most one process is executing inside the monitor at any moment

Mutual exclusion is built in. A process can enter only by calling one of the monitor's procedures, and only one process may be executing inside at a time.

2. No process is waiting on condition variable `c`. Three processes call `csignal(c)`, one after another. Then four processes each call `cwait(c)`. How many of those four end up suspended?

Answer: 4 processes

A condition variable keeps no count, so all three signals found nobody waiting and were lost; every one of the four `cwait` calls suspends. A semaphore starting at 0 would have saved the three signals (count = 3), so only the fourth `semWait` would block.

3. Which statements about `cwait(c)` are true?

- A. It always suspends the calling process
- B. It releases the monitor so another process can enter
- C. It tests a counter and returns at once if the counter is positive
- D. Monitor code normally checks its own data (such as count) before calling it

Answer: A, B, D

`cwait` is unconditional: it suspends the caller on `c` and frees the monitor. Because it tests nothing, monitor code checks its own variables first and calls `cwait` only when it really must wait.

4. Match each part of a monitor to its job.

Entrance queue	Callers waiting to get in while someone is inside
Condition queue	Processes suspended by <code>cwait</code> on one condition
Urgent queue	Hoare signalers waiting to get back in
Local data	Variables that only the monitor's procedures can touch
Initialization code	Sets the starting state once, when the monitor is created

The entrance queue enforces one-at-a-time entry, each condition variable has its own queue, the urgent queue holds Hoare signalers, and the local data and initialization code live privately inside the module.

5. In a Hoare monitor, process Q waits on condition `c`. Process P later signals `c` while it still has work left inside the monitor. Put the events in order.

Correct order:

1. Q calls `cwait(c)`: it is suspended and the monitor is released
2. P enters the monitor and calls `csignal(c)`
3. Q resumes inside the monitor at once, while P waits in the urgent queue
4. Q returns from its procedure and leaves the monitor
5. P re-enters from the urgent queue, ahead of any newcomers
6. P finishes its remaining work and leaves

Hoare rules hand the monitor straight to the woken process and park an unfinished signaler in the urgent queue. When the woken process leaves, the urgent queue is served before the entrance queue, so P gets back in first.

6. Under Hoare rules, a process signals a condition (and someone is waiting) while it still has more work to do inside the monitor. How many extra process switches does that signal cause?

Answer: 2 switches

One switch suspends the signaler so the woken process can run at once, and a second one resumes the signaler later. A Mesa-style `cnotify` causes neither.

7. In a Mesa (Lampson/Redell) monitor, why should a consumer write `while (count == 0) cwait(notempty);` rather than `if ?`

- A. `cnotify` only makes the waiter ready, and another process may change count before the waiter runs again
- B. A while loop makes the consumer run faster
- C. Mesa monitors do not provide mutual exclusion
- D. `cwait` returns immediately in Mesa monitors

Answer: A. `cnotify` only makes the waiter ready, and another process may change count before the waiter runs again

Between the notify and the moment the waiter gets the monitor back, a third process can slip in and take the item. Only a re-test catches that.

8. Hoare-style or Mesa-style monitor?

Hoare (<code>csignal</code>)	Mesa (<code>cnotify</code>)
An unfinished signaler steps aside into an urgent queue A waiter can safely use if Needs perfectly reliable scheduling	The signaler keeps running after it signals A waiter must re-test with while Offers broadcast and wait timeouts

Hoare signals hand the monitor over at once, which makes it safe but costs switches and demands reliable scheduling. Mesa notifies are hints, so waiters loop, and broadcast and timeouts become safe additions.

9. A memory manager frees a block. Several processes are waiting for blocks of different sizes. Why is `cbroadcast` a better choice than `cnotify` ?

- A. The manager cannot tell which waiters can now proceed, so it wakes them all and each re-tests
- B. `cbroadcast` hands the memory to the largest request first
- C. `cbroadcast` guarantees every waiting process gets its memory
- D. `cnotify` would wake all of them anyway

Answer: A. The manager cannot tell which waiters can now proceed, so it wakes them all and each re-tests

Waking a single waiter may pick one that still does not fit, leaving memory idle while others could run. Broadcasting lets every waiter re-test: those that fit proceed, the rest sleep again.

10. A watchdog timer on a Mesa-style wait lets a process resume after a time limit even if no notify arrives, and this is safe because the process re-tests its condition.

Answer: True

Since Mesa waiters always re-check with `while`, waking early does no harm. The timer turns a forgotten notify into a delay instead of a process stuck forever.

11. A bounded buffer has $N = 5$ slots, and `nextin` is 4. After one more `append`, what is `nextin` ?

Answer: 0

`nextin = (4 + 1) % 5 = 0` : the index wraps around to the first slot, so the array is used as a ring.

12. Semaphores can already solve any synchronization problem. Why were monitors introduced?

- A. Semaphore calls end up scattered across many processes, so one misplaced call is easy to make and hard to find
- B. Semaphores cannot solve the producer/consumer problem
- C. Monitors always run faster than semaphores
- D. Semaphores work only on single-processor machines

Answer: A. Semaphore calls end up scattered across many processes, so one misplaced call is easy to make and hard to find

A monitor gathers the shared data and all of its synchronization into one module, so correctness is easier to control and verify.

5.6 Message Passing

You should be able to

- Explain how `send` and `receive` give processes both synchronization and communication, on one machine or many.
- Predict who blocks under each mix of blocking and nonblocking `send` and `receive`, and name the risk of each.
- Compare direct and indirect addressing, the four sender/receiver patterns, and how mailboxes and ports are bound and owned.
- Describe the header and body of a message and choose a queuing discipline for a mailbox.
- Use messages to enforce mutual exclusion and to solve the bounded-buffer producer/consumer problem.

What message passing is

Cooperating processes need two things: **synchronization** (staying in step, for example not using data before it exists) and **communication** (handing data over). Semaphores and monitors provide synchronization and rely on shared variables for the data. **Message passing** provides both at once: a message carries the data, and a process waiting to receive it cannot run ahead of the sender. It works on a single processor, on a shared-memory multiprocessor and across a network of machines that share no memory at all.

Message passing is how processes cooperate when they share no memory: across a network, and inside microkernel operating systems whose services run as separate processes. Two primitives do the work:

- `send(destination, message)` : deliver this message to a process or mailbox.
- `receive(source, message)` : collect a message from a process or mailbox into the caller's message variable.

Design characteristics

Question	Options
Synchronization	send blocking or nonblocking; receive blocking, nonblocking, or test for arrival
Addressing	direct (explicit or implicit) or indirect (mailbox), with static or dynamic association and some owner
Format	header + body; fixed or variable length
Queuing discipline	FIFO, priority, or receiver's choice

Blocking and nonblocking

- **Blocking send:** the sender waits until its message has been received. **Nonblocking send:** the sender continues at once; the message waits to be picked up.
- **Blocking receive:** with no message waiting, the caller sleeps until one arrives. **Nonblocking receive:** it returns at once, with a message or with "no message". **Test for arrival:** check whether anything is waiting without taking it or blocking.

Combination	Behaviour	Risk
Blocking send + blocking receive	Rendezvous: whoever arrives first waits; tight synchronization; only the message being handed over needs storing.	Either side can wait forever if the partner crashes or the message is lost.
Nonblocking send + blocking receive	The most useful mix: senders never stall; an idle receiver (such as a server) sleeps without using the processor.	A faulty sender can flood the system with messages; the sender learns of delivery only if the receiver replies.
Nonblocking send + nonblocking receive	Nobody ever waits.	The receiver must keep polling; a message arriving just after an empty receive can go unnoticed.

The fourth mix, blocking send with nonblocking receive, is possible but rarely chosen. A blocking receive that could wait forever is usually protected by a **timeout** (give up and return an error after a time limit) or by testing for arrival first.

Addressing

Direct addressing names a process. `send` gives the destination process ID. `receive` either names the one sender it will accept (explicit, fine when the partner is known in advance) or accepts anyone (**implicit**): the source parameter comes back holding the sender's ID. A server needs implicit addressing, because it cannot list every client in advance. With direct addressing there is no way to name a group, so reaching three receivers takes three sends, and a sender must pick a specific receiver by name.

Indirect addressing sends to a shared **mailbox**, a queue of messages. Senders and receivers never name each other; they only share the mailbox name, which decouples them. It supports four patterns:

- **One-to-one**: a private link between two processes (for example two pipeline stages).
- **Many-to-one**: client/server; many clients send requests to one server. Such a mailbox is usually a **port**.
- **One-to-many**: broadcast; one send and every member of the group gets a copy.
- **Many-to-many**: several servers share one mailbox; whichever asks first gets the next request.

Binding and ownership of mailboxes

- **Static association**: the link between processes and a mailbox is fixed once, when it is set up. Typical for a permanent one-to-one link.
- **Dynamic association**: processes attach and detach while running, with `connect` and `disconnect`. Needed when senders come and go, as with a server's clients.

Owner	The mailbox is destroyed when
The receiving process (a port)	that receiver ends; later sends fail and queued messages are lost
The process that created it	its creator ends, even if others still use it
The operating system	someone explicitly destroys it; it outlives any process, so a new receiver can take over the queued messages

Message format

A message has a **header** that describes it and a **body** that holds the contents. Header fields:

- **Message type**: what kind of message it is (request, reply, alarm...).
- **Destination ID** and **source ID**: where it goes and who sent it.
- **Message length**: size of the body, essential for variable-length messages.
- **Control information**: for example a sequence number, a priority, or a pointer that links messages into a queue.

Fixed-length messages are easy to store and queue but waste space on short data and force long data to be split. **Variable-length** messages waste nothing but need the length field and per-message space

allocation. Worked example: with a 64-byte fixed body, a 200-byte record needs 4 messages ($200 \div 64 = 3.125$, round up), and the last one carries 8 bytes plus 56 bytes of padding.

Queuing discipline

The rule for which waiting message a receiver gets next. **FIFO** (arrival order) is the simple, fair default, but urgent messages wait behind routine ones. **Priority** order lets urgent messages go first (by type or a sender-chosen priority; ties by arrival), at the risk of low-priority messages waiting a long time. Or the **receiver chooses**: it inspects the queue and selects the message it wants. Example: messages #1 (priority 1), #2 (3), #3 (2), #4 (3) in arrival order: FIFO delivers #1 first; priority delivers #2 (highest priority, older than #4).

Mutual exclusion with one message

```
create_mailbox(box); // shared by all processes
send(box, null);     // exactly one message: the token
while (true) {       // each process loops forever
    receive(box, msg); // take the token, or block
    critical_section(); // only the token holder is here
    send(box, msg);    // return the token
    remainder();       // work that needs no token
}
```

Receive is blocking and send is nonblocking. Whoever holds the single message is the only process in its critical section. If several processes call receive while the message is in the mailbox, exactly one gets it; if several are blocked when it is sent back, exactly one wakes. The number of messages placed in the mailbox at the start is how many processes may be inside together, just like a semaphore's initial value: 0 means nobody can ever enter, 2 breaks mutual exclusion. A process that crashes while holding the token (or forgets to send it back) leaves everyone else blocked forever.

Producer/consumer with two mailboxes

For a bounded buffer of capacity N, create mailboxes **mayproduce** and **mayconsume**, then send N empty (null) messages to mayproduce. Each empty message is permission to produce: one free slot.

```
producer: while (true) { // loop forever
    receive(mayproduce, pmsg); // wait for a free slot
    pmsg = produce();        // fill the message
    send(mayconsume, pmsg);  // pass it on
}                             // and repeat
consumer: while (true) { // loop forever
    receive(mayconsume, cmsg); // wait for data
    consume(cmsg);           // use the item
    send(mayproduce, null);    // return a free slot
}                             // and repeat
```

The full messages in mayconsume **are** the buffer. The producer blocks when mayproduce is empty (buffer full); the consumer blocks when mayconsume is empty (no data). A send from one side wakes the other.

The total number of messages in circulation always equals N . Worked example: $N = 5$, producer has sent 7, consumer has consumed 4 and returned 4 empties, neither holds one: $\text{mayproduce} \text{ has } 5 - 7 + 4 = 2$ empties and mayconsume holds 3 full messages ($2 + 3 = 5$). The scheme works unchanged with several producers and several consumers.

Key terms

Message passing

Cooperation in which processes hand each other information by calling send and receive instead of sharing variables. Because a receiver can wait for a message, the same calls also synchronize the processes.

Message

A packet of information passed from one process to another: a header that describes it plus a body that holds the actual contents.

Send primitive

`send(destination, message)`: hands a message to a named process or mailbox.

Receive primitive

`receive(source, message)`: collects a message from a named process or mailbox and stores it in the caller's message variable.

Blocking send

A send after which the sending process is suspended until its message has been received.

Nonblocking send

A send that returns at once; the sender keeps running while its message waits to be received.

Blocking receive

A receive that suspends the caller until a message is available, then returns with it.

Nonblocking receive

A receive that returns at once: with a message if one is waiting, otherwise with a "no message" result. The caller continues either way.

Test for arrival

A check that reports whether a message is waiting, without taking it and without blocking, so the process can decide whether to call receive.

Rendezvous

The meeting that happens when send and receive are both blocking: whichever process arrives first waits, and the message passes when both are present.

Receive timeout

A time limit placed on a blocking receive: if no message arrives in time, the call gives up and returns an error instead of waiting forever.

Direct addressing

Naming a specific process: send gives the destination process's ID, and receive either names the expected sender or accepts any sender.

Implicit addressing

A receive that does not name a sender. It accepts a message from any process and fills in the source parameter with the sender's ID when it completes.

Indirect addressing

Sending to a shared mailbox instead of to a process. Receivers take messages from the mailbox, so senders and receivers do not need to know each other.

Mailbox

A queue of messages kept as a shared data structure. Senders put messages in; receivers take them out.

Port

A mailbox tied to one receiving process, usually created and owned by it, that many senders can use. It is destroyed when its owner ends.

Dynamic association

Linking processes to a mailbox while the system runs, with connect and disconnect calls, so senders can come and go. Its opposite, static association, fixes the link once and for all when the mailbox is set up.

Message header

The descriptive part of a message: message type, destination ID, source ID, message length and control information such as a sequence number or priority.

Queuing discipline

The rule that picks which waiting message a receiver gets next: first-in-first-out by default, or by priority, or by the receiver's own choice.

Bounded buffer

A buffer with a fixed number of slots; a producer must wait when all slots are full and a consumer must wait when all are empty.

Review questions with answers

1. Which combination of send and receive produces a **rendezvous**?

- A. Blocking send and blocking receive
- B. Nonblocking send and blocking receive
- C. Nonblocking send and nonblocking receive
- D. Blocking send and nonblocking receive

Answer: A. Blocking send and blocking receive

When both calls block, whichever process arrives first waits for the other, and the message passes only when both are present. That tight meeting point is the rendezvous.

2. A server uses **direct** addressing and must accept requests from any client, then reply to each one. How should its receive work?

- A. Name one specific client as the source
- B. Leave the source open, accept any sender and find out the sender's ID when the message arrives
- C. Call receive once for each known client, naming them in turn
- D. Use static association

Answer: B. Leave the source open, accept any sender and find out the sender's ID when the message arrives

Implicit addressing lets a receive accept a message from anyone; the source parameter is filled in on return, so the server knows whom to answer.

3. With a nonblocking send, a faulty process stuck in a loop can flood the system with messages, because nothing ever makes it wait.

Answer: True

A nonblocking send returns at once, so no delay ever slows the sender. The system's buffers and processor time can be eaten up by the flood.

4. Match each way of owning or binding a mailbox with what it means.

Port (owned by its receiver)	Destroyed when that receiving process ends
Mailbox owned by the process that created it	Destroyed when its creator ends
Mailbox owned by the operating system	Lasts until someone explicitly destroys it
Static association	The link is fixed once, when the mailbox is set up
Dynamic association	Processes connect and disconnect while running

Ownership decides a mailbox's lifetime: a port dies with its receiver, a creator-owned mailbox with its creator, and an OS-owned mailbox only on an explicit destroy. Association decides whether the set of attached processes is fixed (static) or can change with connect and disconnect (dynamic).

5. Which of these belong in a message's **header**?

- A. Message type
- B. Destination ID
- C. Source ID
- D. Message length
- E. The data being delivered
- F. Control information such as a sequence number or priority

Answer: A, B, C, D, F

The header describes the message: its type, where it goes, who sent it, how long it is, and bookkeeping such as sequence numbers and priority. The data itself travels in the body.

6. Match each communication pattern with a typical use.

One-to-one	A private link between two stages of a pipeline
Many-to-one	Many clients sending requests to one server
One-to-many	Warning every member of a group about a shutdown
Many-to-many	Several servers taking requests from one shared mailbox

The patterns count senders and receivers. Many-to-one is client/server (usually a port), one-to-many is broadcast, and many-to-many lets a pool of servers share the work.

7. Does each call **block** the caller, or return at once?

Can block the caller	Always returns at once
Blocking receive when the mailbox is empty Blocking send before the message has been received	Nonblocking receive when the mailbox is empty Nonblocking send Test for arrival

Only the blocking forms ever suspend the caller. A nonblocking receive returns “no message”, and a test for arrival just reports whether something is waiting.

8. Put the steps of mutual exclusion with a single token message in order.

Correct order:

1. Create a mailbox that every process shares
2. Send exactly one (null) message into it
3. A process calls receive and gets the message
4. It runs its critical section
5. It sends the message back to the mailbox

The single message is the token: set it up once, take it to enter, and put it back on the way out so the next process can enter.

9. Messages have a fixed-length body of 64 bytes. How many messages are needed to send a 200-byte record?

Answer: 4 messages

$200 \div 64 = 3.125$, so three full messages carry 192 bytes and a fourth carries the last 8 (with 56 bytes of padding). Fixed length is simple to manage but wastes space and forces long data to be split.

10. A producer and consumer share a buffer of capacity 5, built from mailboxes `mayproduce` and `mayconsume`. So far the producer has sent 7 full messages and the consumer has consumed 4 of them, returning an empty message each time. Neither process is holding a message. How many empty messages are in `mayproduce`?

Answer: 2 messages

`mayproduce` started with 5. The producer took 7 out and the consumer put 4 back: $5 - 7 + 4 = 2$. Check: 3 full messages wait in `mayconsume`, and $2 + 3 = 5$, the capacity.

11. A mailbox holds four messages, oldest first: #1 (priority 1), #2 (priority 3), #3 (priority 2), #4 (priority 3). It uses priority order, with ties broken by arrival. Which message does the next receive get?

- A. #1
- B. #2
- C. #3
- D. #4

Answer: B. #2

Priority order picks the highest priority (3), and among #2 and #4 the older one, #2, goes first.

12. In the two-mailbox producer/consumer solution, what does a message waiting in `mayproduce` stand for?

- A. An item that is ready to be consumed
- B. Permission to produce: one free slot in the buffer
- C. A request from the consumer to stop producing
- D. A lock on the whole buffer

Answer: B. Permission to produce: one free slot in the buffer

`mayproduce` starts with one empty message per slot. The producer must receive one before producing, so when all slots are full it blocks until the consumer returns an empty.

5.7 Readers/Writers Problem

You should be able to

- State the three readers/writers rules and explain why this is neither plain mutual exclusion nor producer/consumer.
- Trace the readers-priority semaphore solution line by line and explain why a writer can starve.
- Explain how `rsem`, `y` and `z` give writers priority, and describe the semaphore queues in the four classic situations.
- Trace the message-passing controller and read the meaning of its count variable (above zero, zero, below zero).

1. The problem

A **shared data area** (a file, a block of main memory, or even a bank of processor registers) is used by two kinds of processes. **Readers** only read it and never change it, so they may share access. **Writers** modify it (a writer may also read the data as part of its update), so each writer needs exclusive access. Three rules must hold:

1. Any number of readers may read at the same time.
2. Only one writer at a time may write.
3. While a writer is writing, no reader may read.

Rule 3 exists because a reader could otherwise see an update half-done. Example: a board says "09:40, gate B7"; the writer has changed the time to 10:15 but not yet the gate, so a reader sees "10:15, gate B7", a combination that was never true.

Why it is its own problem

- **Not plain mutual exclusion.** If we could not tell which processes only read, one lock (one critical section) around every access would be the only safe choice. Knowing that readers never change the data lets them share. Example: 4 readers of 2 ms each take $4 \times 2 = 8$ ms under one lock, but 2 ms when they overlap (a processor each).
- **Not producer/consumer.** It can look like one writer (producer) and one reader (consumer), but the consumer is not read-only: it removes an item and updates the buffer's pointers. The producer reads those pointers to find a free slot and check for "full", then updates them. Both sides change the buffer, so neither may share it, and each waits for the other (an item to take, a slot to fill). Readers never change the data, which is exactly why many may share it.

2. Solution 1: readers have priority

Shared: `int readcount = 0; semaphore x = 1, wsem = 1;` . `x` protects `readcount`; `wsem` protects the data.

```
/* reader, each time it reads */
semWait(x); // lock the counter
readcount++; // one more reader inside
if (readcount == 1) semWait(wsem); // first reader bars writers
semSignal(x); // unlock the counter
READUNIT(); // read (others may too)
semWait(x); // lock the counter again
readcount--; // one fewer reader inside
if (readcount == 0) semSignal(wsem); // last out lets writers in
semSignal(x); // unlock the counter

/* writer, each time it writes */
semWait(wsem); // wait until data is free
WRITEUNIT(); // write, completely alone
semSignal(wsem); // hand the data back
```

- Every writer does `semWait(wsem) / semSignal(wsem)` around its write.
- Among readers only the **first in** (`readcount 0 → 1`) takes `wsem` and only the **last out** (`1 → 0`) returns it. The readers hold `wsem` as one group.
- Where they wait: a writer waits on `wsem`. If a writer is writing, the first arriving reader blocks on `wsem` *while still holding x*, so any further readers block on `x`.
- **Weakness: writers can starve.** A newly arriving reader joins whenever other readers are inside, even if a writer is waiting. If readers keep overlapping, `readcount` never returns to 0 and the writer waits indefinitely.

3. Solution 2: writers have priority

Goal: once a writer has declared that it wants to write, no new reader may start. Shared: `int readcount = 0, writecount = 0; semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;`

Name	Job
wsem	Guards the data: held by one writer or by the group of readers (first reader in, last reader out).
rsem	Reader gate. The writer that takes writecount 0 → 1 closes it; the writers then hold it as a group, and whichever writer takes writecount 1 → 0 reopens it.
z	A reader keeps z while it waits on rsem, so only one reader at a time can wait on rsem; the others wait on z. A writer calling semWait(rsem) is never behind a crowd of readers: at worst it waits briefly for that one reader, still holding y, so other writers briefly wait on y.
x / y	Protect readcount / writecount.
writecount	Writers that are waiting or writing.

```

/* reader */
semWait(z); // readers line up here
semWait(rsem); // closed while writers wait
semWait(x); // lock readcount
readcount++; // one more reader inside
if (readcount == 1) semWait(wsem); // first reader bars writers
semSignal(x); // unlock readcount
semSignal(rsem); // reopen the reader gate
semSignal(z); // next reader may go on
READUNIT(); // read (others may too)
semWait(x); // lock readcount
readcount--; // one fewer reader inside
if (readcount == 0) semSignal(wsem); // last out lets writers in
semSignal(x); // unlock readcount

/* writer */
semWait(y); // lock writecount
writecount++; // one more writer wants in
if (writecount == 1) semWait(rsem); // first one shuts the gate
semSignal(y); // unlock writecount
semWait(wsem); // then wait for the data
WRITEUNIT(); // write, completely alone
semSignal(wsem); // hand the data back
semWait(y); // lock writecount
writecount--; // one fewer writer
if (writecount == 0) semSignal(rsem); // last one reopens the gate
semSignal(y); // unlock writecount

```

The four queue pictures

Situation	Semaphores and queues
Only readers	The readers hold wsem; nobody waits anywhere.
Only writers	A writer holds wsem, the writers keep rsem closed; the other writers line up on wsem.
Both, read first	The readers hold wsem, a writer has closed rsem; every writer lines up on wsem, one reader waits on rsem, the other readers wait on z.
Both, write first	A writer holds wsem, the writers keep rsem closed; writers line up on wsem, one reader waits on rsem, the other readers wait on z.

Readers already inside when a writer arrives finish normally; the last one out hands wsem to the writer. The price: a stream of writers can keep readers waiting.

4. Solution 3: message passing

A **controller process** owns access. Readers and writers send requests to the mailboxes **readrequest** or **writerequest**, block on their own mailbox (mbox[i]) until the controller sends them "OK", use the data, then send a message to **finished**. The controller keeps **count**, a private variable no other process touches, which starts at 100 (any number above the largest possible number of readers):

- **count > 0**: no writer accepted yet (a request may sit unread in writerequest); count = 100 – readers inside. The controller serves **finished** first (count++), then **writerequest** (count -= 100), then **readrequest** (count--, send OK). Checking writers before readers gives writers priority.
- **count == 0**: only the accepted writer is left. Send it OK, wait for its finished message, reset count to 100.
- **count < 0**: a writer is waiting for –count readers to leave. The controller receives only finished messages, adding 1 each time, until count reaches 0.

Worked example. Three readers inside: count = 97. A write request is accepted: $97 - 100 = -3$. Three finished messages: -2, -1, 0. The writer gets OK, writes, reports finished; count = 100. Only now is a waiting read request served (count = 99).

5. Comparison

Solution	Tools	Who can wait a long time?
Readers priority	x, wsem, readcount	Writers (can starve)
Writers priority	+ rsem, y, z, writecount	Readers, if writers keep coming
Message passing	controller, 3 mailboxes, count	Readers (writers are served first)

Key terms

Readers/writers problem

A classic synchronization problem: several processes share one data area. Readers only look at it, so any number may read together. Writers modify it (a writer may also read it while updating), so a writer must have the data entirely to itself.

Reader

A process that only reads the shared data and never changes it. Because reading changes nothing, readers may share access with each other.

Writer

A process that modifies the shared data. It may read the data as part of its update, but because it changes it, it needs exclusive access: no reader and no other writer at the same time.

Shared data area

Whatever the readers and writers use in common: a file, a block of main memory, or even a bank of processor registers.

Semaphore

A shared integer that processes may change only through atomic operations: `semWait` subtracts 1 and blocks the caller if the result is negative; `semSignal` adds 1 and wakes one blocked process if any are waiting.

readcount

A shared counter of how many readers are currently reading. The reader that raises it from 0 to 1 locks writers out; the reader that brings it back to 0 lets them in.

writecount

In the writers-priority solution, a shared counter of how many writers are waiting to write or writing.

wsem

The semaphore that guards the data itself. Whenever the data is in use, `wsem` is held either by one writer or by the whole group of current readers.

rsem

In the writers-priority solution, the "reader gate": the first writer that shows up closes it so no new reader can start, and the last writer to finish opens it again.

Readers priority

A policy in which a newly arriving reader joins the readers already reading, even if a writer is waiting. It maximizes sharing but can starve writers.

Writers priority

A policy in which, once a writer has announced that it wants to write, no new reader may start reading until every waiting writer has written.

Starvation

A situation in which a process waits indefinitely, not because of a deadlock, but because other processes keep being served ahead of it.

Mutual exclusion

The guarantee that while one process is using a shared resource in its critical section, no other process can be using that same resource.

Producer/consumer problem

A classic problem in which producers put items into a shared buffer and consumers take them out. Both sides change the buffer and its bookkeeping (a producer adds an item after checking for a free slot, a consumer removes one), so neither side is a read-only reader.

Message passing

Cooperation by sending and receiving messages (send and receive operations) instead of sharing variables. A receive can block until a message arrives, so messages also synchronize.

Mailbox

A named queue that holds messages until a process receives them. Many processes may send to the same mailbox.

Controller process

In the message-passing solution, the single process that decides who may use the shared data. Readers and writers ask it for permission and wait for its "OK" reply.

Review questions with answers

1. Which situation breaks the readers/writers rules?

- A. Three readers reading the data at the same time
- B. One writer writing while nobody else uses the data
- C. One reader reading while one writer writes
- D. Nobody using the data at all

Answer: C. One reader reading while one writer writes

While a writer is writing, no reader may read. The reader could see a half-finished update that mixes old and new values.

2. In the readers-priority semaphore solution, every reader performs `semWait(wsem)` before it reads.

Answer: False

Only the reader that raises `readcount` from 0 to 1 calls `semWait(wsem)`, and only the one that brings it back to 0 calls `semSignal(wsem)`. The readers hold `wsem` as a group.

3. In the readers-priority solution, why can a writer starve?

- A. It needs `readcount` to reach 0, but new readers keep joining while others are still reading
- B. It is blocked on `x`, which readers never release
- C. The semaphore `wsem` starts at 0
- D. Readers are faster than writers, so the scheduler prefers them

Answer: A. It needs `readcount` to reach 0, but new readers keep joining while others are still reading

A writer needs `wsem`, and the readers give it back only when the last of them leaves. If readers keep overlapping, that moment never arrives.

4. Which statements about the writers-priority semaphore solution are true?

- A. The writer that raises `writecount` from 0 to 1 closes `rsem`, so no new reader can start
- B. `writecount` counts the writers that are waiting or writing
- C. Two writers may write together as long as no reader is reading
- D. At most one reader is ever queued on `rsem`; other waiting readers queue on `z`
- E. Readers already inside are forced to stop the moment a writer arrives

Answer: A, B, D

`rsem` is the reader gate, driven by `writecount`, and `z` keeps `rsem`'s queue down to one reader. Writers still take `wsem` one at a time, and readers already inside are allowed to finish: they drain out.

5. Match each variable of the writers-priority solution with its job.

rsem	Keeps new readers out while any writer is interested
wsem	Gives the data to one writer or to the group of readers
x	Protects readcount
y	Protects writecount
z	Lets only one reader at a time wait on rsem

wsem guards the data itself; rsem is the gate writers close; x and y protect the two counters; z is the turnstile that stops readers piling up on rsem.

6. Writers-priority semaphore solution: the readers hold wsem, a writer has closed rsem, every writer is queued on wsem, one reader is queued on rsem and the other waiting readers are queued on z. Which situation is this?

- A. Only readers are in the system
- B. Only writers are in the system
- C. Both readers and writers, and a read came first
- D. Both readers and writers, and a write came first

Answer: C. Both readers and writers, and a read came first

A reader got in first and holds wsem for the readers. The first writer then closed rsem and queued on wsem; later readers stop at rsem (just one) and z (the rest).

7. Readers priority: put the steps of the **first** reader to arrive in order.

Correct order:

1. semWait(x): lock the counter
2. readcount++ (it is now 1)
3. semWait(wsem): lock writers out
4. semSignal(x): unlock the counter
5. READUNIT(): read the data

The counter is locked, updated and tested; the first reader claims wsem for the group; only then does it release the counter and read.

8. Which semaphore solution does each feature belong to?

Readers priority	Writers priority
Uses only two semaphores, x and wsem A newly arriving reader may join while a writer waits A waiting writer can starve	Needs a writecount variable A stream of writers can keep readers waiting Uses the semaphores rsem, y and z

Readers priority is the short solution that lets readers join freely, at the risk of starving writers. Writers priority adds rsem, y, z and writecount so a waiting writer is never overtaken by new readers.

9. Message-passing solution: count starts at 100. The controller has granted 6 read requests and received 2 finished messages. It now accepts a write request. What is count right after that?

Answer: -4

$100 - 6 + 2 = 96$, meaning four readers are inside. $96 - 100 = -4$, so the controller now waits for four finished messages before the writer may start.

10. Message-passing readers/writers solution: the controller's count is -2. How many finished messages must it receive before it sends the waiting writer its OK?

Answer: 2

A negative count is minus the number of readers still inside. Each finished message adds 1, and when count reaches 0 the writer gets its OK.

11. In the message-passing solution, when count > 0 the controller checks the finished mailbox first, then writerequest, then readrequest. Checking writerequest before readrequest is what gives writers priority over newly arriving readers.

Answer: True

Serving finished first keeps count accurate; checking writerequest before readrequest means a waiting writer is always taken ahead of a new reader, and once it is taken count ≤ 0 , so no further read request is served until the writer is done.

12. Why is the readers/writers problem not simply the producer/consumer problem?

- A. Readers never change the data, so they may share it; a consumer is not read-only, because it removes an item and updates the buffer's pointers
- B. There are always more readers than writers
- C. Producer/consumer cannot be solved with semaphores
- D. In readers/writers no process ever has to wait

Answer: A. Readers never change the data, so they may share it; a consumer is not read-only, because it removes an item and updates the buffer's pointers

It can look like one writer (the producer) and one reader (the consumer), but the consumer changes the buffer too: it removes an item and updates the pointers, while the producer reads those pointers to find a free slot and then updates them. Both sides modify shared data, so neither may share it. Readers leave the data untouched, which is exactly why many of them may share it at once.